ЯЗЫК С ЧЕРЕЗ ПРОЕКТ EH/UNWIND ANALYZER

Фундаментальный учебник для новичка

От первой строки С до самостоятельного понимания исходников PE x64 EH/Unwind анализатора

В книге С объясняется не отдельно от проекта, а через проект: `main.c`, `util.c`, `pe.c`, `unwind.c`, `heuristics.c`, `report.c`, `dynamic_win.c`. Материал включает базовый язык, системный С, PE/EH/Unwind предметную область, разбор файлов, практикум, маршрут самостоятельной реализации и построчный учебный разбор ключевых исходников.

Объёмная редакция для подготовки к защите и самостоятельного изучения С

Главная задача этой книги - не пересказать проект и не дать шпаргалку. Задача - научить читать и писать C на материале реального системного проекта. Поэтому здесь сначала объясняется язык: строка, тип, значение, память, адрес, указатель, массив, структура, функция, файл, ошибка, владение ресурсом. Потом эти понятия связываются с исходниками EH/Unwind Analyzer: чтением PE-файла, преобразованием RVA в file offset, разбором .pdata/.xdata, эвристиками, JSON/HTML-отчетами и динамической Windows-валидацией.

Как читать эту книгу

Эта книга написана для ситуации, когда читатель пока не знает язык C, но должен понять проект, написанный на C. Поэтому порядок материала выбран обратный обычному учебнику. Мы не начинаем с абстрактной грамматики языка и не бросаем читателя сразу в `unwind.c`. Мы идём от самого маленького действия в программе к реальному модулю проекта. Каждая новая конструкция вводится только после того, как объяснены слова, из которых она состоит. Важно: здесь нет бытовых аналогий. Вместо них используется техническое, но постепенное объяснение. Если появляется английский термин, рядом даётся русский смысл: `source file` - исходный файл, `compiler` - программа, которая переводит текст C в машинный код, `pointer` - значение, хранящее адрес объекта, `buffer` - непрерывная область памяти с байтами, `offset` - смещение от начала области. Термин не должен висеть отдельно; он должен быть связан с действием программы. Читать нужно последовательно. Первые главы могут казаться медленными, но без них невозможно честно понять такие строки проекта, как `if (len > b->size - off) return false;` или `const PeSection* s = pe_find_section_for_rva(img, rva);`. В них одновременно работают типы, указатели, структуры, арифметика размеров, защита от выхода за границы и соглашение о возврате ошибки. Метод книги: сначала смысл конструкции, потом минимальный C-код, потом построчное чтение, потом ошибка новичка, потом связь с конкретным файлом проекта, потом упражнение. Так формируется навык, а не набор слов.

Карта проекта, через который мы учим C

EH/Unwind Analyzer - это системная утилита. Она читает PE32+/x64 файл, извлекает Exception Directory, разбирает таблицу `RUNTIME_FUNCTION`, читает `UNWIND_INFO`, оценивает структурные аномалии и формирует отчеты. Статический слой работает с байтами файла на диске. Динамический слой на Windows запускает целевой процесс под отладчиком и собирает runtime evidence: исключения, hits обработчиков, регистрации dynamic function tables и другие события.

```
src/main.c          - CLI: аргументы, общий сценарий анализа, запуск static/dynamic, запись
    отчетов
src/util.c          - Buf, StrBuf, чтение файла, безопасное чтение байтов, SHA-256, atomic
    write
src/pe.c            - PE parser: MZ/PE, COFF, OptionalHeader, sections, RVA -> file offset
src/unwind.c        - parser .pdata/.xdata: RUNTIME_FUNCTION, UNWIND_INFO, UWOP slots, chain,
    handler
src/heuristics.c    - эвристический слой: признаки, score, suspicion, confidence, finding,
    mitigations
src/report.c        - CSV, JSON, HTML, evidence chains, русские объяснения признаков
src/dynamic_win.c   - Windows debugger validation: debug events, breakpoints, handler/API
    evidence
include/*.h         - публичные структуры и функции модулей
CMakeLists.txt      - правила сборки проекта
```

В книге эта карта будет возвращаться много раз. Сначала она нужна как ориентир, а ближе к концу - как план самостоятельной реализации. После изучения языка ты должен уметь открыть любой из этих файлов и объяснить: какие данные входят, какие данные выходят, кто владеет памятью, где возможна ошибка, почему стоит проверка и что будет, если проверку убрать.

Часть I. Нулевая точка: что такое программа на C

Первая часть строит минимальный словарь. Она нужна, чтобы читатель перестал видеть в C строку непонятных знаков и начал видеть структуру: директива препроцессора, объявление функции, тело функции, инструкция, выражение, значение, тип, имя, оператор, блок. Пока этот уровень не ясен, проектные файлы будут выглядеть как плотная стена текста.

Глава 1. Исходный файл, компилятор и исполняемый файл

Цель главы: понять, что C-программа начинается как текст, но работает как машинный код после компиляции и линковки.

Термины этой главы

`source file` - исходный файл: текстовый файл с кодом C, например `src/main.c` `compiler` - компилятор: программа, которая проверяет синтаксис C и переводит его в объектный код `linker` - линкер: программа, которая соединяет объектные файлы и библиотеки в исполняемый файл `executable` - исполняемый файл: результат сборки, который может запускаться операционной системой

Подробное объяснение

C не хранит за программиста намерение. Если нужно доказать безопасность чтения, это должно быть написано в коде. Если указатель может быть нулевым, это должно быть проверено до разыменования. Если тип поля в формате равен 32 битам, это должно отражаться в типе переменной. Хорошее чтение C всегда идёт в двух направлениях. Сначала вниз по строке: какой тип, какое имя, какой оператор, какое значение. Потом вверх к задаче: зачем это действие нужно модулю проекта и какую ошибку оно предотвращает. Если оба направления понятны, строка перестаёт быть набором символов.

Минимальный код

```
#include <stdio.h>

int main(void) {
    printf("Hello, C.\n");
    return 0;
}
```

Построчное чтение

Строка `#include <stdio.h>` не выполняется во время работы программы. Это директива препроцессора. Она говорит подготовительному этапу компиляции: подключить объявления стандартных функций ввода-вывода, чтобы компилятор знал, что такое `printf`. Строка `int main(void)` объявляет функцию `main`. Слово `int` означает, что функция возвращает целое число. Имя `main` имеет специальный смысл для обычной C-программы: выполнение начинается именно отсюда. Слово `void` внутри скобок означает, что функция не принимает аргументов. Фигурные скобки `{` и `}` ограничивают тело функции. Тело - это набор инструкций. Инструкция `printf(...)` вызывает функцию вывода текста. Инструкция `return 0;` завершает `main` и возвращает операционной системе код успешного завершения.

Связь с проектом

В проекте файл `src/main.c` является входной точкой CLI-инструмента. Он не выполняет весь анализ сам. Его роль - принять аргументы командной строки, загрузить файл в память, вызвать `pe_parse`, затем `unwind_read_runtime_functions_ex`, затем `unwind_parse`, затем при необходимости `heuristics_analyze`, `dynamic_collect_windows` и функции отчета. Когда ты видишь `main.c`, думай не “там всё”, а “там сценарий”. В системном C это нормальная архитектура: `main` связывает модули, но не должен превращаться в огромный файл с каждой деталью формата PE.

Типичные ошибки

Ошибка новичка - думать, что компилятор запускает программу. Компилятор только создаёт исполняемый файл. Вторая ошибка - считать, что `#include` вставляет библиотеку целиком. На уровне компиляции он в первую очередь приносит объявления. Третья ошибка - читать `return 0;` как формальность. Для CLI-инструмента код возврата важен.

Упражнения

1. Собери минимальную программу и измени `return 0;` на `return 2;`. 2. Найди в `src/main.c` функцию `main` и выпиши первые пять действий. 3. Объясни, почему `src/pe.c` компилируется отдельно, но вызывается из `main.c`.

Глава 2. Текст C: символы, инструкции, блоки

Цель главы: научиться видеть границы действий: где заканчивается инструкция, где начинается блок, где имя, где оператор.

Термины этой главы

`statement` - инструкция: завершенное действие C, обычно заканчивается точкой с запятой `block` - блок: группа инструкций в фигурных скобках `expression` - выражение: часть кода, которая вычисляет значение `operator` - оператор: знак или слово, задающее действие над значениями

Подробное объяснение

C не хранит за программиста намерение. Если нужно доказать безопасность чтения, это должно быть написано в коде. Если указатель может быть нулевым, это должно быть проверено до разыменования. Если тип поля в формате равен 32 битам, это должно отражаться в типе переменной. Хорошее чтение C всегда идёт в двух направлениях. Сначала вниз по строке: какой тип, какое имя, какой оператор, какое значение. Потом вверх к задаче: зачем это действие нужно модулю проекта и какую ошибку оно предотвращает. Если оба направления понятны, строка перестаёт быть набором символов.

Минимальный код

```
int value = 0;
value = value + 1;
if (value > 0) {
    value = value + 10;
}
```

Построчное чтение

`int value = 0;` создаёт объект `value` типа `int` и записывает в него начальное значение 0. Точка с запятой завершает инструкцию. `value = value + 1;` сначала вычисляет правую часть: текущее значение `value` плюс 1. Потом оператор = записывает результат обратно в `value`. `if (value > 0)` проверяет условие. Условие - это выражение, результат которого рассматривается как ложь или истина. Если условие истинно, выполняется блок.

Связь с проектом

В проекте блоки используются как границы владения ресурсами и как границы обработки ошибки. Например, после неудачного `pe_parse` в `main.c` программа освобождает уже выделенные ресурсы и возвращает код ошибки. В `unwind.c` вложенные блоки часто нужны, чтобы ограничить временные переменные конкретной операцией `UWOP_ALLOC_LARGE`, `UWOP_SAVE_NONVOL` и так далее.

Типичные ошибки

Ошибка новичка - думать, что перенос строки завершает действие. В C действие обычно завершает `;`. Вторая ошибка - путать = и ==. Третья ошибка - удалять фигурные скобки, когда в блоке одна строка; в системном коде это ухудшает читаемость.

Упражнения

1. Выпиши из `src/main.c` пять инструкций, которые завершаются `;`. 2. Объясни разницу между `value = value + 1` и `value == value + 1`. 3. Найди в `src/unwind.c` один `switch`.

Глава 3. Типы и значения: почему `uint32_t` важнее, чем просто число

Цель главы: понять, что тип задаёт диапазон, размер, способ интерпретации и правила операций над значением.

Термины этой главы

`type` - описание того, какие значения может хранить объект и сколько памяти он занимает `integer` - целое число `signed` - знаковый тип: может хранить отрицательные и положительные значения `unsigned` - беззнаковый тип: хранит только неотрицательные значения `stdint.h` - стандартный заголовок с типами фиксированной ширины

Подробное объяснение

C не хранит за программиста намерение. Если нужно доказать безопасность чтения, это должно быть написано в коде. Если указатель может быть нулевым, это должно быть проверено до разыменования. Если тип поля в формате равен 32 битам, это должно отражаться в типе переменной. Хорошее чтение C всегда идёт в двух направлениях. Сначала вниз по строке: какой тип, какое имя, какой оператор, какое значение. Потом вверх к задаче: зачем это действие нужно модулю проекта и какую ошибку оно предотвращает. Если оба направления понятны, строка перестаёт быть набором символов.

Минимальный код

```
#include <stdint.h>

uint8_t  b = 0x5A;
uint16_t w = 0x4550;
uint32_t rva = 0x00004000;
uint64_t image_base = 0x0000000140000000ULL;
```

Построчное чтение

`uint8_t` занимает 8 бит, то есть один байт. Он подходит для сырого байта файла. `uint16_t` занимает 16 бит. В PE такие поля встречаются в DOS header, COFF header и других структурах. `uint32_t` занимает 32 бита. RVA в PE32+/x64 обычно хранится как 32-битное смещение внутри image. `uint64_t` занимает 64 бита. ImageBase в PE32+ хранится как 64-битное значение.

Связь с проектом

В `include/pe.h` поля `exception_rva`, `exception_size`, `virtual_address`, `raw_ptr` объявлены как `uint32_t`, потому что это соответствует PE-формату. Поле `image_base` объявлено как `uint64_t`, потому что в x64 оно шире. Если заменить эти типы на обычный `int`, код может начать работать случайно на маленьких файлах и ломаться на больших значениях.

Типичные ошибки

Ошибка новичка - считать, что все числа одинаковые. Вторая ошибка - смешивать signed и unsigned без понимания. Третья ошибка - забывать, что размер поля в бинарном формате задаётся документацией, а не удобством программиста.

Упражнения

1. Найди в `include/unwind.h` все поля типа `uint32_t`. 2. Объясни, почему `size_t` используется для размера файла, а `uint32_t` - для RVA. 3. Напиши минимальную программу, где `uint8_t` хранит `0xff`.

Глава 4. Функции: вход, выход и договор между модулями

Цель главы: понять, что функция - это именованное действие с параметрами, результатом и договором об ошибках.

Термины этой главы

`function` - именованный блок кода, который можно вызвать `parameter` - имя входного значения внутри функции `argument` - конкретное значение, переданное при вызове `return value` - возвращаемое значение `prototype` - объявление функции, по которому компилятор проверяет вызовы

Подробное объяснение

C не хранит за программиста намерение. Если нужно доказать безопасность чтения, это должно быть написано в коде. Если указатель может быть нулевым, это должно быть проверено до разыменования. Если тип поля в формате равен 32 битам, это должно отражаться в типе переменной. Хорошее чтение C всегда идёт в двух направлениях. Сначала вниз по строке: какой тип, какое имя, какой оператор, какое значение. Потом вверх к задаче: зачем это действие нужно модулю проекта и какую ошибку оно предотвращает. Если оба направления понятны, строка перестаёт быть набором символов.

Минимальный код

```
static bool safe_add_u32(uint32_t a, uint32_t b, uint32_t* out) {
    uint64_t t = (uint64_t)a + (uint64_t)b;
    if (t > 0xffffffffu) return false;
    *out = (uint32_t)t;
    return true;
}
```

Построчное чтение

Функция называется `safe_add_u32`. Слово `static` здесь означает внутреннюю видимость внутри файла. Тип результата `bool` означает: функция сообщает успех или неуспех. Она не возвращает саму сумму напрямую. Параметры `a` и `b` - входные 32-битные числа. Параметр `out` - указатель на место, куда функция запишет результат при успехе. Сначала сумма считается в 64-битном типе, чтобы увидеть переполнение 32-битного диапазона.

Связь с проектом

Такая модель используется в `src/pe.c` при преобразовании RVA в file offset. Если сложение `raw_ptr + delta` переполнится, функция не должна возвращать мусорный offset. Функции проекта часто возвращают `bool`, а подробность ошибки пишут в `StrBuf* err`.

Типичные ошибки

Ошибка новичка - думать, что функция может вернуть только один результат. Вторая ошибка - записывать в `*out` до проверки `out != NULL`. Третья ошибка - игнорировать возвращаемый `bool`.

Упражнения

1. Перепиши `safe_add_u32` так, чтобы она работала с `size_t`. 2. Найди в `include/pe.h` прототип `pe_rva_to_fileoff`. 3. Объясни, почему `pe_parse` возвращает `bool`, а не `PeImage` напрямую.

Глава 5. Указатели: адрес как значение и ответственность программиста

Цель главы: понять pointer без прыжка в сложные термины: указатель хранит адрес объекта, а обращение по указателю требует проверки.

Термины этой главы

pointer - значение, которое хранит адрес объекта в памяти address - положение объекта в адресном пространстве процесса dereference - разыменование: обращение к объекту по адресу, записанному в указателе NULL - нулевой указатель: специальное значение, означающее отсутствие адреса объекта

Подробное объяснение

C не хранит за программиста намерение. Если нужно доказать безопасность чтения, это должно быть написано в коде. Если указатель может быть нулевым, это должно быть проверено до разыменования. Если тип поля в формате равен 32 битам, это должно отражаться в типе переменной. Хорошее чтение C всегда идёт в двух направлениях. Сначала вниз по строке: какой тип, какое имя, какой оператор, какое значение. Потом вверх к задаче: зачем это действие нужно модулю проекта и какую ошибку оно предотвращает. Если оба направления понятны, строка перестаёт быть набором символов.

Минимальный код

```
bool set_to_10(int* out) {
    if (!out) return false;
    *out = 10;
    return true;
}
```

Построчное чтение

int* out читается так: out - указатель на объект типа int. В самой переменной out лежит не число 10, а адрес места, куда можно записать int. if (!out) return false; проверяет, что адрес не нулевой. До этой строки нельзя выполнять *out. *out = 10; означает: перейти по адресу, который хранится в out, и записать туда значение 10.

Связь с проектом

В проекте указатели повсюду: const Buf* b, PeImage* out, StrBuf* err, RuntimeFunction** out. Это главный способ передавать большие структуры без копирования и записывать результаты из функции. Например, pe_parse(PeImage* out, const Buf* b, StrBuf* err) получает адрес структуры результата, адрес буфера с байтами файла и адрес буфера ошибок.

Типичные ошибки

Ошибка новичка - видеть * только как звездочку в типе. В объявлении int* p она говорит “p хранит адрес int”. В выражении *p она означает “обратиться по этому адресу”. Вторая ошибка - считать адрес обычным числом, с которым можно делать что угодно. Третья ошибка - забыть, что указатель может быть невалидным не только когда он NULL.

Упражнения

1. Нарисуй память для int x; int* p = &x; *p = 7;. 2. Открой src/util.c и найди три функции, которые сначала проверяют входные указатели. 3. Объясни, почему buf_read проверяет b и out до memcpy.

Глава 6. Массивы, байты и границы буфера

Цель главы: понять, почему чтение бинарного файла в C всегда начинается с границ: offset, length, size.

Термины этой главы

array - последовательность элементов одного типа в памяти byte - минимальная единица данных в файле, в проекте представлен типом uint8_t buffer - область памяти, где лежит последовательность байтов offset - смещение от начала буфера bounds check - проверка границ: доказательство, что чтение не выйдет за конец буфера

Подробное объяснение

C не хранит за программиста намерение. Если нужно доказать безопасность чтения, это должно быть написано в коде. Если указатель может быть нулевым, это должно быть проверено до разыменования. Если тип поля в формате равен 32 битам, это должно отражаться в типе переменной. Хорошее чтение C всегда идёт в двух направлениях. Сначала вниз по строке: какой тип, какое имя, какой оператор, какое значение. Потом вверх к задаче: зачем это действие нужно модулю проекта и какую ошибку оно предотвращает. Если оба направления понятны, строка перестаёт быть набором символов.

Минимальный код

```
bool buf_read(const Buf* b, size_t off, void* out, size_t len) {
    if (!b || !out) return false;
    if (off > b->size) return false;
    if (len > b->size - off) return false;
    memcpy(out, b->data + off, len);
    return true;
}
```

Построчное чтение

Buf хранит указатель на байты и размер. off - место, с которого нужно начать чтение. len - сколько байт нужно прочитать. out - адрес, куда байты будут скопированы. Первая проверка отсекает нулевые указатели. Вторая проверка доказывает, что стартовое смещение не находится после конца файла. Третья проверка доказывает, что длина чтения помещается в остаток буфера. Форма len > b->size - off выбрана не случайно. Если написать off + len > b->size, сложение off + len само может переполниться.

Связь с проектом

Это одна из главных строк всего проекта. PE-парсер, unwind-парсер и отчетные функции доверяют buf_read: если она ошибётся, весь анализ начнет читать байты не оттуда. Когда pe.c читает MZ, PE, Machine, NumberOfSections, он вызывает безопасные функции чтения маленьких полей.

Типичные ошибки

Ошибка новичка - думать, что раз файл “должен” иметь поле, его можно читать без проверки. Вторая ошибка - проверять только off < size, но не проверять len. Третья ошибка - использовать int для размера файла.

Упражнения

1. Объясни, почему len > b->size - off лучше, чем off + len > b->size. 2. Напиши функцию read_4_bytes, которая вызывает buf_read с длиной 4. 3. Найди в src/pe.c место, где читается lfanew.

Глава 7. const: как показать, что функция не должна менять объект

Цель главы: понять, почему `const Buf*` защищает намерение чтения и помогает видеть направление данных.

Термины этой главы

`const` - квалификатор, запрещающий изменение объекта через данный доступ

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
void print_size(const Buf* b) {
    if (!b) return;
    printf("%zu\n", b->size);
}
```

Построчное чтение

`const Buf* b` означает: функция получает адрес `Buf`, но не должна менять поля этой структуры через этот указатель. Это не делает объект абсолютно неизменяемым во всей программе. Это локальный договор функции: она читает, а не изменяет. В проекте `pe_parse` получает `const Buf* b`, потому что PE-парсер не должен менять байты исходного файла.

Связь с проектом

Если убрать `const`, читателю кода сложнее понять направление данных. В системном коде направление данных - часть безопасности. В `unwind_parse` исходный `Buf` тоже не должен изменяться; parser читает доказательства, а не правит файл.

Типичные ошибки

Ошибка - думать, что `const` делает объект физически read-only. Ошибка - убирать `const`, когда функция не меняет данные. Ошибка - путать `const char*` и `char* const`: это разные уровни неизменяемости.

Упражнения

1. Найди в заголовках проекта все параметры с `const`. 2. Объясни, почему `const PeImage* img` не должен изменяться в `pe_rva_to_fileoff`.

Глава 8. `struct`: как C собирает связанные поля в один объект

Цель главы: понять структуру как техническую запись состояния, а не как класс.

Термины этой главы

`struct` - составной тип, объединяющий несколько полей под одним именем `field` - поле структуры

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
typedef struct PeSection {
    char name[9];
    uint32_t virtual_address;
    uint32_t raw_ptr;
    uint32_t raw_size;
} PeSection;
```

Построчное чтение

`struct PeSection` описывает форму объекта: имя секции PE, виртуальный адрес, raw offset и raw size. `typedef` позволяет писать `PeSection` вместо `struct PeSection`. Каждый объект `PeSection` содержит все перечисленные поля.

Связь с проектом

`include/pe.h` через структуры задаёт внутреннюю модель PE-файла. Это уже не сырой файл, а разобранная карта секций. `PeImage` объединяет общие поля образа и массив секций. Так верхние слои не перечитывают файл, а используют построенную карту.

Типичные ошибки

Ошибка - путать структуру C с объектом ООП: у структуры нет методов и скрытой логики, только поля. Ошибка - думать, что структура автоматически соответствует байтам файла. В проекте поля заполняются вручную после проверок. Ошибка - забыть инициализировать структуру перед использованием.

Упражнения

1. Создай маленькую структуру `Range` с `begin` и `end`. 2. Объясни, почему `PeImage.sections` требует `pe_free`.

Глава 9. -> и .: доступ к полям напрямую и через указатель

Цель главы: понять, почему img->sections и section.name читаются по-разному.

Термины этой главы

dot operator - оператор . берет поле у объекта структуры arrow operator - оператор -> берет поле у структуры через указатель

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
PeImage img;
PeImage* p = &img;
img.size_of_image = 0;
p->size_of_image = 0;
```

Построчное чтение

img.size_of_image используется, когда есть сам объект структуры. p->size_of_image используется, когда есть указатель на структуру. Это сокращение для (*p).size_of_image. Перед p->... указатель должен быть валидным.

Связь с проектом

В проекте большинство больших структур передается по указателю, поэтому -> встречается часто: b->size, img->sections, out->valid. Когда код пишет s->raw_ptr, нужно понимать: s является адресом одной секции внутри массива секций.

Типичные ошибки

Ошибка - применять . к указателю или -> к объекту. Ошибка - забыть, что -> выполняет разыменование. Ошибка - читать p->field без понимания, кто владеет p.

Упражнения

1. Открой src/util.c и найди все обращения sb->.... 2. Перепиши p->size_of_image как (*p).size_of_image.

Глава 10. `sizeof`: размер типа и размер объекта

Цель главы: понять, почему `sizeof(*out)` безопаснее, чем повторять имя типа вручную.

Термины этой главы

`sizeof` - оператор C, возвращающий размер типа или объекта в байтах

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
PeImage img;
memset(&img, 0, sizeof(img));
PeImage* out = &img;
memset(out, 0, sizeof(*out));
```

Построчное чтение

`sizeof(img)` возвращает размер объекта `img`. `sizeof(*out)` возвращает размер объекта, на который указывает `out`, то есть тоже `PeImage`. Форма `sizeof(*out)` устойчива к изменению типа указателя.

Связь с проектом

В `pe_parse` используется `memset(out, 0, sizeof(*out));`, чтобы обнулить структуру результата перед заполнением. В `dynamic_win.c` часто используется `sizeof(struct)` перед Windows API, потому что API ожидает корректный размер структуры.

Типичные ошибки

Ошибка - писать размер руками числом. Ошибка - путать размер указателя и размер объекта. Ошибка - думать, что `sizeof` строки всегда даёт длину текста; для `char*` он даёт размер указателя.

Упражнения

1. Объясни, чем `sizeof(ptr)` отличается от `sizeof(*ptr)`. 2. Найди в проекте три примера `sizeof`.

Глава 11. malloc, calloc, realloc, free: владение памятью

Цель главы: понять, кто выделяет память, кто освобождает и почему это должно быть явно.

Термины этой главы

heap - область динамической памяти процесса allocation - выделение памяти ownership - владение: ответственность освободить ресурс

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
uint8_t* mem = (uint8_t*)malloc(size);
if (!mem) return false;
/* работа с mem */
free(mem);
```

Построчное чтение

malloc(size) просит у runtime область памяти размером size байт. Если памяти нет, возвращается NULL, поэтому результат нужно проверять. free(mem) освобождает память. После этого использовать mem как живой буфер нельзя.

Связь с проектом

buf_init_from_file выделяет память для всего файла и возвращает её через owned_mem. Потом main.c вызывает buf_free_owned(owned). pe_parse выделяет массив секций через calloc, а pe_free освобождает его.

Типичные ошибки

Ошибка - забыть free и получить утечку памяти. Ошибка - вызвать free два раза на один адрес. Ошибка - использовать указатель после free.

Упражнения

1. Проследи путь owned в main.c. 2. Объясни, почему pe_free обнуляет структуру после освобождения.

Глава 12. FILE*, бинарное чтение и режим rb

Цель главы: понять, как C читает файл как поток байтов.

Термины этой главы

FILE - тип стандартной библиотеки C для работы с потоком файла fopen - открытие файла fread - чтение байтов из файла

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
FILE* f = fopen(path, "rb");
if (!f) return false;
size_t rd = fread(mem, 1, size, f);
fclose(f);
```

Построчное чтение

fopen(path, "rb") открывает файл для бинарного чтения. fread(mem, 1, size, f) читает size элементов по 1 байту в mem. fclose(f) закрывает файловый ресурс.

Связь с проектом

PE-файл нельзя читать как текст. rb важен, особенно для Windows, где текстовый режим может иначе трактовать переводы строк. buf_init_from_file делает из файлового потока обычный Buf, чтобы дальше весь проект работал с памятью.

Типичные ошибки

Ошибка - забыть закрыть файл при раннем return. Ошибка - не проверить, что fread прочитал нужное число байт. Ошибка - считать ftell безопасным без проверки отрицательного результата.

Упражнения

1. Объясни, почему buf_init_from_file сначала узнаёт размер через fseek/ftell. 2. Сделай тест для пустого файла.

Глава 13. Endian: почему байты `50 45 00 00` становятся `0x00004550`

Цель главы: понять little-endian чтение чисел из PE-файла.

Термины этой главы

`little-endian` - порядок байтов, где младший байт числа лежит раньше в памяти или файле

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
uint32_t le32(const uint8_t* p) {
    return (uint32_t)p[0] |
           ((uint32_t)p[1] << 8) |
           ((uint32_t)p[2] << 16) |
           ((uint32_t)p[3] << 24);
}
```

Построчное чтение

PE хранит многие числа в little-endian. Если в файле лежат байты `50 45 00 00`, то младший байт `0x50` идёт первым, и итоговое число равно `0x00004550`. Сдвиги `<< 8`, `<< 16`, `<< 24` ставят байты на нужные позиции числа.

Связь с проектом

В `util.c` функции `buf_read_u16`, `buf_read_u32`, `buf_read_u64` читают байты и переводят их в числа через little-endian helpers. Так проект не зависит от endian текущей машины и не нарушает alignment.

Типичные ошибки

Ошибка - прочитать байты как строку. Ошибка - просто привести указатель к `uint32_t*`. Ошибка - забыть cast перед сдвигом байта.

Упражнения

1. Возьми байты `4D 5A` и объясни, почему они дают DOS magic `MZ`. 2. Напиши `le16` самостоятельно.

Глава 14. Заголовочные файлы .h и исходные файлы .c

Цель главы: понять интерфейс модуля и его реализацию.

Термины этой главы

header file - заголовочный файл с объявлениями типов и функций implementation file - исходный файл с кодом функций

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
/* include/pe.h */
bool pe_parse(PeImage* out, const Buf* b, StrBuf* err);

/* src/pe.c */
bool pe_parse(PeImage* out, const Buf* b, StrBuf* err) {
    /* реализация */
}
```

Построчное чтение

Заголовок сообщает другим файлам, что функция существует и как её вызывать. Файл .c содержит тело функции. Компилятор проверяет соответствие вызова объявлению.

Связь с проектом

В проекте include/pe.h описывает PeImage и функции PE-парсера, а src/pe.c реализует проверку MZ/PE, секции и RVA-маппинг. Так коллега может использовать ядро через стабильные функции, не меняя внутренности модулей.

Типичные ошибки

Ошибка - класть реализацию большой функции в .h без необходимости. Ошибка - менять сигнатуру в .c, забыв обновить .h. Ошибка - создавать циклические зависимости заголовков без причины.

Упражнения

1. Сопоставь каждый include/*.h с соответствующим src/*.c. 2. Объясни, почему #pragma once стоит в заголовках.

Глава 15. `static` в файле C: внутренняя функция модуля

Цель главы: понять, почему часть функций проекта скрыта внутри файла.

Термины этой главы

`static function` - функция с внутренней связностью: видна только в текущем `.c` файле

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
static bool read_dos_lfanew(const Buf* b, uint32_t* out_lfanew) {
    /* helper только для pe.c */
    return true;
}
```

Построчное чтение

`static` перед функцией в файле C означает: это внутренняя вспомогательная функция. Другие `.c` файлы не должны её вызывать напрямую. Так модуль оставляет наружу только нужный интерфейс.

Связь с проектом

В `pe.c` `read_dos_lfanew`, `safe_add_u32`, `ranges_overlap_u64` являются деталями PE-парсера. Наружу вынесены только `pe_parse`, `pe_free`, `pe_rva_to_fileoff`, `pe_find_section_for_rva`. Это уменьшает поверхность ошибок и делает архитектуру чище.

Типичные ошибки

Ошибка - делать все функции внешними. Ошибка - объявлять helper в заголовке только потому, что он существует. Ошибка - путать `static` функцию и `static` локальную переменную.

Упражнения

1. Найди в `src/unwind.c` три `static` функции. 2. Объясни, почему они не объявлены в `include/unwind.h`.

Глава 16. `bool` и стиль ошибок через `true/false`

Цель главы: понять, почему системная функция часто возвращает не данные, а статус.

Термины этой главы

`bool` - логический тип из `stdbool.h` `true/false` - значения истины и лжи

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
bool read_u32_checked(const Buf* b, size_t off, uint32_t* out) {
    if (!out) return false;
    return buf_read_u32(b, off, out);
}
```

Построчное чтение

Функция возвращает `true`, если действие выполнено. Данные записываются через `out`. Так вызывающий код не путает ошибку с допустимым числовым значением.

Связь с проектом

В проекте `pe_parse`, `unwind_parse`, `report_write_html` используют `bool`. Они выполняют действие, которое может не состояться. Подробность ошибки часто добавляется в `StrBuf* err`.

Типичные ошибки

Ошибка - игнорировать `false` и продолжать как будто данные корректны. Ошибка - возвращать 0 как данные и как ошибку одновременно. Ошибка - не документировать, какие выходные поля валидны при `false`.

Упражнения

1. Найди в `main.c` пять проверок вида `if (!function(...))`. 2. Объясни, что будет, если проигнорировать `pe_parse == false`.

Глава 17. Строки C: char*, нулевой байт и безопасность

Цель главы: понять, что строка C - это массив char, заканчивающийся нулевым байтом.

Термины этой главы

C string - последовательность символов, заканчивающаяся \0 strlen - длина строки до нулевого байта snprintf - форматированная запись с ограничением размера

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
char name[9];
memcpy(name, raw_name, 8);
name[8] = 0;
```

Построчное чтение

Массив name имеет 9 байт: 8 для имени секции PE и один для завершающего нуля. memcpy копирует ровно 8 байт из файла. name[8] = 0 делает массив безопасной C-строкой для печати и сравнения.

Связь с проектом

Так работает чтение имени секции в pe.c. В PE имя секции занимает 8 байт и не обязано быть удобной строкой C. Проект сам добавляет нулевой байт. В отчетах проект использует snprintf, чтобы ограничить запись в фиксированные массивы char.

Типичные ошибки

Ошибка - считать, что байты из файла уже являются безопасной строкой. Ошибка - забыть место под \0. Ошибка - использовать strcpy на данных неизвестной длины.

Упражнения

1. Объясни, почему PeSection.name имеет размер 9, а не 8. 2. Найди в report.c места с snprintf.

Глава 18. memset, memcpy, memcmp: прямые операции с памятью

Цель главы: понять, когда системный C работает не со значениями, а с байтами.

Термины этой главы

memset - заполнение области памяти байтом memcpy - копирование указанного числа байт memory - память процесса

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
PeImage img;
memset(&img, 0, sizeof(img));
memcpy(dst, src, len);
```

Построчное чтение

memset(&img, 0, sizeof(img)) обнуляет все байты структуры. memcpy(dst, src, len) копирует len байт. Она не знает смысл полей, поэтому длина должна быть проверена заранее. Эти функции выполняют ровно то количество байт, которое им передали.

Связь с проектом

buf_read использует memcpy только после проверки границ. pe_parse использует memset, чтобы структура результата начиналась в известном состоянии. В dynamic_win.c memset часто подготавливает структуры Windows API перед заполнением поля dwSize.

Типичные ошибки

Ошибка - вызвать memcpy с неверной длиной. Ошибка - применять memset к объекту, где нулевое состояние не является корректным. Ошибка - забыть, что memcpy не проверяет перекрытие областей.

Упражнения

1. Найди в проекте все memset и объясни, что именно обнуляется. 2. Объясни, почему memcpy в buf_read находится после проверок.

Глава 19. Переполнение целых чисел и безопасная арифметика размеров

Цель главы: понять, почему `a + b` в парсере нельзя считать без проверки.

Термины этой главы

`overflow` - переполнение: результат операции не помещается в тип `range` - диапазон допустимых значений

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
static bool safe_add_u32(uint32_t a, uint32_t b, uint32_t* out) {
    uint64_t t = (uint64_t)a + (uint64_t)b;
    if (t > 0xffffffffu) return false;
    *out = (uint32_t)t;
    return true;
}
```

Построчное чтение

Сложение выполняется в `uint64_t`, чтобы не потерять результат. Потом проверяется, помещается ли сумма обратно в 32 бита. Только после этого выполняется приведение к `uint32_t`.

Связь с проектом

В `pe_rva_to_fileoff` offset получается как `raw_ptr + delta`. Если это сложение переполнится, парсер может указать в начало файла вместо конца. В `StrBuf` похожая дисциплина нужна при увеличении capacity.

Типичные ошибки

Ошибка - писать `uint32_t off = raw + delta;` без проверки. Ошибка - думать, что unsigned overflow безопасен для смысла parser-a. Ошибка - проверять переполнение уже после поврежденного результата.

Упражнения

1. Составь пример, где `0xffffffffu + 1u` даёт неверный для offset смысл. 2. Объясни проверку `n > SIZE_MAX - sb->len - 1u`.

Глава 20. enum и битовые маски

Цель главы: понять, как проект хранит набор флагов и аномалий в одном числе.

Термины этой главы

enum - перечисление именованных целых констант bitmask - число, где отдельные биты означают отдельные признаки flag - отдельный признак, включенный или выключенный

Подробное объяснение

Эта глава уже связывает синтаксис C с инженерной дисциплиной проекта. Важно не просто запомнить конструкцию, а уметь ответить: какая ошибка произойдет, если её использовать неправильно, и какой слой проекта от этой ошибки зависит. Когда читаешь исходник, задавай три вопроса: что хранит переменная, кто имеет право её менять, и что считается ошибкой для этой операции. Такой способ чтения превращает C из набора знаков в систему проверяемых договоров.

Минимальный код

```
enum UnwindAnom {
    UA_TRUNCATED = 1u << 1,
    UA_BAD_OPCODE = 1u << 2
};
uint32_t anomalies = 0;
anomalies |= UA_TRUNCATED;
```

Построчное чтение

1u << 1 создаёт число, где установлен второй бит. anomalies |= UA_TRUNCATED включает этот бит в маске. Одна переменная может хранить сразу несколько включенных признаков.

Связь с проектом

В include/unwind.h перечислены UA_VERSION_UNKNOWN, UA_TRUNCATED, UA_BAD_OPCODE, UA_ALIGNMENT и другие внутренние аномалии разбора UNWIND_INFO. В report.c функция anomalies_to_str переводит маску в читаемый список строк.

Типичные ошибки

Ошибка - складывать флаги обычным + вместо битового |. Ошибка - проверять маску через ==, когда могут быть включены несколько флагов. Ошибка - забыть, что битовая маска требует уникального бита для каждого признака.

Упражнения

1. Объясни выражение if (u->anomalies & UA_TRUNCATED). 2. Добавь мысленно новый флаг UA_TEST = 1u << 17.

Часть II. Системный C на материале EH/Unwind Analyzer

Теперь базовые конструкции языка можно соединять в настоящие проектные механизмы. В этой части мы разбираем конкретные приёмы, которые делают проект рабочим: безопасное чтение файла, буфер строк, парсинг чисел little-endian, построение PE-карты, разбор .pdata/.xdata, накопление evidence, запись отчетов и освобождение ресурсов.

Глава 21. Buf: модель файла как массива байтов

Проект не работает с PE-файлом как с набором готовых структур Windows headers. Он сначала читает весь файл в память и представляет его как Buf: указатель на байты и размер. Это фундаментальная модель для безопасного парсера. Пока файл является Buf, каждое чтение должно доказывать: смещение существует, длина помещается, копирование не выйдет за границы.

Контракт Buf

```
typedef struct Buf {
    const uint8_t* data;
    size_t size;
} Buf;

bool buf_init_from_file(Buf* out, const char* path, uint8_t** owned_mem);
void buf_free_owned(uint8_t* owned_mem);

bool buf_read(const Buf* b, size_t off, void* out, size_t len);
bool buf_read_u8 (const Buf* b, size_t off, uint8_t*  out);
bool buf_read_u16(const Buf* b, size_t off, uint16_t* out);
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out);
bool buf_read_u64(const Buf* b, size_t off, uint64_t* out);

static inline uint16_t le16(const uint8_t* p) {
    return (uint16_t)(p[0] | ((uint16_t)p[1] << 8));
}
static inline uint32_t le32(const uint8_t* p) {
    return (uint32_t)(p[0] | ((uint32_t)p[1] << 8) | ((uint32_t)p[2] << 16) | ((uint32_t)p[3]
        << 24));
}
static inline uint64_t le64(const uint8_t* p) {
    return (uint64_t)le32(p) | ((uint64_t)le32(p + 4) << 32);
```

Реальное чтение

```
bool buf_read(const Buf* b, size_t off, void* out, size_t len) {
    if (!b || !out) return false;
    if (off > b->size) return false;
    if (len > b->size - off) return false;
    memcpy(out, b->data + off, len);
    return true;
}

bool buf_read_u8(const Buf* b, size_t off, uint8_t* out) {
    return buf_read(b, off, out, 1);
}
bool buf_read_u16(const Buf* b, size_t off, uint16_t* out) {
    uint8_t tmp[2];
    if (!buf_read(b, off, tmp, 2)) return false;
    *out = le16(tmp);
    return true;
}
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out) {
    uint8_t tmp[4];
    if (!buf_read(b, off, tmp, 4)) return false;
    *out = le32(tmp);
    return true;
}
bool buf_read_u64(const Buf* b, size_t off, uint64_t* out) {
```

Почему это фундамент

С точки зрения C здесь соединяются структуры, указатели, size_t, void*, memcpy и возврат bool. void* out означает адрес памяти без конкретного типа на уровне функции. Это нормально для копирования байтов: buf_read не интерпретирует данные, он только переносит байты из файла в место, которое передал вызывающий код. Ключевое учебное место - проверка if (len > b->size - off) return false;. Она выглядит плотной, но теперь её можно прочитать строго. Сначала уже доказано, что off <= b->size. Значит выражение b->size - off не уходит в unsigned underflow и означает количество байт, доступных от смещения off до конца файла. Системный C начинается не с красивых структур, а с дисциплины границ. Любой парсер бинарного формата много раз отвечает на один вопрос: имею ли я право прочитать эти N байт с этого смещения?

Что должен уметь объяснить студент

Объяснить, почему Buf* передается по указателю. Объяснить, почему data имеет тип uint8_t*. Объяснить, почему размер имеет тип size_t. Объяснить, почему memcpy вызывается только после проверок.

Глава 22. StrBuf: строка отчета как управляемый ресурс

Обычная строка C имеет фиксированный буфер или требует ручного управления памятью. Отчет проекта строится постепенно: нужно добавлять текст, JSON-поля, HTML-фрагменты, диагностические сообщения. Для этого используется `StrBuf` - собственный динамический буфер строки. Его задача - хранить текущую длину, ёмкость, указатель на память и флаг ошибки.

Контракт StrBuf

```
typedef struct StrBuf {
    char*  s;
    size_t len;
    size_t cap;
    bool failed;
} StrBuf;

#if defined(__MINGW32__) && (defined(__GNUC__) || defined(__clang__))
#define EHUA_PRINTF_FORMAT(format_index, first_arg) \
    __attribute__((format(gnu_printf, format_index, first_arg)))
#elif defined(__GNUC__) || defined(__clang__)
#define EHUA_PRINTF_FORMAT(format_index, first_arg) \
    __attribute__((format(printf, format_index, first_arg)))
#else
#define EHUA_PRINTF_FORMAT(format_index, first_arg)
#endif

bool sb_init(StrBuf* sb, size_t initial_cap);
```

Рост буфера

```
static bool sb_reserve(StrBuf* sb, size_t need_cap) {
    if (!sb || sb->failed) return false;
    if (need_cap <= sb->cap) return true;
    size_t newcap = sb->cap ? sb->cap : 128;
    while (newcap < need_cap) {
        size_t increment = newcap / 2u + 64u;
        if (increment > SIZE_MAX - newcap) {
            newcap = need_cap;
            break;
        }
        newcap += increment;
    }
    char* p = (char*)realloc(sb->s, newcap);
    if (!p) {
        sb->failed = true;
        return false;
    }
    sb->s = p;
    sb->cap = newcap;
    return true;
}

bool sb_init(StrBuf* sb, size_t initial_cap) {
    if (!sb) return false;
    sb->s = NULL;
    sb->len = 0;
    sb->cap = 0;
    sb->failed = false;
    if (initial_cap == 0) initial_cap = 256;
    if (!sb_reserve(sb, initial_cap) || !sb->s) {
        sb->failed = true;
        return false;
    }
    sb->s[0] = 0;
    return true;
}

void sb_free(StrBuf* sb) {
    if (!sb) return;
    free(sb->s);
    sb->s = NULL;
    sb->len = 0;
    sb->cap = 0;
    sb->failed = false;
}

bool sb_appendn(StrBuf* sb, const char* s, size_t n) {
    if (!sb) return false;
    if (sb->failed || (!s && n != 0)) {
        sb->failed = true;
        return false;
    }
    if (sb->len == SIZE_MAX || n > SIZE_MAX - sb->len - 1u) {
        sb->failed = true;
        return false;
```

Жизненный цикл

`len` - сколько полезных символов уже записано. `cap` - сколько байт выделено. `s` - адрес выделенной памяти. `failed` - флаг, который фиксирует состояние ошибки. Такой объект требует дисциплины: сначала `sb_init`, затем много `sb_append` или `sb_appendf`, в конце `sb_free`. В `sb_reserve` видна системная логика C. Если нужная ёмкость уже есть, функция возвращает `true`. Если нет, она увеличивает capacity. Перед увеличением проверяется переполнение `SIZE_MAX`. Затем вызывается `realloc`. Это не просто строковый помощник. Это пример проектирования маленького ресурса в C: у ресурса есть состояние, жизненный цикл, операции и правила ошибки.

Что должен уметь объяснить студент

Реализовать уменьшенный `StrBuf`. Пояснить `len`, `cap`, `s`, `failed`. Объяснить, почему `realloc` требует проверки результата.

Глава 23. PE parser: превращение байтов в карту образа

Файл PE нельзя анализировать полезно, если видеть только байты. Проект строит внутреннюю карту: DOS header, PE signature, COFF header, OptionalHeader, DataDirectory, sections. Карта нужна всем последующим слоям: без секций нельзя корректно перевести RVA в file offset, без Exception Directory нельзя прочитать `.pdata`, без SizeOfImage нельзя проверить выход адресов за границы image.

Начало `pe_parse`

```
bool pe_parse(PeImage* out, const Buf* b, StrBuf* err) {
    if (!out || !b || !err) return false;
    memset(out, 0, sizeof(*out));

    uint32_t lfanew = 0;
    if (!read_dos_lfanew(b, &lfanew)) {
        sb_append(err, "Not a valid MZ/PE file (DOS header check failed).\n");
        return false;
    }
    if ((size_t)lfanew + 4 + 20 > b->size) {
        sb_append(err, "PE header offset out of range.\n");
        return false;
    }

    uint32_t sig = 0;
    if (!buf_read_u32(b, (size_t)lfanew, &sig) || sig != PE_NT_MAGIC) {
        sb_append(err, "Invalid PE signature.\n");
        return false;
    }

    size_t filehdr = (size_t)lfanew + 4;
    uint16_t machine = 0;
    uint16_t nsects = 0;
    uint16_t opt_size = 0;
    if (!buf_read_u16(b, filehdr + 0, &machine) ||
        !buf_read_u16(b, filehdr + 2, &nsects) ||
        !buf_read_u16(b, filehdr + 16, &opt_size)) {
        sb_append(err, "Failed to read COFF header.\n");
        return false;
    }

    if (machine != IMAGE_FILE_MACHINE_AMD64) {
        sb_appendf(err, "Unsupported Machine: 0x%04x (only x64/AMD64 supported).\n", machine);
        return false;
    }

    size_t opthdr = filehdr + 20;
    if (opthdr + opt_size > b->size) {
        sb_append(err, "Optional header out of range.\n");
        return false;
    }
    if (opt_size < 0x70u) {
        sb_append(err, "PE32+ optional header is too small for required fields.\n");
        return false;
    }

    uint16_t magic = 0;
    if (!buf_read_u16(b, opthdr + 0, &magic)) {
        sb_append(err, "Failed to read optional header magic.\n");
        return false;
    }
    if (magic != IMAGE_NT_OPTIONAL_HDR64_MAGIC) {
        sb_appendf(err, "Unsupported OptionalHeader magic: 0x%04x (expect PE32+ 0x20b).\n",
            magic);
        return false;
    }

    uint64_t image_base = 0;
    uint32_t size_of_image = 0;
    uint32_t size_of_headers = 0;
    uint32_t section_alignment = 0;
    uint32_t file_alignment = 0;
    uint32_t number_of_rva_and_sizes = 0;

    /* PE32+ offsets */
    if (!buf_read_u64(b, opthdr + 0x18, &image_base) ||
        !buf_read_u32(b, opthdr + 0x20, &section_alignment) ||
        !buf_read_u32(b, opthdr + 0x24, &file_alignment) ||
        !buf_read_u32(b, opthdr + 0x38, &size_of_image) ||
        !buf_read_u32(b, opthdr + 0x3c, &size_of_headers) ||
        !buf_read_u32(b, opthdr + 0x6c, &number_of_rva_and_sizes)) {
        sb_append(err, "Failed to read key fields from optional header.\n");
        return false;
    }
    if (size_of_image == 0) {
```

RVA -> file offset

```
bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off) {
    if (!img || !out_off) return false;
    const PeSection* s = pe_find_section_for_rva(img, rva);
    if (s) {
        uint32_t delta = rva - s->virtual_address;
        if (delta >= s->raw_size) {
            /* RVA points into virtual tail (zero-fill) - not present in file */
            return false;
        }
        uint32_t off = 0;
        if (!safe_add_u32(s->raw_ptr, delta, &off)) return false;
```

```
        if (img->file_size > 0 && (uint64_t)off >= (uint64_t)img->file_size) return false;
        *out_off = off;
        return true;
    }

    /* RVAs inside SizeOfHeaders map directly to the corresponding file offset. */
    if (img->size_of_headers > 0 &&
        rva < img->size_of_headers &&
        (img->file_size == 0 || (uint64_t)rva < (uint64_t)img->file_size)) {
        *out_off = rva;
        return true;
    }
    return false;
}
```

Три уровня адресации

File offset - смещение внутри файла на диске. RVA - Relative Virtual Address, смещение внутри загруженного image от ImageBase. VA - Virtual Address, фактический виртуальный адрес в процессе, обычно `ImageBase + RVA`. Статический анализ проекта в основном работает с file offset и RVA. Функция `pe_rva_to_fileoff` является мостом между картой image и байтами файла. Она ищет секцию, которая покрывает заданный RVA, вычисляет `delta`, проверяет raw size и безопасно складывает raw pointer с delta. Проект автоматизирует дисциплину адресов. Он не даёт человеку случайно читать `.xdata` по RVA как по offset файла.

Что должен уметь объяснить студент

Объяснить MZ, e_lfanew, PE signature. Различить VA, RVA и file offset. Показать, почему virtual tail не читается из файла.

Глава 24. Exception Directory и RUNTIME_FUNCTION: таблица функций x64

В PE x64 Exception Directory указывает на массив записей RUNTIME_FUNCTION. Каждая запись описывает диапазон кода функции и RVA структуры UNWIND_INFO. В C проект представляет такую запись структурой из трёх uint32_t: begin_rva, end_rva, unwind_rva.

Структура записи

```
typedef struct RuntimeFunction {
    uint32_t begin_rva;
    uint32_t end_rva;
    uint32_t unwind_rva;
} RuntimeFunction;

#define UNWIND_DIAG_MAX_BYTES 32u
#define UNWIND_DIAG_MAX_CHAIN_PATH 16u

typedef struct UnwindSummary {
    bool     valid;
    uint8_t  version;
    uint8_t  flags;
    uint8_t  prolog_size;
```

Чтение таблицы

```
RuntimeFunctionReadResult unwind_read_runtime_functions_ex(
                                const Buf* b,
                                const PeImage* img,
                                RuntimeFunction** out_rf,
                                size_t* out_count,
                                StrBuf* err) {
    if (!b || !img || !out_rf || !out_count || !err) {
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    *out_rf = NULL;
    *out_count = 0;

    if (img->exception_rva == 0 && img->exception_size == 0) {
        return RUNTIME_FUNCTION_READ_OK;
    }
    if (img->exception_rva == 0 || img->exception_size == 0) {
        sb_append(err, "Malformed exception directory: RVA and size must both be zero or non-
            zero.\n");
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    uint32_t off = 0;
    if (!pe_rva_to_fileoff(img, img->exception_rva, &off)) {
        sb_appendf(err, "Exception directory RVA 0x%08x not mappable.\n", img->exception_rva);
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    if ((size_t)off + img->exception_size > b->size) {
        sb_append(err, "Exception directory out of file range.\n");
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    if (img->exception_size < 12) {
        sb_append(err, "Exception directory size too small.\n");
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    if ((img->exception_size % 12u) != 0) {
        sb_append(err, "Exception directory size is not a multiple of RUNTIME_FUNCTION
            size.\n");
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    size_t n = img->exception_size / 12;
    RuntimeFunction* rf = (RuntimeFunction*)calloc(n, sizeof(RuntimeFunction));
    if (!rf) {
        sb_append(err, "Out of memory.\n");
        return RUNTIME_FUNCTION_READ_OOM;
    }

    for (size_t i = 0; i < n; i++) {
        size_t e = (size_t)off + i*12;
        uint32_t bva=0, eva=0, uva=0;
        if (!buf_read_u32(b, e + 0, &bva) ||
            !buf_read_u32(b, e + 4, &eva) ||
            !buf_read_u32(b, e + 8, &uva)) {
            free(rf);
            sb_append(err, "Failed to read runtime function entry.\n");
            return RUNTIME_FUNCTION_READ_MALFORMED;
        }
        rf[i].begin_rva = bva;
        rf[i].end_rva = eva;
        rf[i].unwind_rva = uva;
    }

    *out_rf = rf;
    *out_count = n;
    return RUNTIME_FUNCTION_READ_OK;
}

bool unwind_read_runtime_functions(const Buf* b,
                                const PeImage* img,
                                RuntimeFunction** out_rf,
                                size_t* out_count,
                                StrBuf* err) {
    return unwind_read_runtime_functions_ex(
```

```
                b, img, out_rf, out_count, err) == RUNTIME_FUNCTION_READ_OK;
}
```

Почему нужен двойной указатель

Каждая запись `RUNTIME_FUNCTION` на диске занимает 12 байт: BeginAddress, EndAddress, UnwindInfoAddress. Проект проверяет размер Exception Directory, выделяет массив и читает записи по одной. Двойной указатель `RuntimeFunction** out` нужен потому, что функция должна изменить указатель вызывающего кода: выделить массив и вернуть его адрес наружу. `size_t* out_count` аналогично возвращает число записей. С точки зрения предметной области эта таблица является картой функций. Если диапазоны не отсортированы, пересекаются или указывают на невалидный unwind RVA, карта становится ненадёжной.

Что должен уметь объяснить студент

Объяснить размер одной записи 12 байт. Объяснить `RuntimeFunction** out`. Объяснить, почему BeginAddress должен быть меньше EndAddress.

Глава 25. UNWIND_INFO: компактная структура, где ошибка на 2 байта ломает всё

UNWIND_INFO описывает, как Windows может виртуально отмотать состояние функции: восстановить стек, регистры и перейти к вызывающему контексту при исключении или stack walking. Формат компактный: header, массив UNWIND_CODE, затем tail, который может содержать handler RVA или chained RUNTIME_FUNCTION-like запись.

Начало unwind_parse

```
static bool unwind_parse_fail(UnwindSummary* out,
                              StrBuf* err,
                              size_t err_start,
                              const char* fallback) {
    if (!out) return false;
    out->valid = false;

    const char* message = fallback ? fallback : "UNWIND_INFO parse failed.";
    if (err && err->s && err->len > err_start) {
        message = err->s + err_start;
    } else if (err && fallback) {
        sb_appendf(err, "%s\n", fallback);
    }

    size_t n = strlen(message);
    while (n > 0 && (message[n - 1] == '\n' || message[n - 1] == '\r')) n--;
    if (n >= sizeof(out->parse_error)) n = sizeof(out->parse_error) - 1;
    memcpy(out->parse_error, message, n);
    out->parse_error[n] = '\0';
    return false;
}

bool unwind_parse(const Buf* b,
                  const PeImage* img,
                  uint32_t unwind_rva,
                  const UnwindParseOpts* opts,
                  UnwindSummary* out,
                  StrBuf* ops_text,
                  StrBuf* err) {
    if (!b || !img || !out || !err) return false;
    memset(out, 0, sizeof(*out));
    size_t err_start = err->len;
    if (ops_text) { ops_text->len = 0; if (ops_text->s) ops_text->s[0]=0; }

    int max_depth = 8;
    bool follow_chain = true;
    bool strict_abi = false;
    if (opts) {
        if (opts->max_chain_depth > 0) max_depth = opts->max_chain_depth;
        follow_chain = opts->follow_chain;
        strict_abi = opts->strict_abi;
    }
    if (max_depth > 16) max_depth = 16;

    uint8_t hdr[4];
    uint32_t uoff = 0;
    uint32_t attempted_uoff = 0;
    bool attempted_mapped = pe_rva_to_fileoff(img, unwind_rva, &attempted_uoff);
    if ((unwind_rva & 3u) != 0) {
        out->anomalies |= UA_ALIGNMENT;
    }
    if (!parse_unwindinfo_header(b, img, unwind_rva, &uoff, hdr, err)) {
        out->anomalies |= UA_TRUNCATED;
        unwind_diag_set(out, b, "UnwindInfoAddress / UNWIND_INFO header",
                        attempted_mapped, attempted_mapped ? (size_t)attempted_uoff : 0u,
                        attempted_mapped, 4u,
                        attempted_mapped ? available_until_limit(b, (size_t)attempted_uoff,
                b->size) : 0u,
                        false, 0, 0);
        return unwind_parse_fail(out, err, err_start, "Failed to read UNWIND_INFO header.");
    }
    size_t record_limit = unwind_record_limit(b, img, unwind_rva);

    uint8_t ver = (uint8_t)(hdr[0] & 0x7);
    uint8_t flags = (uint8_t)(hdr[0] >> 3);
    uint8_t prolog = hdr[1];
    uint8_t count = hdr[2];
    uint8_t fr = (uint8_t)(hdr[3] & 0x0f);
    uint8_t foff = (uint8_t)(hdr[3] >> 4);

    out->version = ver;
    out->flags = flags;
    out->prolog_size = prolog;
    out->count_codes = count;
    out->frame_reg = fr;
    out->frame_off = foff;

    if (!(ver == 1 || ver == 2)) {
        out->anomalies |= UA_VERSION_UNKNOWN;
        unwind_diag_set(out, b, "Version", true, (size_t)uoff, true, 1u, 1u, false, 0, 0);
        return unwind_parse_fail(out, err, err_start, "Unsupported UNWIND_INFO version.");
    }
    if (flags & (uint8_t)~(UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER | UNW_FLAG_CHAININFO)) {
        out->anomalies |= UA_BAD_OPERAND;
    }
    if ((flags & UNW_FLAG_CHAININFO) && (flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER))) {
        out->anomalies |= UA_BAD_OPERAND;
    }
    if (fr == 0 && foff != 0) {
```

```
        out->anomalies |= UA_BAD_OPERAND;
    }
    if (fr != 0 && !is_unwindable_gpr(fr)) {
        out->anomalies |= UA_BAD_OPERAND;
    }
    if (strict_abi && fr != 0 && !is_abi_nonvolatile_gpr(fr)) {
        out->anomalies |= UA_ABI_REGISTER_CLASS;
    }

    /* total slots stored on disk (aligned to even number) */
    uint32_t slots_even = (uint32_t)((count + 1u) & ~1u);
    size_t codes_off = (size_t)uoff + 4;
    size_t codes_bytes = (size_t)slots_even * 2u;
    size_t codes_available = available_until_limit(b, codes_off, record_limit);
    if (codes_bytes > codes_available) {
        out->anomalies |= UA_TRUNCATED;
        unwind_diag_set(out, b, "CountOfCodes / UNWIND_CODE array",
                        true, codes_off, true, codes_bytes, codes_available,
                        true, slots_even, (uint32_t)(codes_available / 2u));
        sb_append(err, "Unwind codes array out of file range.\n");
        return unwind_parse_fail(out, err, err_start, "Unwind codes array out of file
            range.");
    }

    uint32_t fixed_stack = 0;
    uint32_t pushed = 0;
    uint16_t nonvol_mask = 0;
    uint16_t xmm_mask = 0;
    bool have_prev_code_offset = false;
    uint8_t prev_code_offset = 0;
    bool saw_set_fpreg = false;
    bool saw_push_nonvol = false;
    bool v2_seen_epilog = false;
```

Slots для операций

```
static bool slots_for_code(uint8_t version,
                           uint8_t op,
                           uint8_t opinfo,
                           bool v2_seen_epilog,
                           uint8_t* out_slots) {
    /* based on documented v1 ops + v2 epilog/spare, and common implementations */
    switch (op) {
        case 0: /* UWOP_PUSH_NONVOL */ *out_slots = 1; return true;
        case 1: /* UWOP_ALLOC_LARGE */ if (opinfo > 1) return false; *out_slots = (uint8_t)(2
            + opinfo); return true;
        case 2: /* UWOP_ALLOC_SMALL */ *out_slots = 1; return true;
        case 3: /* UWOP_SET_FPREG */   *out_slots = 1; return true;
        case 4: /* UWOP_SAVE_NONVOL */ *out_slots = 2; return true;
        case 5: /* UWOP_SAVE_NONVOL_FAR */ *out_slots = 3; return true;
        case 6: /* v2 UWOP_EPILOG, v1 UWOP_SAVE_XMM (undoc) */
            if (version == 2) {
                /* The first long-form epilog descriptor has one operand slot.
                   Short-form and subsequent descriptors occupy one slot. */
                *out_slots = (!v2_seen_epilog && (opinfo & 1u) == 0u) ? 2u : 1u;
                return true;
            }
            *out_slots = 2; return true;
        case 7: /* v2 UWOP_SPARE, v1 UWOP_SAVE_XMM_FAR (undoc) */
            if (version == 2) { *out_slots = 1; return true; }
            *out_slots = 3; return true;
        case 8: /* UWOP_SAVE_XMM128 */ *out_slots = 2; return true;
        case 9: /* UWOP_SAVE_XMM128_FAR */ *out_slots = 3; return true;
        case 10: /* UWOP_PUSH_MACHFRAME */ *out_slots = 1; return true;
        default:
            return false;
    }
}

static bool parse_unwindinfo_header(const Buf* b, const PeImage* img, uint32_t unwind_rva,
                                    uint32_t* out_fileoff, uint8_t hdr[4], StrBuf* err) {
    uint32_t off = 0;
    if (!pe_rva_to_fileoff(img, unwind_rva, &off)) {
        sb_appendf(err, "UnwindInfo RVA 0x%08x not mappable to file offset.\n", unwind_rva);
        return false;
    }
    if ((size_t)off + 4 > b->size) {
        sb_append(err, "UnwindInfo header out of file range.\n");
        return false;
    }
    if (!buf_read(b, off, hdr, 4)) {
        sb_append(err, "Failed to read UnwindInfo header bytes.\n");
        return false;
    }
    *out_fileoff = off;
    return true;
}
```

Цикл по slots

```
    for (uint32_t i = 0; i < count; ) {
        uint16_t slot = 0;
        size_t slot_fileoff = codes_off + (size_t)i * 2u;
        if (!read_u16_slot(b, slot_fileoff, &slot)) {
            out->anomalies |= UA_TRUNCATED;
            unwind_diag_set_read_failure(out, b, "UNWIND_CODE slot", slot_fileoff, 2u,
                record_limit);
            sb_append(err, "Failed to read unwind slot.\n");
            return unwind_parse_fail(out, err, err_start, "Failed to read unwind slot.");
        }

        uint8_t code_offset = (uint8_t)(slot & 0xff);
        uint8_t op_byte = (uint8_t)(slot >> 8);
```

```
uint8_t op = (uint8_t)(op_byte & 0x0f);
uint8_t opinfo = (uint8_t)(op_byte >> 4);

uint8_t slots = 0;
if (!slots_for_code(ver, op, opinfo, v2_seen_epilog, &slots)) {
    out->anomalies |= UA_BAD_OPCODE;
    /* try to continue safely as 1 slot */
    slots = 1;
}
if (i + slots > count) {
    out->anomalies |= UA_TRUNCATED;
    unwind_diag_set(out, b, "CountOfCodes / UWOP operand slots",
                    true, slot_fileoff, true, (size_t)slots * 2u,
                    (size_t)(count - i) * 2u, true, slots, count - i);
    sb_append(err, "Unwind opcode spans beyond CountOfCodes.\n");
    return unwind_parse_fail(out, err, err_start, "Unwind opcode spans beyond
        CountOfCodes.");
}
bool v2_descriptor = ver == 2 && (op == 6 || op == 7);
if (!v2_descriptor) {
    if (code_offset > prolog) {
        out->anomalies |= UA_CODEOFFSET_OUTSIDE;
    }
    if (have_prev_code_offset && code_offset > prev_code_offset) {
        out->anomalies |= UA_CODEOFFSET_ORDER;
    }
    have_prev_code_offset = true;
    prev_code_offset = code_offset;
}
if (strict_abi && !v2_descriptor) {
    if (saw_push_nonvol && op != 0 && op != 10) {
        out->anomalies |= UA_ABI_PUSH_ORDER;
    }
    if (op == 0) saw_push_nonvol = true;
    if ((flags & UNW_FLAG_CHAININFO) && op != 4) {
        out->anomalies |= UA_ABI_CHAIN_RESTRICTION;
    }
}

if (ops_text) {
    /* print one-line per op start */
    switch (op) {
        case 0: sb_appendf(ops_text, "  @%02x PUSH_NONVOL %s\n", code_offset,
        unwind_reg_name(opinfo)); break;
        case 1: sb_appendf(ops_text, "  @%02x ALLOC_LARGE mode=%u\n", code_offset,
        (unsigned)opinfo); break;
        case 2: sb_appendf(ops_text, "  @%02x ALLOC_SMALL %u\n", code_offset,
        (unsigned)(opinfo*8u + 8u)); break;
        case 3: sb_appendf(ops_text, "  @%02x SET_FPREG %s + %u\n", code_offset,
        unwind_reg_name(fr), (unsigned)(foff*16u)); break;
        case 4: sb_appendf(ops_text, "  @%02x SAVE_NONVOL %s\n", code_offset,
        unwind_reg_name(opinfo)); break;
        case 5: sb_appendf(ops_text, "  @%02x SAVE_NONVOL_FAR %s\n", code_offset,
        unwind_reg_name(opinfo)); break;
        case 6:
            if (ver == 2) {
                if (!v2_seen_epilog) {
                    if ((opinfo & 1u) == 0u) {
                        uint16_t epilog_slot = 0;
                        if (read_u16_slot(b, codes_off + (size_t)(i + 1) * 2,
                                          &epilog_slot)) {
                            unsigned epilog_offset =
                                (unsigned)(epilog_slot & 0xffu) |
                                ((unsigned)(epilog_slot >> 12) << 8);
                            sb_appendf(ops_text,
                                       "  EPILOG size=%u offset_from_end=%u flags=%u
        (v2)\n",
                                       (unsigned)code_offset, epilog_offset,
                                       (unsigned)opinfo);
                        }
                    } else {
                        sb_appendf(ops_text, "  EPILOG size=%u flags=%u (v2)\n",
                                   (unsigned)code_offset, (unsigned)opinfo);
                    }
                } else {
                    unsigned epilog_offset =
                        (unsigned)code_offset | ((unsigned)opinfo << 8);
                    sb_appendf(ops_text, "  EPILOG offset_from_end=%u (v2)\n",
                               epilog_offset);
                }
            } else {
                sb_appendf(ops_text, "  @%02x SAVE_XMM (undoc/v1)\n", code_offset);
            }
            break;
        case 7:
            if (ver == 2) sb_appendf(ops_text, "  @%02x SPARE (v2)\n", code_offset);
            else sb_appendf(ops_text, "  @%02x SAVE_XMM_FAR (undoc/v1)\n",
        code_offset);
            break;
        case 8: sb_appendf(ops_text, "  @%02x SAVE_XMM128 XMM%u\n", code_offset,
        (unsigned)opinfo); break;
        case 9: sb_appendf(ops_text, "  @%02x SAVE_XMM128_FAR XMM%u\n", code_offset,
        (unsigned)opinfo); break;
        case 10: sb_appendf(ops_text, "  @%02x PUSH_MACHFRAME err=%u\n", code_offset,
        (unsigned)opinfo); break;
        default: sb_appendf(ops_text, "  @%02x OP_%u info=%u\n", code_offset,
        (unsigned)op, (unsigned)opinfo); break;
    }
}

/* compute effects */
switch (op) {
    case 0: /* push nonvol */
        add_fixed_stack(&fixed_stack, 8, &out->anomalies);
```

```
            pushed++;
            if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;
            if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {
                out->anomalies |= UA_ABI_REGISTER_CLASS;
            }
            if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);
            break;
        case 1: { /* alloc large */
            if (opinfo == 0) {
                uint16_t scaled = 0;
                if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {
                    out->anomalies |= UA_TRUNCATED;
                    unwind_diag_set_read_failure(out, b, "UWOP_ALLOC_LARGE operand",
                                                 codes_off + (size_t)(i + 1) * 2u,
                                                 2u, record_limit);
                    return unwind_parse_fail(out, err, err_start, "Truncated
            UWOP_ALLOC_LARGE operand.");
                }
```

CountOfCodes, slots и padding

`CountOfCodes` говорит, сколько двухбайтовых slots логически занято массивом unwind-кодов. Но не каждая unwind-операция занимает один slot. `UWOP_ALLOC_LARGE` может занимать два или три slots, `UWOP_SAVE_NONVOL_FAR` - три, `UWOP_SAVE_XMM128` - два. Проект не делает наивное “один opcode - один slot”. Он вызывает `slots_for_code`, проверяет, что `i + slots <= count`, и только потом читает operands. Padding учтён через `slots_even = (count + 1u) & ~1u`. Если `CountOfCodes` нечётный, tail начинается после дополнительного padding slot. Без этого handler RVA или chained info были бы прочитаны со смещением на два байта.

Что должен уметь объяснить студент

Объяснить `Version`, `Flags`, `SizeOfProlog`, `CountOfCodes`. Объяснить, почему одна UWOP может занимать несколько slots. Объяснить, как вычисляется начало tail.

Глава 26. Диагностика: как parser объясняет, где именно сломалось чтение

Хороший системный parser не должен просто сказать parse failed. Он должен сохранить поле, file offset, expected size, available size, expected slots, available slots и байты вокруг ошибки. В проекте это сделано в `UnwindSummary`: помимо итоговых полей там есть диагностическая часть.

Диагностические поля

```
    char     diagnostic_field[64];   /* exact field/area that caused the strongest diagnostic
        */
    bool     diagnostic_has_file_offset;
    uint32_t diagnostic_file_offset;
    bool     diagnostic_has_size;
    uint32_t diagnostic_expected_size;
    uint32_t diagnostic_available_size;
    bool     diagnostic_has_slots;
    uint32_t diagnostic_expected_slots;
    uint32_t diagnostic_available_slots;
    bool     diagnostic_has_bytes;
    uint32_t diagnostic_bytes_file_offset;
    uint8_t  diagnostic_bytes[UNWIND_DIAG_MAX_BYTES];
    uint8_t  diagnostic_bytes_count;

    uint32_t chain_path[UNWIND_DIAG_MAX_CHAIN_PATH];
    uint8_t  chain_path_count;
    bool     chain_loop_detected;
    uint32_t chain_loop_rva;
} UnwindSummary;

enum UnwindFlags {
    UNW_FLAG_EHANDLER  = 0x01,
    UNW_FLAG_UHANDLER  = 0x02,
    UNW_FLAG_CHAININFO = 0x04
};

/* anomalies flags (for heuristic analysis / diagnostics) */
enum UnwindAnom {
    UA_VERSION_UNKNOWN       = 1u << 0,
    UA_TRUNCATED             = 1u << 1,
    UA_BAD_OPCODE            = 1u << 2,
    UA_BAD_OPERAND           = 1u << 3,
    UA_CODEOFFSET_OUTSIDE    = 1u << 4,
    UA_CHAIN_LOOP            = 1u << 5,
    UA_CHAIN_DEPTH_EXCEEDED  = 1u << 6,
    UA_HANDLER_OOB           = 1u << 7,
    UA_CODEOFFSET_ORDER      = 1u << 8,
    UA_OPERAND_SANITY        = 1u << 9,
    UA_ALIGNMENT             = 1u << 10,
    UA_CHAIN_INVALID         = 1u << 11,
```

Заполнение диагностики

```
static void unwind_diag_set(UnwindSummary* out,
                            const Buf* b,
                            const char* field,
                            bool has_file_offset,
                            size_t file_offset,
                            bool has_size,
                            size_t expected_size,
                            size_t available_size,
                            bool has_slots,
                            uint32_t expected_slots,
                            uint32_t available_slots) {
    if (!out) return;
    if (field && field[0]) {
        size_t n = strlen(field);
        if (n >= sizeof(out->diagnostic_field)) n = sizeof(out->diagnostic_field) - 1;
        memcpy(out->diagnostic_field, field, n);
        out->diagnostic_field[n] = '\0';
    }
    out->diagnostic_has_file_offset = has_file_offset;
    if (has_file_offset) out->diagnostic_file_offset = clamp_size_to_u32(file_offset);
    out->diagnostic_has_size = has_size;
    if (has_size) {
        out->diagnostic_expected_size = clamp_size_to_u32(expected_size);
        out->diagnostic_available_size = clamp_size_to_u32(available_size);
    }
    out->diagnostic_has_slots = has_slots;
    if (has_slots) {
        out->diagnostic_expected_slots = expected_slots;
        out->diagnostic_available_slots = available_slots;
    }
    if (has_file_offset) {
        size_t max_bytes = has_size ? available_size : (size_t)UNWIND_DIAG_MAX_BYTES;
        unwind_diag_capture_bytes(out, b, file_offset, max_bytes);
    }
}

static void unwind_diag_set_read_failure(UnwindSummary* out,
                                         const Buf* b,
                                         const char* field,
                                         size_t file_offset,
                                         size_t expected_size,
                                         size_t limit) {
    size_t available = available_until_limit(b, file_offset, limit);
    unwind_diag_set(out, b, field, true, file_offset, true,
```

```
                    expected_size, available, false, 0, 0);
}

static uint32_t read_u32_from_slots(uint16_t lo, uint16_t hi) {
    return (uint32_t)lo | ((uint32_t)hi << 16);
}

static bool is_unwindable_gpr(uint8_t reg) {
    /* Real MSVC, Rust and OpenSSL records also describe volatile registers. */
    return reg < 16 && reg != 4;
}

static bool is_unwindable_xmm(uint8_t reg) {
    return reg < 16;
}

static bool is_abi_nonvolatile_gpr(uint8_t reg) {
    return reg == 3 || reg == 5 || reg == 6 || reg == 7 || reg >= 12;
}

static bool is_abi_nonvolatile_xmm(uint8_t reg) {
    return reg >= 6 && reg < 16;
}

static void add_fixed_stack(uint32_t* total, uint32_t amount, uint32_t* anomalies) {
    if (!total || !anomalies) return;
```

Зачем это нужно

`unwind_diag_set` получает `UnwindSummary* out` и аккуратно заполняет только те диагностические поля, для которых есть данные. `bool has_file_offset`, `bool has_size`, `bool has_slots` нужны, чтобы отличать “значение равно нулю” от “значение отсутствует”. В отчете такие данные превращаются в evidence. Если `UWOP_ALLOC_LARGE` требует operand slot, которого нет внутри `CountOfCodes`, отчет может показать failed field, file offset, expected slots и available slots. Структура результата может хранить не только успешные данные, но и подробности неуспешного анализа. Это важно для инструмента, который должен быть полезен на поврежденных файлах.

Что должен уметь объяснить студент

Объяснить `failed_field`. Объяснить expected/available size. Объяснить, почему parse fail не должен превращаться в мусорный handler вывод.

Глава 27. Heuristics layer: признаки, score, suspicion, confidence

Эвристический слой не читает файл напрямую. Он получает уже построенную PE-карту, массив RUNTIME_FUNCTION, массив UnwindSummary и массив us_ok. Его задача - агрегировать признаки: нарушения таблицы, ошибки unwind, нетипичное размещение metadata, reuse UnwindInfoAddress, dominance handler-а.

HeuristicReport

```
typedef struct HeuristicReport {
    int    score; /* 0..100 (higher => more suspicious) */
    double suspicion;  /* 0..1 derived from score */
    double confidence; /* 0..1 confidence in the static verdict */
    bool   requires_manual_review;
    uint32_t evidence_families_mask;
    char   verdict[48];

    size_t total;

    size_t pe_section_raw_oob;
    size_t pe_section_virtual_oob;
    size_t pe_section_raw_overlaps;
    size_t pe_section_virtual_overlaps;
    size_t pe_section_misaligned;
    bool   pe_exception_directory_oob;

    size_t rf_invalid_ranges;
    size_t rf_oob;
    size_t rf_unwind_oob;
    size_t rf_unsorted;
    size_t rf_overlaps;
    size_t rf_duplicate_begins;

    size_t unwind_parse_fail;
    size_t unwind_anomalies;
    size_t unwind_abi_warnings;

    size_t unwind_unusual_section;
    size_t unwind_exec_section;
    size_t unwind_writable_section;
    size_t handler_nonexec_section;
    size_t handler_writable_section;

    double unwind_reuse_ratio;    /* max_count / total */
    double unwind_unique_ratio;   /* unique / total */

    double handler_reuse_ratio;   /* max_count / handler_count (if any) */
    size_t handler_count;

    size_t finding_count;
    HeuristicFinding findings[HEURISTICS_MAX_FINDINGS];
    size_t weak_finding_count;
    size_t medium_finding_count;
    size_t strong_finding_count;
    char evidence_level[16];

    size_t mitigation_count;
    HeuristicMitigation mitigations[HEURISTICS_MAX_MITIGATIONS];

    StrBuf details; /* human-readable findings */
} HeuristicReport;

bool heuristics_analyze(const PeImage* img,
                        const RuntimeFunction* rf,
                        const UnwindSummary* us,
                        const bool* us_ok,
                        size_t n,
                        HeuristicReport* out,
                        StrBuf* err);

void heuristics_free(HeuristicReport* r);
const char* heuristics_evidence_family_name(uint32_t bit);
size_t heuristics_evidence_family_count(uint32_t mask);

#ifdef __cplusplus
}
#endif
```

Калибровка

```
 * Heuristic calibration notes
 * --------------------------
 * These constants intentionally form a conservative engineering model rather
 * than a mathematically validated classifier. The weights are grouped by the
 * evidence family discussed in the thesis so the rationale stays stable and
 * reviewable:
 *   - table/unwind integrity violations => strongest static weights
 *   - placement anomalies => medium/high depending on prevalence
 *   - reuse/dominance patterns => moderate unless they are very strong
 *   - mitigations reduce confidence when evidence is isolated or sample size is tiny
 */
static const int SCORE_EMPTY_PDATA = 10;
static const int SCORE_PE_SECTION_RAW_OOB = 25;
static const int SCORE_PE_SECTION_VIRTUAL_OOB = 20;
static const int SCORE_PE_SECTION_RAW_OVERLAP = 15;
static const int SCORE_PE_SECTION_VIRTUAL_OVERLAP = 20;
static const int SCORE_PE_SECTION_MISALIGNED = 8;
```

```
static const int SCORE_PE_EXCEPTION_DIRECTORY_OOB = 25;
static const int SCORE_INVALID_RANGE_LOW = 10;
static const int SCORE_INVALID_RANGE_HIGH = 20;
static const int SCORE_RF_OOB_LOW = 10;
static const int SCORE_RF_OOB_HIGH = 25;
static const int SCORE_UNWIND_RVA_OOB_LOW = 10;
static const int SCORE_UNWIND_RVA_OOB_HIGH = 25;
static const int SCORE_RF_UNSORTED = 15;
static const int SCORE_RF_OVERLAP = 25;
static const int SCORE_DUPLICATE_BEGIN = 10;
static const int SCORE_UNWIND_PARSE_FAIL_LOW = 10;
static const int SCORE_UNWIND_PARSE_FAIL_HIGH = 25;
static const int SCORE_UNWIND_ANOM_LOW = 10;
static const int SCORE_UNWIND_ANOM_HIGH = 20;
static const int SCORE_UNWIND_REUSE_MEDIUM = 10;
static const int SCORE_UNWIND_REUSE_HIGH = 20;
static const int SCORE_HANDLER_DOMINANCE = 15;
static const int SCORE_UNWIND_EXEC_SECTION = 20;
static const int SCORE_UNWIND_UNUSUAL_SECTION_LOW = 8;
static const int SCORE_UNWIND_UNUSUAL_SECTION_HIGH = 15;
static const int SCORE_UNWIND_WRITABLE_SECTION_LOW = 10;
static const int SCORE_UNWIND_WRITABLE_SECTION_HIGH = 18;
static const int SCORE_HANDLER_NONEXEC = 15;
static const int SCORE_HANDLER_WRITABLE = 10;
static const int SCORE_COMBO_PLACEMENT_EXEC_AND_HANDLER = 10;
static const int SCORE_COMBO_PLACEMENT_AND_INTEGRITY = 8;
static const int SCORE_COMBO_REUSE_AND_DOMINANCE = 8;
static const int SCORE_COMBO_WRITABLE_METADATA_AND_HANDLER = 10;

static const double CONFIDENCE_BASE = 0.15;
static const double CONFIDENCE_ZERO_SCORE_BASE = 0.20;
static const double CONFIDENCE_PER_FAMILY = 0.11;
static const double CONFIDENCE_PER_STRONG = 0.08;
static const double CONFIDENCE_PER_MEDIUM = 0.04;
static const double CONFIDENCE_BONUS_TABLE_HARD = 0.08;
static const double CONFIDENCE_BONUS_UNWIND_HARD = 0.08;
static const double CONFIDENCE_PENALTY_SINGLE_SIGNAL = 0.12;
static const double CONFIDENCE_PENALTY_SMALL_SAMPLE = 0.06;
static const double CONFIDENCE_PENALTY_PLACEMENT_ONLY = 0.10;
static const double CONFIDENCE_PENALTY_SINGLE_FAMILY = 0.08;
static const double CONFIDENCE_MIN = 0.05;
static const double CONFIDENCE_MAX = 0.90;

static const double THRESHOLD_HIGH_PERCENT = 0.01;   /* >1% of entries => not just a one-off
    glitch */
static const double THRESHOLD_UNWIND_ANOM_HIGH = 0.05; /* >5% anomalies => broad integrity
    issue */
static const double THRESHOLD_UNWIND_REUSE_MEDIUM = 0.10;
static const double THRESHOLD_UNWIND_REUSE_HIGH = 0.30;
static const double THRESHOLD_HANDLER_DOMINANCE = 0.80;

static bool sec_name_eq(const char* a, const char* b) {
    if (!a || !b) return false;
    while (*a && *b) {
        char ca = *a;
```

Финализация

```
static void finalize_verdict(HeuristicReport* out) {
    if (!out) return;

    out->suspicion = safe_ratio((double)out->score, 100.0);

    size_t family_count = heuristics_evidence_family_count(out->evidence_families_mask);
    size_t strong_count = count_findings_with_severity(out, "high") +
        count_findings_with_severity(out, "critical");
    size_t medium_count = count_findings_with_severity(out, "medium");
    size_t weak_count = 0;
    if (out->finding_count >= strong_count + medium_count) weak_count = out->finding_count -
        strong_count - medium_count;

    out->strong_finding_count = strong_count;
    out->medium_finding_count = medium_count;
    out->weak_finding_count = weak_count;
    safe_copy(out->evidence_level, sizeof(out->evidence_level),
              evidence_level_from_counts(weak_count, medium_count, strong_count, out->score));

    double confidence = CONFIDENCE_BASE;
    confidence += (double)family_count * CONFIDENCE_PER_FAMILY;
    confidence += (double)strong_count * CONFIDENCE_PER_STRONG;
    confidence += (double)medium_count * CONFIDENCE_PER_MEDIUM;

    if (out->rf_invalid_ranges || out->rf_oob || out->rf_unwind_oob || out->rf_overlaps)
        confidence += CONFIDENCE_BONUS_TABLE_HARD;
    if (out->unwind_parse_fail || out->unwind_anomalies) confidence +=
        CONFIDENCE_BONUS_UNWIND_HARD;
    if (out->score == 0) confidence = CONFIDENCE_ZERO_SCORE_BASE;

    /* Weakening rules: lower confidence when only isolated / non-correlated static signals
        exist. */
    if (out->finding_count == 1 && strong_count == 0) {
        add_mitigation(out, "single_signal_only",
                       "Only one non-strong signal contributed; treat as weak standalone
                evidence.");
        confidence -= CONFIDENCE_PENALTY_SINGLE_SIGNAL;
    }
    if (out->total < 32 && strong_count == 0 && out->finding_count > 0) {
        add_mitigation(out, "small_sample_size",
                       "Very small .pdata sample can overstate ratios; confidence reduced
                until more functions are observed.");
        confidence -= CONFIDENCE_PENALTY_SMALL_SAMPLE;
    }
    if (out->unwind_unusual_section > 0 &&
        out->unwind_exec_section == 0 &&
```

```
        out->unwind_writable_section == 0 &&
        out->handler_nonexec_section == 0 &&
        out->handler_writable_section == 0 &&
        out->rf_invalid_ranges == 0 &&
        out->rf_oob == 0 &&
        out->rf_unwind_oob == 0 &&
        out->unwind_parse_fail == 0 &&
        out->unwind_anomalies == 0) {
        add_mitigation(out, "placement_only_signal",
                       "Placement-only anomalies may come from uncommon but benign layouts;
                manual verification required.");
        confidence -= CONFIDENCE_PENALTY_PLACEMENT_ONLY;
    }
    if ((out->strong_finding_count == 0) && (family_count <= 1) && out->finding_count > 0) {
        add_mitigation(out, "single_family_only",
                       "Evidence comes from only one family; cross-layer corroboration is
                recommended before strong conclusions.");
        confidence -= CONFIDENCE_PENALTY_SINGLE_FAMILY;
    }

    if (confidence < CONFIDENCE_MIN) confidence = CONFIDENCE_MIN;
    if (confidence > CONFIDENCE_MAX) confidence = CONFIDENCE_MAX;
    out->confidence = confidence;

    out->requires_manual_review =
        (out->score >= 20) ||
        (out->unwind_exec_section > 0) ||
        (out->unwind_writable_section > 0) ||
        (out->handler_nonexec_section > 0) ||
        (out->handler_writable_section > 0) ||
        (out->unwind_parse_fail > 0) ||
        (out->mitigation_count > 0);

    if (out->score >= 70) safe_copy(out->verdict, sizeof(out->verdict), "high_suspicion");
    else if (out->score >= 40) safe_copy(out->verdict, sizeof(out->verdict),
        "moderate_suspicion");
    else if (out->score >= 15) safe_copy(out->verdict, sizeof(out->verdict), "low_suspicion");
    else safe_copy(out->verdict, sizeof(out->verdict), "no_strong_static_anomalies");
}

bool heuristics_analyze(const PeImage* img,
                        const RuntimeFunction* rf,
                        const UnwindSummary* us,
                        const bool* us_ok,
                        size_t n,
                        HeuristicReport* out,
                        StrBuf* err) {
    if (!img || !out || !err) return false;
    if (n > 0 && (!rf || !us || !us_ok)) return false;
    memset(out, 0, sizeof(*out));
    if (!sb_init(&out->details, 1024)) {
        sb_append(err, "Out of memory (details buffer).\n");
        return false;
    }
    out->total = n;
    out->pe_section_raw_oob = img->section_raw_oob;
    out->pe_section_virtual_oob = img->section_virtual_oob;
    out->pe_section_raw_overlaps = img->section_raw_overlaps;
    out->pe_section_virtual_overlaps = img->section_virtual_overlaps;
    out->pe_section_misaligned = img->section_misaligned;
    out->pe_exception_directory_oob = img->exception_directory_oob;

    /* RUNTIME_FUNCTION sanity */
    uint32_t prev_begin = 0;
    bool prev_valid = false;
    size_t unsorted = 0;
    FindingLocation invalid_range_loc = {0};
    FindingLocation rf_oob_loc = {0};
    FindingLocation unwind_oob_loc = {0};
    FindingLocation unsorted_loc = {0};
    FindingLocation overlap_loc = {0};
    FindingLocation duplicate_loc = {0};
    FindingLocation parse_fail_loc = {0};
    FindingLocation anomaly_loc = {0};
    FindingLocation unusual_unwind_loc = {0};
    FindingLocation executable_unwind_loc = {0};
    FindingLocation writable_unwind_loc = {0};
    FindingLocation nonexec_handler_loc = {0};
    FindingLocation writable_handler_loc = {0};
```

Score и confidence

С точки зрения C `HeuristicReport` - большая структура состояния: счетчики, доли, итоговый score, confidence, verdict, массив findings и буфер details. С точки зрения методологии `score` и `confidence` нельзя путать. `score` отражает подозрительность, а `confidence` отражает уверенность в выводе. В `finalize_verdict` итоговая уверенность не является точностью машинного обучения. Это инженерная оценка статического evidence с усилениями и снижениями для одиночных сигналов, малого sample size или placement-only ситуации.

Что должен уметь объяснить студент

Различить score, suspicion, confidence. Пояснить evidence families. Пояснить mitigation small_sample_size.

Глава 28. Finding: как C-структура становится доказательным сообщением

Finding - это не просто строка “подозрительно”. В проекте finding имеет code, family, severity, message, location и affected_count. Report layer расширяет его русским объяснением: правило, ожидаемое состояние, наблюдаемое состояние, последствие и следующий шаг ручной проверки.

Структура Finding

```
typedef struct HeuristicFinding {
    char code[40];
    char family[32];
    char severity[16];
    char message[192];
    bool has_location;
    uint32_t representative_rva;
    size_t affected_count;
} HeuristicFinding;

typedef struct HeuristicMitigation {
    char code[40];
    char message[192];
} HeuristicMitigation;

typedef struct HeuristicReport {
    int    score; /* 0..100 (higher => more suspicious) */
    double suspicion;  /* 0..1 derived from score */
    double confidence; /* 0..1 confidence in the static verdict */
    bool   requires_manual_review;
```

Добавление finding

```
static HeuristicFinding* add_finding(HeuristicReport* out,
                                     const char* code,
                                     const char* family,
                                     const char* severity,
                                     const char* fmt,
                                     ...) {
    if (!out || !fmt) return NULL;

    char msg[192];
    va_list ap;
    va_start(ap, fmt);
    (void)vsnprintf(msg, sizeof(msg), fmt, ap);
    va_end(ap);

    out->evidence_families_mask |= family_bit_from_name(family);

    HeuristicFinding* stored = NULL;
    if (out->finding_count < HEURISTICS_MAX_FINDINGS) {
        HeuristicFinding* f = &out->findings[out->finding_count++];
        memset(f, 0, sizeof(*f));
        safe_copy(f->code, sizeof(f->code), code ? code : "unspecified");
        safe_copy(f->family, sizeof(f->family), family ? family : "unknown");
        safe_copy(f->severity, sizeof(f->severity), severity ? severity : "info");
        safe_copy(f->message, sizeof(f->message), msg);
        stored = f;
    }

    sb_appendf(&out->details, "- [%s/%s] %s\n",
               severity ? severity : "info",
               code ? code : "unspecified",
               msg);
    return stored;
}

static void annotate_finding(HeuristicFinding* finding,
                             FindingLocation location,
                             size_t affected_count) {
    if (!finding) return;
    finding->has_location = location.set;
    finding->representative_rva = location.rva;
    finding->affected_count = affected_count;
}

static void append_note(HeuristicReport* out, const char* fmt, ...) {
    if (!out || !fmt) return;

    char msg[192];
    va_list ap;
    va_start(ap, fmt);
    (void)vsnprintf(msg, sizeof(msg), fmt, ap);
    va_end(ap);

    sb_appendf(&out->details, "- [info] %s\n", msg);
}

static size_t count_findings_with_severity(const HeuristicReport* out, const char* sev) {
    size_t n = 0;
    if (!out || !sev) return 0;
    for (size_t i = 0; i < out->finding_count; i++) {
        if (strcmp(out->findings[i].severity, sev) == 0) n++;
    }
    return n;
}

static bool has_finding_code(const HeuristicReport* out, const char* code) {
```

```
    if (!out || !code) return false;
    for (size_t i = 0; i < out->finding_count; i++) {
        if (strcmp(out->findings[i].code, code) == 0) return true;
    }
    return false;
```

Русские объяснения

```
static const char* finding_rule_text(const char* code) {
    if (!code) return "Код признака отсутствует, поэтому отчет не может сопоставить его с
        правилом.";
    if (strcmp(code, "pe_section_raw_oob") == 0) return "Raw-диапазон каждой секции должен
        лежать внутри файла: PointerToRawData + SizeOfRawData <= размер файла.";
    if (strcmp(code, "pe_section_virtual_oob") == 0) return "Virtual-диапазон каждой секции
        должен лежать внутри загруженного образа: VirtualAddress + max(VirtualSize,
        SizeOfRawData) <= SizeOfImage.";
    if (strcmp(code, "pe_section_raw_overlap") == 0) return "Raw-диапазоны секций не должны
        пересекаться, иначе один участок файла получает двух владельцев.";
    if (strcmp(code, "pe_section_virtual_overlap") == 0) return "Virtual-диапазоны секций не
        должны пересекаться, иначе один RVA неоднозначно относится сразу к нескольким областям
         image.";
    if (strcmp(code, "pe_section_misaligned") == 0) return "Инварианты PE-выравнивания должны
        выполняться: FileAlignment, SectionAlignment, заголовки и секции должны быть
        согласованы.";
    if (strcmp(code, "pe_exception_directory_oob") == 0) return "Exception Directory должна
        либо корректно отсутствовать как RVA=0 и Size=0, либо целиком помещаться в
        SizeOfImage.";
    if (strcmp(code, "no_runtime_functions") == 0) return "Для нетривиального x64-образа с
        исполняемым кодом обычно ожидаются записи RUNTIME_FUNCTION для non-leaf функций.";
    if (strcmp(code, "invalid_ranges") == 0) return "Каждая запись RUNTIME_FUNCTION должна
        описывать положительный диапазон: EndAddress должен быть больше BeginAddress.";
    if (strcmp(code, "rf_oob") == 0) return "Диапазон BeginAddress/EndAddress каждой
        RUNTIME_FUNCTION должен находиться внутри SizeOfImage.";
    if (strcmp(code, "unwind_rva_oob") == 0) return "UnwindInfoAddress каждой RUNTIME_FUNCTION
         должен указывать на доступную UNWIND_INFO внутри image.";
    if (strcmp(code, "rf_unsorted") == 0) return "Таблица RUNTIME_FUNCTION должна быть
        отсортирована по BeginAddress, чтобы поиск по ControlPc был однозначным.";
    if (strcmp(code, "rf_overlaps") == 0) return "Диапазоны RUNTIME_FUNCTION не должны
        пересекаться, потому что один RVA кода должен иметь одну unwind-запись.";
    if (strcmp(code, "duplicate_begin") == 0) return "Разные записи RUNTIME_FUNCTION не должны
         повторно использовать один и тот же BeginAddress.";
    if (strcmp(code, "unwind_parse_fail") == 0) return "UNWIND_INFO должна структурно
        разбираться по формату x64 unwind без обрыва и выхода за границы.";
    if (strcmp(code, "unwind_anomalies") == 0) return "Поля UNWIND_INFO и unwind-коды должны
        соответствовать кодировке Microsoft x64 unwind и ограничениям ABI.";
    if (strcmp(code, "unwind_reuse_strong") == 0) return "Массовое повторное использование
        одного UnwindInfoAddress многими функциями нетипично и требует подтверждения другими
        признаками.";
    if (strcmp(code, "unwind_reuse_moderate") == 0) return "Умеренное повторное использование
        UnwindInfoAddress становится доказательством только при наличии другого статического
        сигнала.";
    if (strcmp(code, "handler_dominance") == 0) return "Один handler target не должен
        доминировать над большой группой защищенных функций без понятного runtime/compiler-
        объяснения.";
    if (strcmp(code, "unwind_in_exec_section") == 0) return "UNWIND_INFO не должна размещаться
         в исполняемой секции кода.";
    if (strcmp(code, "unwind_unusual_section") == 0) return "UNWIND_INFO обычно лежит в
        metadata-секциях вроде .xdata, .rdata или .pdata.";
    if (strcmp(code, "unwind_writable_section") == 0) return "UNWIND_INFO обычно должна быть
        read-only metadata, а не writable-областью.";
    if (strcmp(code, "handler_nonexec_section") == 0) return "Exception handler должен
        указывать на исполняемый код.";
    if (strcmp(code, "handler_writable_section") == 0) return "Exception handler в writable-
        памяти нетипичен для обычного compiler-produced образа.";
    if (strcmp(code, "placement_exec_nonexec_handler_combo") == 0) return "Подозрительное
        размещение unwind metadata и handler в non-executable области усиливают друг друга как
         единый комбо-признак.";
    if (strcmp(code, "placement_integrity_combo") == 0) return "Подозрительное размещение
        metadata вместе с ошибками целостности unwind/table усиливает итоговый вывод.";
    if (strcmp(code, "reuse_dominance_combo") == 0) return "Повторное использование UnwindInfo
         и доминирующий handler вместе похожи на dispatcher-style схему.";
    if (strcmp(code, "writable_metadata_handler_combo") == 0) return "Writable unwind metadata
         и writable handler targets вместе образуют более сильный признак подозрительного
        размещения.";
    return "Эвристическое правило определяется кодом найденного признака.";
}

static const char* finding_expected_text(const char* code) {
    if (!code) return "Признак должен иметь известный код и достаточно контекста для
        объяснения.";
    if (strcmp(code, "pe_section_raw_oob") == 0) return "Для каждой секции с raw-данными
        должно выполняться raw_end <= размер файла.";
    if (strcmp(code, "pe_section_virtual_oob") == 0) return "Для каждой секции должно
        выполняться virtual_end <= SizeOfImage.";
    if (strcmp(code, "pe_section_raw_overlap") == 0) return "Raw-интервалы секций должны быть
        непересекающимися или пустыми.";
    if (strcmp(code, "pe_section_virtual_overlap") == 0) return "Virtual-интервалы секций
        должны быть непересекающимися.";
    if (strcmp(code, "pe_section_misaligned") == 0) return "Заголовки, SizeOfImage, ImageBase
        и raw/virtual адреса секций должны выполнять правила PE-выравнивания.";
    if (strcmp(code, "pe_exception_directory_oob") == 0) return "Exception Directory должна
        корректно отсутствовать или иметь валидный читаемый диапазон [RVA, RVA+Size).";
    if (strcmp(code, "no_runtime_functions") == 0) return "x64-код со stack frame, сохранением
         nonvolatile регистров, handlers или вызовами обычно имеет .pdata записи.";
    if (strcmp(code, "invalid_ranges") == 0) return "Для каждой RUNTIME_FUNCTION должно
        выполняться BeginAddress < EndAddress.";
    if (strcmp(code, "rf_oob") == 0) return "BeginAddress и EndAddress должны разрешаться
        внутри загруженного image.";
    if (strcmp(code, "unwind_rva_oob") == 0) return "UnwindInfoAddress должен быть RVA внутри
        image и должен мапиться в байты файла при статическом чтении.";
    if (strcmp(code, "rf_unsorted") == 0) return "Записи должны идти по возрастанию
        BeginAddress.";
    if (strcmp(code, "rf_overlaps") == 0) return "После сортировки EndAddress предыдущей
        записи должен быть <= BeginAddress следующей.";
```

```
    if (strcmp(code, "duplicate_begin") == 0) return "Каждый BeginAddress должен
        идентифицировать одну запись function table.";
    if (strcmp(code, "unwind_parse_fail") == 0) return "Version, Flags, CountOfCodes, массив
        unwind-кодов, padding и tail-данные должны разбираться без усечения.";
    if (strcmp(code, "unwind_anomalies") == 0) return "Unwind opcodes, operands, code offsets,
         chained info и handler data должны быть внутренне согласованы.";
    if (strcmp(code, "unwind_reuse_strong") == 0) return "Большинство функций должны иметь
        собственную UNWIND_INFO или делить ее только в небольших compiler-plausible
        шаблонах.";
    if (strcmp(code, "unwind_reuse_moderate") == 0) return "Reuse должен оставаться ниже
        умеренного порога, если нет понятного benign compiler/runtime объяснения.";
    if (strcmp(code, "handler_dominance") == 0) return "Handler references должны
        распределяться согласно обычным language/runtime структурам или объясняться известными
         runtime helpers.";
    if (strcmp(code, "unwind_in_exec_section") == 0) return "UNWIND_INFO должна лежать в non-
        executable metadata, обычно .xdata/.rdata/.pdata.";
    if (strcmp(code, "unwind_unusual_section") == 0) return "UNWIND_INFO должна находиться в
        секции, роль которой соответствует read-only exception/unwind metadata.";
    if (strcmp(code, "unwind_writable_section") == 0) return "Секция с UNWIND_INFO обычно
        должна быть readable и не должна быть writable/executable.";
    if (strcmp(code, "handler_nonexec_section") == 0) return "Handler RVA должен разрешаться в
         executable-секцию.";
    if (strcmp(code, "handler_writable_section") == 0) return "Handler RVA должен указывать на
         executable-код без неожиданного writable-права.";
    if (strcmp(code, "placement_exec_nonexec_handler_combo") == 0) return "Placement-признаки
        не должны одновременно усиливать друг друга по unwind metadata и handler targets.";
    if (strcmp(code, "placement_integrity_combo") == 0) return "Placement-признаки не должны
        совпадать с parse/OOB/integrity-проблемами.";
    if (strcmp(code, "reuse_dominance_combo") == 0) return "Reuse-признаки не должны совпадать
         с одним dominant handler target без доброкачественного объяснения.";
    if (strcmp(code, "writable_metadata_handler_combo") == 0) return "Writable metadata и
        writable handler targets не должны вместе встречаться в нормальном compiler output.";
    return "Наблюдаемое состояние должно удовлетворять соответствующему PE/unwind-
        инварианту.";
}

static const char* finding_impact_text(const char* code) {
    if (!code) return "Неизвестный признак сложнее проверить, поэтому его нужно рассматривать
        вручную.";
    if (strncmp(code, "pe_section_", 11) == 0 || strcmp(code, "pe_exception_directory_oob") ==
         0) return "Это может сломать перевод RVA в file offset, сделать дальнейшие
        доказательства неоднозначными или показать поврежденную/враждебную PE-разметку.";
    if (strcmp(code, "no_runtime_functions") == 0) return "Stack walking может быть неполным
        для non-leaf кода, поэтому аналитику нужно подтвердить, что образ действительно leaf-
        only или специального назначения.";
    if (strcmp(code, "invalid_ranges") == 0 || strcmp(code, "rf_oob") == 0 || strcmp(code,
        "rf_unsorted") == 0 || strcmp(code, "rf_overlaps") == 0 || strcmp(code,
        "duplicate_begin") == 0) return "Function table может перестать давать уникальную и
        надежную карту от instruction RVA к unwind metadata.";
    if (strcmp(code, "unwind_rva_oob") == 0) return "Windows или эмуляция unwind может не
        найти metadata функции, из-за чего ломается virtual unwind и восстановление стека.";
    if (strcmp(code, "unwind_parse_fail") == 0 || strcmp(code, "unwind_anomalies") == 0)
        return "Virtual unwind может вычислить неверный caller context, оборваться раньше
        времени или скрыть реальный exception/handler path.";
    if (strcmp(code, "unwind_reuse_strong") == 0 || strcmp(code, "unwind_reuse_moderate") ==
        0) return "Сильный metadata reuse может указывать на синтетическую или dispatcher-
        oriented разметку, особенно при совпадении с другими признаками.";
    if (strcmp(code, "handler_dominance") == 0) return "Один handler, контролирующий много
        функций, может указывать на централизованный exception dispatch, обфускацию или
        защитную логику.";
    if (strcmp(code, "unwind_in_exec_section") == 0 || strcmp(code, "unwind_unusual_section")
        == 0 || strcmp(code, "unwind_writable_section") == 0) return "Нетипичное размещение
        metadata снижает доверие к unwind map и может указывать на ручную разметку, packing
        или protected code.";
    if (strcmp(code, "handler_nonexec_section") == 0 || strcmp(code,
        "handler_writable_section") == 0) return "Handler target с неожиданными правами делает
         exception dispatch небезопасным, неоднозначным или похожим на защитную схему.";
    if (strstr(code, "_combo") != NULL) return "Итоговый verdict считает взаимно
        подтвержденные признаки сильнее, чем отдельные слабые сигналы.";
    return "Признак влияет на доверие к PE/unwind-анализу и должен проверяться вместе с
        соседними таблицами отчета.";
}

static const char* finding_title_ru(const char* code) {
```

Почему это не просто текст

В `HeuristicFinding` нельзя хранить бесконечный текст: используются массивы `char code[40]`, `char family[32]`, `char severity[16]`, `char message[192]`. Это фиксированные буферы. Поэтому запись в них должна быть ограниченной. Код finding-а является стабильным идентификатором. Например, `unwind_parse_fail` связывает эвристику, JSON, HTML, объяснение на русском, evidence chain и тестовый бинарник. Без стабильного кода отчет был бы просто текстом; со стабильным кодом он становится машинно обрабатываемым результатом анализа.

Что должен уметь объяснить студент

Объяснить code/family/severity. Объяснить affected_count. Объяснить, как finding связывается с отчетом.

Глава 29. Report layer: JSON, HTML и опасность строк в C

Отчетный слой показывает, насколько C требователен к строкам. JSON нельзя собирать простым склеиванием без escaping. HTML тоже требует аккуратного escaping, иначе данные файла могут превратиться в исполняемый HTML-контент.

JSON escaping

```
static void json_escape(StrBuf* sb, const char* s) {
    if (!s) return;
    size_t length = strlen(s);
    for (size_t i = 0; i < length; i++) {
        unsigned char c = (unsigned char)s[i];
        switch (c) {
            case '\\': sb_append(sb, "\\\\"); break;
            case '"':  sb_append(sb, "\\\""); break;
            case '\n': sb_append(sb, "\\n"); break;
            case '\r': sb_append(sb, "\\r"); break;
            case '\t': sb_append(sb, "\\t"); break;
            default:
                if (c < 0x20) sb_appendf(sb, "\\u%04x", (unsigned)c);
                else if (c < 0x80) sb_appendn(sb, &s[i], 1);
                else {
                    size_t sequence = valid_utf8_sequence_length(
                        (const unsigned char*)&s[i], length - i);
                    if (sequence) {
                        sb_appendn(sb, &s[i], sequence);
                        i += sequence - 1;
                    } else {
                        sb_appendf(sb, "\\u%04x", (unsigned)c);
                    }
                }
        }
    }
}

static void json_key(StrBuf* sb, const char* k) {
    sb_append(sb, "\"");
    json_escape(sb, k);
    sb_append(sb, "\":");
}

static void json_string(StrBuf* sb, const char* s) {
    sb_append(sb, "\"");
    json_escape(sb, s ? s : "");
    sb_append(sb, "\"");
}

static void json_string_field(StrBuf* sb, const char* k, const char* v, bool trailing_comma) {
    json_key(sb, k);
    json_string(sb, v);
    if (trailing_comma) sb_append(sb, ",");
}

static void json_bool_field(StrBuf* sb, const char* k, bool v, bool trailing_comma) {
    json_key(sb, k);
    sb_append(sb, v ? "true" : "false");
    if (trailing_comma) sb_append(sb, ",");
}


static void json_nullable_hex32(StrBuf* sb, bool has_value, uint32_t value) {
    if (has_value) sb_appendf(sb, "\"0x%08x\"", (unsigned)value);
    else sb_append(sb, "null");
}

static void json_nullable_count(StrBuf* sb, bool has_value, uint32_t value) {
    if (has_value) sb_appendf(sb, "%u", (unsigned)value);
    else sb_append(sb, "null");
}

static void bytes_to_hex(const uint8_t* bytes, size_t count, char* out, size_t out_size) {
    static const char hex[] = "0123456789abcdef";
    if (!out || out_size == 0) return;
    size_t pos = 0;
    for (size_t i = 0; i < count && bytes && pos + 3 < out_size; i++) {
        if (i > 0) out[pos++] = ' ';
        out[pos++] = hex[(bytes[i] >> 4) & 0x0f];
        out[pos++] = hex[bytes[i] & 0x0f];
    }
    out[pos] = '\0';
}

static void json_unwind_diagnostic(StrBuf* sb, const UnwindSummary* u) {
    char bytes_hex[UNWIND_DIAG_MAX_BYTES * 3u + 1u];
    bytes_hex[0] = '\0';
    if (u && u->diagnostic_has_bytes) {
        bytes_to_hex(u->diagnostic_bytes, u->diagnostic_bytes_count, bytes_hex,
            sizeof(bytes_hex));
    }
    sb_append(sb, "{");
    json_string_field(sb, "failed_field", u ? u->diagnostic_field : "", true);
    json_key(sb, "failure_file_offset");
    json_nullable_hex32(sb, u && u->diagnostic_has_file_offset, u ? u->diagnostic_file_offset
        : 0u);
    sb_append(sb, ",");
    json_key(sb, "expected_size");
    json_nullable_count(sb, u && u->diagnostic_has_size, u ? u->diagnostic_expected_size :
        0u);
    sb_append(sb, ",");
    json_key(sb, "available_size");
```

```
    json_nullable_count(sb, u && u->diagnostic_has_size, u ? u->diagnostic_available_size :
        0u);
    sb_append(sb, ",");
```

Evidence chain

```
static void json_finding_evidence_chain(StrBuf* sb,
                                        const HeuristicFinding* finding,
                                        const PeImage* img,
                                        const RuntimeFunction* rf,
                                        const UnwindSummary* us,
                                        const bool* us_ok,
                                        size_t n) {
    size_t related = finding_related_function_index(finding, rf, us, us_ok, n);
    const RuntimeFunction* related_rf = (related != (size_t)-1) ? &rf[related] : NULL;
    const UnwindSummary* related_us = (related != (size_t)-1 && us) ? &us[related] : NULL;
    const PeSection* unwind_section = related_rf ? pe_find_section_for_rva(img,
        related_rf->unwind_rva) : NULL;
    uint32_t unwind_file_offset = 0;
    bool has_unwind_file_offset = related_rf && pe_rva_to_fileoff(img, related_rf->unwind_rva,
         &unwind_file_offset);
    char permissions[4] = "---";
    if (unwind_section) section_permissions(unwind_section->characteristics, permissions);

    sb_append(sb, "{");
    json_key(sb, "exception_directory_rva");
    sb_appendf(sb, "\"0x%08x\",", (unsigned)(img ? img->exception_rva : 0u));
    json_key(sb, "exception_directory_size");
    sb_appendf(sb, "\"0x%x\",", (unsigned)(img ? img->exception_size : 0u));
    json_key(sb, "runtime_function_index");
    if (related != (size_t)-1) sb_appendf(sb, "%zu,", related);
    else sb_append(sb, "null,");
    json_key(sb, "function_begin_rva");
    json_nullable_hex32(sb, related_rf != NULL, related_rf ? related_rf->begin_rva : 0u);
    sb_append(sb, ",");
    json_key(sb, "function_end_rva");
    json_nullable_hex32(sb, related_rf != NULL, related_rf ? related_rf->end_rva : 0u);
    sb_append(sb, ",");
    json_key(sb, "unwind_rva");
    json_nullable_hex32(sb, related_rf != NULL, related_rf ? related_rf->unwind_rva : 0u);
    sb_append(sb, ",");
    json_string_field(sb, "unwind_section", related_rf ? section_name_or_dash(img,
        related_rf->unwind_rva) : "-", true);
    json_key(sb, "unwind_file_offset");
    json_nullable_hex32(sb, has_unwind_file_offset, unwind_file_offset);
    sb_append(sb, ",");
    json_string_field(sb, "unwind_section_permissions", permissions, true);
    json_string_field(sb, "failed_field", related_us ? related_us->diagnostic_field : "",
        false);
    sb_append(sb, "}");
}

static void json_heuristic_families(StrBuf* sb, uint32_t mask) {
    sb_append(sb, "[");
    bool first = true;
    uint32_t bits[] = {
        HEF_TABLE_INTEGRITY,
        HEF_UNWIND_INTEGRITY,
        HEF_METADATA_PLACEMENT,
        HEF_UNWIND_REUSE,
        HEF_HANDLER_DOMINANCE
    };
    for (size_t i = 0; i < sizeof(bits)/sizeof(bits[0]); i++) {
        if (!(mask & bits[i])) continue;
        if (!first) sb_append(sb, ",");
        json_string(sb, heuristics_evidence_family_name(bits[i]));
        first = false;
    }
    sb_append(sb, "]");
}

static unsigned dynamic_signal_count(const DynamicReport* dr) {
    unsigned n = 0;
    if (!dr) return 0;
    if (dr->unwind_handler_hits) n++;
    if (dr->veh_hits || dr->veh_registrations) n++;
    if (dr->vch_hits || dr->vch_registrations) n++;
    if (dr->uef_hits || dr->uef_set_calls) n++;
    if (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls) n++;
    if (dr->second_chance_seen) n++;
    return n;
}

static bool dynamic_session_complete(const DynamicReport* dr) {
    return dr &&
           dr->ran &&
           dr->ok &&
           !dr->timed_out &&
           dr->target_exited &&
           dr->instrumentation_complete;
}

static bool has_static_dynamic_correlation(const HeuristicReport* hr, const DynamicReport* dr)
     {
    bool suspicious_static_handler_context = false;
    if (!hr || !dynamic_session_complete(dr)) return false;
    suspicious_static_handler_context =
        (hr->handler_nonexec_section > 0) ||
        (hr->handler_writable_section > 0) ||
        ((hr->evidence_families_mask & HEF_HANDLER_DOMINANCE) != 0u);
    if (suspicious_static_handler_context && (dr->unwind_handler_hits || dr->veh_hits ||
        dr->vch_hits || dr->uef_hits)) return true;
    if ((hr->unwind_exec_section || hr->unwind_unusual_section || hr->unwind_writable_section
        ||
```

```
         hr->handler_nonexec_section || hr->handler_writable_section) &&
        (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls)) return
            true;
    return false;
}


static const char* runtime_evidence_level(const DynamicReport* dr) {
    if (!dr || !dr->ran) return "none";
    unsigned strong = 0, medium = 0, weak = 0;
    if (dr->unwind_handler_hits || dr->veh_hits || dr->vch_hits || dr->uef_hits) strong++;
    if (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls) medium++;
    if (dr->second_chance_seen) medium++;
    if (dr->veh_registrations || dr->vch_registrations || dr->uef_set_calls ||
        dr->first_chance_seen) weak++;
    const char* level = "none";
```

Представления отчета

Функция `json_escape` проходит строку посимвольно. Для обратного слэша, кавычки, перевода строки, возврата каретки и табуляции добавляются escape-последовательности. Управляющие символы ниже `0x20` записываются как `\u00xx`. HTML-отчет нужен не только для красоты. Он делает результаты анализа обозримыми: metadata, PE summary, таблица функций, статические признаки, динамические evidence, ручная валидация. JSON нужен для интеграции с будущим backend/database/frontend. CSV нужен для Ghidra и табличного анализа. Один анализ - несколько представлений.

Что должен уметь объяснить студент

Объяснить, почему JSON требует escaping. Объяснить, чем CSV для Ghidra отличается от полного CSV. Объяснить, что такое evidence chain.

Глава 30. Dynamic layer: Windows debugger API как отдельный системный мир

Динамическая часть проекта компилируется и работает полноценно только на Windows, потому что использует Windows API: debug events, process memory, breakpoints, Toolhelp module snapshots. На Linux или macOS статический анализ PE-файлов возможен, но динамический Windows-debugger слой не может работать нативно без Windows-среды.

Платформенная граница

```
#ifdef _WIN32

#include <windows.h>
#include <tlhelp32.h>

/* MinGW exposes exception_code as a compatibility macro. */
#ifdef exception_code
#undef exception_code
#endif

/* ------------------------------------------------------------------------ */

typedef struct ModInfo {
    char name[MAX_PATH];
    uint64_t base;
    uint64_t size;
} ModInfo;

typedef struct ModList {
    ModInfo mods[128];
    size_t count;
    bool truncated;
} ModList;

static void mods_init(ModList* ml) { if (ml) memset(ml, 0, sizeof(*ml)); }

static const char* basename_lower(const char* path) {
    const char* base = path;
    for (const char* p = path; *p; p++) {
        if (*p == '\\' || *p == '/') base = p + 1;
    }
    return base;
}

static void strlower_inplace(char* s) {
    for (; s && *s; s++) {
        if (*s >= 'A' && *s <= 'Z') *s = (char)(*s - 'A' + 'a');
    }
}

static void mods_add(ModList* ml, const char* name, uint64_t base, uint64_t size) {
    if (!ml || !name) return;
    if (ml->count >= sizeof(ml->mods)/sizeof(ml->mods[0])) {
        ml->truncated = true;
        return;
    }
    ModInfo* m = &ml->mods[ml->count++];
    strncpy(m->name, name, sizeof(m->name)-1);
    m->name[sizeof(m->name)-1] = 0;
    strlower_inplace(m->name);
    m->base = base;
    m->size = size;
}

static const ModInfo* mods_find(const ModList* ml, const char* name) {
    if (!ml || !name) return NULL;
    char tmp[MAX_PATH];
    strncpy(tmp, name, sizeof(tmp)-1);
    tmp[sizeof(tmp)-1] = 0;
    strlower_inplace(tmp);
    for (size_t i = 0; i < ml->count; i++) {
        if (strcmp(ml->mods[i].name, tmp) == 0) return &ml->mods[i];
    }
    return NULL;
}

static const ModInfo* mods_find_by_addr(const ModList* ml, uint64_t addr) {
    if (!ml) return NULL;
    for (size_t i = 0; i < ml->count; i++) {
        uint64_t b = ml->mods[i].base;
        uint64_t e = ml->mods[i].size > UINT64_MAX - b
            ? UINT64_MAX
            : b + ml->mods[i].size;
        if (addr >= b && addr < e) return &ml->mods[i];
    }
    return NULL;
}

static bool mods_refresh_toolhelp(ModList* ml, DWORD pid) {
    if (!ml) return false;
    ml->count = 0;
    ml->truncated = false;
    HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPMODULE | TH32CS_SNAPMODULE32, pid);
    if (snap == INVALID_HANDLE_VALUE) return false;

    MODULEENTRY32 me;
    memset(&me, 0, sizeof(me));
    me.dwSize = sizeof(me);
    if (!Module32First(snap, &me)) {
```

```
        CloseHandle(snap);
```

Breakpoint logic

```
typedef enum BpKind {
    BP_UNWIND_HANDLER       = 1u << 0,
    BP_API_ADD_VEH          = 1u << 1,
    BP_API_ADD_VCH          = 1u << 2,
    BP_API_SET_UEF          = 1u << 3,
    BP_API_RTL_ADD_FT       = 1u << 4,
    BP_API_RTL_INSTALL_FTCB = 1u << 5,
    BP_CUSTOM_VEH_HANDLER   = 1u << 6,
    BP_CUSTOM_VCH_HANDLER   = 1u << 7,
    BP_CUSTOM_UEF           = 1u << 8
} BpKind;

typedef struct Breakpoint {
    uint64_t addr;
    uint8_t  orig;
    bool     armed;
    bool     rearm_pending;
    uint32_t kinds;
} Breakpoint;

typedef struct BpList {
    Breakpoint bps[512];
    size_t count;
} BpList;

static void bps_init(BpList* bl) { if (bl) memset(bl, 0, sizeof(*bl)); }

static Breakpoint* bps_find(BpList* bl, uint64_t addr) {
    if (!bl) return NULL;
    for (size_t i = 0; i < bl->count; i++) {
        if (bl->bps[i].addr == addr) return &bl->bps[i];
    }
    return NULL;
}

static bool write_one_byte(HANDLE hProcess, uint64_t addr, uint8_t val, StrBuf* err) {
    SIZE_T nw = 0;
    uint8_t b = val;
    if (WriteProcessMemory(hProcess, (LPVOID)(uintptr_t)addr, &b, 1, &nw) && nw == 1) {
        FlushInstructionCache(hProcess, (LPCVOID)(uintptr_t)addr, 1);
        return true;
    }
    DWORD werr = GetLastError();

    /* Retry with temporary RWX. Some targets map code pages RX. */
    DWORD oldProt = 0;
    if (!VirtualProtectEx(hProcess, (LPVOID)(uintptr_t)addr, 1, PAGE_EXECUTE_READWRITE,
        &oldProt)) {
        if (err) sb_appendf(err,
            "WriteProcessMemory failed at 0x%llx (err=%lu) and VirtualProtectEx failed
                (err=%lu)\n",
            (unsigned long long)addr, (unsigned long)werr, (unsigned long)GetLastError());
        return false;
    }

    nw = 0;
    if (!WriteProcessMemory(hProcess, (LPVOID)(uintptr_t)addr, &b, 1, &nw) || nw != 1) {
        DWORD werr2 = GetLastError();
        DWORD tmp = 0;
        VirtualProtectEx(hProcess, (LPVOID)(uintptr_t)addr, 1, oldProt, &tmp);
        if (err) sb_appendf(err,
            "WriteProcessMemory failed at 0x%llx after VirtualProtectEx (err=%lu)\n",
            (unsigned long long)addr, (unsigned long)werr2);
        return false;
    }

    FlushInstructionCache(hProcess, (LPCVOID)(uintptr_t)addr, 1);
    DWORD tmp = 0;
    VirtualProtectEx(hProcess, (LPVOID)(uintptr_t)addr, 1, oldProt, &tmp);
    return true;
}

static bool bp_set(HANDLE hProcess, Breakpoint* bp, StrBuf* err) {
    SIZE_T nread = 0;
    uint8_t b = 0;
    if (!ReadProcessMemory(hProcess, (LPCVOID)(uintptr_t)bp->addr, &b, 1, &nread) || nread !=
        1) {
        if (err) sb_appendf(err, "ReadProcessMemory failed at 0x%llx (err=%lu)\n",
                            (unsigned long long)bp->addr, (unsigned long)GetLastError());
        return false;
    }
    bp->orig = b;
    if (!write_one_byte(hProcess, bp->addr, 0xCC, err)) return false;
    bp->armed = true;
    bp->rearm_pending = false;
    return true;
}

static bool bp_clear(HANDLE hProcess, Breakpoint* bp, StrBuf* err) {
    if (!bp || !bp->armed) return true;
    if (!write_one_byte(hProcess, bp->addr, bp->orig, err)) return false;
    bp->armed = false;
    return true;
}

static bool bp_add_and_arm(BpList* bl, HANDLE hProcess, uint64_t addr, BpKind kind, StrBuf*
    err) {
    if (!bl) return false;
```

Self-check breakpoint

```
static bool bp_add_and_arm_selfcheck(BpList* bl,
                                     HANDLE hProcess,
                                     const ModList* mods,
                                     DynamicReport* dr,
                                     uint64_t addr,
                                     BpKind kind,
                                     const char* what,
                                     StrBuf* err) {
    if (!bl) return false;

    Breakpoint* existing = bps_find(bl, addr);
    bool first_time = (existing == NULL);
    bool capacity_exhausted =
        first_time && bl->count >= sizeof(bl->bps) / sizeof(bl->bps[0]);
    if (dr && first_time) dr->breakpoint_set_attempts++;

    StrBuf te; sb_init(&te, 256);
    bool ok = bp_add_and_arm(bl, hProcess, addr, kind, &te);

    if (dr && !ok) {
        dr->breakpoint_set_failures++;
        if (capacity_exhausted) dr->breakpoint_capacity_drops++;
        dr->instrumentation_complete = false;
    }

    if (dr && first_time) {
        char loc[192];
        format_bp_location(mods, addr, loc, sizeof(loc));

        uint8_t cur = 0xFF;
        StrBuf peek_err; sb_init(&peek_err, 128);
        bool peek_ok = bp_peek_byte(hProcess, addr, &cur, &peek_err);

        if (ok) {
            if (peek_ok) {
                dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : OK (byte=0x%02x)%s\n",
                          bp_kind_name(kind),
                          (what ? what : ""),
                          loc,
                          (unsigned)cur,
                          (cur == 0xCC ? "" : " [WARN: not INT3]"));
            } else {
                dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : OK (peek failed: %s)\n",
                          bp_kind_name(kind), (what ? what : ""), loc,
                          (peek_err.s ? peek_err.s : "unknown"));
            }
        } else {
            dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : FAIL%s%s",
                      bp_kind_name(kind),
                      (what ? what : ""),
                      loc,
                      (te.len ? " : " : "\n"),
                      (te.len ? te.s : "\n"));
        }

        sb_free(&peek_err);
    }

    if (!ok && err && te.len) sb_append(err, te.s);
    sb_free(&te);
    return ok;
}


/* ------------------------------------------------------------------------ */

/* Pending breakpoint re-arming must be tracked per-thread.
   With multiple threads, a single global pending pointer can cause lost rearms. */
typedef struct PendingRearmEntry {
    DWORD tid;
    Breakpoint* bp;
} PendingRearmEntry;

typedef struct PendingRearmList {
    PendingRearmEntry e[64];
} PendingRearmList;

static void pending_rearm_init(PendingRearmList* pr) {
    if (!pr) return;
    memset(pr, 0, sizeof(*pr));
}

static bool pending_rearm_set(PendingRearmList* pr, DWORD tid, Breakpoint* bp) {
    if (!pr || !bp) return false;
    /* overwrite existing */
    for (size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++) {
        if (pr->e[i].bp && pr->e[i].tid == tid) {
            pr->e[i].bp = bp;
            return true;
        }
    }
    /* take a free slot */
    for (size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++) {
        if (!pr->e[i].bp) {
            pr->e[i].tid = tid;
            pr->e[i].bp = bp;
            return true;
        }
    }
    return false;
}
```

```
static Breakpoint* pending_rearm_take(PendingRearmList* pr, DWORD tid) {
    if (!pr) return NULL;
    for (size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++) {
        if (pr->e[i].bp && pr->e[i].tid == tid) {
            Breakpoint* bp = pr->e[i].bp;
            pr->e[i].bp = NULL;
            pr->e[i].tid = 0;
            return bp;
```

Static и dynamic

`#ifdef _WIN32` означает: следующий код включается в сборку только если компилятор собирает под Windows. Для других ОС должна быть альтернативная реализация или stub, который сообщает, что dynamic mode недоступен. Динамика в проекте не заменяет статический анализ. Она собирает runtime evidence: возникали ли исключения, были ли hits handler-ов, регистрировались ли dynamic function tables, удалось ли поставить breakpoints, завершился ли target, был ли timeout. Для изучения C динамический слой важен как пример работы с внешней ОС: handles, process memory, threads, modules, debug loop, breakpoints, error codes.

Что должен уметь объяснить студент

Объяснить `#ifdef _WIN32`. Объяснить breakpoint `0xCC`. Объяснить, почему dynamic нельзя полноценно запускать на macOS/Linux без Windows.

Часть III. Проект по файлам: как читать исходники после изучения C

Теперь можно разобрать файлы проекта как систему. Для каждого файла важно видеть: входные данные, выходные данные, главные структуры, ключевые функции, ошибки, владение памятью и связь с другими модулями.

Файл include/util.h

Назначение файла: контракт базовых типов `Buf` и `StrBuf`, объявление функций чтения и записи. Размер файла в текущем проекте: 72 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
#pragma once
/*
  util.h - small helpers for safe parsing and dynamic arrays
  Project: EH/Unwind Analyzer (PE x64)
*/

#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

#ifdef __cplusplus
extern "C" {
#endif

typedef struct Buf {
    const uint8_t* data;
    size_t size;
} Buf;

bool buf_init_from_file(Buf* out, const char* path, uint8_t** owned_mem);
void buf_free_owned(uint8_t* owned_mem);

bool buf_read(const Buf* b, size_t off, void* out, size_t len);
bool buf_read_u8 (const Buf* b, size_t off, uint8_t*  out);
bool buf_read_u16(const Buf* b, size_t off, uint16_t* out);
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out);
bool buf_read_u64(const Buf* b, size_t off, uint64_t* out);

static inline uint16_t le16(const uint8_t* p) {
    return (uint16_t)(p[0] | ((uint16_t)p[1] << 8));
}
static inline uint32_t le32(const uint8_t* p) {
    return (uint32_t)(p[0] | ((uint32_t)p[1] << 8) | ((uint32_t)p[2] << 16) | ((uint32_t)p[3]
        << 24));
}
static inline uint64_t le64(const uint8_t* p) {
    return (uint64_t)le32(p) | ((uint64_t)le32(p + 4) << 32);
}

typedef struct StrBuf {
    char*  s;
    size_t len;
    size_t cap;
    bool failed;
} StrBuf;

#if defined(__MINGW32__) && (defined(__GNUC__) || defined(__clang__))
#define EHUA_PRINTF_FORMAT(format_index, first_arg) \
    __attribute__((format(gnu_printf, format_index, first_arg)))
#elif defined(__GNUC__) || defined(__clang__)
#define EHUA_PRINTF_FORMAT(format_index, first_arg) \
    __attribute__((format(printf, format_index, first_arg)))
#else
#define EHUA_PRINTF_FORMAT(format_index, first_arg)
#endif

bool sb_init(StrBuf* sb, size_t initial_cap);
void sb_free(StrBuf* sb);
bool sb_appendf(StrBuf* sb, const char* fmt, ...)
    EHUA_PRINTF_FORMAT(2, 3);
bool sb_append(StrBuf* sb, const char* s);
bool sb_appendn(StrBuf* sb, const char* s, size_t n);
bool sb_ok(const StrBuf* sb);
bool file_write_atomic(const char* path, const char* data, size_t len);

/* SHA-256 digest encoded as 64 lowercase hexadecimal characters plus NUL. */
bool sha256_hex(const uint8_t* data, size_t size, char out_hex[65]);

#undef EHUA_PRINTF_FORMAT

#ifdef __cplusplus
}
#endif
```

Как этот файл учит C

Этот файл учит базовой системной дисциплине: байты, размеры, указатели, heap, строки, varargs, переполнение и освобождение ресурсов. Если понять `util`, остальной проект станет значительно проще.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет

памятью, если файл выделяет ресурс?

Файл src/util.c

Назначение файла: реализация чтения файла, безопасного доступа к байтам, динамической строки, SHA-256 и atomic write. Размер файла в текущем проекте: 348 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
bool buf_init_from_file(Buf* out, const char* path, uint8_t** owned_mem) {
    if (!out || !path || !owned_mem) return false;
    *owned_mem = NULL;
    FILE* f = fopen(path, "rb");
    if (!f) return false;

    if (fseek(f, 0, SEEK_END) != 0) { fclose(f); return false; }
    long sz = ftell(f);
    if (sz < 0) { fclose(f); return false; }
    if (fseek(f, 0, SEEK_SET) != 0) { fclose(f); return false; }

    size_t alloc_size = sz > 0 ? (size_t)sz : 1u;
    uint8_t* mem = (uint8_t*)malloc(alloc_size);
    if (!mem) { fclose(f); return false; }

    size_t rd = fread(mem, 1, (size_t)sz, f);
    fclose(f);
    if (rd != (size_t)sz) { free(mem); return false; }

    out->data = mem;
    out->size = (size_t)sz;
    *owned_mem = mem;
    return true;
}

void buf_free_owned(uint8_t* owned_mem) {
    free(owned_mem);
}

bool buf_read(const Buf* b, size_t off, void* out, size_t len) {
    if (!b || !out) return false;
    if (off > b->size) return false;
    if (len > b->size - off) return false;
    memcpy(out, b->data + off, len);
    return true;
}

bool buf_read_u8(const Buf* b, size_t off, uint8_t* out) {
    return buf_read(b, off, out, 1);
}
bool buf_read_u16(const Buf* b, size_t off, uint16_t* out) {
    uint8_t tmp[2];
    if (!buf_read(b, off, tmp, 2)) return false;
    *out = le16(tmp);
    return true;
}
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out) {
    uint8_t tmp[4];
    if (!buf_read(b, off, tmp, 4)) return false;
    *out = le32(tmp);
    return true;
}
bool buf_read_u64(const Buf* b, size_t off, uint64_t* out) {
    uint8_t tmp[8];
    if (!buf_read(b, off, tmp, 8)) return false;
    *out = le64(tmp);
    return true;
}

static bool sb_reserve(StrBuf* sb, size_t need_cap) {
    if (!sb || sb->failed) return false;
    if (need_cap <= sb->cap) return true;
    size_t newcap = sb->cap ? sb->cap : 128;
    while (newcap < need_cap) {
        size_t increment = newcap / 2u + 64u;
        if (increment > SIZE_MAX - newcap) {
            newcap = need_cap;
            break;
        }
        newcap += increment;
    }
    char* p = (char*)realloc(sb->s, newcap);
    if (!p) {
        sb->failed = true;
        return false;
    }
    sb->s = p;
    sb->cap = newcap;
    return true;
}

bool sb_init(StrBuf* sb, size_t initial_cap) {
    if (!sb) return false;
    sb->s = NULL;
    sb->len = 0;
    sb->cap = 0;
    sb->failed = false;
    if (initial_cap == 0) initial_cap = 256;
    if (!sb_reserve(sb, initial_cap) || !sb->s) {
```

```
            sb->failed = true;
            return false;
        }
        sb->s[0] = 0;
        return true;
    }
```

Как этот файл учит C

Этот файл учит базовой системной дисциплине: байты, размеры, указатели, heap, строки, varargs, переполнение и освобождение ресурсов. Если понять `util`, остальной проект станет значительно проще.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет памятью, если файл выделяет ресурс?

Файл include/pe.h

Назначение файла: структуры `PeSection` и `PeImage`, public API PE-парсера. Размер файла в текущем проекте: 67 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
#pragma once
/*
  pe.h - minimal PE32+/x64 parser (no Windows headers)
*/

#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>
#include "util.h"

#ifdef __cplusplus
extern "C" {
#endif

#define PE_DOS_MAGIC 0x5A4D      /* 'MZ' */
#define PE_NT_MAGIC  0x00004550  /* 'PE\0\0' */

#define IMAGE_FILE_MACHINE_AMD64 0x8664
#define IMAGE_NT_OPTIONAL_HDR64_MAGIC 0x20b

#define IMAGE_NUMBEROF_DIRECTORY_ENTRIES 16
#define IMAGE_DIRECTORY_ENTRY_EXCEPTION 3

typedef struct PeSection {
    char     name[9]; /* null-terminated */
    uint32_t virtual_address;
    uint32_t virtual_size;
    uint32_t raw_ptr;
    uint32_t raw_size;
    uint32_t characteristics;
} PeSection;

typedef struct PeImage {
    bool     ok;
    bool     is_pe64;
    uint16_t machine;
    uint64_t image_base;
    uint32_t size_of_image;
    uint32_t size_of_headers;
    uint32_t section_alignment;
    uint32_t file_alignment;
    uint32_t exception_rva;
    uint32_t exception_size;
    size_t   file_size;
    size_t   num_sections;
    PeSection* sections; /* owned */

    size_t section_raw_oob;
    size_t section_virtual_oob;
    size_t section_raw_overlaps;
    size_t section_virtual_overlaps;
    size_t section_misaligned;
    bool   exception_directory_oob;
} PeImage;

bool pe_parse(PeImage* out, const Buf* b, StrBuf* err);
void pe_free(PeImage* img);

/* Map RVA -> file offset. Returns false if not mappable. */
bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off);

/* Find section containing RVA (or NULL). */
const PeSection* pe_find_section_for_rva(const PeImage* img, uint32_t rva);

#ifdef __cplusplus
}
#endif
```

Как этот файл учит C

Этот файл учит читать бинарный формат через проверяемую карту. Здесь особенно важны fixed-width integer types, endian, offset arithmetic, sections и строгое разделение RVA/file offset.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет памятью, если файл выделяет ресурс?

Файл src/pe.c

Назначение файла: разбор MZ/PE/COFF/OptionalHeader/sections, проверка секций, преобразование RVA в file offset. Размер файла в текущем проекте: 301 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
bool pe_parse(PeImage* out, const Buf* b, StrBuf* err) {
    if (!out || !b || !err) return false;
    memset(out, 0, sizeof(*out));

    uint32_t lfanew = 0;
    if (!read_dos_lfanew(b, &lfanew)) {
        sb_append(err, "Not a valid MZ/PE file (DOS header check failed).\n");
        return false;
    }
    if ((size_t)lfanew + 4 + 20 > b->size) {
        sb_append(err, "PE header offset out of range.\n");
        return false;
    }

    uint32_t sig = 0;
    if (!buf_read_u32(b, (size_t)lfanew, &sig) || sig != PE_NT_MAGIC) {
        sb_append(err, "Invalid PE signature.\n");
        return false;
    }

    size_t filehdr = (size_t)lfanew + 4;
    uint16_t machine = 0;
    uint16_t nsects = 0;
    uint16_t opt_size = 0;
    if (!buf_read_u16(b, filehdr + 0, &machine) ||
        !buf_read_u16(b, filehdr + 2, &nsects) ||
        !buf_read_u16(b, filehdr + 16, &opt_size)) {
        sb_append(err, "Failed to read COFF header.\n");
        return false;
    }

    if (machine != IMAGE_FILE_MACHINE_AMD64) {
        sb_appendf(err, "Unsupported Machine: 0x%04x (only x64/AMD64 supported).\n", machine);
        return false;
    }

    size_t opthdr = filehdr + 20;
    if (opthdr + opt_size > b->size) {
        sb_append(err, "Optional header out of range.\n");
        return false;
    }
    if (opt_size < 0x70u) {
        sb_append(err, "PE32+ optional header is too small for required fields.\n");
        return false;
    }

    uint16_t magic = 0;
    if (!buf_read_u16(b, opthdr + 0, &magic)) {
        sb_append(err, "Failed to read optional header magic.\n");
        return false;
    }
    if (magic != IMAGE_NT_OPTIONAL_HDR64_MAGIC) {
        sb_appendf(err, "Unsupported OptionalHeader magic: 0x%04x (expect PE32+ 0x20b).\n",
            magic);
        return false;
    }

    uint64_t image_base = 0;
    uint32_t size_of_image = 0;
    uint32_t size_of_headers = 0;
    uint32_t section_alignment = 0;
    uint32_t file_alignment = 0;
    uint32_t number_of_rva_and_sizes = 0;

    /* PE32+ offsets */
    if (!buf_read_u64(b, opthdr + 0x18, &image_base) ||
        !buf_read_u32(b, opthdr + 0x20, &section_alignment) ||
        !buf_read_u32(b, opthdr + 0x24, &file_alignment) ||
        !buf_read_u32(b, opthdr + 0x38, &size_of_image) ||
        !buf_read_u32(b, opthdr + 0x3c, &size_of_headers) ||
        !buf_read_u32(b, opthdr + 0x6c, &number_of_rva_and_sizes)) {
        sb_append(err, "Failed to read key fields from optional header.\n");
        return false;
    }
    if (size_of_image == 0) {
        sb_append(err, "SizeOfImage is zero.\n");
        return false;
    }
    if (size_of_headers == 0 || (uint64_t)size_of_headers > (uint64_t)b->size) {
        sb_append(err, "SizeOfHeaders is zero or exceeds the file size.\n");
        return false;
    }
    if (section_alignment == 0 || file_alignment == 0) {
        sb_append(err, "SectionAlignment/FileAlignment must be non-zero.\n");
        return false;
    }
    if (number_of_rva_and_sizes < IMAGE_DIRECTORY_ENTRY_EXCEPTION + 1) {
        sb_append(err, "DataDirectory too small.\n");
        return false;
```

```
}

/* DataDirectory starts at 0x70 */
size_t dd_rel = 0x70u + IMAGE_DIRECTORY_ENTRY_EXCEPTION * 8u;
if (dd_rel + 8u > opt_size) {
    sb_append(err, "Exception data directory lies outside the optional header.\n");
    return false;
```

Как этот файл учит C

Этот файл учит читать бинарный формат через проверяемую карту. Здесь особенно важны fixed-width integer types, endian, offset arithmetic, sections и строгое разделение RVA/file offset.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет памятью, если файл выделяет ресурс?

Файл include/unwind.h

Назначение файла: структуры `RuntimeFunction`, `UnwindSummary`, флаги и аномалии x64 unwind. Размер файла в текущем проекте: 136 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
#pragma once
/*
  unwind.h - x64 UNWIND_INFO parsing (PE exception directory)
*/

#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>
#include "util.h"
#include "pe.h"

#ifdef __cplusplus
extern "C" {
#endif

typedef struct RuntimeFunction {
    uint32_t begin_rva;
    uint32_t end_rva;
    uint32_t unwind_rva;
} RuntimeFunction;

#define UNWIND_DIAG_MAX_BYTES 32u
#define UNWIND_DIAG_MAX_CHAIN_PATH 16u

typedef struct UnwindSummary {
    bool     valid;
    uint8_t  version;
    uint8_t  flags;
    uint8_t  prolog_size;
    uint8_t  count_codes;
    uint8_t  frame_reg;
    uint8_t  frame_off; /* scaled by 16 */
    bool     has_handler;
    uint32_t handler_rva;

    bool     has_chain;
    RuntimeFunction chain;

    uint32_t fixed_stack_bytes;     /* estimated fixed stack frame delta: push+alloc+machframe
        */
    uint32_t pushed_nonvol_count;
    uint16_t saved_nonvol_mask;     /* bits 0..15 => reg numbers */
    uint16_t saved_xmm_mask;        /* bits 0..15 => XMM reg numbers */

    uint32_t anomalies;             /* bitmask of internal anomalies */
    char     parse_error[160];       /* non-empty when unwind_parse returns false */

    char     diagnostic_field[64];   /* exact field/area that caused the strongest diagnostic
       */
    bool     diagnostic_has_file_offset;
    uint32_t diagnostic_file_offset;
    bool     diagnostic_has_size;
    uint32_t diagnostic_expected_size;
    uint32_t diagnostic_available_size;
    bool     diagnostic_has_slots;
    uint32_t diagnostic_expected_slots;
    uint32_t diagnostic_available_slots;
    bool     diagnostic_has_bytes;
    uint32_t diagnostic_bytes_file_offset;
    uint8_t  diagnostic_bytes[UNWIND_DIAG_MAX_BYTES];
    uint8_t  diagnostic_bytes_count;

    uint32_t chain_path[UNWIND_DIAG_MAX_CHAIN_PATH];
    uint8_t  chain_path_count;
    bool     chain_loop_detected;
    uint32_t chain_loop_rva;
} UnwindSummary;

enum UnwindFlags {
    UNW_FLAG_EHANDLER  = 0x01,
    UNW_FLAG_UHANDLER  = 0x02,
    UNW_FLAG_CHAININFO = 0x04
};

/* anomalies flags (for heuristic analysis / diagnostics) */
enum UnwindAnom {
    UA_VERSION_UNKNOWN       = 1u << 0,
    UA_TRUNCATED             = 1u << 1,
    UA_BAD_OPCODE            = 1u << 2,
    UA_BAD_OPERAND           = 1u << 3,
    UA_CODEOFFSET_OUTSIDE    = 1u << 4,
    UA_CHAIN_LOOP            = 1u << 5,
    UA_CHAIN_DEPTH_EXCEEDED  = 1u << 6,
    UA_HANDLER_OOB           = 1u << 7,
    UA_CODEOFFSET_ORDER      = 1u << 8,
    UA_OPERAND_SANITY        = 1u << 9,
    UA_ALIGNMENT             = 1u << 10,
```

Как этот файл учит C

Этот файл учит работать с компактным бинарным layout, битовыми полями, флагами, массивами slots, padding, tail-data и chain-ссылками.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет памятью, если файл выделяет ресурс?

Файл src/unwind.c

Назначение файла: чтение `.pdata`, разбор `UNWIND_INFO`, UWOP slots, padding, handler, chain, diagnostics. Размер файла в текущем проекте: 836 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
static bool unwind_parse_fail(UnwindSummary* out,
                              StrBuf* err,
                              size_t err_start,
                              const char* fallback) {
    if (!out) return false;
    out->valid = false;

    const char* message = fallback ? fallback : "UNWIND_INFO parse failed.";
    if (err && err->s && err->len > err_start) {
        message = err->s + err_start;
    } else if (err && fallback) {
        sb_appendf(err, "%s\n", fallback);
    }

    size_t n = strlen(message);
    while (n > 0 && (message[n - 1] == '\n' || message[n - 1] == '\r')) n--;
    if (n >= sizeof(out->parse_error)) n = sizeof(out->parse_error) - 1;
    memcpy(out->parse_error, message, n);
    out->parse_error[n] = '\0';
    return false;
}

bool unwind_parse(const Buf* b,
                  const PeImage* img,
                  uint32_t unwind_rva,
                  const UnwindParseOpts* opts,
                  UnwindSummary* out,
                  StrBuf* ops_text,
                  StrBuf* err) {
    if (!b || !img || !out || !err) return false;
    memset(out, 0, sizeof(*out));
    size_t err_start = err->len;
    if (ops_text) { ops_text->len = 0; if (ops_text->s) ops_text->s[0]=0; }

    int max_depth = 8;
    bool follow_chain = true;
    bool strict_abi = false;
    if (opts) {
        if (opts->max_chain_depth > 0) max_depth = opts->max_chain_depth;
        follow_chain = opts->follow_chain;
        strict_abi = opts->strict_abi;
    }
    if (max_depth > 16) max_depth = 16;

    uint8_t hdr[4];
    uint32_t uoff = 0;
    uint32_t attempted_uoff = 0;
    bool attempted_mapped = pe_rva_to_fileoff(img, unwind_rva, &attempted_uoff);
    if ((unwind_rva & 3u) != 0) {
        out->anomalies |= UA_ALIGNMENT;
    }
    if (!parse_unwindinfo_header(b, img, unwind_rva, &uoff, hdr, err)) {
        out->anomalies |= UA_TRUNCATED;
        unwind_diag_set(out, b, "UnwindInfoAddress / UNWIND_INFO header",
                        attempted_mapped, attempted_mapped ? (size_t)attempted_uoff : 0u,
                        attempted_mapped, 4u,
                        attempted_mapped ? available_until_limit(b, (size_t)attempted_uoff,
                b->size) : 0u,
                        false, 0, 0);
        return unwind_parse_fail(out, err, err_start, "Failed to read UNWIND_INFO header.");
    }
    size_t record_limit = unwind_record_limit(b, img, unwind_rva);

    uint8_t ver = (uint8_t)(hdr[0] & 0x7);
    uint8_t flags = (uint8_t)(hdr[0] >> 3);
    uint8_t prolog = hdr[1];
    uint8_t count = hdr[2];
    uint8_t fr = (uint8_t)(hdr[3] & 0x0f);
    uint8_t foff = (uint8_t)(hdr[3] >> 4);

    out->version = ver;
    out->flags = flags;
    out->prolog_size = prolog;
    out->count_codes = count;
    out->frame_reg = fr;
    out->frame_off = foff;

    if (!(ver == 1 || ver == 2)) {
        out->anomalies |= UA_VERSION_UNKNOWN;
        unwind_diag_set(out, b, "Version", true, (size_t)uoff, true, 1u, 1u, false, 0, 0);
        return unwind_parse_fail(out, err, err_start, "Unsupported UNWIND_INFO version.");
    }
    if (flags & (uint8_t)~(UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER | UNW_FLAG_CHAININFO)) {
        out->anomalies |= UA_BAD_OPERAND;
    }
    if ((flags & UNW_FLAG_CHAININFO) && (flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER))) {
        out->anomalies |= UA_BAD_OPERAND;
    }
    if (fr == 0 && foff != 0) {
```

```
        out->anomalies |= UA_BAD_OPERAND;
    }
    if (fr != 0 && !is_unwindable_gpr(fr)) {
        out->anomalies |= UA_BAD_OPERAND;
    }
    if (strict_abi && fr != 0 && !is_abi_nonvolatile_gpr(fr)) {
        out->anomalies |= UA_ABI_REGISTER_CLASS;
```

Как этот файл учит C

Этот файл учит работать с компактным бинарным layout, битовыми полями, флагами, массивами slots, padding, tail-data и chain-ссылками.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет памятью, если файл выделяет ресурс?

Файл include/heuristics.h

Назначение файла: модель findings, mitigations и итогового heuristic report. Размер файла в текущем проекте: 109 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
#pragma once
/*
  heuristics.h - heuristic assessment of broken / shadowed / low-informative unwind metadata
*/

#include <stddef.h>
#include <stdbool.h>
#include <stdint.h>
#include "pe.h"
#include "unwind.h"
#include "util.h"

#ifdef __cplusplus
extern "C" {
#endif

enum HeuristicEvidenceFamily {
    HEF_TABLE_INTEGRITY    = 1u << 0,
    HEF_UNWIND_INTEGRITY   = 1u << 1,
    HEF_METADATA_PLACEMENT = 1u << 2,
    HEF_UNWIND_REUSE       = 1u << 3,
    HEF_HANDLER_DOMINANCE  = 1u << 4
};

enum { HEURISTICS_MAX_FINDINGS = 32, HEURISTICS_MAX_MITIGATIONS = 16 };

typedef struct HeuristicFinding {
    char code[40];
    char family[32];
    char severity[16];
    char message[192];
    bool has_location;
    uint32_t representative_rva;
    size_t affected_count;
} HeuristicFinding;

typedef struct HeuristicMitigation {
    char code[40];
    char message[192];
} HeuristicMitigation;

typedef struct HeuristicReport {
    int    score; /* 0..100 (higher => more suspicious) */
    double suspicion;  /* 0..1 derived from score */
    double confidence; /* 0..1 confidence in the static verdict */
    bool   requires_manual_review;
    uint32_t evidence_families_mask;
    char   verdict[48];

    size_t total;

    size_t pe_section_raw_oob;
    size_t pe_section_virtual_oob;
    size_t pe_section_raw_overlaps;
    size_t pe_section_virtual_overlaps;
    size_t pe_section_misaligned;
    bool   pe_exception_directory_oob;

    size_t rf_invalid_ranges;
    size_t rf_oob;
    size_t rf_unwind_oob;
    size_t rf_unsorted;
    size_t rf_overlaps;
    size_t rf_duplicate_begins;

    size_t unwind_parse_fail;
    size_t unwind_anomalies;
    size_t unwind_abi_warnings;

    size_t unwind_unusual_section;
    size_t unwind_exec_section;
    size_t unwind_writable_section;
    size_t handler_nonexec_section;
    size_t handler_writable_section;

    double unwind_reuse_ratio;    /* max_count / total */
    double unwind_unique_ratio;   /* unique / total */

    double handler_reuse_ratio;   /* max_count / handler_count (if any) */
    size_t handler_count;

    size_t finding_count;
    HeuristicFinding findings[HEURISTICS_MAX_FINDINGS];
    size_t weak_finding_count;
    size_t medium_finding_count;
```

Как этот файл учит C

Этот файл учит агрегировать результаты в аналитическую модель: счетчики, ratios, thresholds, findings, confidence и mitigation. Это уже не чтение байтов, а интерпретация evidence.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет памятью, если файл выделяет ресурс?

Файл src/heuristics.c

Назначение файла: подсчёт признаков, score/confidence/verdict, evidence families. Размер файла в текущем проекте: 1044 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
bool heuristics_analyze(const PeImage* img,
                        const RuntimeFunction* rf,
                        const UnwindSummary* us,
                        const bool* us_ok,
                        size_t n,
                        HeuristicReport* out,
                        StrBuf* err) {
    if (!img || !out || !err) return false;
    if (n > 0 && (!rf || !us || !us_ok)) return false;
    memset(out, 0, sizeof(*out));
    if (!sb_init(&out->details, 1024)) {
        sb_append(err, "Out of memory (details buffer).\n");
        return false;
    }
    out->total = n;
    out->pe_section_raw_oob = img->section_raw_oob;
    out->pe_section_virtual_oob = img->section_virtual_oob;
    out->pe_section_raw_overlaps = img->section_raw_overlaps;
    out->pe_section_virtual_overlaps = img->section_virtual_overlaps;
    out->pe_section_misaligned = img->section_misaligned;
    out->pe_exception_directory_oob = img->exception_directory_oob;

    /* RUNTIME_FUNCTION sanity */
    uint32_t prev_begin = 0;
    bool prev_valid = false;
    size_t unsorted = 0;
    FindingLocation invalid_range_loc = {0};
    FindingLocation rf_oob_loc = {0};
    FindingLocation unwind_oob_loc = {0};
    FindingLocation unsorted_loc = {0};
    FindingLocation overlap_loc = {0};
    FindingLocation duplicate_loc = {0};
    FindingLocation parse_fail_loc = {0};
    FindingLocation anomaly_loc = {0};
    FindingLocation unusual_unwind_loc = {0};
    FindingLocation executable_unwind_loc = {0};
    FindingLocation writable_unwind_loc = {0};
    FindingLocation nonexec_handler_loc = {0};
    FindingLocation writable_handler_loc = {0};

    HMap unwind_counts = {0};
    HMap handler_counts = {0};
    if (n > (SIZE_MAX - 1u) / 2u) {
        sb_append(err, "Runtime function count is too large for reuse analysis.\n");
        sb_free(&out->details);
        return false;
    }
    if (!hmap_init(&unwind_counts, n * 2 + 1u) ||
        !hmap_init(&handler_counts, 128)) {
        sb_append(err, "Out of memory (hash maps).\n");
        hmap_free(&unwind_counts);
        hmap_free(&handler_counts);
        sb_free(&out->details);
        return false;
    }

    for (size_t i = 0; i < n; i++) {
        uint32_t b = rf[i].begin_rva;
        uint32_t e = rf[i].end_rva;
        uint32_t u = rf[i].unwind_rva;

        if (e <= b) {
            out->rf_invalid_ranges++;
            remember_location(&invalid_range_loc, b);
        }
        if (b >= img->size_of_image || e > img->size_of_image) {
            out->rf_oob++;
            remember_location(&rf_oob_loc, b);
        }
        if (u >= img->size_of_image || u == 0) {
            out->rf_unwind_oob++;
            remember_location(&unwind_oob_loc, u);
        }

        if (prev_valid && b < prev_begin) {
            unsorted++;
            remember_location(&unsorted_loc, b);
        }
        prev_begin = b;
        prev_valid = true;

        if (!hmap_inc(&unwind_counts, u, NULL)) {
            sb_append(err, "Out of memory while counting unwind metadata reuse.\n");
            hmap_free(&unwind_counts);
            hmap_free(&handler_counts);
            sb_free(&out->details);
            return false;
        }
```

```
if (us_ok[i] && us[i].has_handler) {
    out->handler_count++;
    if (!hmap_inc(&handler_counts, us[i].handler_rva, NULL)) {
        sb_append(err, "Out of memory while counting handler reuse.\n");
        hmap_free(&unwind_counts);
        hmap_free(&handler_counts);
```

Как этот файл учит C

Этот файл учит агрегировать результаты в аналитическую модель: счетчики, ratios, thresholds, findings, confidence и mitigation. Это уже не чтение байтов, а интерпретация evidence.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет памятью, если файл выделяет ресурс?

Файл include/report.h

Назначение файла: API записи CSV/JSON/HTML и metadata отчета. Размер файла в текущем проекте: 72 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
#pragma once
/*
  report.h - output helpers (stdout, CSV, JSON)
*/

#include <stddef.h>
#include <stdbool.h>
#include "pe.h"
#include "unwind.h"
#include "heuristics.h"
#include "dynamic.h"
#include "util.h"

#ifdef __cplusplus
extern "C" {
#endif

typedef struct ReportMetadata {
    const char* tool_version;
    const char* report_schema_version;
    const char* generated_at_utc;
    const char* input_sha256;
    uint64_t input_size;
    const char* command_line;
    const char* build_compiler;
    const char* build_platform;
    const char* build_mode;
    bool strict_abi;
    bool follow_chain;
    uint32_t max_chain_depth;
} ReportMetadata;

void report_print_brief(const char* path, const PeImage* img, size_t rf_count);

bool report_write_csv(const char* path,
                      const PeImage* img,
                      const RuntimeFunction* rf,
                      const UnwindSummary* us,
                      const bool* us_ok,
                      size_t n,
                      bool ghidra_mode,
                      StrBuf* err);

bool report_write_json_ex(const char* path,
                          const char* input_path,
                          const PeImage* img,
                          const RuntimeFunction* rf,
                          const UnwindSummary* us,
                          const bool* us_ok,
                          size_t n,
                          const HeuristicReport* hr,
                          const DynamicReport* dr,
                          const ReportMetadata* metadata,
                          StrBuf* err);

bool report_write_html(const char* path,
                       const char* input_path,
                       const PeImage* img,
                       const RuntimeFunction* rf,
                       const UnwindSummary* us,
                       const bool* us_ok,
                       size_t n,
                       const HeuristicReport* hr,
                       const DynamicReport* dr,
                       const ReportMetadata* metadata,
                       StrBuf* err);

void report_print_heuristics(const HeuristicReport* hr);

#ifdef __cplusplus
}
#endif
```

Как этот файл учит C

Этот файл учит превращать структурированные данные в человеко- и машинно-читаемые отчеты. Здесь важны строки, escaping, форматирование, metadata и стабильность схемы.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет памятью, если файл выделяет ресурс?

Файл src/report.c

Назначение файла: человеческое представление результата: escaping, evidence chains, HTML, JSON, Ghidra CSV. Размер файла в текущем проекте: 2792 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
bool report_write_json_ex(const char* path,
                      const char* input_path,
                      const PeImage* img,
                      const RuntimeFunction* rf,
                      const UnwindSummary* us,
                      const bool* us_ok,
                      size_t n,
                      const HeuristicReport* hr,
                      const DynamicReport* dr,
                      const ReportMetadata* metadata,
                      StrBuf* err) {
    if (!path || !input_path || (!rf && n) || (!us && n) || (!us_ok && n) ||
        !report_model_valid(img, hr, dr) || !report_metadata_valid(metadata)) {
        sb_append(err, "Invalid arguments or metadata for JSON report.\n");
        return false;
    }

    StrBuf sb;
    if (!sb_init(&sb, 8192)) { sb_append(err, "OOM.\n"); return false; }
    StrBuf comment;
    if (!sb_init(&comment, 512)) { sb_free(&sb); sb_append(err, "OOM.\n"); return false; }
    StrBuf anom;
    if (!sb_init(&anom, 128)) { sb_free(&comment); sb_free(&sb); sb_append(err, "OOM.\n");
        return false; }

    sb_append(&sb, "{");
    json_string_field(&sb, "input", input_path, true);
    json_key(&sb, "report_metadata");
    sb_append(&sb, "{");
    json_string_field(&sb, "schema_version", metadata->report_schema_version, true);
    json_string_field(&sb, "tool_version", metadata->tool_version, true);
    json_string_field(&sb, "generated_at_utc", metadata->generated_at_utc, true);
    json_string_field(&sb, "input_sha256", metadata->input_sha256, true);
    json_key(&sb, "input_size"); sb_appendf(&sb, "%llu,", (unsigned long
        long)metadata->input_size);
    json_string_field(&sb, "command_line", metadata->command_line, true);
    json_string_field(&sb, "build_compiler", metadata->build_compiler, true);
    json_string_field(&sb, "build_platform", metadata->build_platform, true);
    json_string_field(&sb, "build_mode", metadata->build_mode, true);
    json_bool_field(&sb, "strict_abi", metadata->strict_abi, true);
    json_bool_field(&sb, "follow_chain", metadata->follow_chain, true);
    json_key(&sb, "max_chain_depth");
    sb_appendf(&sb, "%u", (unsigned)metadata->max_chain_depth);
    sb_append(&sb, "},");
    json_key(&sb, "image_base"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long
        long)img->image_base);
    json_key(&sb, "size_of_image"); sb_appendf(&sb, "\"0x%x\",",
        (unsigned)img->size_of_image);
    json_key(&sb, "size_of_headers"); sb_appendf(&sb, "\"0x%x\",",
        (unsigned)img->size_of_headers);
    json_key(&sb, "section_alignment"); sb_appendf(&sb, "\"0x%x\",",
        (unsigned)img->section_alignment);
    json_key(&sb, "file_alignment"); sb_appendf(&sb, "\"0x%x\",",
        (unsigned)img->file_alignment);
    json_key(&sb, "exception_directory");
    sb_appendf(&sb, "{\"rva\":\"0x%08x\",\"size\":\"0x%x\"}",
               (unsigned)img->exception_rva, (unsigned)img->exception_size);
    sb_append(&sb, ",");
    json_key(&sb, "pe_validation");
    sb_append(&sb, "{");
    json_key(&sb, "section_raw_oob"); sb_appendf(&sb, "%zu,", img->section_raw_oob);
    json_key(&sb, "section_virtual_oob"); sb_appendf(&sb, "%zu,", img->section_virtual_oob);
    json_key(&sb, "section_raw_overlaps"); sb_appendf(&sb, "%zu,", img->section_raw_overlaps);
    json_key(&sb, "section_virtual_overlaps"); sb_appendf(&sb, "%zu,",
        img->section_virtual_overlaps);
    json_key(&sb, "section_misaligned"); sb_appendf(&sb, "%zu,", img->section_misaligned);
    json_bool_field(&sb, "exception_directory_oob", img->exception_directory_oob, false);
    sb_append(&sb, "},");
    json_key(&sb, "sections");
    sb_append(&sb, "[");
    for (size_t i = 0; i < img->num_sections; i++) {
        const PeSection* section = &img->sections[i];
        if (i) sb_append(&sb, ",");
        sb_append(&sb, "{");
        json_string_field(&sb, "name", section->name, true);
        json_key(&sb, "virtual_address"); sb_appendf(&sb, "\"0x%08x\",",
            (unsigned)section->virtual_address);
        json_key(&sb, "virtual_size"); sb_appendf(&sb, "\"0x%x\",",
            (unsigned)section->virtual_size);
        json_key(&sb, "raw_offset"); sb_appendf(&sb, "\"0x%x\",", (unsigned)section->raw_ptr);
        json_key(&sb, "raw_size"); sb_appendf(&sb, "\"0x%x\",", (unsigned)section->raw_size);
        json_key(&sb, "characteristics"); sb_appendf(&sb, "\"0x%08x\",",
            (unsigned)section->characteristics);
        json_bool_field(&sb, "readable", (section->characteristics & IMAGE_SCN_MEM_READ) != 0,
             true);
        json_bool_field(&sb, "writable", (section->characteristics & IMAGE_SCN_MEM_WRITE) !=
            0, true);
        json_bool_field(&sb, "executable", (section->characteristics & IMAGE_SCN_MEM_EXECUTE)
            != 0, false);
```

```
        sb_append(&sb, "}");
    }
    sb_append(&sb, "],");

    if (hr) {
        json_key(&sb, "heuristic_analysis");
        sb_append(&sb, "{");
        json_key(&sb, "score"); sb_appendf(&sb, "%d,", hr->score);
        json_key(&sb, "suspicion"); sb_appendf(&sb, "%.6f,", hr->suspicion);
        json_key(&sb, "confidence"); sb_appendf(&sb, "%.6f,", hr->confidence);
        json_bool_field(&sb, "requires_manual_review", hr->requires_manual_review, true);
        json_string_field(&sb, "verdict", hr->verdict, true);
        json_string_field(&sb, "evidence_level", hr->evidence_level, true);
        json_key(&sb, "weak_finding_count"); sb_appendf(&sb, "%zu,", hr->weak_finding_count);
        json_key(&sb, "medium_finding_count"); sb_appendf(&sb, "%zu,",
            hr->medium_finding_count);
        json_key(&sb, "strong_finding_count"); sb_appendf(&sb, "%zu,",
            hr->strong_finding_count);
        json_key(&sb, "total"); sb_appendf(&sb, "%zu,", hr->total);
        json_key(&sb, "pe_section_raw_oob"); sb_appendf(&sb, "%zu,", hr->pe_section_raw_oob);
        json_key(&sb, "pe_section_virtual_oob"); sb_appendf(&sb, "%zu,",
            hr->pe_section_virtual_oob);
        json_key(&sb, "pe_section_raw_overlaps"); sb_appendf(&sb, "%zu,",
            hr->pe_section_raw_overlaps);
```

Как этот файл учит C

Этот файл учит превращать структурированные данные в человеко- и машинно-читаемые отчеты. Здесь важны строки, escaping, форматирование, metadata и стабильность схемы.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет памятью, если файл выделяет ресурс?

Файл include/dynamic.h

Назначение файла: структура динамического отчета и public API dynamic collection. Размер файла в текущем проекте: 122 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
#pragma once
/*
  dynamic.h - Windows-only dynamic evidence collector (Debug API)

  Defensive intent:
    Collect runtime evidence of exception-handling redirection (e.g., to an
    interceptor or VM dispatcher) by observing:
      - hits of handlers referenced from UNWIND_INFO (x64)
      - registration/hits of vectored/unhandled exception handlers
      - registration of dynamic function tables
      - first-chance vs second-chance exception flow

  This module does NOT attempt to "break" protections or recover hidden data.
*/

#include <stdint.h>
#include <stdbool.h>
#include "util.h"
#include "pe.h"
#include "unwind.h"

#ifdef __cplusplus
extern "C" {
#endif

enum { DYNAMIC_MAX_EVENTS = 256 };

typedef struct DynamicEvent {
    char type[40];
    char module[64];
    uint32_t thread_id;
    uint32_t exception_code;
    bool first_chance;
    uint64_t address;
    uint64_t arg0;
    uint64_t arg1;
    uint64_t arg2;
    uint64_t arg3;
} DynamicEvent;

typedef struct DynamicReport {
    bool enabled;
    bool ran;
    bool ok;
    bool timed_out;
    bool target_exited;
    uint32_t target_exit_code;

    uint32_t timeout_ms;
    uint32_t trace_steps;

    bool instrumentation_complete;
    uint32_t modules_observed_peak;
    uint32_t module_refresh_failures;
    uint32_t module_list_truncations;
    uint32_t unwind_handlers_discovered;
    uint32_t breakpoint_set_attempts;
    uint32_t breakpoint_set_failures;
    uint32_t breakpoint_capacity_drops;
    uint32_t pending_rearm_failures;
    uint32_t breakpoint_rearm_failures;

    uint32_t exceptions_seen;
    uint32_t first_chance_seen;
    uint32_t second_chance_seen;

    uint32_t unwind_handler_bp_set;
    uint32_t unwind_handler_hits;

    uint32_t veh_registrations;
    uint32_t veh_hits;
    uint32_t vch_registrations;
    uint32_t vch_hits;
    uint32_t uef_set_calls;
    uint32_t uef_hits;

    uint32_t rtl_add_function_table_calls;
    uint32_t rtl_install_function_table_cb_calls;

    uint32_t event_count;
    uint32_t dropped_events;
    DynamicEvent events[DYNAMIC_MAX_EVENTS];

    /* Execution trace sampled after first handler hit (RIP addresses). */
    uint32_t trace_count;
```

Как этот файл учит C

Этот файл учит C на границе с операционной системой: handles, debug events, process memory, breakpoints, platform-specific compilation. Это самый платформенно-зависимый слой проекта.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет памятью, если файл выделяет ресурс?

Файл src/dynamic_win.c

Назначение файла: Windows-only debugger validation, breakpoints, runtime events, dynamic function table evidence. Размер файла в текущем проекте: 1253 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
bool dynamic_collect_windows(const char* pe_path,
                             const char* const* child_argv,
                             int child_argc,
                             const PeImage* img,
                             const RuntimeFunction* rf,
                             const UnwindSummary* us,
                             const bool* us_ok,
                             size_t rf_count,
                             uint32_t timeout_ms,
                             uint32_t trace_steps,
                             DynamicReport* out,
                             StrBuf* err) {
    if (!out || !err || !pe_path || !img) return false;
    if (rf_count > 0 && (!rf || !us || !us_ok)) return false;

    dynamic_report_init(out);
    if (!sb_ok(&out->details)) {
        sb_append(err, "Out of memory (dynamic report buffer).\n");
        return false;
    }
    out->enabled = true;
    out->timeout_ms = timeout_ms;
    out->trace_steps = trace_steps > 256u ? 256u : trace_steps;

    StrBuf cmd;
    if (!sb_init(&cmd, 1024)) {
        sb_append(err, "Out of memory (command line buffer).\n");
        return false;
    }
    if (!build_cmdline(pe_path, child_argv, child_argc, &cmd)) {
        sb_append(err, "Failed to build command line.\n");
        sb_free(&cmd);
        return false;
    }

    STARTUPINFOA si;
    PROCESS_INFORMATION pi;
    memset(&si, 0, sizeof(si));
    memset(&pi, 0, sizeof(pi));
    si.cb = sizeof(si);

    /*
     * Do not use CREATE_SUSPENDED.
     *
     * We want the standard Windows Debug API event stream to start
     * immediately after CreateProcess. On some configurations,
     * creating the debuggee suspended can result in no debug events
     * being delivered (no CREATE_PROCESS_DEBUG_EVENT), which causes
     * a deadlock (the tool waits forever, never resumes the thread).
     */
    DWORD flags = DEBUG_ONLY_THIS_PROCESS;

    if (!CreateProcessA(NULL, cmd.s, NULL, NULL, FALSE, flags, NULL, NULL, &si, &pi)) {
        sb_appendf(err, "CreateProcess failed (err=%lu)\n", GetLastError());
        sb_free(&cmd);
        return false;
    }

    out->ran = true;

    HANDLE hProcess = pi.hProcess;
    DWORD pid = pi.dwProcessId;

    ModList mods;
    mods_init(&mods);

    BpList bps;
    bps_init(&bps);

    /* Collect unique handler RVAs from static parse. */
    uint32_t* handler_rvas = (uint32_t*)calloc(rf_count ? rf_count : 1, sizeof(uint32_t));
    if (!handler_rvas) {
        sb_append(err, "Out of memory (handler RVA list).\n");
        TerminateProcess(hProcess, 0);
        if (pi.hThread) CloseHandle(pi.hThread);
        if (pi.hProcess) CloseHandle(pi.hProcess);
        sb_free(&cmd);
        return false;
    }
    size_t handler_count = 0;
    for (size_t i = 0; i < rf_count; i++) {
        if (!us_ok[i] || !us[i].has_handler) continue;
        uint32_t hr = us[i].handler_rva;
        bool seen = false;
        for (size_t j = 0; j < handler_count; j++) {
            if (handler_rvas[j] == hr) { seen = true; break; }
        }
        if (!seen) handler_rvas[handler_count++] = hr;
    }
```

```
    out->unwind_handlers_discovered = (uint32_t)handler_count;

    DEBUG_EVENT ev;
    bool got_create = false;
    bool debug_loop_ok = true;
    bool process_exited = false;
```

Как этот файл учит C

Этот файл учит C на границе с операционной системой: handles, debug events, process memory, breakpoints, platform-specific compilation. Это самый платформенно-зависимый слой проекта.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет памятью, если файл выделяет ресурс?

Файл src/main.c

Назначение файла: CLI orchestration: аргументы, запуск модулей, освобождение ресурсов, коды выхода. Размер файла в текущем проекте: 527 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
int main(int argc, char** argv) {
    Args a;
    int exit_code = 0;
    ParseResult pres = parse_args(argc, argv, &a);
    if (pres != PARSE_OK) {
        if (pres == PARSE_VERSION) {
            print_version();
            return 0;
        }
        usage(argv[0]);
        return (pres == PARSE_HELP) ? 0 : 2;
    }

    uint8_t* owned = NULL;
    Buf b;
    if (!buf_init_from_file(&b, a.input, &owned)) {
        fprintf(stderr, "Failed to read file: %s\n", a.input);
        return 1;
    }

    StrBuf err;
    sb_init(&err, 1024);

    PeImage img;
    if (!pe_parse(&img, &b, &err)) {
        fprintf(stderr, "PE parse error:\n%s", err.s ? err.s : "");
        sb_free(&err);
        buf_free_owned(owned);
        return 1;
    }

    RuntimeFunction* rf = NULL;
    size_t rf_count = 0;
    StrBuf err2;
    sb_init(&err2, 1024);

    RuntimeFunctionReadResult rf_read = unwind_read_runtime_functions_ex(
        &b, &img, &rf, &rf_count, &err2);
    if (rf_read != RUNTIME_FUNCTION_READ_OK) {
        fprintf(stderr, "Failed to read exception directory:\n%s", err2.s ? err2.s : "");
        if (rf_read == RUNTIME_FUNCTION_READ_OOM) {
            pe_free(&img);
            sb_free(&err);
            sb_free(&err2);
            buf_free_owned(owned);
            return 1;
        }
        /*
         * Keep the established report field/finding code for compatibility.
         * It now covers every structurally unreadable exception directory.
         */
        img.exception_directory_oob = true;
        fprintf(stderr, "Continuing with a partial structural report.\n");
        exit_code = 1;
    }

    report_print_brief(a.input, &img, rf_count);

    UnwindSummary* us = (UnwindSummary*)calloc(rf_count ? rf_count : 1,
        sizeof(UnwindSummary));
    bool* us_ok = (bool*)calloc(rf_count ? rf_count : 1, sizeof(bool));
    if (!us || !us_ok) {
        fprintf(stderr, "Out of memory.\n");
        free(us);
        free(us_ok);
        free(rf);
        pe_free(&img);
        sb_free(&err);
        sb_free(&err2);
        buf_free_owned(owned);
        return 1;
    }

    UnwindParseOpts uopts = {
        .max_chain_depth = a.max_chain_depth,
        .follow_chain = a.follow_chain,
        .strict_abi = a.strict_abi
    };

    StrBuf ops;
    sb_init(&ops, 1024);

    for (size_t i = 0; i < rf_count; i++) {
        StrBuf perr;
        sb_init(&perr, 256);
        if (a.show_ops) { ops.len = 0; if (ops.s) ops.s[0]=0; }
        bool ok = unwind_parse(&b, &img, rf[i].unwind_rva, &uopts, &us[i], a.show_ops ? &ops :
             NULL, &perr);
        us_ok[i] = ok;
```

```
    if (a.dump && a.show_ops && ok) {
        /* ops will be printed in dump loop below */
    }
    sb_free(&perr);
}

if (a.dump) {
    size_t limit = (a.dump_limit < 0) ? rf_count : (size_t)a.dump_limit;
```

Как этот файл учит C

Этот файл учит orchestration: как CLI собирает сценарий из модулей, как обрабатывает ошибки и как в конце освобождает всё, что было выделено.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет памятью, если файл выделяет ресурс?

Файл CMakeLists.txt

Назначение файла: описание сборки, зависимость от платформы и компилятора. Размер файла в текущем проекте: 296 строк. Читать его нужно как часть цепочки: входные байты -> структурная карта -> unwind summary -> heuristic report -> пользовательский отчет. Первый вопрос при чтении файла: какие данные входят? Второй вопрос: какие данные выходят? Третий вопрос: где находятся проверки ошибок? Четвертый вопрос: кто владеет памятью? Пятый вопрос: какие функции являются внешним API, а какие являются локальными helper-функциями?

Ключевой фрагмент

```
cmake_minimum_required(VERSION 3.23)

project(
    eh_unwind_analyzer
    VERSION 1.0.0
    DESCRIPTION "Static and dynamic analyzer for Windows x64 EH/Unwind metadata"
    LANGUAGES C
)

include(CTest)

option(EHUA_WARNINGS_AS_ERRORS "Treat compiler warnings as errors" ON)
option(EHUA_ENABLE_SANITIZERS "Enable AddressSanitizer and UndefinedBehaviorSanitizer" OFF)
option(EHUA_ENABLE_COVERAGE "Enable GCC/Clang coverage instrumentation" OFF)
option(EHUA_BUILD_FUZZER "Build the Clang libFuzzer parser harness" OFF)

if(EHUA_ENABLE_SANITIZERS AND EHUA_ENABLE_COVERAGE)
    message(FATAL_ERROR "Sanitizer and coverage instrumentation cannot be enabled together")
endif()

if(WIN32 AND NOT CMAKE_SIZEOF_VOID_P EQUAL 8)
    message(FATAL_ERROR "EH/Unwind Analyzer supports Windows x64 builds only")
endif()

function(ehua_apply_warnings target)
    if(MSVC)
        target_compile_options(${target} PRIVATE /W4 /permissive- /utf-8)
        target_compile_definitions(${target} PRIVATE _CRT_SECURE_NO_WARNINGS=1)
        if(EHUA_WARNINGS_AS_ERRORS)
            target_compile_options(${target} PRIVATE /WX)
        endif()
    else()
        target_compile_options(
            ${target}
            PRIVATE
                -Wall
                -Wextra
                -Wpedantic
                -Wconversion
                -Wshadow
                -Wformat=2
        )
        if(CMAKE_C_COMPILER_ID STREQUAL "GNU")
            target_compile_options(${target} PRIVATE -finput-charset=UTF-8 -fexec-
                charset=UTF-8)
        endif()
        if(EHUA_WARNINGS_AS_ERRORS)
            target_compile_options(${target} PRIVATE -Werror)
        endif()
    endif()
endfunction()

function(ehua_apply_instrumentation target)
    if(EHUA_ENABLE_SANITIZERS)
        if(MSVC)
            message(FATAL_ERROR "EHUA_ENABLE_SANITIZERS is currently supported with GCC or
                Clang")
        endif()
        target_compile_options(${target} PRIVATE -fsanitize=address,undefined -fno-omit-frame-
            pointer)
        target_link_options(${target} PRIVATE -fsanitize=address,undefined)
    elseif(EHUA_ENABLE_COVERAGE)
        if(MSVC)
            message(FATAL_ERROR "EHUA_ENABLE_COVERAGE is currently supported with GCC or
                Clang")
        endif()
        target_compile_options(${target} PRIVATE --coverage -O0 -g)
        target_link_options(${target} PRIVATE --coverage)
    endif()
endfunction()

set(EHUA_CORE_SOURCES
    src/app_info.c
    src/dynamic_win.c
    src/heuristics.c
    src/pe.c
    src/report.c
    src/unwind.c
    src/util.c
)

add_library(ehua_core STATIC ${EHUA_CORE_SOURCES})
target_include_directories(
    ehua_core
    PUBLIC
        $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
        $<INSTALL_INTERFACE:include>
)
target_compile_features(ehua_core PUBLIC c_std_17)
```

```
if(WIN32)
    target_compile_definitions(ehua_core PUBLIC WIN32_LEAN_AND_MEAN=1)
endif()

if(MINGW)
    target_compile_definitions(ehua_core PUBLIC __USE_MINGW_ANSI_STDIO=1)
endif()
ehua_apply_warnings(ehua_core)
```

Как этот файл учит C

Этот файл показывает вспомогательный слой проекта и помогает понять, как разные части сборки соединяются в единый инструмент.

Вопросы для самопроверки

Какие данные входят в этот файл из других модулей? Какие данные или эффекты он отдаёт наружу? Какая функция является главным входом файла? Какие проверки предотвращают повреждение памяти или неверный анализ? Кто владеет памятью, если файл выделяет ресурс?

Часть IV. Как самому написать мини-версию проекта

После изучения C через исходники полезно не просто перечитать проект, а воспроизвести его уменьшенную версию. Цель не в том, чтобы сразу написать полный EH/Unwind Analyzer, а в том, чтобы пройти тот же путь архитектурно: файл -> buffer -> PE header -> sections -> RVA map -> Exception Directory -> RUNTIME_FUNCTION -> UNWIND_INFO header -> диагностика -> JSON.

Этап 1. Минимальный CLI

Написать `main`, который принимает путь к файлу, проверяет количество аргументов, печатает usage и возвращает разные коды выхода для success/error. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 2. Чтение файла в `Buf`

Реализовать структуру с `uint8_t* data` и `size_t size`, функцию чтения файла и функцию освобождения памяти. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 3. Безопасные чтения

Реализовать `buf_read`, `buf_read_u16`, `buf_read_u32`, `buf_read_u64` с проверкой границ и little-endian. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 4. Проверка MZ и PE signature

Прочитать DOS magic, `e_lfanew`, проверить, что PE header находится внутри файла, прочитать `PE\0\0`. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 5. COFF и OptionalHeader

Прочитать Machine, NumberOfSections, SizeOfOptionalHeader, PE32+ magic, ImageBase, SizeOfImage, SizeOfHeaders, alignments. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 6. Sections

Выделить массив секций, прочитать name, VirtualAddress, VirtualSize, raw ptr, raw size, characteristics, добавить проверки OOB/overlap/alignment. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 7. RVA -> file offset

Реализовать поиск секции по RVA и преобразование в file offset с учётом virtual tail и переполнения. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 8. Exception Directory

Прочитать DataDirectory[Exception], проверить RVA/Size, размер кратный 12, выделить массив `RuntimeFunction`. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 9. `UNWIND_INFO` header

Для каждой функции прочитать UnwindInfoAddress, header, version, flags, prolog, count codes, frame register. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 10. `UNWIND_CODE` slots

Реализовать `slots_for_code`, пройти массив, проверять выход операции за CountOfCodes, считать fixed stack и регистры. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за

границы.

Этап 11. Padding и tail

Вычислить even slot count, после массива прочитать handler RVA или chain info в зависимости от flags. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 12. Diagnostics

При каждой ошибке заполнять failed_field, expected/available bytes, file offset, slots и byte dump. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 13. Heuristics

Посчитать invalid ranges, oob, unsorted, overlaps, duplicate begins, parse fails, anomalies, placement, reuse, handler dominance. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 14. JSON report

Сформировать JSON с metadata, sections, functions, summaries, findings и diagnostics. Реализовать escaping. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 15. HTML report

Построить человекочитаемую страницу с таблицами и русскими объяснениями findings. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Этап 16. Dynamic stub и Windows dynamic

Сначала сделать корректный stub для non-Windows, затем на Windows добавить debug loop и сбор runtime evidence. Критерий готовности этапа: программа должна не только работать на нормальном файле, но и корректно отвечать на поврежденный вход. Для каждого этапа нужен хотя бы один тест: нормальный случай, обрыв данных, неверное поле, выход за границы.

Часть V. Словарь терминов проекта и языка C

ABI

Application Binary Interface - соглашение двоичного уровня: как функции передают параметры, какие регистры сохраняются, как устроен stack frame. Для x64 unwind ABI задает, какие unwind-коды допустимы и что они означают.

address

адрес - положение объекта или инструкции в адресном пространстве. В проекте нужно различать VA, RVA и file offset.

allocation

выделение памяти. В C обычно выполняется через `malloc`, `calloc` или `realloc`; освобождение через `free`.

binary parser

парсер бинарного формата - программа, которая читает байты и превращает их в проверенную внутреннюю модель структур.

bounds check

проверка границ. Доказательство, что чтение N байт с offset помещается в буфер.

buffer

непрерывная область памяти. В проекте `Buf` хранит байты PE-файла.

caller context

контекст вызывающей функции, который virtual unwind пытается восстановить: RIP, RSP и сохраненные регистры.

CHAININFO

флаг `UNW_FLAG_CHAININFO`, означающий, что tail `UNWIND_INFO` содержит chained RUNTIME_FUNCTION-like запись.

CLI

Command Line Interface - интерфейс командной строки. В проекте им управляет `src/main.c`.

confidence

уверенность анализатора в наблюдении или итоговом выводе. Не равна severity и не является ML-точностью.

CountOfCodes

поле `UNWIND_INFO`, указывающее число двухбайтовых slots массива unwind codes. Одна операция может занимать несколько slots.

debug event

событие отладчика Windows: загрузка модуля, исключение, завершение процесса и другие runtime-факты.

dynamic function table

таблица unwind-информации, зарегистрированная во время выполнения, например через `RtlAddFunctionTable`. Статика видит файл, динамика может наблюдать такую регистрацию.

EH

Exception Handling - обработка исключений. В контексте проекта связано с Windows x64 SEH и handler metadata.

evidence

доказательное наблюдение: конкретные поля, адреса, секции, counters, bytes, events, на которых основан finding.

file offset

смещение в файле на диске. Не равно RVA, если файл загружается в память по секциям.

finding

структурированное сообщение о найденном признаке: code, family, severity, evidence, consequence, confidence, next step.

handler RVA

RVA обработчика исключений, который может быть указан в tail `UNWIND_INFO` при EHANDLER/UHANDLER.

heap

динамическая память процесса, из которой C-программа вручную выделяет объекты.

ImageBase

базовый адрес PE-образа. VA часто вычисляется как ImageBase + RVA.

little-endian

порядок байтов, где младший байт числа хранится первым.

metadata

данные о коде, не являющиеся самим исполняемым кодом. `.pdata` и `.xdata` - metadata для unwind/exception механизма.

padding

выравнивающее заполнение. В `UNWIND_INFO` при нечётном CountOfCodes tail начинается после дополнительного slot.

PE32+

64-битный вариант PE optional header, используемый для x64 Windows executables.

pointer

указатель - значение, хранящее адрес объекта. Разыменование указателя требует валидного адреса.

RUNTIME_FUNCTION

запись `.pdata`, описывающая диапазон функции и RVA связанной `UNWIND_INFO`.

RVA

Relative Virtual Address - смещение внутри image от ImageBase.

section

секция PE-образа: `.text`, `.pdata`, `.xdata`, `.rdata` и другие области с raw и virtual параметрами.

severity

серьезность признака: насколько значимым является нарушение, если оно реально.

slot

двухбайтовая ячейка массива `UNWIND_CODE`. Некоторые UWOP занимают дополнительные slots.

static analysis

анализ файла без запуска. В проекте читает PE и строит карту обещанной unwind/exception metadata.

StrBuf

динамический строковый буфер проекта для накопления ошибок и отчетов.

UNWIND_INFO

структура `.xdata`, описывающая prolog effects, stack allocation, сохраненные регистры, handler или chain.

UWOP

Unwind Operation - код операции внутри массива `UNWIND_CODE`: push, alloc, save, set frame register и т.д.

VA

Virtual Address - фактический адрес в виртуальном адресном пространстве процесса.

Часть VI. Большой практикум

Практикум нужен, чтобы знания стали действием. Ответы лучше писать письменно: не “понял”, а конкретно объяснил строку, тип, указатель, проверку, вход, выход и ошибку.

Базовый C

1. Написать программу с `main`, которая возвращает разные коды выхода. 2. Объявить переменные `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t` и объяснить размер каждой. 3. Написать функцию с `bool` результатом и output pointer. 4. Показать разницу между = и == на маленьком примере. 5. Объяснить, что делает `sizeof(*ptr)`. 6. Написать структуру `Range` и функцию проверки диапазона. 7. Объяснить `const` на примере `const Buf*`. 8. Написать функцию безопасного сложения `uint32_t`. 9. Реализовать bitmask из трех флагов. 10. Показать, почему `char name[9]` нужен для 8-байтового имени PE секции.

Память и строки

1. Реализовать чтение файла в heap buffer. 2. Написать `free`-путь для всех ошибок чтения. 3. Реализовать минимальный `StrBuf`. 4. Добавить `appendf` через `vsnprintf`. 5. Проверить переполнение capacity. 6. Написать JSON escape для кавычек и обратного слэша. 7. Объяснить, почему `memcpy` не знает границ буфера. 8. Сделать тест, где `buf_read` должен вернуть false. 9. Сделать тест для пустого файла. 10. Сделать тест для строки без завершающего нуля.

PE и адреса

1. Прочитать MZ magic из файла. 2. Прочитать `e_lfanew` и проверить границы. 3. Проверить PE signature. 4. Прочитать Machine и отклонить не-AMD64. 5. Прочитать OptionalHeader magic PE32+. 6. Прочитать ImageBase и SizeOfImage. 7. Прочитать Exception Directory. 8. Прочитать section headers. 9. Реализовать поиск секции по RVA. 10. Реализовать RVA -> file offset и тест для virtual tail.

Unwind

1. Прочитать массив `RUNTIME_FUNCTION`. 2. Проверить кратность Exception Directory размеру 12. 3. Проверить BeginAddress < EndAddress. 4. Найти overlaps после сортировки. 5. Прочитать header `UNWIND_INFO`. 6. Разложить Version и Flags из первого байта. 7. Реализовать `slots_for_code` для UWOP_ALLOC_LARGE. 8. Проверить выход UWOP за CountOfCodes. 9. Учесть padding при нечётном CountOfCodes. 10. Прочитать handler RVA при EHANDLER.

Heuristics и отчеты

1. Создать структуру Finding. 2. Добавить code/family/severity/message. 3. Сформировать score из двух признаков. 4. Разделить severity и confidence. 5. Добавить mitigation для small sample. 6. Сформировать JSON finding. 7. Сформировать HTML section. 8. Сделать Ghidra CSV строку. 9. Добавить русское объяснение finding. 10. Проверить, что отчет показывает expected/available bytes при parse fail.

Dynamic и платформа

1. Объяснить `#ifdef _WIN32`. 2. Сделать non-Windows stub dynamic-функции. 3. Описать, что такое debug event. 4. Описать, зачем breakpoint `0xCC`. 5. Объяснить, почему WriteProcessMemory требует проверки. 6. Объяснить timeout в dynamic mode. 7. Разделить static evidence и dynamic evidence. 8. Объяснить dynamic function table registration. 9. Сформировать dynamic summary без запуска target. 10. Объяснить, почему вредоносные образцы запускать только в изоляции.

Часть VII. Как объяснять проект через C на защите

Если комиссия спрашивает по C, отвечать нужно не набором терминов, а связкой “конструкция языка - зачем она здесь - какую ошибку предотвращает - что будет без неё”. Ниже приведены типовые вопросы и правильный технический ход ответа.

Почему проект написан на C?

Потому что задача находится на уровне бинарных форматов, байтов, фиксированных integer widths, PE layout, Windows API и переносимой CLI-сборки. C позволяет явно контролировать чтение bytes, layout структур результата, управление памятью и зависимость от платформенных API. Это не означает, что C безопасен автоматически; наоборот, проект показывает дисциплину проверок, которая нужна системному инструменту.

Почему нельзя просто наложить структуру PE поверх файла?

Потому что файл может быть не выровнен как структура процесса, может быть обрезан, может иметь злонамеренные поля, а компилятор может добавить padding в C-структуры. Проект читает поля через `buf_read_u16/u32/u64`, проверяя границы и endian, поэтому не зависит от опасного type punning.

Почему так много `uint32_t`?

Потому что PE и x64 unwind metadata задают конкретные размеры полей. RVA, section fields, BeginAddress, EndAddress и UnwindInfoAddress хранятся как 32-битные значения. `uint32_t` фиксирует это в коде и связывает реализацию с форматом.

Почему функции возвращают `bool`, а данные через указатель?

Так разделяются статус операции и результат. Например, `pe_rva_to_fileoff` может не суметь преобразовать RVA; если она возвращала бы только число, было бы сложно отличить offset 0 от ошибки. Поэтому статус - `true/false`, результат - через `out_off`.

Почему нужен `StrBuf`?

Потому что отчет и ошибки собираются постепенно, а обычная C-строка не растет сама. `StrBuf` хранит pointer, length, capacity и failed state, что позволяет безопасно добавлять текст и отслеживать OOM/overflow.

Почему dynamic отдельно от static?

Потому что static читает файл на диске, а dynamic наблюдает живой процесс. На уровне C это разные источники данных и разные API. Static должен оставаться переносимым, dynamic использует Windows debugger API и поэтому находится под `#ifdef _WIN32`.

Почему в unwind parser столько проверок?

Потому что `UNWIND_INFO` компактный и смещённый на два байта tail полностью меняет смысл handler/chain. Проверки CountOfCodes, slots, padding, flags и bounds защищают parser от чтения мусора и делают отчет доказательным.

Почему confidence не точность модели?

Потому что проект не является ML-классификатором на размеченной выборке. Confidence здесь инженерная уверенность в статическом evidence: сколько семейств признаков сработало, насколько они сильны, есть ли mitigation из-за малого sample или одиночного сигнала.

Приложение A. Ключевые фрагменты проекта для построчного разбора

Безопасное чтение буфера

```
bool buf_read(const Buf* b, size_t off, void* out, size_t len) {
    if (!b || !out) return false;
    if (off > b->size) return false;
    if (len > b->size - off) return false;
    memcpy(out, b->data + off, len);
    return true;
}

bool buf_read_u8(const Buf* b, size_t off, uint8_t* out) {
    return buf_read(b, off, out, 1);
}
bool buf_read_u16(const Buf* b, size_t off, uint16_t* out) {
    uint8_t tmp[2];
    if (!buf_read(b, off, tmp, 2)) return false;
    *out = le16(tmp);
    return true;
}
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out) {
    uint8_t tmp[4];
    if (!buf_read(b, off, tmp, 4)) return false;
    *out = le32(tmp);
    return true;
}
bool buf_read_u64(const Buf* b, size_t off, uint64_t* out) {
    uint8_t tmp[8];
    if (!buf_read(b, off, tmp, 8)) return false;
    *out = le64(tmp);
    return true;
```

Разбирай этот фрагмент по одному действию: типы, входные указатели, проверки, основное действие, ошибка, освобождение ресурса, связь с верхним слоем проекта. Если строка содержит сразу несколько операций, перепиши её на бумаге в несколько более простых шагов.

RVA в file offset

```
bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off) {
    if (!img || !out_off) return false;
    const PeSection* s = pe_find_section_for_rva(img, rva);
    if (s) {
        uint32_t delta = rva - s->virtual_address;
        if (delta >= s->raw_size) {
            /* RVA points into virtual tail (zero-fill) - not present in file */
            return false;
        }
        uint32_t off = 0;
        if (!safe_add_u32(s->raw_ptr, delta, &off)) return false;
        if (img->file_size > 0 && (uint64_t)off >= (uint64_t)img->file_size) return false;
        *out_off = off;
        return true;
    }

    /* RVAs inside SizeOfHeaders map directly to the corresponding file offset. */
    if (img->size_of_headers > 0 &&
        rva < img->size_of_headers &&
        (img->file_size == 0 || (uint64_t)rva < (uint64_t)img->file_size)) {
        *out_off = rva;
        return true;
    }
    return false;
}
```

Разбирай этот фрагмент по одному действию: типы, входные указатели, проверки, основное действие, ошибка, освобождение ресурса, связь с верхним слоем проекта. Если строка содержит сразу несколько операций, перепиши её на бумаге в несколько более простых шагов.

Slots для UWOP

```
static bool slots_for_code(uint8_t version,
                           uint8_t op,
                           uint8_t opinfo,
                           bool v2_seen_epilog,
                           uint8_t* out_slots) {
    /* based on documented v1 ops + v2 epilog/spare, and common implementations */
    switch (op) {
        case 0: /* UWOP_PUSH_NONVOL */ *out_slots = 1; return true;
        case 1: /* UWOP_ALLOC_LARGE */ if (opinfo > 1) return false; *out_slots = (uint8_t)(2
            + opinfo); return true;
        case 2: /* UWOP_ALLOC_SMALL */ *out_slots = 1; return true;
        case 3: /* UWOP_SET_FPREG */   *out_slots = 1; return true;
        case 4: /* UWOP_SAVE_NONVOL */ *out_slots = 2; return true;
        case 5: /* UWOP_SAVE_NONVOL_FAR */ *out_slots = 3; return true;
        case 6: /* v2 UWOP_EPILOG, v1 UWOP_SAVE_XMM (undoc) */
            if (version == 2) {
                /* The first long-form epilog descriptor has one operand slot.
                   Short-form and subsequent descriptors occupy one slot. */
                *out_slots = (!v2_seen_epilog && (opinfo & 1u) == 0u) ? 2u : 1u;
                return true;
            }
            *out_slots = 2; return true;
        case 7: /* v2 UWOP_SPARE, v1 UWOP_SAVE_XMM_FAR (undoc) */
```

```
            if (version == 2) { *out_slots = 1; return true; }
            *out_slots = 3; return true;
        case 8: /* UWOP_SAVE_XMM128 */ *out_slots = 2; return true;
        case 9: /* UWOP_SAVE_XMM128_FAR */ *out_slots = 3; return true;
        case 10: /* UWOP_PUSH_MACHFRAME */ *out_slots = 1; return true;
        default:
            return false;
    }
}

static bool parse_unwindinfo_header(const Buf* b, const PeImage* img, uint32_t unwind_rva,
                                    uint32_t* out_fileoff, uint8_t hdr[4], StrBuf* err) {
    uint32_t off = 0;
    if (!pe_rva_to_fileoff(img, unwind_rva, &off)) {
        sb_appendf(err, "UnwindInfo RVA 0x%08x not mappable to file offset.\n", unwind_rva);
        return false;
    }
    if ((size_t)off + 4 > b->size) {
        sb_append(err, "UnwindInfo header out of file range.\n");
        return false;
    }
    if (!buf_read(b, off, hdr, 4)) {
        sb_append(err, "Failed to read UnwindInfo header bytes.\n");
        return false;
    }
    *out_fileoff = off;
    return true;
}

static bool read_u16_slot(const Buf* b, size_t off, uint16_t* out) {
    uint8_t tmp[2];
    if (!buf_read(b, off, tmp, 2)) return false;
    *out = (uint16_t)(tmp[0] | ((uint16_t)tmp[1] << 8));
```

Разбирай этот фрагмент по одному действию: типы, входные указатели, проверки, основное действие, ошибка, освобождение ресурса, связь с верхним слоем проекта. Если строка содержит сразу несколько операций, перепиши её на бумаге в несколько более простых шагов.

Главный цикл unwind parser

```
    for (uint32_t i = 0; i < count; ) {
        uint16_t slot = 0;
        size_t slot_fileoff = codes_off + (size_t)i * 2u;
        if (!read_u16_slot(b, slot_fileoff, &slot)) {
            out->anomalies |= UA_TRUNCATED;
            unwind_diag_set_read_failure(out, b, "UNWIND_CODE slot", slot_fileoff, 2u,
                record_limit);
            sb_append(err, "Failed to read unwind slot.\n");
            return unwind_parse_fail(out, err, err_start, "Failed to read unwind slot.");
        }

        uint8_t code_offset = (uint8_t)(slot & 0xff);
        uint8_t op_byte = (uint8_t)(slot >> 8);
        uint8_t op = (uint8_t)(op_byte & 0x0f);
        uint8_t opinfo = (uint8_t)(op_byte >> 4);

        uint8_t slots = 0;
        if (!slots_for_code(ver, op, opinfo, v2_seen_epilog, &slots)) {
            out->anomalies |= UA_BAD_OPCODE;
            /* try to continue safely as 1 slot */
            slots = 1;
        }
        if (i + slots > count) {
            out->anomalies |= UA_TRUNCATED;
            unwind_diag_set(out, b, "CountOfCodes / UWOP operand slots",
                            true, slot_fileoff, true, (size_t)slots * 2u,
                            (size_t)(count - i) * 2u, true, slots, count - i);
            sb_append(err, "Unwind opcode spans beyond CountOfCodes.\n");
            return unwind_parse_fail(out, err, err_start, "Unwind opcode spans beyond
                CountOfCodes.");
        }
        bool v2_descriptor = ver == 2 && (op == 6 || op == 7);
        if (!v2_descriptor) {
            if (code_offset > prolog) {
                out->anomalies |= UA_CODEOFFSET_OUTSIDE;
            }
            if (have_prev_code_offset && code_offset > prev_code_offset) {
                out->anomalies |= UA_CODEOFFSET_ORDER;
            }
            have_prev_code_offset = true;
            prev_code_offset = code_offset;
        }
        if (strict_abi && !v2_descriptor) {
            if (saw_push_nonvol && op != 0 && op != 10) {
                out->anomalies |= UA_ABI_PUSH_ORDER;
            }
            if (op == 0) saw_push_nonvol = true;
            if ((flags & UNW_FLAG_CHAININFO) && op != 4) {
                out->anomalies |= UA_ABI_CHAIN_RESTRICTION;
            }
        }

        if (ops_text) {
            /* print one-line per op start */
            switch (op) {
                case 0: sb_appendf(ops_text, "  @%02x PUSH_NONVOL %s\n", code_offset,
                unwind_reg_name(opinfo)); break;
                case 1: sb_appendf(ops_text, "  @%02x ALLOC_LARGE mode=%u\n", code_offset,
                (unsigned)opinfo); break;
                case 2: sb_appendf(ops_text, "  @%02x ALLOC_SMALL %u\n", code_offset,
                (unsigned)(opinfo*8u + 8u)); break;
                case 3: sb_appendf(ops_text, "  @%02x SET_FPREG %s + %u\n", code_offset,
```

```
                unwind_reg_name(fr), (unsigned)(foff*16u)); break;
                case 4: sb_appendf(ops_text, "  @%02x SAVE_NONVOL %s\n", code_offset,
                unwind_reg_name(opinfo)); break;
                case 5: sb_appendf(ops_text, "  @%02x SAVE_NONVOL_FAR %s\n", code_offset,
                unwind_reg_name(opinfo)); break;
                case 6:
                    if (ver == 2) {
                        if (!v2_seen_epilog) {
                            if ((opinfo & 1u) == 0u) {
                                uint16_t epilog_slot = 0;
                                if (read_u16_slot(b, codes_off + (size_t)(i + 1) * 2,
                                                  &epilog_slot)) {
                                    unsigned epilog_offset =
                                        (unsigned)(epilog_slot & 0xffu) |
                                        ((unsigned)(epilog_slot >> 12) << 8);
                                    sb_appendf(ops_text,
                                               "  EPILOG size=%u offset_from_end=%u flags=%u
                (v2)\n",
                                               (unsigned)code_offset, epilog_offset,
                                               (unsigned)opinfo);
                                }
                            } else {
                                sb_appendf(ops_text, "  EPILOG size=%u flags=%u (v2)\n",
                                           (unsigned)code_offset, (unsigned)opinfo);
                            }
                        } else {
                            unsigned epilog_offset =
                                (unsigned)code_offset | ((unsigned)opinfo << 8);
                            sb_appendf(ops_text, "  EPILOG offset_from_end=%u (v2)\n",
                                       epilog_offset);
                        }
                    } else {
                        sb_appendf(ops_text, "  @%02x SAVE_XMM (undoc/v1)\n", code_offset);
                    }
                    break;
                case 7:
                    if (ver == 2) sb_appendf(ops_text, "  @%02x SPARE (v2)\n", code_offset);
                    else sb_appendf(ops_text, "  @%02x SAVE_XMM_FAR (undoc/v1)\n",
                code_offset);
                    break;
                case 8: sb_appendf(ops_text, "  @%02x SAVE_XMM128 XMM%u\n", code_offset,
                (unsigned)opinfo); break;
                case 9: sb_appendf(ops_text, "  @%02x SAVE_XMM128_FAR XMM%u\n", code_offset,
                (unsigned)opinfo); break;
                case 10: sb_appendf(ops_text, "  @%02x PUSH_MACHFRAME err=%u\n", code_offset,
                (unsigned)opinfo); break;
                default: sb_appendf(ops_text, "  @%02x OP_%u info=%u\n", code_offset,
                (unsigned)op, (unsigned)opinfo); break;
            }
        }

        /* compute effects */
        switch (op) {
            case 0: /* push nonvol */
                add_fixed_stack(&fixed_stack, 8, &out->anomalies);
                pushed++;
                if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {
                    out->anomalies |= UA_ABI_REGISTER_CLASS;
                }
                if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);
                break;
            case 1: { /* alloc large */
                if (opinfo == 0) {
                    uint16_t scaled = 0;
                    if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {
                        out->anomalies |= UA_TRUNCATED;
```

Разбирай этот фрагмент по одному действию: типы, входные указатели, проверки, основное действие, ошибка, освобождение ресурса, связь с верхним слоем проекта. Если строка содержит сразу несколько операций, перепиши её на бумаге в несколько более простых шагов.

Финализация verdict

```
static void finalize_verdict(HeuristicReport* out) {
    if (!out) return;

    out->suspicion = safe_ratio((double)out->score, 100.0);

    size_t family_count = heuristics_evidence_family_count(out->evidence_families_mask);
    size_t strong_count = count_findings_with_severity(out, "high") +
        count_findings_with_severity(out, "critical");
    size_t medium_count = count_findings_with_severity(out, "medium");
    size_t weak_count = 0;
    if (out->finding_count >= strong_count + medium_count) weak_count = out->finding_count -
        strong_count - medium_count;

    out->strong_finding_count = strong_count;
    out->medium_finding_count = medium_count;
    out->weak_finding_count = weak_count;
    safe_copy(out->evidence_level, sizeof(out->evidence_level),
              evidence_level_from_counts(weak_count, medium_count, strong_count, out->score));

    double confidence = CONFIDENCE_BASE;
    confidence += (double)family_count * CONFIDENCE_PER_FAMILY;
    confidence += (double)strong_count * CONFIDENCE_PER_STRONG;
    confidence += (double)medium_count * CONFIDENCE_PER_MEDIUM;

    if (out->rf_invalid_ranges || out->rf_oob || out->rf_unwind_oob || out->rf_overlaps)
        confidence += CONFIDENCE_BONUS_TABLE_HARD;
    if (out->unwind_parse_fail || out->unwind_anomalies) confidence +=
        CONFIDENCE_BONUS_UNWIND_HARD;
```

```
    if (out->score == 0) confidence = CONFIDENCE_ZERO_SCORE_BASE;

    /* Weakening rules: lower confidence when only isolated / non-correlated static signals
        exist. */
    if (out->finding_count == 1 && strong_count == 0) {
        add_mitigation(out, "single_signal_only",
                       "Only one non-strong signal contributed; treat as weak standalone
                evidence.");
        confidence -= CONFIDENCE_PENALTY_SINGLE_SIGNAL;
    }
    if (out->total < 32 && strong_count == 0 && out->finding_count > 0) {
        add_mitigation(out, "small_sample_size",
                       "Very small .pdata sample can overstate ratios; confidence reduced
                until more functions are observed.");
        confidence -= CONFIDENCE_PENALTY_SMALL_SAMPLE;
    }
    if (out->unwind_unusual_section > 0 &&
        out->unwind_exec_section == 0 &&
        out->unwind_writable_section == 0 &&
        out->handler_nonexec_section == 0 &&
        out->handler_writable_section == 0 &&
        out->rf_invalid_ranges == 0 &&
        out->rf_oob == 0 &&
        out->rf_unwind_oob == 0 &&
        out->unwind_parse_fail == 0 &&
        out->unwind_anomalies == 0) {
        add_mitigation(out, "placement_only_signal",
                       "Placement-only anomalies may come from uncommon but benign layouts;
                manual verification required.");
        confidence -= CONFIDENCE_PENALTY_PLACEMENT_ONLY;
    }
    if ((out->strong_finding_count == 0) && (family_count <= 1) && out->finding_count > 0) {
        add_mitigation(out, "single_family_only",
                       "Evidence comes from only one family; cross-layer corroboration is
                recommended before strong conclusions.");
        confidence -= CONFIDENCE_PENALTY_SINGLE_FAMILY;
    }

    if (confidence < CONFIDENCE_MIN) confidence = CONFIDENCE_MIN;
    if (confidence > CONFIDENCE_MAX) confidence = CONFIDENCE_MAX;
    out->confidence = confidence;

    out->requires_manual_review =
        (out->score >= 20) ||
        (out->unwind_exec_section > 0) ||
        (out->unwind_writable_section > 0) ||
        (out->handler_nonexec_section > 0) ||
        (out->handler_writable_section > 0) ||
        (out->unwind_parse_fail > 0) ||
        (out->mitigation_count > 0);

    if (out->score >= 70) safe_copy(out->verdict, sizeof(out->verdict), "high_suspicion");
    else if (out->score >= 40) safe_copy(out->verdict, sizeof(out->verdict),
        "moderate_suspicion");
    else if (out->score >= 15) safe_copy(out->verdict, sizeof(out->verdict), "low_suspicion");
    else safe_copy(out->verdict, sizeof(out->verdict), "no_strong_static_anomalies");
}

bool heuristics_analyze(const PeImage* img,
                        const RuntimeFunction* rf,
                        const UnwindSummary* us,
                        const bool* us_ok,
                        size_t n,
                        HeuristicReport* out,
                        StrBuf* err) {
    if (!img || !out || !err) return false;
    if (n > 0 && (!rf || !us || !us_ok)) return false;
    memset(out, 0, sizeof(*out));
    if (!sb_init(&out->details, 1024)) {
        sb_append(err, "Out of memory (details buffer).\n");
        return false;
    }
    out->total = n;
    out->pe_section_raw_oob = img->section_raw_oob;
    out->pe_section_virtual_oob = img->section_virtual_oob;
    out->pe_section_raw_overlaps = img->section_raw_overlaps;
    out->pe_section_virtual_overlaps = img->section_virtual_overlaps;
    out->pe_section_misaligned = img->section_misaligned;
    out->pe_exception_directory_oob = img->exception_directory_oob;

    /* RUNTIME_FUNCTION sanity */
    uint32_t prev_begin = 0;
    bool prev_valid = false;
    size_t unsorted = 0;
    FindingLocation invalid_range_loc = {0};
    FindingLocation rf_oob_loc = {0};
    FindingLocation unwind_oob_loc = {0};
```

Разбирай этот фрагмент по одному действию: типы, входные указатели, проверки, основное действие, ошибка, освобождение ресурса, связь с верхним слоем проекта. Если строка содержит сразу несколько операций, перепиши её на бумаге в несколько более простых шагов.

JSON escaping

```
static void json_escape(StrBuf* sb, const char* s) {
    if (!s) return;
    size_t length = strlen(s);
    for (size_t i = 0; i < length; i++) {
        unsigned char c = (unsigned char)s[i];
        switch (c) {
            case '\\': sb_append(sb, "\\\\"); break;
            case '"':  sb_append(sb, "\\\""); break;
```

```
            case '\n': sb_append(sb, "\\n"); break;
            case '\r': sb_append(sb, "\\r"); break;
            case '\t': sb_append(sb, "\\t"); break;
            default:
                if (c < 0x20) sb_appendf(sb, "\\u%04x", (unsigned)c);
                else if (c < 0x80) sb_appendn(sb, &s[i], 1);
                else {
                    size_t sequence = valid_utf8_sequence_length(
                        (const unsigned char*)&s[i], length - i);
                    if (sequence) {
                        sb_appendn(sb, &s[i], sequence);
                        i += sequence - 1;
                    } else {
                        sb_appendf(sb, "\\u%04x", (unsigned)c);
                    }
                }
        }
    }
}

static void json_key(StrBuf* sb, const char* k) {
    sb_append(sb, "\"");
    json_escape(sb, k);
    sb_append(sb, "\":");
}

static void json_string(StrBuf* sb, const char* s) {
    sb_append(sb, "\"");
    json_escape(sb, s ? s : "");
    sb_append(sb, "\"");
}

static void json_string_field(StrBuf* sb, const char* k, const char* v, bool trailing_comma) {
    json_key(sb, k);
    json_string(sb, v);
    if (trailing_comma) sb_append(sb, ",");
}

static void json_bool_field(StrBuf* sb, const char* k, bool v, bool trailing_comma) {
    json_key(sb, k);
    sb_append(sb, v ? "true" : "false");
    if (trailing_comma) sb_append(sb, ",");
}


static void json_nullable_hex32(StrBuf* sb, bool has_value, uint32_t value) {
    if (has_value) sb_appendf(sb, "\"0x%08x\"", (unsigned)value);
    else sb_append(sb, "null");
}

static void json_nullable_count(StrBuf* sb, bool has_value, uint32_t value) {
    if (has_value) sb_appendf(sb, "%u", (unsigned)value);
    else sb_append(sb, "null");
}

static void bytes_to_hex(const uint8_t* bytes, size_t count, char* out, size_t out_size) {
    static const char hex[] = "0123456789abcdef";
    if (!out || out_size == 0) return;
    size_t pos = 0;
    for (size_t i = 0; i < count && bytes && pos + 3 < out_size; i++) {
        if (i > 0) out[pos++] = ' ';
        out[pos++] = hex[(bytes[i] >> 4) & 0x0f];
        out[pos++] = hex[bytes[i] & 0x0f];
    }
    out[pos] = '\0';
}

static void json_unwind_diagnostic(StrBuf* sb, const UnwindSummary* u) {
    char bytes_hex[UNWIND_DIAG_MAX_BYTES * 3u + 1u];
    bytes_hex[0] = '\0';
    if (u && u->diagnostic_has_bytes) {
        bytes_to_hex(u->diagnostic_bytes, u->diagnostic_bytes_count, bytes_hex,
            sizeof(bytes_hex));
    }
    sb_append(sb, "{");
    json_string_field(sb, "failed_field", u ? u->diagnostic_field : "", true);
    json_key(sb, "failure_file_offset");
    json_nullable_hex32(sb, u && u->diagnostic_has_file_offset, u ? u->diagnostic_file_offset
        : 0u);
```

Разбирай этот фрагмент по одному действию: типы, входные указатели, проверки, основное действие, ошибка, освобождение ресурса, связь с верхним слоем проекта. Если строка содержит сразу несколько операций, перепиши её на бумаге в несколько более простых шагов.

CLI main

```
int main(int argc, char** argv) {
    Args a;
    int exit_code = 0;
    ParseResult pres = parse_args(argc, argv, &a);
    if (pres != PARSE_OK) {
        if (pres == PARSE_VERSION) {
            print_version();
            return 0;
        }
        usage(argv[0]);
        return (pres == PARSE_HELP) ? 0 : 2;
    }

    uint8_t* owned = NULL;
    Buf b;
    if (!buf_init_from_file(&b, a.input, &owned)) {
```

```
    fprintf(stderr, "Failed to read file: %s\n", a.input);
    return 1;
}

StrBuf err;
sb_init(&err, 1024);

PeImage img;
if (!pe_parse(&img, &b, &err)) {
    fprintf(stderr, "PE parse error:\n%s", err.s ? err.s : "");
    sb_free(&err);
    buf_free_owned(owned);
    return 1;
}

RuntimeFunction* rf = NULL;
size_t rf_count = 0;
StrBuf err2;
sb_init(&err2, 1024);

RuntimeFunctionReadResult rf_read = unwind_read_runtime_functions_ex(
    &b, &img, &rf, &rf_count, &err2);
if (rf_read != RUNTIME_FUNCTION_READ_OK) {
    fprintf(stderr, "Failed to read exception directory:\n%s", err2.s ? err2.s : "");
    if (rf_read == RUNTIME_FUNCTION_READ_OOM) {
        pe_free(&img);
        sb_free(&err);
        sb_free(&err2);
        buf_free_owned(owned);
        return 1;
    }
    /*
     * Keep the established report field/finding code for compatibility.
     * It now covers every structurally unreadable exception directory.
     */
    img.exception_directory_oob = true;
    fprintf(stderr, "Continuing with a partial structural report.\n");
    exit_code = 1;
}

report_print_brief(a.input, &img, rf_count);

UnwindSummary* us = (UnwindSummary*)calloc(rf_count ? rf_count : 1,
    sizeof(UnwindSummary));
bool* us_ok = (bool*)calloc(rf_count ? rf_count : 1, sizeof(bool));
if (!us || !us_ok) {
    fprintf(stderr, "Out of memory.\n");
    free(us);
    free(us_ok);
    free(rf);
    pe_free(&img);
    sb_free(&err);
    sb_free(&err2);
    buf_free_owned(owned);
    return 1;
}

UnwindParseOpts uopts = {
    .max_chain_depth = a.max_chain_depth,
    .follow_chain = a.follow_chain,
    .strict_abi = a.strict_abi
};

StrBuf ops;
sb_init(&ops, 1024);

for (size_t i = 0; i < rf_count; i++) {
    StrBuf perr;
    sb_init(&perr, 256);
    if (a.show_ops) { ops.len = 0; if (ops.s) ops.s[0]=0; }
    bool ok = unwind_parse(&b, &img, rf[i].unwind_rva, &uopts, &us[i], a.show_ops ? &ops :
         NULL, &perr);
    us_ok[i] = ok;
    if (a.dump && a.show_ops && ok) {
        /* ops will be printed in dump loop below */
    }
    sb_free(&perr);
}

if (a.dump) {
    size_t limit = (a.dump_limit < 0) ? rf_count : (size_t)a.dump_limit;
    if (limit > rf_count) limit = rf_count;
    printf("\n[Dump] Showing %zu / %zu entries\n", limit, rf_count);
    for (size_t i = 0; i < limit; i++) {
        print_one(i, &rf[i], &us[i], us_ok[i]);
        if (a.show_ops && us_ok[i]) {
            StrBuf ops2;
            StrBuf perr;
            sb_init(&ops2, 1024);
            sb_init(&perr, 256);
            (void)unwind_parse(&b, &img, rf[i].unwind_rva, &uopts, &us[i], &ops2, &perr);
            if (ops2.s && ops2.len) printf("%s", ops2.s);
            sb_free(&ops2);
            sb_free(&perr);
        }
    }
}

HeuristicReport hr;
HeuristicReport* hr_ptr = NULL;
DynamicReport dr;
memset(&dr, 0, sizeof(dr));
DynamicReport* dr_ptr = NULL;
bool dr_initialized = false;
```

```
    if (a.heuristics) {
        StrBuf perr;
```

Разбирай этот фрагмент по одному действию: типы, входные указатели, проверки, основное действие, ошибка, освобождение ресурса, связь с верхним слоем проекта. Если строка содержит сразу несколько операций, перепиши её на бумаге в несколько более простых шагов.

Приложение B. Расширенные учебные разборы типовых строк проекта

if (!out || !b || !err) return false;

Эта строка проверяет три входных указателя до использования. В C выражение !out истинно, если указатель равен нулю. Оператор || означает логическое “или”: если хотя бы один из указателей отсутствует, функция не может безопасно продолжать. Возврат false сообщает вызывающему коду, что договор функции не выполнен. Без этой строки последующее обращение out->..., b->... или sb_append(err, ...) могло бы разыменовать нулевой указатель. Чтобы закрепить это место, выпиши тип каждого имени в строке, затем подпиши действие каждого оператора, затем сформулируй, какую ошибку предотвращает строка. Такой порядок должен стать автоматическим при чтении проекта.

memset(out, 0, sizeof(*out));

Эта строка приводит структуру результата к известному начальному состоянию. out - указатель на структуру, *out - сама структура, sizeof(*out) - её фактический размер. memset заполняет все байты нулем. Для PeImage это означает: флаги false, числовые поля 0, указатели NULL. После этого при ошибке освобождение через pe_free не увидит случайный мусорный указатель. Чтобы закрепить это место, выпиши тип каждого имени в строке, затем подпиши действие каждого оператора, затем сформулируй, какую ошибку предотвращает строка. Такой порядок должен стать автоматическим при чтении проекта.

uint32_t slots_even = (uint32_t)((count + 1u) & ~1u);

Эта строка округляет число slots до чётного значения. count + 1u увеличивает значение на один, ~1u создаёт маску со всеми единицами кроме младшего бита, & сбрасывает младший бит. Если count уже чётный, результат не меняется; если нечётный, результат становится следующим чётным. Это нужно для padding в UNWIND_INFO: tail должен начинаться после выровненного массива slots. Чтобы закрепить это место, выпиши тип каждого имени в строке, затем подпиши действие каждого оператора, затем сформулируй, какую ошибку предотвращает строка. Такой порядок должен стать автоматическим при чтении проекта.

if (i + slots > count) return false;

Эта проверка защищает parser от чтения operands за пределами массива UNWIND_CODE. Переменная i - индекс текущего slot, slots - сколько slots занимает текущая операция. Если операция требует больше slots, чем осталось до CountOfCodes, parser должен остановиться. Иначе он примет tail или чужие bytes за operands и сформирует ложный вывод. Чтобы закрепить это место, выпиши тип каждого имени в строке, затем подпиши действие каждого оператора, затем сформулируй, какую ошибку предотвращает строка. Такой порядок должен стать автоматическим при чтении проекта.

const PeSection* s = pe_find_section_for_rva(img, rva);

Эта строка получает адрес секции, покрывающей заданный RVA. const PeSection* означает: функция получает указатель на секцию, но не должна менять её поля. Если секция не найдена, вернётся NULL, и это состояние нужно обработать. Такой код показывает переход от числового адреса к структурной карте PE. Чтобы закрепить это место, выпиши тип каждого имени в строке, затем подпиши действие каждого оператора, затем сформулируй, какую ошибку предотвращает строка. Такой порядок должен стать автоматическим при чтении проекта.

if (delta >= s->raw_size) return false;

Эта проверка отличает RVA, который лежит в виртуальном диапазоне секции, от RVA, который реально присутствует в raw bytes файла. В PE секция может иметь virtual tail: область есть в памяти после загрузки, но отсутствует в файле. Static parser не имеет права читать такую область из файла. Чтобы закрепить это место, выпиши тип каждого имени в строке, затем подпиши действие каждого оператора, затем сформулируй, какую ошибку предотвращает строка. Такой порядок должен стать автоматическим при чтении проекта.

out->anomalies |= UA_TRUNCATED;

Эта строка включает бит аномалии в маске. |= не заменяет все остальные признаки, а добавляет новый. Если ранее уже были включены UA_ALIGNMENT или другие флаги, они сохраняются. Так один UnwindSummary может хранить несколько проблем одновременно. Чтобы закрепить это место, выпиши тип каждого имени в строке, затем подпиши действие каждого оператора, затем сформулируй, какую ошибку предотвращает строка. Такой порядок должен стать автоматическим при чтении проекта.

if (!file_write_atomic(path, output.s, output.len))

Эта строка проверяет запись отчета. Atomic write означает, что проект сначала пишет временный файл, а потом заменяет итоговый путь. Это снижает риск получить наполовину записанный отчет, если запись оборвётся. Даже отчетный слой в системной утилите должен проверять ошибки ввода-вывода. Чтобы закрепить это место, выпиши тип каждого имени в строке, затем подпиши действие каждого оператора, затем сформулируй, какую ошибку предотвращает строка. Такой порядок должен

стать автоматическим при чтении проекта.

Заключение: что должно измениться после чтения

После этой книги цель не в том, чтобы заучить каждую строку проекта. Цель - научиться читать системный C как цепочку доказательств. В `util.c` ты должен видеть безопасность байтов. В `pe.c` - построение карты PE. В `unwind.c` - строгий layout `.xdata`. В `heuristics.c` - инженерную модель evidence. В `report.c` - перевод структурированных данных в объяснимый отчет. В `dynamic_win.c` - границу между файлом и живым процессом Windows. Если ты можешь открыть `src/main.c` и объяснить путь от `argv[1]` до HTML/JSON отчета, открыть `src/pe.c` и объяснить RVA mapping, открыть `src/unwind.c` и объяснить CountOfCodes, slots и padding, открыть `src/heuristics.c` и объяснить score/confidence, значит C начал работать как инструмент мышления, а не как набор непонятных символов.

Часть VIII. Большой учебный курс C через реальные конструкции проекта

Эта часть добавлена специально как учебный курс, а не как справочник. Каждый модуль берёт одну конструкцию C, которая реально встречается в EH/Unwind Analyzer, объясняет её до использования, показывает фрагмент из проекта и связывает с задачей анализа PE x64 EH/Unwind metadata. Если предыдущие части дают маршрут, эта часть даёт многократную тренировку чтения настоящего C-кода.

Учебный модуль 1. argc и argv: вход программы из командной строки

Сначала смысл

CLI-программа получает данные не через графическое окно, а через массив строк командной строки. argc хранит количество строк, argv хранит адрес массива указателей на строки. Это первая точка, где внешний мир входит в C-программу.

Фрагмент из проекта

```
int main(int argc, char** argv) {
    Args a;
    int exit_code = 0;
    ParseResult pres = parse_args(argc, argv, &a);
    if (pres != PARSE_OK) {
        if (pres == PARSE_VERSION) {
            print_version();
            return 0;
        }
        usage(argv[0]);
        return (pres == PARSE_HELP) ? 0 : 2;
    }

    uint8_t* owned = NULL;
    Buf b;
    if (!buf_init_from_file(&b, a.input, &owned)) {
        fprintf(stderr, "Failed to read file: %s\n", a.input);
        return 1;
    }

    StrBuf err;
    sb_init(&err, 1024);

    PeImage img;
    if (!pe_parse(&img, &b, &err)) {
        fprintf(stderr, "PE parse error:\n%s", err.s ? err.s : "");
        sb_free(&err);
        buf_free_owned(owned);
        return 1;
    }

    RuntimeFunction* rf = NULL;
    size_t rf_count = 0;
    StrBuf err2;
```

Какие элементы C здесь работают

Работают указатель на указатель char**, массив указателей, индексирование argv[i], сравнение строк, проверка границ массива через argc. Важно: argv[1] допустим только если argc >= 2.

Почему это нужно именно этому проекту

Проекту нужно принять путь к PE-файлу и опции: --json, --html, --heuristics, --dynamic, --strict-abi. Поэтому корректный parsing аргументов является частью надёжности инструмента.

Как читать такие строки

Сначала найди, где проверяется argc. Затем выпиши, какие элементы argv считаются именем файла, а какие опциями. После этого для каждой опции найди поле структуры Args, которое она меняет.

Что ломается при непонимании

Если читать argv[1] без проверки количества аргументов, программа может обратиться за пределы массива. Если сравнивать строки оператором ==, будет сравниваться адрес, а не текст. Если не обработать неизвестную опцию, пользователь получит непредсказуемое поведение.

Практическое задание

Добавь мысленно новую опцию --dry-run. Опиши, какое поле появится в Args, где его нужно разобрать и как оно повлияет на основной сценарий.

Учебный модуль 2. Структура `Args`: состояние CLI без глобальных переменных

Сначала смысл

Когда опций много, нельзя хранить их как россыпь независимых переменных по всему `main`. Структура собирает всё состояние запуска в один объект. Это делает договор между parser-ом аргументов и основным сценарием явным.

Фрагмент из проекта

```
typedef struct Args {
    const char* input;
    bool dump;
    bool show_ops;
    bool heuristics;
    const char* csv_path;
    const char* ghidra_csv_path;
    const char* json_path;
    const char* html_path;
    int  dump_limit;        /* -1 => all */
    bool follow_chain;
    bool strict_abi;
    int  max_chain_depth;
    bool dynamic;
    uint32_t timeout_ms;
    uint32_t trace_steps;
    const char** child_argv;
    int child_argc;
} Args;

typedef enum ParseResult {
    PARSE_OK = 0,
    PARSE_HELP = 1,
    PARSE_VERSION = 2,
    PARSE_ERR = 3
} ParseResult;

static char* dup_cstr(const char* s) {
    if (!s) return NULL;
    size_t n = strlen(s);
    char* out = (char*)malloc(n + 1u);
    if (!out) return NULL;
    memcpy(out, s, n + 1u);
    return out;
```

Какие элементы C здесь работают

Работают `typedef struct`, поля разных типов, указатели на строки, `bool`, `int`, `uint32_t`, массив указателей `const char**`. Также важна инициализация через `memset` перед заполнением.

Почему это нужно именно этому проекту

Проект использует `Args`, чтобы отделить чтение командной строки от выполнения анализа. Сначала `parse_args` заполняет структуру, затем `main` читает её как готовую конфигурацию.

Как читать такие строки

Прочитай каждое поле `Args` как вопрос: какой режим оно включает, какой файл задаёт, какое числовое ограничение хранит, к какому модулю относится.

Что ломается при непонимании

Если состояние опций размазать по глобальным переменным, тестировать parsing труднее. Если структуру не обнулить, поля могут содержать мусор и включать режимы случайно.

Практическое задание

Составь таблицу: поле `Args`, тип, значение по умолчанию, какая опция его меняет.

Учебный модуль 3. `enum ParseResult`: именованные результаты вместо магических чисел

Сначала смысл

Функция разбора аргументов может закончиться несколькими способами: успех, help, version, ошибка. Если вернуть просто 0, 1, 2, 3, код будет труднее читать. `enum` даёт именованные константы.

Фрагмент из проекта

```
typedef enum ParseResult {
    PARSE_OK = 0,
    PARSE_HELP = 1,
    PARSE_VERSION = 2,
    PARSE_ERR = 3
} ParseResult;

static char* dup_cstr(const char* s) {
    if (!s) return NULL;
    size_t n = strlen(s);
    char* out = (char*)malloc(n + 1u);
    if (!out) return NULL;
    memcpy(out, s, n + 1u);
    return out;
}

static char* argv_text_to_utf8(const char* s) {
    if (!s) return NULL;
#if defined(_WIN32)
    int wide_len = MultiByteToWideChar(CP_ACP, 0, s, -1, NULL, 0);
    if (wide_len <= 0) return dup_cstr(s);
    wchar_t* wide = (wchar_t*)calloc((size_t)wide_len, sizeof(wchar_t));
    if (!wide) return NULL;
    if (MultiByteToWideChar(CP_ACP, 0, s, -1, wide, wide_len) <= 0) {
        free(wide);
        return dup_cstr(s);
    }
    int utf8_len = WideCharToMultiByte(CP_UTF8, 0, wide, -1, NULL, 0, NULL, NULL);
    if (utf8_len <= 0) {
        free(wide);
        return dup_cstr(s);
    }
    char* out = (char*)malloc((size_t)utf8_len);
    if (!out) {
```

Какие элементы C здесь работают

Работает перечисление `enum`, присваивание явных числовых значений, сравнение результата с именованной константой. Это повышает читаемость и снижает риск перепутать смысл числа.

Почему это нужно именно этому проекту

В `main` результат `parse_args` определяет, печатать ли help/version или запускать анализ. Это ранний routing сценария.

Как читать такие строки

Найди, где `parse_args` возвращает каждый вариант `ParseResult`, и где `main` на него реагирует.

Что ломается при непонимании

Магические числа плохо защищают от ошибки: `if (pres == 2)` непонятно без поиска. `if (pres == PARSE_VERSION)` читается сразу.

Практическое задание

Добавь вариант `PARSE_WARN` на бумаге и опиши, почему это плохая идея, если warning не должен останавливать запуск.

Учебный модуль 4. `strcmp`: сравнение текста, а не адресов строк

Сначала смысл

Строка C хранится как массив `char`, а переменная `char*` хранит адрес первого символа. Оператор == сравнил бы адреса, а не содержимое. Для сравнения текста используется `strcmp`.

Фрагмент из проекта

```
if (strcmp(argv[1], "--help") == 0 || strcmp(argv[1], "-h") == 0) return PARSE_HELP;
if (strcmp(argv[1], "--version") == 0) return PARSE_VERSION;
a->input = argv[1];

for (int i = 2; i < argc; i++) {
    const char* s = argv[i];
    if (strcmp(s, "--") == 0) {
        a->child_argc = argc - (i + 1);
        a->child_argv = (const char**)&argv[i + 1];
        break;
    }
    if (strcmp(s, "--help") == 0 || strcmp(s, "-h") == 0) return PARSE_HELP;
    else if (strcmp(s, "--version") == 0) return PARSE_VERSION;
    else if (strcmp(s, "--dump") == 0) a->dump = true;
    else if (strcmp(s, "--ops") == 0) a->show_ops = true;
    else if (strcmp(s, "--heuristics") == 0) a->heuristics = true;
    else if (strcmp(s, "--dynamic") == 0) a->dynamic = true;
    else if (strcmp(s, "--no-follow-chain") == 0) a->follow_chain = false;
    else if (strcmp(s, "--strict-abi") == 0) a->strict_abi = true;
    else if (strcmp(s, "--limit") == 0 && i + 1 < argc) {
        long value = 0;
        if (!parse_long_range(argv[++i], 0, INT_MAX, &value)) {
            fprintf(stderr, "Invalid value for --limit: %s (expected 0..%d)\n", argv[i],
            INT_MAX);
            return PARSE_ERR;
        }
        a->dump_limit = (int)value;
    }
    else if (strcmp(s, "--max-chain-depth") == 0 && i + 1 < argc) {
        long value = 0;
        if (!parse_long_range(argv[++i], 1, 16, &value)) {
            fprintf(stderr, "Invalid value for --max-chain-depth: %s (expected 1..16)\n",
            argv[i]);
            return PARSE_ERR;
        }
        a->max_chain_depth = (int)value;
```

Какие элементы C здесь работают

Работают указатели на `char`, вызов функции стандартной библиотеки, нулевой байт строки, результат сравнения: 0 означает равенство текстов.

Почему это нужно именно этому проекту

Все CLI-опции проекта распознаются через сравнение текста: `--help`, `--version`, `--dump`, `--json`, `--dynamic`. Без корректного сравнения интерфейс командной строки не работает.

Как читать такие строки

Читай `strcmp(s, "--json") == 0` как “текст s точно равен строке --json”. Не переводи это как “s истинно”.

Что ломается при непонимании

Если сравнить `argv[i] == "--json"`, может случайно не сработать даже при одинаковом тексте. Если передать в `strcmp` NULL, поведение будет ошибочным.

Практическое задание

Выпиши все опции, которые разбирает `parse_args`, и сгруппируй их на флаги, опции с числом и опции с путем.

Учебный модуль 5. strtol и проверка числовых опций

Сначала смысл

Командная строка передаёт числа как текст. Чтобы получить число C, текст нужно разобрать, проверить отсутствие мусорных символов, переполнение и диапазон.

Фрагмент из проекта

```
static bool parse_long_range(const char* text, long min_value, long max_value, long* out) {
    if (!text || !*text || !out) return false;
    errno = 0;
    char* end = NULL;
    long value = strtol(text, &end, 10);
    if (errno == ERANGE || end == text || *end != '\0') return false;
    if (value < min_value || value > max_value) return false;
    *out = value;
    return true;
}

static bool parse_u32_range(const char* text, uint32_t min_value, uint32_t max_value, uint32_t* out)
     {
    if (!text || !*text || !out || text[0] == '-') return false;
    errno = 0;
    char* end = NULL;
    unsigned long value = strtoul(text, &end, 10);
    if (errno == ERANGE || end == text || *end != '\0') return false;
    if (value < (unsigned long)min_value || value > (unsigned long)max_value) return false;
    *out = (uint32_t)value;
    return true;
}

static ParseResult parse_args(int argc, char** argv, Args* a) {
    memset(a, 0, sizeof(*a));
    a->dump_limit = -1;
    a->follow_chain = true;
    a->max_chain_depth = 8;
    a->timeout_ms = 15000;
    a->trace_steps = 64;

    if (argc < 2) return PARSE_ERR;
    if (strcmp(argv[1], "--help") == 0 || strcmp(argv[1], "-h") == 0) return PARSE_HELP;
    if (strcmp(argv[1], "--version") == 0) return PARSE_VERSION;
    a->input = argv[1];
```

Какие элементы C здесь работают

Работают `errno`, `strtol`, указатель `end`, сравнение границ, выходной указатель `long* out`. Это классический C-паттерн: функция возвращает `bool`, а результат пишет через pointer.

Почему это нужно именно этому проекту

Проект применяет это для `--limit` и `--max-chain-depth`, чтобы пользователь не мог передать отрицательное, слишком большое или текстовое значение там, где ожидается диапазон.

Как читать такие строки

Читай функцию сверху вниз: проверка пустого текста, сброс errno, вызов `strtol`, проверка результата разбора, проверка диапазона, запись в `out`.

Что ломается при непонимании

Если просто вызвать `atoi`, нельзя нормально отличить ошибку от значения 0. Если не проверить `end`, строка `123abc` может быть принята как 123.

Практическое задание

Напиши аналог для `uint32_t`, который отклоняет отрицательные строки.

Учебный модуль 6. `uint32_t`, `size_t` и границы числовых миров

Сначала смысл

В проекте есть два числовых мира. Первый - поля PE-формата фиксированной ширины, например RVA как 32-битное значение. Второй - размеры памяти текущего процесса, для которых используется `size_t`.

Фрагмент из проекта

```
typedef struct PeImage {
    bool     ok;
    bool     is_pe64;
    uint16_t machine;
    uint64_t image_base;
    uint32_t size_of_image;
    uint32_t size_of_headers;
    uint32_t section_alignment;
    uint32_t file_alignment;
    uint32_t exception_rva;
    uint32_t exception_size;
    size_t   file_size;
    size_t   num_sections;
    PeSection* sections; /* owned */

    size_t section_raw_oob;
    size_t section_virtual_oob;
    size_t section_raw_overlaps;
    size_t section_virtual_overlaps;
    size_t section_misaligned;
    bool   exception_directory_oob;
} PeImage;

bool pe_parse(PeImage* out, const Buf* b, StrBuf* err);
void pe_free(PeImage* img);

/* Map RVA -> file offset. Returns false if not mappable. */
bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off);

/* Find section containing RVA (or NULL). */
const PeSection* pe_find_section_for_rva(const PeImage* img, uint32_t rva);

#ifdef __cplusplus
}
```

Какие элементы C здесь работают

Работают fixed-width integer types из `stdint.h`, машинный тип размера `size_t`, явные casts и проверки перед приведением. Нужно понимать, где число описывает файл, а где память процесса.

Почему это нужно именно этому проекту

Если поле документации PE имеет 4 байта, код использует `uint32_t`. Если код считает размер выделенного массива или файла в памяти, он использует `size_t`.

Как читать такие строки

Когда видишь число, спрашивай: это поле PE, offset в файле, длина буфера, количество элементов или адрес? Тип должен соответствовать ответу.

Что ломается при непонимании

Смешивание этих типов без проверок может привести к переполнению, усечению или неверному сравнению. Особенно опасны переходы `size_t -> uint32_t` и сложение RVA с размером.

Практическое задание

Найди в `PeImage` поля `uint32_t`, `uint64_t`, `size_t` и объясни назначение каждого.

Учебный модуль 7. `Buf`: файл как неизменяемый источник bytes

Сначала смысл

Чтобы анализировать PE, проект сначала создаёт простую модель: указатель на bytes и размер. Эта модель не знает PE, секций или unwind. Она знает только непрерывную область памяти.

Фрагмент из проекта

```
typedef struct Buf {
    const uint8_t* data;
    size_t size;
} Buf;

bool buf_init_from_file(Buf* out, const char* path, uint8_t** owned_mem);
void buf_free_owned(uint8_t* owned_mem);

bool buf_read(const Buf* b, size_t off, void* out, size_t len);
bool buf_read_u8 (const Buf* b, size_t off, uint8_t*  out);
bool buf_read_u16(const Buf* b, size_t off, uint16_t* out);
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out);
bool buf_read_u64(const Buf* b, size_t off, uint64_t* out);

static inline uint16_t le16(const uint8_t* p) {
    return (uint16_t)(p[0] | ((uint16_t)p[1] << 8));
}
static inline uint32_t le32(const uint8_t* p) {
    return (uint32_t)(p[0] | ((uint32_t)p[1] << 8) | ((uint32_t)p[2] << 16) | ((uint32_t)p[3] <<
        24));
}
static inline uint64_t le64(const uint8_t* p) {
    return (uint64_t)le32(p) | ((uint64_t)le32(p + 4) << 32);
}

typedef struct StrBuf {
    char*  s;
    size_t len;
    size_t cap;
    bool failed;
} StrBuf;

#if defined(__MINGW32__) && (defined(__GNUC__) || defined(__clang__))
#define EHUA_PRINTF_FORMAT(format_index, first_arg) \
    __attribute__((format(gnu_printf, format_index, first_arg)))
```

Какие элементы C здесь работают

Работают структура, указатель `const uint8_t*`, размер `size_t`, передача структуры по указателю. `const` показывает, что байты исходного файла не должны изменяться.

Почему это нужно именно этому проекту

Все парсеры выше доверяют `Buf` как источнику данных. Поэтому правильность `Buf` и функций чтения является фундаментом всего инструмента.

Как читать такие строки

Перед чтением любого поля спрашивай: какой offset, сколько bytes, что будет при обрыве файла?

Что ломается при непонимании

Если `Buf.size` неверен или `Buf.data` указывает не на полный файл, все дальнейшие проверки будут строиться на неправильном основании.

Практическое задание

Сделай схему: файл на диске -> heap memory -> `Buf` -> `buf_read_u32` -> поле структуры.

Учебный модуль 8. buf_read: главная проверка границ

Сначала смысл

Эта функция - базовый “сторож” парсера. Она не знает, что такое PE, но она знает, можно ли безопасно скопировать N bytes с данного offset.

Фрагмент из проекта

```
bool buf_read(const Buf* b, size_t off, void* out, size_t len) {
    if (!b || !out) return false;
    if (off > b->size) return false;
    if (len > b->size - off) return false;
    memcpy(out, b->data + off, len);
    return true;
}

bool buf_read_u8(const Buf* b, size_t off, uint8_t* out) {
    return buf_read(b, off, out, 1);
}
bool buf_read_u16(const Buf* b, size_t off, uint16_t* out) {
    uint8_t tmp[2];
    if (!buf_read(b, off, tmp, 2)) return false;
    *out = le16(tmp);
    return true;
}
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out) {
    uint8_t tmp[4];
    if (!buf_read(b, off, tmp, 4)) return false;
    *out = le32(tmp);
    return true;
}
bool buf_read_u64(const Buf* b, size_t off, uint64_t* out) {
    uint8_t tmp[8];
    if (!buf_read(b, off, tmp, 8)) return false;
    *out = le64(tmp);
    return true;
}

static bool sb_reserve(StrBuf* sb, size_t need_cap) {
    if (!sb || sb->failed) return false;
    if (need_cap <= sb->cap) return true;
    size_t newcap = sb->cap ? sb->cap : 128;
```

Какие элементы C здесь работают

Работают указатели, `size_t`, условные проверки, арифметика без переполнения, `memcpy`, возврат `bool`.

Почему это нужно именно этому проекту

Каждый слой проекта вызывает функции чтения, которые в итоге опираются на `buf_read`. Поэтому одна аккуратная проверка защищает много кода выше.

Как читать такие строки

Читай `buf_read` как доказательство: входные указатели существуют, start offset допустим, length помещается, только после этого copy.

Что ломается при непонимании

Если проверка будет неполной, поврежденный файл сможет заставить parser читать за пределами памяти. Если проверка будет слишком слабой, отчет может строиться на мусорных bytes.

Практическое задание

Докажи письменно, почему `len > b->size - off` не переполняется после проверки `off > b->size`.

Учебный модуль 9. Little-endian helpers: чтение чисел из bytes

Сначала смысл

PE хранит числовые поля в little-endian порядке. Нельзя читать bytes как текст и нельзя бездумно приводить указатель к `uint32_t*`.

Фрагмент из проекта

```
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out) {
    uint8_t tmp[4];
    if (!buf_read(b, off, tmp, 4)) return false;
    *out = le32(tmp);
    return true;
}
bool buf_read_u64(const Buf* b, size_t off, uint64_t* out) {
    uint8_t tmp[8];
    if (!buf_read(b, off, tmp, 8)) return false;
    *out = le64(tmp);
    return true;
}

static bool sb_reserve(StrBuf* sb, size_t need_cap) {
    if (!sb || sb->failed) return false;
    if (need_cap <= sb->cap) return true;
    size_t newcap = sb->cap ? sb->cap : 128;
    while (newcap < need_cap) {
        size_t increment = newcap / 2u + 64u;
        if (increment > SIZE_MAX - newcap) {
            newcap = need_cap;
            break;
        }
        newcap += increment;
    }
    char* p = (char*)realloc(sb->s, newcap);
    if (!p) {
        sb->failed = true;
        return false;
    }
    sb->s = p;
    sb->cap = newcap;
    return true;
}
```

Какие элементы C здесь работают

Работают локальный массив `uint8_t tmp[4]`, вызов `buf_read`, функция `le32`, запись результата через pointer.

Почему это нужно именно этому проекту

Благодаря этому проект одинаково читает PE-поля независимо от alignment и особенностей платформы, на которой запущен статический анализ.

Как читать такие строки

Разбирай чтение числа как две операции: сначала безопасно получить bytes, потом интерпретировать их по формату.

Что ломается при непонимании

Если перепутать endian, PE, RVA и размеры станут другими числами. Это приведёт к ложным OOB, неверным секциям и сломанному unwind parsing.

Практическое задание

Возьми bytes `00 40 00 00` и вручную получи `0x00004000`.

Учебный модуль 10. StrBuf: динамическая строка без магии

Сначала смысл

В C строка не растёт сама. Если отчет строится постепенно, нужен объект, который хранит pointer, length, capacity и состояние ошибки.

Фрагмент из проекта

```
bool sb_init(StrBuf* sb, size_t initial_cap) {
    if (!sb) return false;
    sb->s = NULL;
    sb->len = 0;
    sb->cap = 0;
    sb->failed = false;
    if (initial_cap == 0) initial_cap = 256;
    if (!sb_reserve(sb, initial_cap) || !sb->s) {
        sb->failed = true;
        return false;
    }
    sb->s[0] = 0;
    return true;
}

void sb_free(StrBuf* sb) {
    if (!sb) return;
    free(sb->s);
    sb->s = NULL;
    sb->len = 0;
    sb->cap = 0;
    sb->failed = false;
}

bool sb_appendn(StrBuf* sb, const char* s, size_t n) {
    if (!sb) return false;
    if (sb->failed || (!s && n != 0)) {
        sb->failed = true;
        return false;
    }
    if (sb->len == SIZE_MAX || n > SIZE_MAX - sb->len - 1u) {
        sb->failed = true;
        return false;
    }
```

Какие элементы C здесь работают

Работают heap allocation, структура состояния, invariant `len < cap`, завершающий нулевой байт, функции append/free.

Почему это нужно именно этому проекту

Проект использует `StrBuf` для ошибок, details, JSON и HTML. Это единый способ безопасно накапливать текст.

Как читать такие строки

Читай операции `StrBuf` как управление ресурсом: init создаёт, append меняет, free завершает жизненный цикл.

Что ломается при непонимании

Если забыть поддерживать завершающий ноль, строковые функции будут читать дальше буфера. Если неверно считать capacity, append может выйти за память.

Практическое задание

Реализуй `sb_append_char` и объясни, какие проверки она должна сделать.

Учебный модуль 11. realloc и рост буфера

Сначала смысл

Рост динамического буфера - опасное место: нужно не потерять старую память при ошибке, не переполнить capacity и сохранить уже записанный текст.

Фрагмент из проекта

```
static bool sb_reserve(StrBuf* sb, size_t need_cap) {
    if (!sb || sb->failed) return false;
    if (need_cap <= sb->cap) return true;
    size_t newcap = sb->cap ? sb->cap : 128;
    while (newcap < need_cap) {
        size_t increment = newcap / 2u + 64u;
        if (increment > SIZE_MAX - newcap) {
            newcap = need_cap;
            break;
        }
        newcap += increment;
    }
    char* p = (char*)realloc(sb->s, newcap);
    if (!p) {
        sb->failed = true;
        return false;
    }
    sb->s = p;
    sb->cap = newcap;
    return true;
}

bool sb_init(StrBuf* sb, size_t initial_cap) {
    if (!sb) return false;
    sb->s = NULL;
    sb->len = 0;
    sb->cap = 0;
    sb->failed = false;
    if (initial_cap == 0) initial_cap = 256;
    if (!sb_reserve(sb, initial_cap) || !sb->s) {
        sb->failed = true;
        return false;
    }
    sb->s[0] = 0;
```

Какие элементы C здесь работают

Работают цикл роста capacity, проверка SIZE_MAX, realloc, присваивание нового указателя только после успеха.

Почему это нужно именно этому проекту

Без sb_reserve каждый append должен был бы вручную проверять место. Проект выносит эту ответственность в одну функцию.

Как читать такие строки

Ищи invariant: после успешного reserve need_cap <= sb->cap; после ошибки failed = true.

Что ломается при непонимании

Если присвоить результат realloc прямо в sb->s без временной переменной, при ошибке можно потерять адрес старого буфера и получить утечку.

Практическое задание

Объясни, почему capacity растёт не на один байт, а более крупными шагами.

Учебный модуль 12. `file_write_atomic`: отчёт не должен записываться наполовину

Сначала смысл

Запись результата тоже может оборваться. Atomic write снижает риск оставить на месте итогового отчета частично записанный файл.

Фрагмент из проекта

```
bool file_write_atomic(const char* path, const char* data, size_t len) {
    if (!path || (!data && len != 0)) return false;
    size_t path_len = strlen(path);
    if (path_len > SIZE_MAX - 48u) return false;

    size_t temp_size = path_len + 48u;
    char* temp_path = (char*)malloc(temp_size);
    if (!temp_path) return false;
#if defined(_WIN32)
    unsigned long process_id = (unsigned long)GetCurrentProcessId();
#else
    unsigned long process_id = (unsigned long)getpid();
#endif
    int written = snprintf(temp_path, temp_size, "%s.tmp.%lu", path, process_id);
    if (written < 0 || (size_t)written >= temp_size) {
        free(temp_path);
        return false;
    }

    FILE* f = fopen(temp_path, "wb");
    if (!f) {
        free(temp_path);
        return false;
    }
    size_t wr = len ? fwrite(data, 1, len, f) : 0u;
    bool ok = wr == len && ferror(f) == 0 && fflush(f) == 0;
    if (fclose(f) != 0) ok = false;

    if (ok) {
#if defined(_WIN32)
        ok = MoveFileExA(
                 temp_path,
                 path,
                 MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRITE_THROUGH) != 0;
```

Какие элементы C здесь работают

Работают временный путь, `snprintf`, `fopen`, `fwrite`, `fflush`, `fclose`, `rename` или Windows `MoveFileExA`.

Почему это нужно именно этому проекту

JSON/HTML отчет может читать другой человек или backend. Частичный файл хуже явной ошибки, потому что выглядит как результат, но содержит обрыв.

Как читать такие строки

Читай эту функцию как транзакцию: создать temp, записать, flush, закрыть, заменить итоговый путь, удалить temp при ошибке.

Что ломается при непонимании

Если не проверить `fwrite` и `fclose`, можно сообщить “успешно”, хотя данные не попали на диск.

Практическое задание

Опиши, чем atomic write полезен при падении программы во время записи HTML.

Учебный модуль 13. PeImage: внутренняя карта PE-файла

Сначала смысл

После парсинга PE проекту нужна не россыпь локальных чисел, а объект, описывающий образ: ImageBase, SizeOfImage, alignments, Exception Directory, sections и структурные проблемы секций.

Фрагмент из проекта

```
typedef struct PeImage {
    bool     ok;
    bool     is_pe64;
    uint16_t machine;
    uint64_t image_base;
    uint32_t size_of_image;
    uint32_t size_of_headers;
    uint32_t section_alignment;
    uint32_t file_alignment;
    uint32_t exception_rva;
    uint32_t exception_size;
    size_t   file_size;
    size_t   num_sections;
    PeSection* sections; /* owned */

    size_t section_raw_oob;
    size_t section_virtual_oob;
    size_t section_raw_overlaps;
    size_t section_virtual_overlaps;
    size_t section_misaligned;
    bool   exception_directory_oob;
} PeImage;

bool pe_parse(PeImage* out, const Buf* b, StrBuf* err);
void pe_free(PeImage* img);

/* Map RVA -> file offset. Returns false if not mappable. */
bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off);

/* Find section containing RVA (or NULL). */
const PeSection* pe_find_section_for_rva(const PeImage* img, uint32_t rva);

#ifdef __cplusplus
}
```

Какие элементы C здесь работают

Работают структура с разными типами полей, owned pointer `PeSection* sections`, counters ошибок, bool-флаги.

Почему это нужно именно этому проекту

Эта структура передаётся в unwind parser, heuristics и report. Она является общей картой файла для всех слоёв.

Как читать такие строки

Читай `PeImage` сверху вниз: сначала нормальные поля образа, потом массив секций, потом диагностические counters.

Что ломается при непонимании

Если карта неполная или не освобождается через `pe_free`, проект либо ошибётся в анализе, либо утечёт память.

Практическое задание

Составь список полей `PeImage`, которые нужны для RVA -> file offset.

Учебный модуль 14. MZ и `e_lfanew`: первый мост к PE-заголовку

Сначала смысл

PE-файл начинается с DOS header. Поле `e_lfanew` по offset `0x3c` указывает, где находится PE header. Нельзя перейти туда, не проверив MZ и границы.

Фрагмент из проекта

```
static bool read_dos_lfanew(const Buf* b, uint32_t* out_lfanew) {
    uint16_t mz = 0;
    if (!buf_read_u16(b, 0, &mz)) return false;
    if (mz != PE_DOS_MAGIC) return false;
    uint32_t lfanew = 0;
    if (!buf_read_u32(b, 0x3c, &lfanew)) return false;
    *out_lfanew = lfanew;
    return true;
}

static bool safe_add_u32(uint32_t a, uint32_t b, uint32_t* out) {
    uint64_t t = (uint64_t)a + (uint64_t)b;
    if (t > 0xffffffffu) return false;
    *out = (uint32_t)t;
    return true;
}

static bool ranges_overlap_u64(uint64_t a_begin,
                               uint64_t a_size,
                               uint64_t b_begin,
                               uint64_t b_size) {
    if (a_size == 0 || b_size == 0) return false;
    return a_begin < b_begin + b_size && b_begin < a_begin + a_size;
}

bool pe_parse(PeImage* out, const Buf* b, StrBuf* err) {
    if (!out || !b || !err) return false;
    memset(out, 0, sizeof(*out));

    uint32_t lfanew = 0;
    if (!read_dos_lfanew(b, &lfanew)) {
        sb_append(err, "Not a valid MZ/PE file (DOS header check failed).\n");
        return false;
    }
```

Какие элементы C здесь работают

Работают `buf_read_u16`, `buf_read_u32`, сравнение magic, output pointer.

Почему это нужно именно этому проекту

Проект начинает PE parse именно с этого, потому что без корректного `e_lfanew` нельзя доверять расположению COFF/OptionalHeader.

Как читать такие строки

Читай функцию как три шага: прочитать MZ, сравнить, прочитать lfanew, вернуть его через `out_lfanew`.

Что ломается при непонимании

Если принять неверный `e_lfanew`, parser будет читать PE signature из произвольных bytes.

Практическое задание

Найди offset `0x3c` в описании DOS header и объясни, почему он читается как `uint32_t`.

Учебный модуль 15. Проверка Machine: почему только AMD64

Сначала смысл

Проект специализирован на PE x64 EH/Unwind metadata. Для x86 модель исключений и unwind устроена иначе, поэтому parser честно отклоняет неподдерживаемый Machine.

Фрагмент из проекта

```
if (machine != IMAGE_FILE_MACHINE_AMD64) {
    sb_appendf(err, "Unsupported Machine: 0x%04x (only x64/AMD64 supported).\n", machine);
    return false;
}

size_t opthdr = filehdr + 20;
if (opthdr + opt_size > b->size) {
    sb_append(err, "Optional header out of range.\n");
    return false;
}
if (opt_size < 0x70u) {
    sb_append(err, "PE32+ optional header is too small for required fields.\n");
    return false;
}

uint16_t magic = 0;
if (!buf_read_u16(b, opthdr + 0, &magic)) {
    sb_append(err, "Failed to read optional header magic.\n");
    return false;
}
if (magic != IMAGE_NT_OPTIONAL_HDR64_MAGIC) {
    sb_appendf(err, "Unsupported OptionalHeader magic: 0x%04x (expect PE32+ 0x20b).\n", magic);
    return false;
}

uint64_t image_base = 0;
uint32_t size_of_image = 0;
uint32_t size_of_headers = 0;
uint32_t section_alignment = 0;
uint32_t file_alignment = 0;
uint32_t number_of_rva_and_sizes = 0;

/* PE32+ offsets */
if (!buf_read_u64(b, opthdr + 0x18, &image_base) ||
```

Какие элементы C здесь работают

Работают константы, `uint16_t machine`, сравнение, сообщение ошибки через `StrBuf`.

Почему это нужно именно этому проекту

Так инструмент не делает вид, что анализирует формат, которого не понимает. Это снижает риск ложных выводов.

Как читать такие строки

Когда видишь проверку формата, спрашивай: это ограничение реализации или требование предметной области?

Что ломается при непонимании

Если не проверить Machine, x86 бинарник может быть разобран как x64, и все выводы по `.pdata/.xdata` будут методически неверными.

Практическое задание

Сформулируй ответ комиссии: почему x64 является осознанным ограничением, а не багом.

Учебный модуль 16. Section table: чтение массива структур из файла

Сначала смысл

Секции PE идут массивом заголовков фиксированного размера. Проект читает каждую секцию в свою `PeSection`, но не накладывает C-структуру напрямую на bytes.

Фрагмент из проекта

```
for (uint16_t i = 0; i < nsects; i++) {
    size_t sh = secthdr + (size_t)i * 40;
    uint8_t name8[8];
    if (!buf_read(b, sh + 0, name8, 8)) { free(secs); sb_append(err, "Failed to read section
        name.\n"); return false; }
    memcpy(secs[i].name, name8, 8);
    secs[i].name[8] = 0;

    if (!buf_read_u32(b, sh + 8, &secs[i].virtual_size) ||
        !buf_read_u32(b, sh + 12, &secs[i].virtual_address) ||
        !buf_read_u32(b, sh + 16, &secs[i].raw_size) ||
        !buf_read_u32(b, sh + 20, &secs[i].raw_ptr) ||
        !buf_read_u32(b, sh + 36, &secs[i].characteristics)) {
        free(secs);
        sb_append(err, "Failed to read section header.\n");
        return false;
    }
}

size_t raw_oob = 0;
size_t virtual_oob = 0;
size_t raw_overlaps = 0;
size_t virtual_overlaps = 0;
size_t misaligned = 0;

bool compact_alignment =
    section_alignment < 0x1000u && file_alignment == section_alignment;
bool file_alignment_valid =
    is_power_of_two_u32(file_alignment) &&
    ((file_alignment >= 0x200u && file_alignment <= 0x10000u) ||
     compact_alignment);
if (!file_alignment_valid) misaligned++;
if (section_alignment < file_alignment) misaligned++;
if (section_alignment < 0x1000u && file_alignment != section_alignment) {
    misaligned++;
```

Какие элементы C здесь работают

Работают цикл `for`, индекс, вычисление offset `secthdr + i * 40`, локальный массив имени, безопасные чтения полей.

Почему это нужно именно этому проекту

Секции нужны для поиска RVA, определения permissions и проверки размещения `.pdata/.xdata/handler`.

Как читать такие строки

Читай цикл как повторение одного доказанного шаблона: offset заголовка, name, fields, запись в массив.

Что ломается при непонимании

Если неверно прочитать секции, последующий анализ будет путать код, metadata и raw bytes.

Практическое задание

Объясни, почему размер section header равен 40 bytes и где в коде это используется.

Учебный модуль 17. Alignment checks: когда формат требует кратности

Сначала смысл

PE имеет требования к выравниванию секций, заголовков, ImageBase и размеров. Проект фиксирует нарушения как структурные признаки.

Фрагмент из проекта

```
bool compact_alignment =
    section_alignment < 0x1000u && file_alignment == section_alignment;
bool file_alignment_valid =
    is_power_of_two_u32(file_alignment) &&
    ((file_alignment >= 0x200u && file_alignment <= 0x10000u) ||
     compact_alignment);
if (!file_alignment_valid) misaligned++;
if (section_alignment < file_alignment) misaligned++;
if (section_alignment < 0x1000u && file_alignment != section_alignment) {
    misaligned++;
}
if ((size_of_image % section_alignment) != 0) misaligned++;
if ((size_of_headers % file_alignment) != 0 ||
    (uint64_t)size_of_headers < (uint64_t)needed) {
    misaligned++;
}
if ((image_base & 0xffffu) != 0) misaligned++;

for (size_t i = 0; i < (size_t)nsects; i++) {
    const PeSection* s = &secs[i];
    uint64_t raw_end = (uint64_t)s->raw_ptr + (uint64_t)s->raw_size;
    uint64_t virtual_span = (uint64_t)u32_max(s->virtual_size, s->raw_size);
    uint64_t virtual_end = (uint64_t)s->virtual_address + virtual_span;
    if (s->raw_size > 0 && raw_end > (uint64_t)b->size) raw_oob++;
    if (virtual_span > 0 && virtual_end > (uint64_t)size_of_image) virtual_oob++;
    if ((s->virtual_address % section_alignment) != 0 ||
        (s->raw_size > 0 &&
         ((s->raw_ptr % file_alignment) != 0 ||
          (s->raw_size % file_alignment) != 0))) {
        misaligned++;
    }
    if (i > 0 && s->virtual_address <= secs[i - 1].virtual_address) {
        misaligned++;
    }
```

Какие элементы C здесь работают

Работают арифметика остатка %, helper `is_power_of_two_u32`, counters misaligned.

Почему это нужно именно этому проекту

Такие проверки помогают отличить нормальный compiler output от поврежденной или нестандартной PE-разметки.

Как читать такие строки

Читай alignment check как проверку инварианта формата, а не как эстетическое требование.

Что ломается при непонимании

Если игнорировать alignment, часть поврежденных файлов может пройти как нормальная карта, а потом сломать RVA mapping.

Практическое задание

Выпиши все условия, которые увеличивают `misaligned`.

Учебный модуль 18. Overlaps: пересечение raw и virtual диапазонов

Сначала смысл

Два диапазона пересекаются, если начало каждого лежит раньше конца другого. Проект проверяет raw и virtual overlaps секций.

Фрагмент из проекта

```
static bool ranges_overlap_u64(uint64_t a_begin,
                               uint64_t a_size,
                               uint64_t b_begin,
                               uint64_t b_size) {
    if (a_size == 0 || b_size == 0) return false;
    return a_begin < b_begin + b_size && b_begin < a_begin + a_size;
}

bool pe_parse(PeImage* out, const Buf* b, StrBuf* err) {
    if (!out || !b || !err) return false;
    memset(out, 0, sizeof(*out));

    uint32_t lfanew = 0;
    if (!read_dos_lfanew(b, &lfanew)) {
        sb_append(err, "Not a valid MZ/PE file (DOS header check failed).\n");
        return false;
    }
    if ((size_t)lfanew + 4 + 20 > b->size) {
        sb_append(err, "PE header offset out of range.\n");
        return false;
    }

    uint32_t sig = 0;
    if (!buf_read_u32(b, (size_t)lfanew, &sig) || sig != PE_NT_MAGIC) {
        sb_append(err, "Invalid PE signature.\n");
        return false;
    }

    size_t filehdr = (size_t)lfanew + 4;
    uint16_t machine = 0;
    uint16_t nsects = 0;
    uint16_t opt_size = 0;
    if (!buf_read_u16(b, filehdr + 0, &machine) ||
        !buf_read_u16(b, filehdr + 2, &nsects) ||
```

Какие элементы C здесь работают

Работают `uint64_t`, защита от нулевых размеров, логическое `&&`, сравнение интервалов.

Почему это нужно именно этому проекту

Пересечение секций делает ownership bytes/RVA неоднозначным. Для анализатора это снижает доверие к карте.

Как читать такие строки

Читай диапазоны как полуинтервалы `[begin, begin+size)`, где конец не включается.

Что ломается при непонимании

Если один RVA относится к двум секциям, невозможно однозначно сказать, где находится metadata или handler.

Практическое задание

Нарисуй два диапазона и проверь формулу overlap вручную.

Учебный модуль 19. RVA mapping: центральная операция PE-анализа

Сначала смысл

RVA - это адрес внутри image, а file offset - место в файле на диске. Проект должен переводить одно в другое через section table.

Фрагмент из проекта

```
bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off) {
    if (!img || !out_off) return false;
    const PeSection* s = pe_find_section_for_rva(img, rva);
    if (s) {
        uint32_t delta = rva - s->virtual_address;
        if (delta >= s->raw_size) {
            /* RVA points into virtual tail (zero-fill) - not present in file */
            return false;
        }
        uint32_t off = 0;
        if (!safe_add_u32(s->raw_ptr, delta, &off)) return false;
        if (img->file_size > 0 && (uint64_t)off >= (uint64_t)img->file_size) return false;
        *out_off = off;
        return true;
    }

    /* RVAs inside SizeOfHeaders map directly to the corresponding file offset. */
    if (img->size_of_headers > 0 &&
        rva < img->size_of_headers &&
        (img->file_size == 0 || (uint64_t)rva < (uint64_t)img->file_size)) {
        *out_off = rva;
        return true;
    }
    return false;
}
```

Какие элементы C здесь работают

Работают поиск секции, вычисление delta, проверка raw size, безопасное сложение, output pointer.

Почему это нужно именно этому проекту

Без этой функции нельзя прочитать `UNWIND_INFO`, потому что `UnwindInfoAddress` хранится как RVA, а bytes лежат в файле по raw offset.

Как читать такие строки

Читай mapping как цепочку: RVA -> section -> delta -> raw_ptr + delta -> file offset.

Что ломается при непонимании

Главная ошибка - использовать RVA как file offset напрямую. На реальных PE это часто даст неверные bytes.

Практическое задание

Возьми условную секцию VA=0x4000 raw_ptr=0x1000 и RVA=0x4020. Посчитай file offset.

Учебный модуль 20. `RuntimeFunction** out`: возврат выделенного массива

Сначала смысл

Функция читает таблицу неизвестной заранее длины. Она должна выделить массив и вернуть его вызывающему коду. Для изменения указателя вызывающего нужен двойной pointer.

Фрагмент из проекта

```
RuntimeFunctionReadResult unwind_read_runtime_functions_ex(
                                  const Buf* b,
                                  const PeImage* img,
                                  RuntimeFunction** out_rf,
                                  size_t* out_count,
                                  StrBuf* err) {
    if (!b || !img || !out_rf || !out_count || !err) {
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    *out_rf = NULL;
    *out_count = 0;

    if (img->exception_rva == 0 && img->exception_size == 0) {
        return RUNTIME_FUNCTION_READ_OK;
    }
    if (img->exception_rva == 0 || img->exception_size == 0) {
        sb_append(err, "Malformed exception directory: RVA and size must both be zero or non-
            zero.\n");
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    uint32_t off = 0;
    if (!pe_rva_to_fileoff(img, img->exception_rva, &off)) {
        sb_appendf(err, "Exception directory RVA 0x%08x not mappable.\n", img->exception_rva);
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    if ((size_t)off + img->exception_size > b->size) {
        sb_append(err, "Exception directory out of file range.\n");
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    if (img->exception_size < 12) {
        sb_append(err, "Exception directory size too small.\n");
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    if ((img->exception_size % 12u) != 0) {
        sb_append(err, "Exception directory size is not a multiple of RUNTIME_FUNCTION size.\n");
```

Какие элементы C здесь работают

Работают `RuntimeFunction**`, `size_t*`, `malloc/calloc`, проверка overflow, enum результата.

Почему это нужно именно этому проекту

В `main.c` массив `rf` после вызова становится владельцем данных и должен быть освобожден через `free(rf)`.

Как читать такие строки

Читай `*out = arr` как запись нового адреса в переменную вызывающего кода.

Что ломается при непонимании

Если неправильно понять двойной указатель, можно выделить память внутри функции и потерять адрес после возврата.

Практическое задание

Сделай маленький пример `make_array(int** out, size_t* n)`.

Учебный модуль 21. `slots_for_code`: форматная таблица в виде switch

Сначала смысл

Каждый UWOP имеет правило размера. `switch` превращает номер операции в количество slots и одновременно проверяет допустимость operands.

Фрагмент из проекта

```
static bool slots_for_code(uint8_t version,
                           uint8_t op,
                           uint8_t opinfo,
                           bool v2_seen_epilog,
                           uint8_t* out_slots) {
    /* based on documented v1 ops + v2 epilog/spare, and common implementations */
    switch (op) {
        case 0: /* UWOP_PUSH_NONVOL */ *out_slots = 1; return true;
        case 1: /* UWOP_ALLOC_LARGE */ if (opinfo > 1) return false; *out_slots = (uint8_t)(2 +
            opinfo); return true;
        case 2: /* UWOP_ALLOC_SMALL */ *out_slots = 1; return true;
        case 3: /* UWOP_SET_FPREG */   *out_slots = 1; return true;
        case 4: /* UWOP_SAVE_NONVOL */ *out_slots = 2; return true;
        case 5: /* UWOP_SAVE_NONVOL_FAR */ *out_slots = 3; return true;
        case 6: /* v2 UWOP_EPILOG, v1 UWOP_SAVE_XMM (undoc) */
            if (version == 2) {
                /* The first long-form epilog descriptor has one operand slot.
                   Short-form and subsequent descriptors occupy one slot. */
                *out_slots = (!v2_seen_epilog && (opinfo & 1u) == 0u) ? 2u : 1u;
                return true;
            }
            *out_slots = 2; return true;
        case 7: /* v2 UWOP_SPARE, v1 UWOP_SAVE_XMM_FAR (undoc) */
            if (version == 2) { *out_slots = 1; return true; }
            *out_slots = 3; return true;
        case 8: /* UWOP_SAVE_XMM128 */ *out_slots = 2; return true;
        case 9: /* UWOP_SAVE_XMM128_FAR */ *out_slots = 3; return true;
        case 10: /* UWOP_PUSH_MACHFRAME */ *out_slots = 1; return true;
        default:
            return false;
    }
}

static bool parse_unwindinfo_header(const Buf* b, const PeImage* img, uint32_t unwind_rva,
                                    uint32_t* out_fileoff, uint8_t hdr[4], StrBuf* err) {
```

Какие элементы C здесь работают

Работают `switch/case`, output pointer, `uint8_t`, return bool, comments с предметным смыслом.

Почему это нужно именно этому проекту

Это место защищает parser от наивной ошибки “один UWOP - один slot”. Проект знает, что некоторые операции занимают дополнительные operand slots.

Как читать такие строки

Читай каждый `case` как строку спецификации: opcode -> сколько slots -> допустимый opinfo.

Что ломается при непонимании

Если slots посчитаны неверно, tail `UNWIND_INFO` будет прочитан со смещением и handler/chain станет мусором.

Практическое задание

Выпиши все UWOP, которые занимают 3 slots.

Учебный модуль 22. Header UNWIND_INFO: битовые поля в одном байте

Сначала смысл

Первый byte UNWIND_INFO содержит version в младших битах и flags в старших. C извлекает их битовыми операциями.

Фрагмент из проекта

```
uint8_t ver = (uint8_t)(hdr[0] & 0x7);
uint8_t flags = (uint8_t)(hdr[0] >> 3);
uint8_t prolog = hdr[1];
uint8_t count = hdr[2];
uint8_t fr = (uint8_t)(hdr[3] & 0x0f);
uint8_t foff = (uint8_t)(hdr[3] >> 4);

out->version = ver;
out->flags = flags;
out->prolog_size = prolog;
out->count_codes = count;
out->frame_reg = fr;
out->frame_off = foff;

if (!(ver == 1 || ver == 2)) {
    out->anomalies |= UA_VERSION_UNKNOWN;
    unwind_diag_set(out, b, "Version", true, (size_t)uoff, true, 1u, 1u, false, 0, 0);
    return unwind_parse_fail(out, err, err_start, "Unsupported UNWIND_INFO version.");
}
if (flags & (uint8_t)~(UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER | UNW_FLAG_CHAININFO)) {
    out->anomalies |= UA_BAD_OPERAND;
}
if ((flags & UNW_FLAG_CHAININFO) && (flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER))) {
    out->anomalies |= UA_BAD_OPERAND;
}
if (fr == 0 && foff != 0) {
    out->anomalies |= UA_BAD_OPERAND;
}
if (fr != 0 && !is_unwindable_gpr(fr)) {
    out->anomalies |= UA_BAD_OPERAND;
}
if (strict_abi && fr != 0 && !is_abi_nonvolatile_gpr(fr)) {
    out->anomalies |= UA_ABI_REGISTER_CLASS;
}
```

Какие элементы C здесь работают

Работают &, >>, uint8_t, casts, маски. Это пример чтения packed binary layout.

Почему это нужно именно этому проекту

Проект должен точно разложить version/flags, потому что от flags зависит, есть ли handler или chain tail.

Как читать такие строки

Читай hdr[0] & 0x7 как “взять младшие три бита”, а hdr[0] >> 3 как “сдвинуть старшие биты вниз”.

Что ломается при непонимании

Если перепутать биты, parser может искать handler там, где его нет, или пропустить chaininfo.

Практическое задание

Возьми byte 0x19 и вычисли version/flags.

Учебный модуль 23. Padding after CountOfCodes: почему tail начинается не сразу

Сначала смысл

Массив `UNWIND_CODE` выравнивается до чётного числа slots. Если CountOfCodes нечётный, появляется padding slot.

Фрагмент из проекта

```
uint32_t slots_even = (uint32_t)((count + 1u) & ~1u);
size_t codes_off = (size_t)uoff + 4;
size_t codes_bytes = (size_t)slots_even * 2u;
size_t codes_available = available_until_limit(b, codes_off, record_limit);
if (codes_bytes > codes_available) {
    out->anomalies |= UA_TRUNCATED;
    unwind_diag_set(out, b, "CountOfCodes / UNWIND_CODE array",
                    true, codes_off, true, codes_bytes, codes_available,
                    true, slots_even, (uint32_t)(codes_available / 2u));
    sb_append(err, "Unwind codes array out of file range.\n");
    return unwind_parse_fail(out, err, err_start, "Unwind codes array out of file range.");
}

uint32_t fixed_stack = 0;
uint32_t pushed = 0;
uint16_t nonvol_mask = 0;
uint16_t xmm_mask = 0;
bool have_prev_code_offset = false;
uint8_t prev_code_offset = 0;
bool saw_set_fpreg = false;
bool saw_push_nonvol = false;
bool v2_seen_epilog = false;

/* parse slots 0..count-1 as unwind code starts */
for (uint32_t i = 0; i < count; ) {
    uint16_t slot = 0;
    size_t slot_fileoff = codes_off + (size_t)i * 2u;
    if (!read_u16_slot(b, slot_fileoff, &slot)) {
        out->anomalies |= UA_TRUNCATED;
        unwind_diag_set_read_failure(out, b, "UNWIND_CODE slot", slot_fileoff, 2u,
            record_limit);
        sb_append(err, "Failed to read unwind slot.\n");
        return unwind_parse_fail(out, err, err_start, "Failed to read unwind slot.");
    }
```

Какие элементы C здесь работают

Работают битовая маска, арифметика offset, понимание layout.

Почему это нужно именно этому проекту

Это один из самых важных нюансов проекта: ошибка на 2 bytes полностью ломает чтение handler RVA или chain info.

Как читать такие строки

Читай вычисление `after_codes` как итог layout: header 4 bytes + even slots * 2 bytes.

Что ломается при непонимании

Если padding не учесть, parser будет принимать padding за младшую часть handler RVA или читать chain с неверного места.

Практическое задание

Для CountOfCodes=1, 2, 3, 4 посчитай `slots_even` и offset tail от начала структуры.

Учебный модуль 24. Anomaly bitmask: несколько проблем в одном `UnwindSummary`

Сначала смысл

Одна структура unwind может иметь сразу несколько проблем: alignment, bad operand, truncated tail, chain loop. Маска позволяет хранить их вместе.

Фрагмент из проекта

```
enum UnwindAnom {
    UA_VERSION_UNKNOWN       = 1u << 0,
    UA_TRUNCATED             = 1u << 1,
    UA_BAD_OPCODE            = 1u << 2,
    UA_BAD_OPERAND           = 1u << 3,
    UA_CODEOFFSET_OUTSIDE    = 1u << 4,
    UA_CHAIN_LOOP            = 1u << 5,
    UA_CHAIN_DEPTH_EXCEEDED  = 1u << 6,
    UA_HANDLER_OOB           = 1u << 7,
    UA_CODEOFFSET_ORDER      = 1u << 8,
    UA_OPERAND_SANITY        = 1u << 9,
    UA_ALIGNMENT             = 1u << 10,
    UA_CHAIN_INVALID         = 1u << 11,
    UA_ABI_REGISTER_CLASS    = 1u << 12,
    UA_ABI_RESERVED_OPINFO   = 1u << 13,
    UA_ABI_PUSH_ORDER        = 1u << 14,
    UA_ABI_CHAIN_RESTRICTION = 1u << 15,
    UA_ABI_SAVE_OFFSET       = 1u << 16
};

typedef struct UnwindParseOpts {
    int  max_chain_depth; /* default 8 */
    bool follow_chain;    /* compute + annotate chain info */
    bool strict_abi;      /* flag formal Microsoft x64 unwind ABI violations */
} UnwindParseOpts;

typedef enum RuntimeFunctionReadResult {
    RUNTIME_FUNCTION_READ_OK = 0,
    RUNTIME_FUNCTION_READ_MALFORMED,
    RUNTIME_FUNCTION_READ_OOM
} RuntimeFunctionReadResult;

/* Parse UNWIND_INFO at unwind_rva. Produces summary + optional text of unwind ops. */
bool unwind_parse(const Buf* b,
```

Какие элементы C здесь работают

Работают enum constants, shift, OR assignment, проверка через bitwise AND.

Почему это нужно именно этому проекту

Проект передаёт эту маску в heuristics и report, где она превращается в counters и читаемые строки.

Как читать такие строки

Читай `anomalies` как набор включенных битов, а не как одно число severity.

Что ломается при непонимании

Если хранить только один код ошибки, parser потеряет вторичные признаки. Если хранить строку вместо маски, будет сложнее агрегировать статистику.

Практическое задание

Добавь новый флаг мысленно и объясни, где его нужно отобразить в `report.c`.

Учебный модуль 25. Handler tail: когда flags указывают на EHANDLER/UHANDLER

Сначала смысл

Если в flags стоит EHANDLER или UHANDLER, tail содержит RVA обработчика. Проект читает его после учёта slots и padding.

Фрагмент из проекта

```
if (flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER)) {
    out->has_handler = true;
    uint32_t handler_rva = 0;
    if (!buf_read_u32(b, after_codes + 0, &handler_rva)) {
        out->anomalies |= UA_TRUNCATED;
        unwind_diag_set_read_failure(out, b, "exception handler RVA tail", after_codes, 4u,
            record_limit);
        return unwind_parse_fail(out, err, err_start, "Truncated exception handler RVA.");
    } else {
        out->handler_rva = handler_rva;
        if (handler_rva >= img->size_of_image) {
            out->anomalies |= UA_HANDLER_OOB;
            unwind_diag_set(out, b, "exception handler RVA", true, after_codes,
                            true, 4u, 4u, false, 0, 0);
        }
    }
}

out->fixed_stack_bytes = fixed_stack;
out->pushed_nonvol_count = pushed;
out->saved_nonvol_mask = nonvol_mask;
out->saved_xmm_mask = xmm_mask;

/* follow chain (lightly): detect loops and depth. */
if (follow_chain && out->has_chain) {
    uint32_t seen[16]; /* depth max 16 */
    int seen_n = 0;
    out->chain_path[0] = unwind_rva;
    out->chain_path_count = 1;
    uint32_t cur = out->chain.unwind_rva;
    for (int depth = 0; depth < max_depth; depth++) {
        bool loop = false;
        for (int j = 0; j < seen_n; j++) if (seen[j] == cur) loop = true;
        if (out->chain_path_count < UNWIND_DIAG_MAX_CHAIN_PATH) {
            out->chain_path[out->chain_path_count++] = cur;
```

Какие элементы C здесь работают

Работают bitmask flags, `buf_read_u32`, запись `handler_rva`, проверка `handler_rva < SizeOfImage`.

Почему это нужно именно этому проекту

Handler RVA важен для оценки exception-based control flow и placement-признаков. Report показывает, куда ведёт handler.

Как читать такие строки

Читай этот блок только после понимания tail offset: handler читается не из произвольного места, а из точно вычисленного `after_codes`.

Что ломается при непонимании

Если handler RVA выходит за SizeOfImage, это серьёзный признак недостоверной metadata. Если handler лежит в writable/nonexec секции, это требует ручной проверки.

Практическое задание

Объясни, почему проект не парсит универсально language-specific handler data без знания handler-а.

Учебный модуль 26. CHAININFO: связанная unwind metadata и loops

Сначала смысл

CHAININFO означает, что tail содержит запись, похожую на `RUNTIME_FUNCTION`, указывающую на связанную unwind metadata.

Фрагмент из проекта

```
    if (flags & UNW_FLAG_CHAININFO){ strcat(out, first ? "CHAININFO" : "|CHAININFO"); first = false;
       }
    if (first) strcpy(out, "0");
}

static bool slots_for_code(uint8_t version,
                           uint8_t op,
                           uint8_t opinfo,
                           bool v2_seen_epilog,
                           uint8_t* out_slots) {
    /* based on documented v1 ops + v2 epilog/spare, and common implementations */
    switch (op) {
        case 0: /* UWOP_PUSH_NONVOL */ *out_slots = 1; return true;
        case 1: /* UWOP_ALLOC_LARGE */ if (opinfo > 1) return false; *out_slots = (uint8_t)(2 +
            opinfo); return true;
        case 2: /* UWOP_ALLOC_SMALL */ *out_slots = 1; return true;
        case 3: /* UWOP_SET_FPREG */   *out_slots = 1; return true;
        case 4: /* UWOP_SAVE_NONVOL */ *out_slots = 2; return true;
        case 5: /* UWOP_SAVE_NONVOL_FAR */ *out_slots = 3; return true;
        case 6: /* v2 UWOP_EPILOG, v1 UWOP_SAVE_XMM (undoc) */
            if (version == 2) {
                /* The first long-form epilog descriptor has one operand slot.
                   Short-form and subsequent descriptors occupy one slot. */
                *out_slots = (!v2_seen_epilog && (opinfo & 1u) == 0u) ? 2u : 1u;
                return true;
            }
            *out_slots = 2; return true;
        case 7: /* v2 UWOP_SPARE, v1 UWOP_SAVE_XMM_FAR (undoc) */
            if (version == 2) { *out_slots = 1; return true; }
            *out_slots = 3; return true;
        case 8: /* UWOP_SAVE_XMM128 */ *out_slots = 2; return true;
        case 9: /* UWOP_SAVE_XMM128_FAR */ *out_slots = 3; return true;
        case 10: /* UWOP_PUSH_MACHFRAME */ *out_slots = 1; return true;
        default:
            return false;
    }
```

Какие элементы C здесь работают

Работают структура `RuntimeFunction`, чтение трёх `uint32_t`, follow-chain loop, массив seen, ограничение depth.

Почему это нужно именно этому проекту

Проект может вычислять chain path и обнаруживать loop/depth exceeded. Это важно, потому что chain loop делает восстановление primary info ненадёжным.

Как читать такие строки

Читай chain как граф ссылок: текущий unwind RVA -> следующий unwind RVA -> ...; parser должен помнить посещённые nodes.

Что ломается при непонимании

Если follow chain без ограничения, поврежденный файл может создать бесконечный цикл анализа.

Практическое задание

Нарисуй цепочку из трех RVA и покажи, где возникает loop.

Учебный модуль 27. Diagnostics fields: evidence вместо “поверь мне”

Сначала смысл

Диагностика сохраняет конкретное место ошибки: поле, file offset, expected/available sizes, expected/available slots, bytes.

Фрагмент из проекта

```
static void unwind_diag_set(UnwindSummary* out,
                            const Buf* b,
                            const char* field,
                            bool has_file_offset,
                            size_t file_offset,
                            bool has_size,
                            size_t expected_size,
                            size_t available_size,
                            bool has_slots,
                            uint32_t expected_slots,
                            uint32_t available_slots) {
    if (!out) return;
    if (field && field[0]) {
        size_t n = strlen(field);
        if (n >= sizeof(out->diagnostic_field)) n = sizeof(out->diagnostic_field) - 1;
        memcpy(out->diagnostic_field, field, n);
        out->diagnostic_field[n] = '\0';
    }
    out->diagnostic_has_file_offset = has_file_offset;
    if (has_file_offset) out->diagnostic_file_offset = clamp_size_to_u32(file_offset);
    out->diagnostic_has_size = has_size;
    if (has_size) {
        out->diagnostic_expected_size = clamp_size_to_u32(expected_size);
        out->diagnostic_available_size = clamp_size_to_u32(available_size);
    }
    out->diagnostic_has_slots = has_slots;
    if (has_slots) {
        out->diagnostic_expected_slots = expected_slots;
        out->diagnostic_available_slots = available_slots;
    }
    if (has_file_offset) {
        size_t max_bytes = has_size ? available_size : (size_t)UNWIND_DIAG_MAX_BYTES;
        unwind_diag_capture_bytes(out, b, file_offset, max_bytes);
    }
```

Какие элементы C здесь работают

Работают фиксированные массивы char, bool has-value flags, копирование bytes, clamp size to uint32.

Почему это нужно именно этому проекту

Report использует эти поля, чтобы объяснить parse fail профессионально: где именно обрыв и сколько данных не хватило.

Как читать такие строки

Читай diagnostic structure как доказательство, которое передаётся из parser-а в report.

Что ломается при непонимании

Без diagnostics пользователь увидит только “UNWIND parse failed” и не сможет проверить вывод вручную.

Практическое задание

Смоделируй ошибку: CountOfCodes требует 4 bytes, available 2. Какие поля diagnostics должны заполниться?

Учебный модуль 28. Heuristic hash map: подсчёт reuse без готовой библиотеки

Сначала смысл

В C нет встроенного словаря. Проект реализует маленькую hash map для подсчёта повторов `UnwindInfoAddress` и handler RVA.

Фрагмент из проекта

```
typedef struct HMap {
    HEntry* e;
    size_t cap;
    size_t len;
} HMap;

static uint32_t h32(uint32_t x) {
    /* xorshift-ish */
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static bool hmap_init(HMap* m, size_t cap) {
    if (!m) return false;
    m->cap = cap < 64 ? 64 : cap;
    m->len = 0;
    m->e = (HEntry*)calloc(m->cap, sizeof(HEntry));
    return m->e != NULL;
}

static void hmap_free(HMap* m) {
    if (!m) return;
    free(m->e);
    m->e = NULL;
    m->cap = 0;
    m->len = 0;
}

static bool hmap_grow(HMap* m);
```

Какие элементы C здесь работают

Работают структуры `HEntry/HMap`, heap allocation, hash function, open addressing, linear probing, grow.

Почему это нужно именно этому проекту

Reuse является важным признаком: если один unwind RVA используется слишком многими функциями, это может быть compiler pattern или suspicious dispatcher-style metadata.

Как читать такие строки

Читай hash map как массив ячеек: каждая ячейка либо пустая, либо хранит key/count. Collision решается переходом к следующей ячейке.

Что ломается при непонимании

Если счетчик reuse ошибётся, score/confidence будут неверными. Если map не grow-ится, можно потерять данные на больших таблицах.

Практическое задание

Выпиши, что происходит при `hmap_inc`, если key уже существует.

Учебный модуль 29. `qsort` и comparator для диапазонов

Сначала смысл

Чтобы найти overlaps и duplicate begins, удобно отсортировать диапазоны по begin/end. C стандартная библиотека даёт `qsort`, но comparator нужно написать самому.

Фрагмент из проекта

```
static int cmp_rfidx(const void* a, const void* b) {
    const RfIdx* x = (const RfIdx*)a;
    const RfIdx* y = (const RfIdx*)b;
    if (x->begin < y->begin) return -1;
    if (x->begin > y->begin) return 1;
    if (x->end < y->end) return -1;
    if (x->end > y->end) return 1;
    return 0;
}

static int clamp_int(int v, int lo, int hi) {
    if (v < lo) return lo;
    if (v > hi) return hi;
    return v;
}

static void add_score(int* score, int add) {
    if (!score) return;
    *score = clamp_int(*score + add, 0, 100);
}

static double safe_ratio(double a, double b) { return (b == 0.0) ? 0.0 : (a / b); }


const char* heuristics_evidence_family_name(uint32_t bit) {
    switch (bit) {
        case HEF_TABLE_INTEGRITY: return "table_integrity";
        case HEF_UNWIND_INTEGRITY: return "unwind_integrity";
        case HEF_METADATA_PLACEMENT: return "metadata_placement";
        case HEF_UNWIND_REUSE: return "unwind_reuse";
        case HEF_HANDLER_DOMINANCE: return "handler_dominance";
        default: return "unknown";
    }
}
```

Какие элементы C здесь работают

Работают `const void*`, cast к нужному типу, возврат -1/0/1, порядок сравнения.

Почему это нужно именно этому проекту

После сортировки проект может линейно пройти function ranges и увидеть пересечения независимо от исходного порядка `.pdata`.

Как читать такие строки

Читай comparator как формализацию порядка: сначала begin, при равенстве end.

Что ломается при непонимании

Неверный comparator может нарушить сортировку и дать ложные overlaps или пропустить реальные.

Практическое задание

Напиши comparator для структуры `Range { uint32_t begin; uint32_t end; }`.

Учебный модуль 30. Score constants: инженерная калибровка, а не магия

Сначала смысл

Вес признака задаёт вклад в подозрительность. Это не ML-вес, выученный на датасете, а инженерная модель, основанная на важности нарушения.

Фрагмент из проекта

```
static const int SCORE_EMPTY_PDATA = 10;
static const int SCORE_PE_SECTION_RAW_OOB = 25;
static const int SCORE_PE_SECTION_VIRTUAL_OOB = 20;
static const int SCORE_PE_SECTION_RAW_OVERLAP = 15;
static const int SCORE_PE_SECTION_VIRTUAL_OVERLAP = 20;
static const int SCORE_PE_SECTION_MISALIGNED = 8;
static const int SCORE_PE_EXCEPTION_DIRECTORY_OOB = 25;
static const int SCORE_INVALID_RANGE_LOW = 10;
static const int SCORE_INVALID_RANGE_HIGH = 20;
static const int SCORE_RF_OOB_LOW = 10;
static const int SCORE_RF_OOB_HIGH = 25;
static const int SCORE_UNWIND_RVA_OOB_LOW = 10;
static const int SCORE_UNWIND_RVA_OOB_HIGH = 25;
static const int SCORE_RF_UNSORTED = 15;
static const int SCORE_RF_OVERLAP = 25;
static const int SCORE_DUPLICATE_BEGIN = 10;
static const int SCORE_UNWIND_PARSE_FAIL_LOW = 10;
static const int SCORE_UNWIND_PARSE_FAIL_HIGH = 25;
static const int SCORE_UNWIND_ANOM_LOW = 10;
static const int SCORE_UNWIND_ANOM_HIGH = 20;
static const int SCORE_UNWIND_REUSE_MEDIUM = 10;
static const int SCORE_UNWIND_REUSE_HIGH = 20;
static const int SCORE_HANDLER_DOMINANCE = 15;
static const int SCORE_UNWIND_EXEC_SECTION = 20;
static const int SCORE_UNWIND_UNUSUAL_SECTION_LOW = 8;
static const int SCORE_UNWIND_UNUSUAL_SECTION_HIGH = 15;
static const int SCORE_UNWIND_WRITABLE_SECTION_LOW = 10;
static const int SCORE_UNWIND_WRITABLE_SECTION_HIGH = 18;
static const int SCORE_HANDLER_NONEXEC = 15;
static const int SCORE_HANDLER_WRITABLE = 10;
static const int SCORE_COMBO_PLACEMENT_EXEC_AND_HANDLER = 10;
static const int SCORE_COMBO_PLACEMENT_AND_INTEGRITY = 8;
static const int SCORE_COMBO_REUSE_AND_DOMINANCE = 8;
static const int SCORE_COMBO_WRITABLE_METADATA_AND_HANDLER = 10;
```

Какие элементы C здесь работают

Работают `static const int`, именование констант, группировка признаков по семействам evidence.

Почему это нужно именно этому проекту

Проект делает веса reviewable: можно увидеть, почему malformed table сильнее, чем isolated unusual section.

Как читать такие строки

Читай веса вместе с comments calibration notes: они показывают логику выбора.

Что ломается при непонимании

Если веса спрятать прямо в коде как числа, будет трудно объяснить комиссии и поддерживать проект.

Практическое задание

Выбери три признака и объясни, почему их веса отличаются.

Учебный модуль 31. Severity и confidence: два разных ответа

Сначала смысл

Severity отвечает на вопрос “насколько серьёзно, если наблюдение реально”. Confidence отвечает “насколько мы уверены в наблюдении и выводе”.

Фрагмент из проекта

```
static void finalize_verdict(HeuristicReport* out) {
    if (!out) return;

    out->suspicion = safe_ratio((double)out->score, 100.0);

    size_t family_count = heuristics_evidence_family_count(out->evidence_families_mask);
    size_t strong_count = count_findings_with_severity(out, "high") +
        count_findings_with_severity(out, "critical");
    size_t medium_count = count_findings_with_severity(out, "medium");
    size_t weak_count = 0;
    if (out->finding_count >= strong_count + medium_count) weak_count = out->finding_count -
        strong_count - medium_count;

    out->strong_finding_count = strong_count;
    out->medium_finding_count = medium_count;
    out->weak_finding_count = weak_count;
    safe_copy(out->evidence_level, sizeof(out->evidence_level),
              evidence_level_from_counts(weak_count, medium_count, strong_count, out->score));

    double confidence = CONFIDENCE_BASE;
    confidence += (double)family_count * CONFIDENCE_PER_FAMILY;
    confidence += (double)strong_count * CONFIDENCE_PER_STRONG;
    confidence += (double)medium_count * CONFIDENCE_PER_MEDIUM;

    if (out->rf_invalid_ranges || out->rf_oob || out->rf_unwind_oob || out->rf_overlaps) confidence
        += CONFIDENCE_BONUS_TABLE_HARD;
    if (out->unwind_parse_fail || out->unwind_anomalies) confidence += CONFIDENCE_BONUS_UNWIND_HARD;
    if (out->score == 0) confidence = CONFIDENCE_ZERO_SCORE_BASE;

    /* Weakening rules: lower confidence when only isolated / non-correlated static signals exist.
        */
    if (out->finding_count == 1 && strong_count == 0) {
        add_mitigation(out, "single_signal_only",
                       "Only one non-strong signal contributed; treat as weak standalone
                evidence.");
        confidence -= CONFIDENCE_PENALTY_SINGLE_SIGNAL;
    }
    if (out->total < 32 && strong_count == 0 && out->finding_count > 0) {
        add_mitigation(out, "small_sample_size",
```

Какие элементы C здесь работают

Работают counters findings, family count, floating point double, penalties/bonuses, clamp to min/max.

Почему это нужно именно этому проекту

Проект честно снижает confidence для одиночных сигналов и small sample. Это важно для защиты от ложных категоричных выводов.

Как читать такие строки

Читай `finalize_verdict` как расчет итоговой позиции: сначала evidence, потом усиления, потом ослабления, потом verdict.

Что ломается при непонимании

Если смешать severity и confidence, отчет начнет говорить либо слишком резко, либо слишком слабо.

Практическое задание

Придумай пример высокого severity и низкого confidence.

Учебный модуль 32. JSON escaping: текст отчета как bytes

Сначала смысл

JSON требует, чтобы кавычки, обратные слэши, управляющие символы и некорректные bytes были представлены безопасно.

Фрагмент из проекта

```
static void json_escape(StrBuf* sb, const char* s) {
    if (!s) return;
    size_t length = strlen(s);
    for (size_t i = 0; i < length; i++) {
        unsigned char c = (unsigned char)s[i];
        switch (c) {
            case '\\': sb_append(sb, "\\\\"); break;
            case '"':  sb_append(sb, "\\\""); break;
            case '\n': sb_append(sb, "\\n"); break;
            case '\r': sb_append(sb, "\\r"); break;
            case '\t': sb_append(sb, "\\t"); break;
            default:
                if (c < 0x20) sb_appendf(sb, "\\u%04x", (unsigned)c);
                else if (c < 0x80) sb_appendn(sb, &s[i], 1);
                else {
                    size_t sequence = valid_utf8_sequence_length(
                        (const unsigned char*)&s[i], length - i);
                    if (sequence) {
                        sb_appendn(sb, &s[i], sequence);
                        i += sequence - 1;
                    } else {
                        sb_appendf(sb, "\\u%04x", (unsigned)c);
                    }
                }
        }
    }
}

static void json_key(StrBuf* sb, const char* k) {
    sb_append(sb, "\"");
    json_escape(sb, k);
    sb_append(sb, "\":");
}
```

Какие элементы C здесь работают

Работают цикл по строке, switch, `sb_append`, проверка UTF-8 sequence length.

Почему это нужно именно этому проекту

Проект может вставлять в JSON пути, ошибки, сообщения findings и bytes. Всё это должно быть корректно экранировано.

Как читать такие строки

Читай escaping как parser наоборот: входная C-строка превращается в JSON-safe представление.

Что ломается при непонимании

Если escaping сломан, backend не сможет прочитать JSON или пользователь увидит повреждённый отчет.

Практическое задание

Покажи, как строка `a"b\c` должна выглядеть в JSON.

Учебный модуль 33. HTML report: объяснение для человека поверх структурированных данных

Сначала смысл

HTML-отчет строится из тех же данных, что JSON, но организует их для человека: summary, tables, findings, ручная валидация.

Фрагмент из проекта

```
bool report_write_html(const char* path,
                       const char* input_path,
                       const PeImage* img,
                       const RuntimeFunction* rf,
                       const UnwindSummary* us,
                       const bool* us_ok,
                       size_t n,
                       const HeuristicReport* hr,
                       const DynamicReport* dr,
                       const ReportMetadata* metadata,
                       StrBuf* err) {
    if (!path || !input_path || (!rf && n) || (!us && n) || (!us_ok && n) ||
        !report_model_valid(img, hr, dr) || !report_metadata_valid(metadata)) {
        sb_append(err, "Invalid arguments or metadata for HTML report.\n");
        return false;
    }

    StrBuf sb = {0};
    StrBuf anomalies = {0};
    if (!sb_init(&sb, 32768) ||
        !sb_init(&anomalies, 128)) {
        sb_free(&anomalies);
        sb_free(&sb);
        sb_append(err, "OOM.\n");
        return false;
    }

    const char* tool_version = metadata->tool_version;
    const char* generated_at = metadata->generated_at_utc;
    const char* schema_version = metadata->report_schema_version;
    const char* input_hash = metadata->input_sha256;
    const char* build_platform = metadata->build_platform;
    const char* build_compiler = metadata->build_compiler;
    const char* build_mode = metadata->build_mode;
```

Какие элементы C здесь работают

Работают `StrBuf`, HTML escaping, форматирование чисел, секции документа, условные блоки dynamic/static.

Почему это нужно именно этому проекту

Это важно для ВКР и заказчика: инструмент должен не только выдавать machine-readable JSON, но и показывать evidence в профессиональной форме.

Как читать такие строки

Читай HTML writer как presentation layer: он не должен заново анализировать файл, а должен правильно объяснить уже посчитанные данные.

Что ломается при непонимании

Если HTML скрывает важные поля, проект выглядит слабее, чем есть. Если HTML выводит сырые неэкранированные данные, появляется риск поломанной страницы.

Практическое задание

Сравни JSON и HTML: какие поля должны совпадать по смыслу?

Учебный модуль 34. Ghidra CSV: интеграционный слой без изменения ядра

Сначала смысл

Ghidra CSV отличается от полного CSV: он ориентирован на импорт функций, names и comments в дизассемблер.

Фрагмент из проекта

```
if (ghidra_mode) {
    sb_append(&output, "begin_rva,end_rva,name,comment\n");
    for (size_t i = 0; i < n; i++) {
        const RuntimeFunction* r = &rf[i];
        const UnwindSummary* u = &us[i];

        char name[64];
        snprintf(name, sizeof(name), "sub_%08X", (unsigned)r->begin_rva);

        if (us_ok[i]) build_unwind_comment(u, &comment);
        else {
            comment.len = 0;
            if (comment.s) comment.s[0] = 0;
            sb_append(&comment, "UNWIND parse failed");
            if (u->parse_error[0]) sb_appendf(&comment, ": %s", u->parse_error);
        }

        char a[32], b[32];
        snprintf(a, sizeof(a), "0x%08X", (unsigned)r->begin_rva);
        snprintf(b, sizeof(b), "0x%08X", (unsigned)r->end_rva);

        csv_cell(&output, a); sb_appendn(&output, ",", 1);
        csv_cell(&output, b); sb_appendn(&output, ",", 1);
        csv_cell(&output, name); sb_appendn(&output, ",", 1);
        csv_cell(&output, comment.s ? comment.s : "");
        sb_appendn(&output, "\n", 1);
    }
} else {
    sb_append(&output,
              "index,begin_rva,end_rva,size,unwind_rva,unwind_ver,"
              "unwind_flags,prolog_size,fixed_stack,frame_reg,frame_off,"
              "handler_rva,has_chain,anomalies,parse_error\n");
    for (size_t i = 0; i < n; i++) {
        const RuntimeFunction* r = &rf[i];
```

Какие элементы C здесь работают

Работают условная ветка, формат CSV, `csv_cell`, построение комментария `build_unwind_comment`.

Почему это нужно именно этому проекту

Проект помогает реверсеру перенести карту unwind metadata в Ghidra, чтобы видеть функции и комментарии рядом с кодом.

Как читать такие строки

Читай ghidra-mode как отдельное представление тех же данных, а не как отдельный анализ.

Что ломается при непонимании

Если CSV плохо экранирует запятые или кавычки, импорт может сломаться.

Практическое задание

Опиши, какие поля нужны Ghidra и почему полный JSON там избыточен.

Учебный модуль 35. ReportMetadata: воспроизводимость отчета

Сначала смысл

Отчет должен содержать не только findings, но и сведения о версии инструмента, схеме отчета, времени, SHA-256 входа и параметрах запуска.

Фрагмент из проекта

```
typedef struct ReportMetadata {
    const char* tool_version;
    const char* report_schema_version;
    const char* generated_at_utc;
    const char* input_sha256;
    uint64_t input_size;
    const char* command_line;
    const char* build_compiler;
    const char* build_platform;
    const char* build_mode;
    bool strict_abi;
    bool follow_chain;
    uint32_t max_chain_depth;
} ReportMetadata;

void report_print_brief(const char* path, const PeImage* img, size_t rf_count);

bool report_write_csv(const char* path,
                      const PeImage* img,
                      const RuntimeFunction* rf,
                      const UnwindSummary* us,
                      const bool* us_ok,
                      size_t n,
                      bool ghidra_mode,
                      StrBuf* err);

bool report_write_json_ex(const char* path,
                          const char* input_path,
                          const PeImage* img,
                          const RuntimeFunction* rf,
                          const UnwindSummary* us,
                          const bool* us_ok,
                          size_t n,
                          const HeuristicReport* hr,
```

Какие элементы C здесь работают

Работают структура с `const char*`, `uint64_t`, bool flags, числовые параметры.

Почему это нужно именно этому проекту

Это нужно, чтобы позже сравнить отчеты, доказать, какой файл анализировался, и понять, с какими опциями был запущен инструмент.

Как читать такие строки

Читай metadata как паспорт анализа: что, чем, когда и с какими настройками было обработано.

Что ломается при непонимании

Без metadata невозможно уверенно связать JSON/HTML с конкретным бинарником и версией ядра.

Практическое задание

Объясни, почему SHA-256 входного файла важнее, чем только имя файла.

Учебный модуль 36. `#ifdef _WIN32`: один проект, разные платформенные возможности

Сначала смысл

Условная компиляция позволяет включать Windows API только при сборке под Windows. Это сохраняет переносимость статического ядра.

Фрагмент из проекта

```
#ifdef _WIN32

#include <windows.h>
#include <tlhelp32.h>

/* MinGW exposes exception_code as a compatibility macro. */
#ifdef exception_code
#undef exception_code
#endif

/* ---------------------------------------------------------------------------- */

typedef struct ModInfo {
    char name[MAX_PATH];
    uint64_t base;
    uint64_t size;
} ModInfo;

typedef struct ModList {
    ModInfo mods[128];
    size_t count;
    bool truncated;
} ModList;

static void mods_init(ModList* ml) { if (ml) memset(ml, 0, sizeof(*ml)); }

static const char* basename_lower(const char* path) {
    const char* base = path;
    for (const char* p = path; *p; p++) {
        if (*p == '\\' || *p == '/') base = p + 1;
    }
    return base;
}
```

Какие элементы C здесь работают

Работают препроцессорные директивы, разные includes, alternative implementation или stubs.

Почему это нужно именно этому проекту

Статический анализ PE может запускаться на macOS/Linux/Windows. Dynamic validation требует Windows debugger API и поэтому зависит от `_WIN32`.

Как читать такие строки

Читай `#ifdef` как разделение мира: что существует в этом build target, а что нет.

Что ломается при непонимании

Если Windows headers подключать без условия на macOS/Linux, проект не соберётся. Если скрыть dynamic ограничения, пользователь неправильно поймёт возможности.

Практическое задание

Сформулируй ответ: почему static cross-platform, а dynamic Windows-only.

Учебный модуль 37. Windows handles и проверка API-вызовов

Сначала смысл

Windows API часто возвращает handles и error codes. C-код обязан проверять каждый вызов и закрывать handle после использования.

Фрагмент из проекта

```
    HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPMODULE | TH32CS_SNAPMODULE32, pid);
    if (snap == INVALID_HANDLE_VALUE) return false;

    MODULEENTRY32 me;
    memset(&me, 0, sizeof(me));
    me.dwSize = sizeof(me);
    if (!Module32First(snap, &me)) {
        CloseHandle(snap);
        return false;
    }
    do {
        const char* base = basename_lower(me.szModule);
        mods_add(ml, base, (uint64_t)(uintptr_t)me.modBaseAddr, (uint64_t)me.modBaseSize);
    } while (Module32Next(snap, &me));

    CloseHandle(snap);
    return true;
}

static bool mods_refresh_accounted(ModList* ml, DWORD pid, DynamicReport* dr) {
    if (!mods_refresh_toolhelp(ml, pid)) {
        if (dr) {
            dr->module_refresh_failures++;
            dr->instrumentation_complete = false;
        }
        return false;
    }
    if (dr) {
        if (ml->count > dr->modules_observed_peak) {
            dr->modules_observed_peak = (uint32_t)ml->count;
        }
        if (ml->truncated) {
            dr->module_list_truncations++;
            dr->instrumentation_complete = false;
```

Какие элементы C здесь работают

Работают тип `HANDLE`, сравнение с `INVALID_HANDLE_VALUE`, `CloseHandle`, структуры Windows API.

Почему это нужно именно этому проекту

Dynamic layer перечисляет modules, читает process memory, ставит breakpoints. Любая ошибка API должна быть записана как incomplete instrumentation, а не скрыта.

Как читать такие строки

Читай Windows API участок как работу с внешним ресурсом: получить, проверить, использовать, закрыть.

Что ломается при непонимании

Если не закрыть handle, будут утечки ресурсов. Если не проверить failed API, отчет динамики будет выглядеть полным, хотя evidence собрано не полностью.

Практическое задание

Найди в `dynamic_win.c` три места, где вызывается `CloseHandle`.

Учебный модуль 38. Breakpoints: изменение памяти процесса и восстановление байта

Сначала смысл

Software breakpoint обычно ставится записью byte `0xCC` в код target process. Перед этим нужно сохранить оригинальный byte, чтобы потом восстановить.

Фрагмент из проекта

```
static bool bp_set(HANDLE hProcess, Breakpoint* bp, StrBuf* err) {
    SIZE_T nread = 0;
    uint8_t b = 0;
    if (!ReadProcessMemory(hProcess, (LPCVOID)(uintptr_t)bp->addr, &b, 1, &nread) || nread != 1) {
        if (err) sb_appendf(err, "ReadProcessMemory failed at 0x%llx (err=%lu)\n",
                            (unsigned long long)bp->addr, (unsigned long)GetLastError());
        return false;
    }
    bp->orig = b;
    if (!write_one_byte(hProcess, bp->addr, 0xCC, err)) return false;
    bp->armed = true;
    bp->rearm_pending = false;
    return true;
}

static bool bp_clear(HANDLE hProcess, Breakpoint* bp, StrBuf* err) {
    if (!bp || !bp->armed) return true;
    if (!write_one_byte(hProcess, bp->addr, bp->orig, err)) return false;
    bp->armed = false;
    return true;
}

static bool bp_add_and_arm(BpList* bl, HANDLE hProcess, uint64_t addr, BpKind kind, StrBuf* err) {
    if (!bl) return false;

    Breakpoint* ex = bps_find(bl, addr);
    if (ex) {
        ex->kinds |= (uint32_t)kind;
        if (ex->armed || ex->rearm_pending) return true;
        return bp_set(hProcess, ex, err);
    }

    if (bl->count >= sizeof(bl->bps)/sizeof(bl->bps[0])) return false;
    Breakpoint* bp = &bl->bps[bl->count];
```

Какие элементы C здесь работают

Работают `ReadProcessMemory`, `WriteProcessMemory`, `FlushInstructionCache`, структура `Breakpoint` с `orig`, `armed`, `rearm_pending`.

Почему это нужно именно этому проекту

Проект использует breakpoints, чтобы фиксировать handler hits и API calls dynamic function table/exception registration.

Как читать такие строки

Читай breakpoint как временную инструментальную правку чужой памяти, требующую строгого rollback.

Что ломается при непонимании

Если не сохранить оригинальный byte или не восстановить его, target process будет повреждён. Если не flush instruction cache, CPU может исполнять старую версию кода.

Практическое задание

Объясни, почему dynamic должен запускать target в контролируемой среде.

Учебный модуль 39. Dynamic report: runtime evidence не равно static finding

Сначала смысл

Динамический отчет хранит факты выполнения: видел ли процесс исключения, hits обработчиков, регистрации VEH/VCH/UEF, вызовы RtlAddFunctionTable, timeout, полноту instrumentation.

Фрагмент из проекта

```
typedef struct DynamicReport {
    bool enabled;
    bool ran;
    bool ok;
    bool timed_out;
    bool target_exited;
    uint32_t target_exit_code;

    uint32_t timeout_ms;
    uint32_t trace_steps;

    bool instrumentation_complete;
    uint32_t modules_observed_peak;
    uint32_t module_refresh_failures;
    uint32_t module_list_truncations;
    uint32_t unwind_handlers_discovered;
    uint32_t breakpoint_set_attempts;
    uint32_t breakpoint_set_failures;
    uint32_t breakpoint_capacity_drops;
    uint32_t pending_rearm_failures;
    uint32_t breakpoint_rearm_failures;

    uint32_t exceptions_seen;
    uint32_t first_chance_seen;
    uint32_t second_chance_seen;

    uint32_t unwind_handler_bp_set;
    uint32_t unwind_handler_hits;

    uint32_t veh_registrations;
    uint32_t veh_hits;
    uint32_t vch_registrations;
    uint32_t vch_hits;
    uint32_t uef_set_calls;
```

Какие элементы C здесь работают

Работают структура со счетчиками, bool flags, `StrBuf details`.

Почему это нужно именно этому проекту

Static говорит, что файл обещает. Dynamic говорит, что произошло в запуске. Они должны связываться, но не заменять друг друга.

Как читать такие строки

Читай dynamic fields как ограничения наблюдения: ran/ok/timed_out/instrumentation_complete важны не меньше counters.

Что ломается при непонимании

Если считать отсутствие dynamic события доказательством отсутствия проблемы, можно ошибиться: подозрительный путь мог не выполниться.

Практическое задание

Сформулируй различие true runtime evidence и absence of evidence.

Учебный модуль 40. Cleanup в `main`: почему конец функции не менее важен начала

Сначала смысл

C не освобождает heap resources автоматически при выходе из блока. `main` должен явно освободить StrBuf, arrays, PeImage sections, dynamic report и file buffer.

Фрагмент из проекта

```
    if (a.heuristics && hr_ptr) heuristics_free(&hr);
    if (dr_initialized) dynamic_report_free(&dr);

    sb_free(&ops);
    free(us_ok);
    free(us);
    free(rf);
    pe_free(&img);
    sb_free(&err);
    sb_free(&err2);
    buf_free_owned(owned);
    return exit_code;
}
```

Какие элементы C здесь работают

Работают `free`, module-specific free functions, условные cleanup paths, порядок освобождения.

Почему это нужно именно этому проекту

Проект анализирует потенциально большие файлы и генерирует отчеты; утечки памяти недопустимы для batch-использования.

Как читать такие строки

Читай конец `main` как зеркальное отражение начала: что было инициализировано, должно быть освобождено.

Что ломается при непонимании

Если забыть cleanup на ошибке, единичный запуск может казаться нормальным, но массовый анализ начнет терять память.

Практическое задание

Составь список всех ресурсов, которые освобождаются в конце `main`.

Учебный модуль 41. CMake: сборка как часть проекта, а не внешняя магия

Сначала смысл

Код C становится продуктом только после воспроизводимой сборки. CMake описывает target, исходники, include paths, compile definitions и платформенные различия.

Фрагмент из проекта

```
cmake_minimum_required(VERSION 3.23)

project(
    eh_unwind_analyzer
    VERSION 1.0.0
    DESCRIPTION "Static and dynamic analyzer for Windows x64 EH/Unwind metadata"
    LANGUAGES C
)

include(CTest)

option(EHUA_WARNINGS_AS_ERRORS "Treat compiler warnings as errors" ON)
option(EHUA_ENABLE_SANITIZERS "Enable AddressSanitizer and UndefinedBehaviorSanitizer" OFF)
option(EHUA_ENABLE_COVERAGE "Enable GCC/Clang coverage instrumentation" OFF)
option(EHUA_BUILD_FUZZER "Build the Clang libFuzzer parser harness" OFF)

if(EHUA_ENABLE_SANITIZERS AND EHUA_ENABLE_COVERAGE)
    message(FATAL_ERROR "Sanitizer and coverage instrumentation cannot be enabled together")
endif()

if(WIN32 AND NOT CMAKE_SIZEOF_VOID_P EQUAL 8)
    message(FATAL_ERROR "EH/Unwind Analyzer supports Windows x64 builds only")
endif()

function(ehua_apply_warnings target)
    if(MSVC)
        target_compile_options(${target} PRIVATE /W4 /permissive- /utf-8)
        target_compile_definitions(${target} PRIVATE _CRT_SECURE_NO_WARNINGS=1)
        if(EHUA_WARNINGS_AS_ERRORS)
            target_compile_options(${target} PRIVATE /WX)
        endif()
    else()
        target_compile_options(
            ${target}
```

Какие элементы C здесь работают

Работают build target, source list, compiler options, conditional platform logic.

Почему это нужно именно этому проекту

Для передачи заказчику важно, чтобы проект собирался понятным способом и не зависел от ручных случайных команд автора.

Как читать такие строки

Читай CMakeLists как карту компиляции: какие `.c` файлы становятся executable, где include directories, какие опции включены.

Что ломается при непонимании

Если сборка не описана, другой человек может не понять, какие файлы входят в ядро и какие флаги нужны.

Практическое задание

Объясни, почему наличие CMake важно для релиза.

Учебный модуль 42. Граница ядра и будущего backend/frontend

Сначала смысл

Ядро не обязано знать о базе данных или frontend. Оно должно выдавать структурированный JSON/HTML/CSV, а внешний слой может запускать CLI или вызывать библиотечный API, если он будет выделен.

Фрагмент из проекта

```
bool report_write_json_ex(const char* path,
                          const char* input_path,
                          const PeImage* img,
                          const RuntimeFunction* rf,
                          const UnwindSummary* us,
                          const bool* us_ok,
                          size_t n,
                          const HeuristicReport* hr,
                          const DynamicReport* dr,
                          const ReportMetadata* metadata,
                          StrBuf* err);

bool report_write_html(const char* path,
                       const char* input_path,
                       const PeImage* img,
                       const RuntimeFunction* rf,
                       const UnwindSummary* us,
                       const bool* us_ok,
                       size_t n,
                       const HeuristicReport* hr,
                       const DynamicReport* dr,
                       const ReportMetadata* metadata,
                       StrBuf* err);

void report_print_heuristics(const HeuristicReport* hr);

#ifdef __cplusplus
}
#endif
```

Какие элементы C здесь работают

Работают стабильные структуры отчета, функции записи, metadata, schema version.

Почему это нужно именно этому проекту

Так проект можно развивать: backend загружает бинарник, запускает ядро, сохраняет JSON в БД, frontend показывает результаты. Ядро при этом остается специализированным анализатором.

Как читать такие строки

Читай report API как контракт с внешним миром.

Что ломается при непонимании

Если ядро смешать с web/backend логикой, его сложнее тестировать и переносить. Если JSON схема нестабильна, интеграция будет ломаться.

Практическое задание

Опиши, какие поля JSON backend должен сохранить для сравнения анализов.

Часть IX. Подробный маршрут самостоятельного написания анализатора с нуля

Эта часть нужна для конечной цели: после изучения книги читатель должен не просто понимать готовый проект, а суметь написать его уменьшенную версию. Каждый шаг ниже построен так: что реализовать, какие конструкции C понадобятся, какие проверки обязательны, какой тест должен пройти этап и какая типичная ошибка новичка разрушит результат.

Шаг 1. CLI и коды выхода

Создать `main(int argc, char** argv)`, проверить наличие пути к файлу, вывести usage, вернуть 2 при ошибке аргументов и 0 при help/version. C-конструкции: argc/argv, `strcmp`, `return`, `fprintf`. Тест: запуск без аргументов должен не падать, а печатать usage.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 2. Чтение файла

Реализовать `buf_init_from_file`. C-конструкции: `FILE*`, `fopen`, `fseek`, `ftell`, `malloc`, `fread`, `fclose`. Обязательная проверка: каждый early return должен закрыть файл и освободить уже выделенную память. Тест: несуществующий файл, пустой файл, нормальный файл.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 3. Безопасный доступ к bytes

Реализовать `buf_read`. C-конструкции: `const Buf*`, `void*`, `size_t`, `memcpy`. Обязательная проверка: `off > size`, затем `len > size - off`. Тест: чтение последнего байта успешно, чтение за последним байтом возвращает false.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 4. Little-endian primitives

Реализовать `read_u16/u32/u64`. C-конструкции: массив `uint8_t tmp[N]`, shifts, casts. Тест: bytes `50 45 00 00` дают `0x4550`.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 5. PE magic

Проверить MZ и PE signature. C-конструкции: constants, compare, error buffer. Тест: файл с неправильным MZ отклоняется.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 6. OptionalHeader PE32+

Прочитать Machine, OptionalHeader magic, ImageBase, SizeOfImage, alignments. Тест: подменить Machine и убедиться, что parser отклоняет не-AMD64.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 7. Sections

Выделить массив `PeSection`, прочитать headers, добавить null terminator к name. Тест: section header за пределами файла должен давать ошибку.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 8. RVA mapping

Реализовать `pe_find_section_for_rva` и `pe_rva_to_fileoff`. Тест: RVA внутри raw range мапится, RVA в virtual tail не мапится.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 9. Exception Directory

Прочитать DataDirectory[3], проверить SizeOfImage и кратность 12. Тест: размер 13 bytes должен давать malformed runtime function table.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 10. RuntimeFunction array

Выделить массив `RuntimeFunction`, заполнить begin/end/unwind. Тест: таблица из двух записей читается и count равен 2.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 11. Unwind header

Прочитать 4 bytes `UNWIND_INFO`, разложить version/flags/prolog/count/frame. Тест: version 3 должен стать parse fail.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 12. UWOP slots

Реализовать таблицу размера операций. Тест: `UWOP_ALLOC_LARGE` mode 1 требует 3 slots; если CountOfCodes=1, parser должен вернуть fail с diagnostics.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 13. Padding и tail

Вычислить `slots_even`, прочитать handler после padding. Тест: CountOfCodes=1 и EHANDLER: handler RVA должен читаться после 4 bytes header + 4 bytes slots area, а не после 6 bytes.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 14. Diagnostics

Заполнять failed_field, file offset, expected/available. Тест: проверить JSON, что parse fail содержит конкретное поле.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 15. Heuristic counters

Посчитать invalid ranges, oob, unsorted, overlaps, duplicate begins. Тест: синтетическая таблица с overlap даёт один finding.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 16. Placement checks

Проверить секцию unwind RVA и handler RVA: executable/writable/unusual. Тест: metadata в `.text` должна дать finding placement.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 17. Reuse и dominance

Посчитать reuse ratios через map. Тест: 100 функций на один unwind RVA дают strong reuse finding.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 18. Report JSON

Сформировать JSON с metadata, sections, functions, summaries, findings. Тест: JSON должен парситься стандартным parser-ом.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 19. Report HTML

Сформировать HTML с русскими объяснениями. Тест: открыть в браузере и найти сработавший finding, evidence и manual validation steps.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 20. Dynamic stub

На non-Windows вернуть понятное сообщение, что dynamic недоступна. Тест: сборка на macOS/Linux не должна падать из-за Windows headers.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Шаг 21. Windows dynamic

На Windows добавить debug loop, breakpoints, counters. Тест: controlled binary с VEH/UEF/RtlAddFunctionTable должен давать соответствующие dynamic counters.

Контрольный вопрос: какие данные этот шаг принимает от предыдущего слоя и какие данные передает следующему? Если ответ не сформулирован, архитектура еще не понята.

Часть X. Построчный учебный разбор ключевых исходников

Эта часть создана как тренажёр чтения C-кода. Она не заменяет исходники, а учит смотреть на строку как на сочетание типа, имени, оператора, проверки, ресурса и предметного смысла. Здесь разобраны ключевые файлы ядра: заголовки, util, PE parser, unwind parser, heuristics и main. Report и dynamic уже разобраны в предыдущих частях как крупные модули, потому что они очень объёмные; при необходимости их можно разбирать тем же методом.

Файл include/util.h

Файл содержит 72 строк. Ниже идут непустые строки с учебным объяснением. Смысл упражнения - не заучить комментарий, а научиться самостоятельно выполнять такой разбор.

Строка 1: `#pragma once` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 2: `/*` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 3: `util.h - small helpers for safe parsing and dynamic arrays` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 4: `Project: EH/Unwind Analyzer (PE x64)` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 5: `*/` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 7: `#include <stdint.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 8: `#include <stddef.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 9: `#include <stdbool.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 11: `#ifdef __cplusplus` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 12: `extern "C" {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 13: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 15: `typedef struct Buf {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 16: `const uint8_t* data;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 17: `size_t size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 18: `} Buf;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 20: `bool buf_init_from_file(Buf* out, const char* path, uint8_t** owned_mem);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 21: `void buf_free_owned(uint8_t* owned_mem);` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 23: `bool buf_read(const Buf* b, size_t off, void* out, size_t len);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 24: `bool buf_read_u8 (const Buf* b, size_t off, uint8_t* out);` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 25: `bool buf_read_u16(const Buf* b, size_t off, uint16_t* out);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или

невозможность безопасно продолжать.

Строка 26: `bool buf_read_u32(const Buf* b, size_t off, uint32_t* out);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 27: `bool buf_read_u64(const Buf* b, size_t off, uint64_t* out);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 29: `static inline uint16_t le16(const uint8_t* p) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 30: `return (uint16_t)(p[0] | ((uint16_t)p[1] << 8));` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 31: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 32: `static inline uint32_t le32(const uint8_t* p) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 33: `return (uint32_t)(p[0] | ((uint32_t)p[1] << 8) | ((uint32_t)p[2] << 16) | ((uint32_t)p[3] << 24));` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 34: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 35: `static inline uint64_t le64(const uint8_t* p) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 36: `return (uint64_t)le32(p) | ((uint64_t)le32(p + 4) << 32);` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 37: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 39: `typedef struct StrBuf {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 40: `char* s;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 41: `size_t len;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 42: `size_t cap;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 43: `bool failed;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 44: `} StrBuf;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 46: `#if defined(__MINGW32__) && (defined(__GNUC__) || defined(__clang__))` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 47: `#define EHUA_PRINTF_FORMAT(format_index, first_arg) \` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 48: `__attribute__((format(gnu_printf, format_index, first_arg)))` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 49: `#elif defined(__GNUC__) || defined(__clang__)` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 50: `#define EHUA_PRINTF_FORMAT(format_index, first_arg) \` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 51: `__attribute__((format(printf, format_index, first_arg)))` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 52: `#else` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 53: `#define EHUA_PRINTF_FORMAT(format_index, first_arg)` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 54: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 56: `bool sb_init(StrBuf* sb, size_t initial_cap);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 57: `void sb_free(StrBuf* sb);` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 58: `bool sb_appendf(StrBuf* sb, const char* fmt, ...)` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 59: `EHUA_PRINTF_FORMAT(2, 3);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 60: `bool sb_append(StrBuf* sb, const char* s);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 61: `bool sb_appendn(StrBuf* sb, const char* s, size_t n);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 62: `bool sb_ok(const StrBuf* sb);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 63: `bool file_write_atomic(const char* path, const char* data, size_t len);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 65: `/* SHA-256 digest encoded as 64 lowercase hexadecimal characters plus NUL. */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 66: `bool sha256_hex(const uint8_t* data, size_t size, char out_hex[65]);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 68: `#undef EHUA_PRINTF_FORMAT` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 70: `#ifdef __cplusplus` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 71: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 72: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Файл include/pe.h

Файл содержит 67 строк. Ниже идут непустые строки с учебным объяснением. Смысл упражнения - не заучить комментарий, а научиться самостоятельно выполнять такой разбор.

Строка 1: `#pragma once` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 2: `/*` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 3: `pe.h - minimal PE32+/x64 parser (no Windows headers)` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с

предыдущими проверками.

Строка 4: `*/` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 6: `#include <stdint.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 7: `#include <stddef.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 8: `#include <stdbool.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 9: `#include "util.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 11: `#ifdef __cplusplus` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 12: `extern "C" {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 13: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 15: `#define PE_DOS_MAGIC 0x5A4D /* 'MZ' */` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 16: `#define PE_NT_MAGIC 0x00004550 /* 'PE\0\0' */` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 18: `#define IMAGE_FILE_MACHINE_AMD64 0x8664` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 19: `#define IMAGE_NT_OPTIONAL_HDR64_MAGIC 0x20b` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 21: `#define IMAGE_NUMBEROF_DIRECTORY_ENTRIES 16` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 22: `#define IMAGE_DIRECTORY_ENTRY_EXCEPTION 3` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 24: `typedef struct PeSection {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 25: `char name[9]; /* null-terminated */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 26: `uint32_t virtual_address;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 27: `uint32_t virtual_size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 28: `uint32_t raw_ptr;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 29: `uint32_t raw_size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 30: `uint32_t characteristics;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 31: `} PeSection;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 33: `typedef struct PeImage {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 34: `bool ok;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 35: `bool is_pe64;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 36: `uint16_t machine;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 37: `uint64_t image_base;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 38: `uint32_t size_of_image;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 39: `uint32_t size_of_headers;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 40: `uint32_t section_alignment;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 41: `uint32_t file_alignment;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 42: `uint32_t exception_rva;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 43: `uint32_t exception_size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 44: `size_t file_size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 45: `size_t num_sections;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 46: `PeSection* sections; /* owned */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 48: `size_t section_raw_oob;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 49: `size_t section_virtual_oob;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 50: `size_t section_raw_overlaps;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 51: `size_t section_virtual_overlaps;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 52: `size_t section_misaligned;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 53: `bool exception_directory_oob;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 54: `} PeImage;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 56: `bool pe_parse(PeImage* out, const Buf* b, StrBuf* err);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 57: `void pe_free(PeImage* img);` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 59: `/* Map RVA -> file offset. Returns false if not mappable. */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 60: `bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 62: `/* Find section containing RVA (or NULL). */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 63: `const PeSection* pe_find_section_for_rva(const PeImage* img, uint32_t rva);` - Эта строка связывает адресную модель PE с фактическими bytes файла. Она помогает не перепутать RVA, VA и file offset.

Строка 65: `#ifdef __cplusplus` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 66: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 67: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Файл include/unwind.h

Файл содержит 136 строк. Ниже идут непустые строки с учебным объяснением. Смысл упражнения - не заучить комментарий, а научиться самостоятельно выполнять такой разбор.

Строка 1: `#pragma once` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 2: `/*` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 3: `unwind.h - x64 UNWIND_INFO parsing (PE exception directory)` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 4: `*/` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 6: `#include <stdint.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 7: `#include <stddef.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 8: `#include <stdbool.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 9: `#include "util.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 10: `#include "pe.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 12: `#ifdef __cplusplus` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 13: `extern "C" {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 14: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 16: `typedef struct RuntimeFunction {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 17: `uint32_t begin_rva;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 18: `uint32_t end_rva;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 19: `uint32_t unwind_rva;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 20: `} RuntimeFunction;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 22: `#define UNWIND_DIAG_MAX_BYTES 32u` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 23: `#define UNWIND_DIAG_MAX_CHAIN_PATH 16u` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 25: `typedef struct UnwindSummary {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 26: `bool valid;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 27: `uint8_t version;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 28: `uint8_t flags;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 29: `uint8_t prolog_size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 30: `uint8_t count_codes;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 31: `uint8_t frame_reg;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 32: `uint8_t frame_off; /* scaled by 16 */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 33: `bool has_handler;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 34: `uint32_t handler_rva;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 36: `bool has_chain;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 37: `RuntimeFunction chain;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 39: `uint32_t fixed_stack_bytes; /* estimated fixed stack frame delta: push+alloc+machframe */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 40: `uint32_t pushed_nonvol_count;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 41: `uint16_t saved_nonvol_mask; /* bits 0..15 => reg numbers */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 42: `uint16_t saved_xmm_mask; /* bits 0..15 => XMM reg numbers */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 44: `uint32_t anomalies; /* bitmask of internal anomalies */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 45: `char parse_error[160]; /* non-empty when unwind_parse returns false */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 47: `char diagnostic_field[64]; /* exact field/area that caused the strongest diagnostic */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 48: `bool diagnostic_has_file_offset;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 49: `uint32_t diagnostic_file_offset;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 50: `bool diagnostic_has_size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 51: `uint32_t diagnostic_expected_size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 52: `uint32_t diagnostic_available_size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 53: `bool diagnostic_has_slots;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 54: `uint32_t diagnostic_expected_slots;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 55: `uint32_t diagnostic_available_slots;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 56: `bool diagnostic_has_bytes;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 57: `uint32_t diagnostic_bytes_file_offset;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 58: `uint8_t diagnostic_bytes[UNWIND_DIAG_MAX_BYTES];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 59: `uint8_t diagnostic_bytes_count;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 61: `uint32_t chain_path[UNWIND_DIAG_MAX_CHAIN_PATH];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 62: `uint8_t chain_path_count;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 63: `bool chain_loop_detected;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 64: `uint32_t chain_loop_rva;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 65: `} UnwindSummary;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 67: `enum UnwindFlags {` - Здесь объявляется перечисление именованных констант. В проекте enum часто используется для кодов результата, flags или битовых признаков, чтобы не писать магические числа.

Строка 68: `UNW_FLAG_EHANDLER = 0x01,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 69: `UNW_FLAG_UHANDLER = 0x02,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 70: `UNW_FLAG_CHAININFO = 0x04` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 71: `};` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 73: `/* anomalies flags (for heuristic analysis / diagnostics) */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 74: `enum UnwindAnom {` - Здесь объявляется перечисление именованных констант. В проекте enum часто используется для кодов результата, flags или битовых признаков, чтобы не писать магические числа.

Строка 75: `UA_VERSION_UNKNOWN = 1u << 0,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 76: `UA_TRUNCATED = 1u << 1,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 77: `UA_BAD_OPCODE = 1u << 2,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 78: `UA_BAD_OPERAND = 1u << 3,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 79: `UA_CODEOFFSET_OUTSIDE = 1u << 4,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 80: `UA_CHAIN_LOOP = 1u << 5,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 81: `UA_CHAIN_DEPTH_EXCEEDED = 1u << 6,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 82: `UA_HANDLER_OOB = 1u << 7,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 83: `UA_CODEOFFSET_ORDER = 1u << 8,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 84: `UA_OPERAND_SANITY = 1u << 9,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 85: `UA_ALIGNMENT = 1u << 10,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 86: `UA_CHAIN_INVALID = 1u << 11,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 87: `UA_ABI_REGISTER_CLASS = 1u << 12,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 88: `UA_ABI_RESERVED_OPINFO = 1u << 13,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 89: `UA_ABI_PUSH_ORDER = 1u << 14,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 90: `UA_ABI_CHAIN_RESTRICTION = 1u << 15,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 91: `UA_ABI_SAVE_OFFSET = 1u << 16` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 92: `};` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 94: `typedef struct UnwindParseOpts {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 95: `int max_chain_depth; /* default 8 */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 96: `bool follow_chain; /* compute + annotate chain info */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 97: `bool strict_abi; /* flag formal Microsoft x64 unwind ABI violations */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 98: `} UnwindParseOpts;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 100: `typedef enum RuntimeFunctionReadResult {` - Здесь объявляется перечисление именованных констант. В проекте enum часто используется для кодов результата, flags или битовых признаков, чтобы не писать магические числа.

Строка 101: `RUNTIME_FUNCTION_READ_OK = 0,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 102: `RUNTIME_FUNCTION_READ_MALFORMED,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 103: `RUNTIME_FUNCTION_READ_OOM` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 104: `} RuntimeFunctionReadResult;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 106: `/* Parse UNWIND_INFO at unwind_rva. Produces summary + optional text of unwind ops. */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 107: `bool unwind_parse(const Buf* b,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 108: `const PeImage* img,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 109: `uint32_t unwind_rva,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 110: `const UnwindParseOpts* opts,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 111: `UnwindSummary* out,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 112: `StrBuf* ops_text,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 113: `StrBuf* err);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 115: `/* Parse exception directory (RUNTIME_FUNCTION table) into array.` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 116: `Caller owns *out (malloc). */` - Эта строка выделяет память в heap. После неё нужно проверить результат на NULL и определить, кто будет освобождать ресурс.

Строка 117: `RuntimeFunctionReadResult unwind_read_runtime_functions_ex(` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 118: `const Buf* b,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 119: `const PeImage* img,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 120: `RuntimeFunction** out,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 121: `size_t* out_count,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 122: `StrBuf* err);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 124: `bool unwind_read_runtime_functions(const Buf* b,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 125: `const PeImage* img,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 126: `RuntimeFunction** out,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 127: `size_t* out_count,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 128: `StrBuf* err);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 130: `/* Pretty helpers */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 131: `const char* unwind_reg_name(uint8_t reg);` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 132: `void unwind_flags_to_string(uint8_t flags, char out[64]);` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 134: `#ifdef __cplusplus` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 135: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 136: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Файл include/heuristics.h

Файл содержит 109 строк. Ниже идут непустые строки с учебным объяснением. Смысл упражнения - не заучить комментарий, а научиться самостоятельно выполнять такой разбор.

Строка 1: `#pragma once` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 2: `/*` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 3: `heuristics.h - heuristic assessment of broken / shadowed / low-informative unwind metadata` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 4: `*/` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 6: `#include <stddef.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 7: `#include <stdbool.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 8: `#include <stdint.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 9: `#include "pe.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 10: `#include "unwind.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 11: `#include "util.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 13: `#ifdef __cplusplus` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 14: `extern "C" {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 15: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 17: `enum HeuristicEvidenceFamily {` - Здесь объявляется перечисление именованных констант. В проекте enum часто используется для кодов результата, flags или битовых признаков, чтобы не писать магические числа.

Строка 18: `HEF_TABLE_INTEGRITY = 1u << 0,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 19: `HEF_UNWIND_INTEGRITY = 1u << 1,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 20: `HEF_METADATA_PLACEMENT = 1u << 2,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 21: `HEF_UNWIND_REUSE = 1u << 3,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 22: `HEF_HANDLER_DOMINANCE = 1u << 4` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 23: `};` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 25: `enum { HEURISTICS_MAX_FINDINGS = 32, HEURISTICS_MAX_MITIGATIONS = 16 };` - Здесь объявляется перечисление именованных констант. В проекте enum часто используется для кодов результата, flags или битовых признаков, чтобы не писать магические числа.

Строка 27: `typedef struct HeuristicFinding {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 28: `char code[40];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 29: `char family[32];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 30: `char severity[16];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 31: `char message[192];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 32: `bool has_location;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 33: `uint32_t representative_rva;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 34: `size_t affected_count;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 35: `} HeuristicFinding;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 37: `typedef struct HeuristicMitigation {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 38: `char code[40];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 39: `char message[192];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 40: `} HeuristicMitigation;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 42: `typedef struct HeuristicReport {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 43: `int score; /* 0..100 (higher => more suspicious) */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 44: `double suspicion; /* 0..1 derived from score */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 45: `double confidence; /* 0..1 confidence in the static verdict */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 46: `bool requires_manual_review;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 47: `uint32_t evidence_families_mask;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 48: `char verdict[48];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 50: `size_t total;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 52: `size_t pe_section_raw_oob;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 53: `size_t pe_section_virtual_oob;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 54: `size_t pe_section_raw_overlaps;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 55: `size_t pe_section_virtual_overlaps;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 56: `size_t pe_section_misaligned;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 57: `bool pe_exception_directory_oob;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 59: `size_t rf_invalid_ranges;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 60: `size_t rf_oob;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 61: `size_t rf_unwind_oob;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 62: `size_t rf_unsorted;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 63: `size_t rf_overlaps;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 64: `size_t rf_duplicate_begins;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 66: `size_t unwind_parse_fail;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 67: `size_t unwind_anomalies;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 68: `size_t unwind_abi_warnings;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 70: `size_t unwind_unusual_section;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 71: `size_t unwind_exec_section;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 72: `size_t unwind_writable_section;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 73: `size_t handler_nonexec_section;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 74: `size_t handler_writable_section;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 76: `double unwind_reuse_ratio; /* max_count / total */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 77: `double unwind_unique_ratio; /* unique / total */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 79: `double handler_reuse_ratio; /* max_count / handler_count (if any) */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 80: `size_t handler_count;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 82: `size_t finding_count;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 83: `HeuristicFinding findings[HEURISTICS_MAX_FINDINGS];` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 84: `size_t weak_finding_count;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 85: `size_t medium_finding_count;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 86: `size_t strong_finding_count;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 87: `char evidence_level[16];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 89: `size_t mitigation_count;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 90: `HeuristicMitigation mitigations[HEURISTICS_MAX_MITIGATIONS];` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 92: `StrBuf details; /* human-readable findings */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 93: `} HeuristicReport;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 95: `bool heuristics_analyze(const PeImage* img,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 96: `const RuntimeFunction* rf,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 97: `const UnwindSummary* us,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 98: `const bool* us_ok,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 99: `size_t n,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 100: `HeuristicReport* out,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 101: `StrBuf* err);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 103: `void heuristics_free(HeuristicReport* r);` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 104: `const char* heuristics_evidence_family_name(uint32_t bit);` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 105: `size_t heuristics_evidence_family_count(uint32_t mask);` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 107: `#ifdef __cplusplus` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 108: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 109: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Файл include/report.h

Файл содержит 72 строк. Ниже идут непустые строки с учебным объяснением. Смысл упражнения - не заучить комментарий, а научиться самостоятельно выполнять такой разбор.

Строка 1: `#pragma once` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 2: `/*` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 3: `report.h - output helpers (stdout, CSV, JSON)` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 4: `*/` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 6: `#include <stddef.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 7: `#include <stdbool.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 8: `#include "pe.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 9: `#include "unwind.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 10: `#include "heuristics.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 11: `#include "dynamic.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 12: `#include "util.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 14: `#ifdef __cplusplus` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 15: `extern "C" {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 16: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 18: `typedef struct ReportMetadata {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 19: `const char* tool_version;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 20: `const char* report_schema_version;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 21: `const char* generated_at_utc;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 22: `const char* input_sha256;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 23: `uint64_t input_size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 24: `const char* command_line;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 25: `const char* build_compiler;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 26: `const char* build_platform;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 27: `const char* build_mode;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 28: `bool strict_abi;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 29: `bool follow_chain;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 30: `uint32_t max_chain_depth;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 31: `} ReportMetadata;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 33: `void report_print_brief(const char* path, const PeImage* img, size_t rf_count);` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 35: `bool report_write_csv(const char* path,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 36: `const PeImage* img,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 37: `const RuntimeFunction* rf,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 38: `const UnwindSummary* us,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 39: `const bool* us_ok,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 40: `size_t n,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 41: `bool ghidra_mode,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 42: `StrBuf* err);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 44: `bool report_write_json_ex(const char* path,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 45: `const char* input_path,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 46: `const PeImage* img,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 47: `const RuntimeFunction* rf,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 48: `const UnwindSummary* us,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 49: `const bool* us_ok,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 50: `size_t n,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 51: `const HeuristicReport* hr,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 52: `const DynamicReport* dr,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 53: `const ReportMetadata* metadata,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 54: `StrBuf* err);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 56: `bool report_write_html(const char* path,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 57: `const char* input_path,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 58: `const PeImage* img,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 59: `const RuntimeFunction* rf,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 60: `const UnwindSummary* us,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 61: `const bool* us_ok,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 62: `size_t n,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 63: `const HeuristicReport* hr,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 64: `const DynamicReport* dr,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 65: `const ReportMetadata* metadata,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 66: `StrBuf* err);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 68: `void report_print_heuristics(const HeuristicReport* hr);` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 70: `#ifdef __cplusplus` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 71: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 72: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Файл include/dynamic.h

Файл содержит 122 строк. Ниже идут непустые строки с учебным объяснением. Смысл упражнения - не заучить комментарий, а научиться самостоятельно выполнять такой разбор.

Строка 1: `#pragma once` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 2: `/*` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 3: `dynamic.h - Windows-only dynamic evidence collector (Debug API)` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 5: `Defensive intent:` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 6: `Collect runtime evidence of exception-handling redirection (e.g., to an` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 7: `interceptor or VM dispatcher) by observing:` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 8: `- hits of handlers referenced from UNWIND_INFO (x64)` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 9: `- registration/hits of vectored/unhandled exception handlers` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 10: `- registration of dynamic function tables` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 11: `- first-chance vs second-chance exception flow` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 13: `This module does NOT attempt to "break" protections or recover hidden data.` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 14: `*/` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 16: `#include <stdint.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 17: `#include <stdbool.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 18: `#include "util.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 19: `#include "pe.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 20: `#include "unwind.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 22: `#ifdef __cplusplus` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 23: `extern "C" {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 24: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 26: `enum { DYNAMIC_MAX_EVENTS = 256 };` - Здесь объявляется перечисление именованных констант. В проекте enum часто используется для кодов результата, flags или битовых признаков, чтобы не писать магические числа.

Строка 28: `typedef struct DynamicEvent {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 29: `char type[40];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 30: `char module[64];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 31: `uint32_t thread_id;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 32: `uint32_t exception_code;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 33: `bool first_chance;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 34: `uint64_t address;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 35: `uint64_t arg0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 36: `uint64_t arg1;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 37: `uint64_t arg2;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 38: `uint64_t arg3;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 39: `} DynamicEvent;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 41: `typedef struct DynamicReport {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 42: `bool enabled;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 43: `bool ran;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 44: `bool ok;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 45: `bool timed_out;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 46: `bool target_exited;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 47: `uint32_t target_exit_code;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 49: `uint32_t timeout_ms;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 50: `uint32_t trace_steps;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 52: `bool instrumentation_complete;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 53: `uint32_t modules_observed_peak;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 54: `uint32_t module_refresh_failures;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 55: `uint32_t module_list_truncations;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 56: `uint32_t unwind_handlers_discovered;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 57: `uint32_t breakpoint_set_attempts;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 58: `uint32_t breakpoint_set_failures;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 59: `uint32_t breakpoint_capacity_drops;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 60: `uint32_t pending_rearm_failures;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 61: `uint32_t breakpoint_rearm_failures;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 63: `uint32_t exceptions_seen;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 64: `uint32_t first_chance_seen;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 65: `uint32_t second_chance_seen;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 67: `uint32_t unwind_handler_bp_set;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 68: `uint32_t unwind_handler_hits;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 70: `uint32_t veh_registrations;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 71: `uint32_t veh_hits;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 72: `uint32_t vch_registrations;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 73: `uint32_t vch_hits;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 74: `uint32_t uef_set_calls;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 75: `uint32_t uef_hits;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 77: `uint32_t rtl_add_function_table_calls;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 78: `uint32_t rtl_install_function_table_cb_calls;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 80: `uint32_t event_count;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 81: `uint32_t dropped_events;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 82: `DynamicEvent events[DYNAMIC_MAX_EVENTS];` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 84: `/* Execution trace sampled after first handler hit (RIP addresses). */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 85: `uint32_t trace_count;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 86: `uint64_t trace[256];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 88: `/* Human readable details */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 89: `StrBuf details;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 90: `} DynamicReport;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 92: `void dynamic_report_init(DynamicReport* dr);` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 93: `void dynamic_report_free(DynamicReport* dr);` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 95: `/*` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 96: `Run target under a debugger and collect evidence.` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 98: `- pe_path: path to target executable` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 99: `- child_argv/child_argc: optional args (may be NULL/0)` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 100: `- img/rf/us/us_ok: results of static parsing (used to set handler breakpoints)` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 101: `- rf_count: number of runtime functions` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 102: `- timeout_ms: hard timeout for debug loop` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 103: `- trace_steps: number of single-steps to record after handler hit (<=256 recommended)` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 105: `Returns true if the dynamic session ran (even if it found no evidence).` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 106: `*/` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 107: `bool dynamic_collect_windows(const char* pe_path,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 108: `const char* const* child_argv,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 109: `int child_argc,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 110: `const PeImage* img,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 111: `const RuntimeFunction* rf,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 112: `const UnwindSummary* us,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 113: `const bool* us_ok,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 114: `size_t rf_count,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 115: `uint32_t timeout_ms,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 116: `uint32_t trace_steps,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 117: `DynamicReport* out,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 118: `StrBuf* err);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 120: `#ifdef __cplusplus` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 121: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 122: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Файл src/util.c

Файл содержит 348 строк. Ниже идут непустые строки с учебным объяснением. Смысл упражнения - не заучить комментарий, а научиться самостоятельно выполнять такой разбор.

Строка 1: `#include "util.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 2: `#include <stdio.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 3: `#include <stdlib.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 4: `#include <string.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 5: `#include <stdarg.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 7: `#if defined(_WIN32)` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 8: `#include <windows.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 9: `#else` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 10: `#include <unistd.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 11: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 13: `bool buf_init_from_file(Buf* out, const char* path, uint8_t** owned_mem) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 14: `if (!out || !path || !owned_mem) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 15: `*owned_mem = NULL;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 16: `FILE* f = fopen(path, "rb");` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 17: `if (!f) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 19: `if (fseek(f, 0, SEEK_END) != 0) { fclose(f); return false; }` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 20: `long sz = ftell(f);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 21: `if (sz < 0) { fclose(f); return false; }` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 22: `if (fseek(f, 0, SEEK_SET) != 0) { fclose(f); return false; }` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 24: `size_t alloc_size = sz > 0 ? (size_t)sz : 1u;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 25: `uint8_t* mem = (uint8_t*)malloc(alloc_size);` - Эта строка выделяет память в heap. После неё нужно проверить результат на NULL и определить, кто будет освобождать ресурс.

Строка 26: `if (!mem) { fclose(f); return false; }` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 28: `size_t rd = fread(mem, 1, (size_t)sz, f);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 29: `fclose(f);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 30: `if (rd != (size_t)sz) { free(mem); return false; }` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 32: `out->data = mem;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 33: `out->size = (size_t)sz;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 34: `*owned_mem = mem;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 35: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 36: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 38: `void buf_free_owned(uint8_t* owned_mem) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 39: `free(owned_mem);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 40: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 42: `bool buf_read(const Buf* b, size_t off, void* out, size_t len) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 43: `if (!b || !out) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 44: `if (off > b->size) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 45: `if (len > b->size - off) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 46: `memcpy(out, b->data + off, len);` - Эта строка копирует указанное число bytes. `memcpy` не знает границ буфера, поэтому корректность должна быть доказана проверками раньше.

Строка 47: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 48: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 50: `bool buf_read_u8(const Buf* b, size_t off, uint8_t* out) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 51: `return buf_read(b, off, out, 1);` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 52: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 53: `bool buf_read_u16(const Buf* b, size_t off, uint16_t* out) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 54: `uint8_t tmp[2];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 55: `if (!buf_read(b, off, tmp, 2)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 56: `*out = le16(tmp);` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 57: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 58: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 59: `bool buf_read_u32(const Buf* b, size_t off, uint32_t* out) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 60: `uint8_t tmp[4];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 61: `if (!buf_read(b, off, tmp, 4)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 62: `*out = le32(tmp);` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 63: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 64: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 65: `bool buf_read_u64(const Buf* b, size_t off, uint64_t* out) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 66: `uint8_t tmp[8];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 67: `if (!buf_read(b, off, tmp, 8)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 68: `*out = le64(tmp);` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 69: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 70: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 72: `static bool sb_reserve(StrBuf* sb, size_t need_cap) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 73: `if (!sb || sb->failed) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 74: `if (need_cap <= sb->cap) return true;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 75: `size_t newcap = sb->cap ? sb->cap : 128;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 76: `while (newcap < need_cap) {` - Это цикл с условием продолжения. При чтении нужно понять, что изменяется внутри цикла и почему цикл обязательно завершится.

Строка 77: `size_t increment = newcap / 2u + 64u;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 78: `if (increment > SIZE_MAX - newcap) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 79: `newcap = need_cap;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 80: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 81: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 82: `newcap += increment;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 83: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 84: `char* p = (char*)realloc(sb->s, newcap);` - Эта строка изменяет размер heap-буфера. Её нужно читать особенно аккуратно: при ошибке нельзя потерять старый указатель, иначе возникнет утечка памяти.

Строка 85: `if (!p) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 86: `sb->failed = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 87: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 88: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 89: `sb->s = p;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 90: `sb->cap = newcap;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 91: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 92: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 94: `bool sb_init(StrBuf* sb, size_t initial_cap) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 95: `if (!sb) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 96: `sb->s = NULL;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 97: `sb->len = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 98: `sb->cap = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 99: `sb->failed = false;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 100: `if (initial_cap == 0) initial_cap = 256;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 101: `if (!sb_reserve(sb, initial_cap) || !sb->s) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 102: `sb->failed = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 103: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 104: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 105: `sb->s[0] = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 106: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 107: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 109: `void sb_free(StrBuf* sb) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 110: `if (!sb) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 111: `free(sb->s);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 112: `sb->s = NULL;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 113: `sb->len = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 114: `sb->cap = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 115: `sb->failed = false;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 116: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 118: `bool sb_appendn(StrBuf* sb, const char* s, size_t n) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 119: `if (!sb) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 120: `if (sb->failed || (!s && n != 0)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 121: `sb->failed = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 122: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 123: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 124: `if (sb->len == SIZE_MAX || n > SIZE_MAX - sb->len - 1u) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 125: `sb->failed = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 126: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 127: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 128: `if (!sb_reserve(sb, sb->len + n + 1)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 129: `if (n != 0) memcpy(sb->s + sb->len, s, n);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 130: `sb->len += n;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 131: `sb->s[sb->len] = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 132: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 133: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 135: `bool sb_append(StrBuf* sb, const char* s) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 136: `if (!sb) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 137: `if (!s) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 138: `sb->failed = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 139: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 140: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 141: `return sb_appendn(sb, s, strlen(s));` - Здесь используется функция работы со строками C. Нужно помнить, что строка C завершается нулевым байтом, а указатель на строку сам по себе не хранит длину.

Строка 142: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 144: `bool sb_appendf(StrBuf* sb, const char* fmt, ...) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 145: `if (!sb) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 146: `if (sb->failed || !fmt) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 147: `sb->failed = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 148: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 149: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 150: `va_list ap;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 151: `va_start(ap, fmt);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 152: `va_list ap2;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 153: `va_copy(ap2, ap);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 155: `#if defined(__GNUC__) || defined(__clang__)` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 156: `#pragma GCC diagnostic push` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 157: `#pragma GCC diagnostic ignored "-Wformat-nonliteral"` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 158: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 159: `int need = vsnprintf(NULL, 0, fmt, ap);` - Эта строка форматирует текст с ограничением размера буфера. В C это важнее, чем неограниченная запись, потому что фиксированные массивы `char` нельзя переполнять.

Строка 160: `va_end(ap);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 161: `if (need < 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 162: `va_end(ap2);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 163: `sb->failed = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 164: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 165: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 167: `if (sb->len == SIZE_MAX || (size_t)need > SIZE_MAX - sb->len - 1u) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 168: `va_end(ap2);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 169: `sb->failed = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 170: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 171: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 172: `if (!sb_reserve(sb, sb->len + (size_t)need + 1u)) { va_end(ap2); return false; }` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 173: `int wrote = vsnprintf(sb->s + sb->len, sb->cap - sb->len, fmt, ap2);` - Эта строка форматирует текст с ограничением размера буфера. В C это важнее, чем неограниченная запись, потому что фиксированные массивы `char` нельзя переполнять.

Строка 174: `#if defined(__GNUC__) || defined(__clang__)` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 175: `#pragma GCC diagnostic pop` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 176: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 177: `va_end(ap2);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 178: `if (wrote != need) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 179: `sb->failed = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 180: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 181: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 182: `sb->len += (size_t)wrote;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 183: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 184: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 186: `bool sb_ok(const StrBuf* sb) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 187: `return sb && !sb->failed && sb->s && sb->len < sb->cap;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 188: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 190: `bool file_write_atomic(const char* path, const char* data, size_t len) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 191: `if (!path || (!data && len != 0)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 192: `size_t path_len = strlen(path);` - Здесь используется функция работы со строками C. Нужно помнить, что строка C завершается нулевым байтом, а указатель на строку сам по себе не хранит длину.

Строка 193: `if (path_len > SIZE_MAX - 48u) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 195: `size_t temp_size = path_len + 48u;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 196: `char* temp_path = (char*)malloc(temp_size);` - Эта строка выделяет память в heap. После неё нужно проверить результат на NULL и определить, кто будет освобождать ресурс.

Строка 197: `if (!temp_path) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 198: `#if defined(_WIN32)` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 199: `unsigned long process_id = (unsigned long)GetCurrentProcessId();` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 200: `#else` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 201: `unsigned long process_id = (unsigned long)getpid();` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 202: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 203: `int written = snprintf(temp_path, temp_size, "%s.tmp.%lu", path, process_id);` - Эта строка форматирует текст с ограничением размера буфера. В C это важнее, чем неограниченная запись, потому что фиксированные массивы `char` нельзя переполнять.

Строка 204: `if (written < 0 || (size_t)written >= temp_size) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 205: `free(temp_path);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 206: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 207: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 209: `FILE* f = fopen(temp_path, "wb");` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 210: `if (!f) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 211: `free(temp_path);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 212: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 213: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 214: `size_t wr = len ? fwrite(data, 1, len, f) : 0u;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 215: `bool ok = wr == len && ferror(f) == 0 && fflush(f) == 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 216: `if (fclose(f) != 0) ok = false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 218: `if (ok) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 219: `#if defined(_WIN32)` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 220: `ok = MoveFileExA(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 221: `temp_path,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 222: `path,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 223: `MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRITE_THROUGH) != 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 224: `#else` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 225: `ok = rename(temp_path, path) == 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 226: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 227: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 228: `if (!ok) (void)remove(temp_path);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 229: `free(temp_path);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 230: `return ok;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 231: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 233: `typedef struct Sha256Ctx {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 234: `uint32_t state[8];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 235: `uint64_t bit_count;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 236: `uint8_t block[64];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 237: `size_t block_len;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 238: `} Sha256Ctx;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 240: `static uint32_t sha256_rotr(uint32_t value, unsigned count) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 241: `return (value >> count) | (value << (32u - count));` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 242: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 244: `static void sha256_transform(Sha256Ctx* ctx, const uint8_t block[64]) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 245: `static const uint32_t k[64] = {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 246: `0x428a2f98u,0x71374491u,0xb5c0fbcfu,0xe9b5dba5u,0x3956c25bu,0x59f111f1u,0x923f82a4u,0xab1c5ed5u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 247: `0xd807aa98u,0x12835b01u,0x243185beu,0x550c7dc3u,0x72be5d74u,0x80deb1feu,0x9bdc06a7u,0xc19bf174u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 248: `0xe49b69c1u,0xefbe4786u,0x0fc19dc6u,0x240ca1ccu,0x2de92c6fu,0x4a7484aau,0x5cb0a9dcu,0x76f988dau,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 249: `0x983e5152u,0xa831c66du,0xb00327c8u,0xbf597fc7u,0xc6e00bf3u,0xd5a79147u,0x06ca6351u,0x14292967u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 250: `0x27b70a85u,0x2e1b2138u,0x4d2c6dfcu,0x53380d13u,0x650a7354u,0x766a0abbu,0x81c2c92eu,0x92722c85u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 251: `0xa2bfe8a1u,0xa81a664bu,0xc24b8b70u,0xc76c51a3u,0xd192e819u,0xd6990624u,0xf40e3585u,0x106aa070u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 252: `0x19a4c116u,0x1e376c08u,0x2748774cu,0x34b0bcb5u,0x391c0cb3u,0x4ed8aa4au,0x5b9cca4fu,0x682e6ff3u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 253: `0x748f82eeu,0x78a5636fu,0x84c87814u,0x8cc70208u,0x90befffau,0xa4506cebu,0xbef9a3f7u,0xc67178f2u` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 254: `};` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 255: `uint32_t w[64];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 256: `for (size_t i = 0; i < 16; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 257: `size_t j = i * 4;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 258: `w[i] = ((uint32_t)block[j] << 24) |` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 259: `((uint32_t)block[j + 1] << 16) |` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 260: `((uint32_t)block[j + 2] << 8) |` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 261: `(uint32_t)block[j + 3];` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 262: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 263: `for (size_t i = 16; i < 64; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 264: `uint32_t s0 = sha256_rotr(w[i - 15], 7) ^ sha256_rotr(w[i - 15], 18) ^ (w[i - 15] >> 3);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 265: `uint32_t s1 = sha256_rotr(w[i - 2], 17) ^ sha256_rotr(w[i - 2], 19) ^ (w[i - 2] >> 10);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 266: `w[i] = w[i - 16] + s0 + w[i - 7] + s1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 267: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 269: `uint32_t a = ctx->state[0], b = ctx->state[1], c = ctx->state[2], d = ctx->state[3];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 270: `uint32_t e = ctx->state[4], f = ctx->state[5], g = ctx->state[6], h = ctx->state[7];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 271: `for (size_t i = 0; i < 64; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 272: `uint32_t s1 = sha256_rotr(e, 6) ^ sha256_rotr(e, 11) ^ sha256_rotr(e, 25);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 273: `uint32_t ch = (e & f) ^ ((~e) & g);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 274: `uint32_t t1 = h + s1 + ch + k[i] + w[i];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 275: `uint32_t s0 = sha256_rotr(a, 2) ^ sha256_rotr(a, 13) ^ sha256_rotr(a, 22);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 276: `uint32_t maj = (a & b) ^ (a & c) ^ (b & c);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 277: `uint32_t t2 = s0 + maj;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 278: `h = g; g = f; f = e; e = d + t1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 279: `d = c; c = b; b = a; a = t1 + t2;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 280: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 281: `ctx->state[0] += a; ctx->state[1] += b; ctx->state[2] += c; ctx->state[3] += d;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 282: `ctx->state[4] += e; ctx->state[5] += f; ctx->state[6] += g; ctx->state[7] += h;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 283: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 285: `static void sha256_init(Sha256Ctx* ctx) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 286: `static const uint32_t initial[8] = {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 287: `0x6a09e667u,0xbb67ae85u,0x3c6ef372u,0xa54ff53au,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 288: `0x510e527fu,0x9b05688cu,0x1f83d9abu,0x5be0cd19u` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 289: `};` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 290: `memcpy(ctx->state, initial, sizeof(initial));` - Эта строка копирует указанное число bytes. memcpy не знает границ буфера, поэтому корректность должна быть доказана проверками раньше.

Строка 291: `ctx->bit_count = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 292: `ctx->block_len = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 293: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 295: `static void sha256_update(Sha256Ctx* ctx, const uint8_t* data, size_t size) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 296: `if (size == 0) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 297: `ctx->bit_count += (uint64_t)size * 8u;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 298: `while (size > 0) {` - Это цикл с условием продолжения. При чтении нужно понять, что изменяется внутри цикла и почему цикл обязательно завершится.

Строка 299: `size_t available = 64u - ctx->block_len;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 300: `size_t take = size < available ? size : available;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 301: `memcpy(ctx->block + ctx->block_len, data, take);` - Эта строка копирует указанное число bytes. memcpy не знает границ буфера, поэтому корректность должна быть доказана проверками раньше.

Строка 302: `ctx->block_len += take;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 303: `data += take;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 304: `size -= take;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 305: `if (ctx->block_len == 64u) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 306: `sha256_transform(ctx, ctx->block);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 307: `ctx->block_len = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 308: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 309: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 310: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 312: `static void sha256_final(Sha256Ctx* ctx, uint8_t digest[32]) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 313: `ctx->block[ctx->block_len++] = 0x80;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 314: `if (ctx->block_len > 56u) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 315: `while (ctx->block_len < 64u) ctx->block[ctx->block_len++] = 0;` - Это цикл с условием продолжения. При чтении нужно понять, что изменяется внутри цикла и почему цикл обязательно завершится.

Строка 316: `sha256_transform(ctx, ctx->block);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 317: `ctx->block_len = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 318: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 319: `while (ctx->block_len < 56u) ctx->block[ctx->block_len++] = 0;` - Это цикл с условием продолжения. При чтении нужно понять, что изменяется внутри цикла и почему цикл обязательно завершится.

Строка 320: `for (int i = 7; i >= 0; i--) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 321: `ctx->block[ctx->block_len++] = (uint8_t)(ctx->bit_count >> (unsigned)(i * 8));` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 322: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 323: `sha256_transform(ctx, ctx->block);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 325: `for (size_t i = 0; i < 8; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 326: `digest[i * 4] = (uint8_t)(ctx->state[i] >> 24);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 327: `digest[i * 4 + 1] = (uint8_t)(ctx->state[i] >> 16);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 328: `digest[i * 4 + 2] = (uint8_t)(ctx->state[i] >> 8);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 329: `digest[i * 4 + 3] = (uint8_t)ctx->state[i];` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 330: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 331: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 333: `bool sha256_hex(const uint8_t* data, size_t size, char out_hex[65]) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 334: `static const char digits[] = "0123456789abcdef";` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 335: `if (!out_hex || (size > 0 && !data)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 336: `if ((uint64_t)size > UINT64_MAX / 8u) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 337: `Sha256Ctx ctx;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 338: `uint8_t digest[32];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 339: `sha256_init(&ctx);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 340: `sha256_update(&ctx, data, size);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 341: `sha256_final(&ctx, digest);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 342: `for (size_t i = 0; i < sizeof(digest); i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 343: `out_hex[i * 2] = digits[digest[i] >> 4];` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 344: `out_hex[i * 2 + 1] = digits[digest[i] & 0x0fu];` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 345: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 346: `out_hex[64] = '\0';` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 347: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 348: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Файл src/pe.c

Файл содержит 301 строк. Ниже идут непустые строки с учебным объяснением. Смысл упражнения - не заучить комментарий, а научиться самостоятельно выполнять такой разбор.

Строка 1: `#include "pe.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 2: `#include <stdlib.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 3: `#include <string.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 5: `static uint32_t u32_max(uint32_t a, uint32_t b) { return a > b ? a : b; }` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 7: `static bool is_power_of_two_u32(uint32_t value) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 8: `return value != 0 && (value & (value - 1u)) == 0;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 9: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 11: `static bool read_dos_lfanew(const Buf* b, uint32_t* out_lfanew) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 12: `uint16_t mz = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 13: `if (!buf_read_u16(b, 0, &mz)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 14: `if (mz != PE_DOS_MAGIC) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 15: `uint32_t lfanew = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 16: `if (!buf_read_u32(b, 0x3c, &lfanew)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше

разыменования и копирования памяти.

Строка 17: `*out_lfanew = lfanew;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 18: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 19: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 21: `static bool safe_add_u32(uint32_t a, uint32_t b, uint32_t* out) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 22: `uint64_t t = (uint64_t)a + (uint64_t)b;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 23: `if (t > 0xffffffffu) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 24: `*out = (uint32_t)t;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 25: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 26: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 28: `static bool ranges_overlap_u64(uint64_t a_begin,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 29: `uint64_t a_size,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 30: `uint64_t b_begin,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 31: `uint64_t b_size) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 32: `if (a_size == 0 || b_size == 0) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 33: `return a_begin < b_begin + b_size && b_begin < a_begin + a_size;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 34: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 36: `bool pe_parse(PeImage* out, const Buf* b, StrBuf* err) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 37: `if (!out || !b || !err) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 38: `memset(out, 0, sizeof(*out));` - Эта строка заполняет область памяти байтом, обычно нулём. В проекте это часто приводит структуру к известному начальному состоянию перед заполнением.

Строка 40: `uint32_t lfanew = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 41: `if (!read_dos_lfanew(b, &lfanew)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 42: `sb_append(err, "Not a valid MZ/PE file (DOS header check failed).\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 43: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 44: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 45: `if ((size_t)lfanew + 4 + 20 > b->size) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 46: `sb_append(err, "PE header offset out of range.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 47: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 48: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 50: `uint32_t sig = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 51: `if (!buf_read_u32(b, (size_t)lfanew, &sig) || sig != PE_NT_MAGIC) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 52: `sb_append(err, "Invalid PE signature.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 53: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 54: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 56: `size_t filehdr = (size_t)lfanew + 4;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 57: `uint16_t machine = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 58: `uint16_t nsects = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 59: `uint16_t opt_size = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 60: `if (!buf_read_u16(b, filehdr + 0, &machine) ||` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 61: `!buf_read_u16(b, filehdr + 2, &nsects) ||` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 62: `!buf_read_u16(b, filehdr + 16, &opt_size)) {` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 63: `sb_append(err, "Failed to read COFF header.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 64: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 65: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 67: `if (machine != IMAGE_FILE_MACHINE_AMD64) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 68: `sb_appendf(err, "Unsupported Machine: 0x%04x (only x64/AMD64 supported).\n", machine);` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 69: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 70: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 72: size_t opthdr = filehdr + 20; - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 73: if (opthdr + opt_size > b->size) { - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 74: sb_append(err, "Optional header out of range.\n"); - Эта строка добавляет текст в динамический StrBuf. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 75: return false; - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 76: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 77: if (opt_size < 0x70u) { - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 78: sb_append(err, "PE32+ optional header is too small for required fields.\n"); - Эта строка добавляет текст в динамический StrBuf. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 79: return false; - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 80: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 82: uint16_t magic = 0; - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 83: if (!buf_read_u16(b, opthdr + 0, &magic)) { - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 84: sb_append(err, "Failed to read optional header magic.\n"); - Эта строка добавляет текст в динамический StrBuf. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 85: return false; - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 86: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 87: if (magic != IMAGE_NT_OPTIONAL_HDR64_MAGIC) { - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 88: sb_appendf(err, "Unsupported OptionalHeader magic: 0x%04x (expect PE32+ 0x20b).\n", magic); - Эта строка добавляет текст в динамический StrBuf. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 89: return false; - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 90: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 92: uint64_t image_base = 0; - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 93: uint32_t size_of_image = 0; - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 94: uint32_t size_of_headers = 0; - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 95: uint32_t section_alignment = 0; - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 96: uint32_t file_alignment = 0; - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 97: `uint32_t number_of_rva_and_sizes = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 99: `/* PE32+ offsets */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 100: `if (!buf_read_u64(b, opthdr + 0x18, &image_base) ||` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 101: `!buf_read_u32(b, opthdr + 0x20, &section_alignment) ||` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 102: `!buf_read_u32(b, opthdr + 0x24, &file_alignment) ||` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 103: `!buf_read_u32(b, opthdr + 0x38, &size_of_image) ||` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 104: `!buf_read_u32(b, opthdr + 0x3c, &size_of_headers) ||` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 105: `!buf_read_u32(b, opthdr + 0x6c, &number_of_rva_and_sizes)) {` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 106: `sb_append(err, "Failed to read key fields from optional header.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 107: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 108: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 109: `if (size_of_image == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 110: `sb_append(err, "SizeOfImage is zero.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 111: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 112: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 113: `if (size_of_headers == 0 || (uint64_t)size_of_headers > (uint64_t)b->size) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 114: `sb_append(err, "SizeOfHeaders is zero or exceeds the file size.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 115: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 116: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 117: `if (section_alignment == 0 || file_alignment == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 118: `sb_append(err, "SectionAlignment/FileAlignment must be non-zero.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 119: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 120: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 121: `if (number_of_rva_and_sizes < IMAGE_DIRECTORY_ENTRY_EXCEPTION + 1) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 122: `sb_append(err, "DataDirectory too small.\n");` - Эта строка добавляет текст в динамический StrBuf. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 123: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 124: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 126: `/* DataDirectory starts at 0x70 */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 127: `size_t dd_rel = 0x70u + IMAGE_DIRECTORY_ENTRY_EXCEPTION * 8u;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 128: `if (dd_rel + 8u > opt_size) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 129: `sb_append(err, "Exception data directory lies outside the optional header.\n");` - Эта строка добавляет текст в динамический StrBuf. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 130: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 131: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 132: `size_t dd = opthdr + dd_rel;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 133: `uint32_t exc_rva = 0, exc_sz = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 134: `if (!buf_read_u32(b, dd + 0, &exc_rva) || !buf_read_u32(b, dd + 4, &exc_sz)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 135: `sb_append(err, "Failed to read exception directory.\n");` - Эта строка добавляет текст в динамический StrBuf. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 136: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 137: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 139: `size_t secthdr = opthdr + opt_size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 140: `if ((size_t)nsects > (SIZE_MAX - secthdr) / 40u) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 141: `sb_append(err, "Section header size overflow.\n");` - Эта строка добавляет текст в динамический StrBuf. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 142: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 143: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 144: `size_t needed = secthdr + (size_t)nsects * 40u;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 145: `if (needed > b->size) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 146: `sb_append(err, "Section headers out of range.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 147: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 148: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 150: `PeSection* secs = (PeSection*)calloc(nsects ? (size_t)nsects : 1, sizeof(PeSection));` - Эта строка выделяет память в heap. После неё нужно проверить результат на NULL и определить, кто будет освобождать ресурс.

Строка 151: `if (!secs) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 152: `sb_append(err, "Out of memory.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 153: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 154: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 156: `for (uint16_t i = 0; i < nsects; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 157: `size_t sh = secthdr + (size_t)i * 40;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 158: `uint8_t name8[8];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 159: `if (!buf_read(b, sh + 0, name8, 8)) { free(secs); sb_append(err, "Failed to read section name.\n"); return false; }` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 160: `memcpy(secs[i].name, name8, 8);` - Эта строка копирует указанное число bytes. `memcpy` не знает границ буфера, поэтому корректность должна быть доказана проверками раньше.

Строка 161: `secs[i].name[8] = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 163: `if (!buf_read_u32(b, sh + 8, &secs[i].virtual_size) ||` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 164: `!buf_read_u32(b, sh + 12, &secs[i].virtual_address) ||` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 165: `!buf_read_u32(b, sh + 16, &secs[i].raw_size) ||` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 166: `!buf_read_u32(b, sh + 20, &secs[i].raw_ptr) ||` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 167: `!buf_read_u32(b, sh + 36, &secs[i].characteristics)) {` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 168: `free(secs);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 169: `sb_append(err, "Failed to read section header.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 170: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 171: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 172: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 174: `size_t raw_oob = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 175: `size_t virtual_oob = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 176: `size_t raw_overlaps = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 177: `size_t virtual_overlaps = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 178: `size_t misaligned = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 180: `bool compact_alignment =` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 181: `section_alignment < 0x1000u && file_alignment == section_alignment;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 182: `bool file_alignment_valid =` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 183: `is_power_of_two_u32(file_alignment) &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 184: `((file_alignment >= 0x200u && file_alignment <= 0x10000u) ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 185: `compact_alignment);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 186: `if (!file_alignment_valid) misaligned++;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 187: `if (section_alignment < file_alignment) misaligned++;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 188: `if (section_alignment < 0x1000u && file_alignment != section_alignment) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 189: `misaligned++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 190: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 191: `if ((size_of_image % section_alignment) != 0) misaligned++;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 192: `if ((size_of_headers % file_alignment) != 0 ||` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 193: `(uint64_t)size_of_headers < (uint64_t)needed) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 194: `misaligned++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 195: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 196: `if ((image_base & 0xffffu) != 0) misaligned++;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 198: `for (size_t i = 0; i < (size_t)nsects; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 199: `const PeSection* s = &secs[i];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 200: `uint64_t raw_end = (uint64_t)s->raw_ptr + (uint64_t)s->raw_size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 201: `uint64_t virtual_span = (uint64_t)u32_max(s->virtual_size, s->raw_size);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 202: `uint64_t virtual_end = (uint64_t)s->virtual_address + virtual_span;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 203: `if (s->raw_size > 0 && raw_end > (uint64_t)b->size) raw_oob++;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 204: `if (virtual_span > 0 && virtual_end > (uint64_t)size_of_image) virtual_oob++;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 205: `if ((s->virtual_address % section_alignment) != 0 ||` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 206: `(s->raw_size > 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 207: `((s->raw_ptr % file_alignment) != 0 ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 208: `(s->raw_size % file_alignment) != 0))) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 209: `misaligned++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 210: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 211: `if (i > 0 && s->virtual_address <= secs[i - 1].virtual_address) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 212: `misaligned++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 213: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 215: `for (size_t j = 0; j < i; j++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 216: `const PeSection* p = &secs[j];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 217: `if (ranges_overlap_u64(s->raw_ptr, s->raw_size, p->raw_ptr, p->raw_size)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 218: `raw_overlaps++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 219: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 220: `if (ranges_overlap_u64(` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 221: `s->virtual_address,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 222: `u32_max(s->virtual_size, s->raw_size),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 223: `p->virtual_address,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 224: `u32_max(p->virtual_size, p->raw_size))) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 225: `virtual_overlaps++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 226: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 227: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 228: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 230: `bool exception_directory_oob = false;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 231: `if (exc_rva != 0 || exc_sz != 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 232: `uint64_t exc_end = (uint64_t)exc_rva + (uint64_t)exc_sz;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 233: `exception_directory_oob =` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 234: `exc_rva == 0 || exc_sz == 0 || exc_end > (uint64_t)size_of_image;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 235: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 237: `out->ok = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 238: `out->is_pe64 = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 239: `out->machine = machine;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 240: `out->image_base = image_base;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 241: `out->size_of_image = size_of_image;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 242: `out->size_of_headers = size_of_headers;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 243: `out->section_alignment = section_alignment;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 244: `out->file_alignment = file_alignment;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 245: `out->exception_rva = exc_rva;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 246: `out->exception_size = exc_sz;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 247: `out->file_size = b->size;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 248: `out->num_sections = nsects;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 249: `out->sections = secs;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 250: `out->section_raw_oob = raw_oob;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 251: `out->section_virtual_oob = virtual_oob;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 252: `out->section_raw_overlaps = raw_overlaps;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 253: `out->section_virtual_overlaps = virtual_overlaps;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 254: `out->section_misaligned = misaligned;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 255: `out->exception_directory_oob = exception_directory_oob;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 256: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 257: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 259: `void pe_free(PeImage* img) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 260: `if (!img) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 261: `free(img->sections);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 262: `memset(img, 0, sizeof(*img));` - Эта строка заполняет область памяти байтом, обычно нулём. В проекте это часто приводит структуру к известному начальному состоянию перед заполнением.

Строка 263: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 265: `const PeSection* pe_find_section_for_rva(const PeImage* img, uint32_t rva) {` - Эта строка связывает адресную модель PE с фактическими bytes файла. Она помогает не перепутать RVA, VA и file offset.

Строка 266: `if (!img || !img->sections) return NULL;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 267: `for (size_t i = 0; i < img->num_sections; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 268: `const PeSection* s = &img->sections[i];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 269: `uint32_t size = u32_max(s->virtual_size, s->raw_size);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 270: `uint64_t start = (uint64_t)s->virtual_address;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 271: `uint64_t end = start + (uint64_t)size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 272: `if ((uint64_t)rva >= start && (uint64_t)rva < end) return s;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает

безопасный путь, фиксирует ошибку или включает признак.

Строка 273: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 274: `return NULL;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 275: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 277: `bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 278: `if (!img || !out_off) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 279: `const PeSection* s = pe_find_section_for_rva(img, rva);` - Эта строка связывает адресную модель PE с фактическими bytes файла. Она помогает не перепутать RVA, VA и file offset.

Строка 280: `if (s) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 281: `uint32_t delta = rva - s->virtual_address;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 282: `if (delta >= s->raw_size) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 283: `/* RVA points into virtual tail (zero-fill) - not present in file */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 284: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 285: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 286: `uint32_t off = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 287: `if (!safe_add_u32(s->raw_ptr, delta, &off)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 288: `if (img->file_size > 0 && (uint64_t)off >= (uint64_t)img->file_size) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 289: `*out_off = off;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 290: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 291: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 293: `/* RVAs inside SizeOfHeaders map directly to the corresponding file offset. */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 294: `if (img->size_of_headers > 0 &&` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 295: `rva < img->size_of_headers &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 296: `(img->file_size == 0 || (uint64_t)rva < (uint64_t)img->file_size)) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 297: `*out_off = rva;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 298: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 299: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 300: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 301: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Файл src/unwind.c

Файл содержит 836 строк. Ниже идут непустые строки с учебным объяснением. Смысл упражнения - не заучить комментарий, а научиться самостоятельно выполнять такой разбор.

Строка 1: `#include "unwind.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 2: `#include <stdlib.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 3: `#include <string.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 5: `const char* unwind_reg_name(uint8_t reg) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 6: `/* mapping per MS x64 unwind encoding */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 7: `static const char* names[16] = {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 8: `"RAX","RCX","RDX","RBX","RSP","RBP","RSI","RDI",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 9: `"R8","R9","R10","R11","R12","R13","R14","R15"` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 10: `};` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 11: `if (reg < 16) return names[reg];` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 12: `return "REG?";` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 13: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 15: `void unwind_flags_to_string(uint8_t flags, char out[64]) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 16: `out[0] = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 17: `bool first = true;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 18: `if (flags & UNW_FLAG_EHANDLER) { strcat(out, "EHANDLER"); first = false; }` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 19: `if (flags & UNW_FLAG_UHANDLER) { strcat(out, first ? "UHANDLER" : "|UHANDLER"); first = false; }` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 20: `if (flags & UNW_FLAG_CHAININFO){ strcat(out, first ? "CHAININFO" : "|CHAININFO"); first = false; }` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 21: `if (first) strcpy(out, "0");` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 22: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 24: `static bool slots_for_code(uint8_t version,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 25: `uint8_t op,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 26: `uint8_t opinfo,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 27: `bool v2_seen_epilog,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 28: `uint8_t* out_slots) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 29: `/* based on documented v1 ops + v2 epilog/spare, and common implementations */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 30: `switch (op) {` - Это выбор поведения по дискретному коду. В проекте `switch` особенно важен для UWOP: номер операции определяет, сколько slots читать и как интерпретировать operands.

Строка 31: `case 0: /* UWOP_PUSH_NONVOL */ *out_slots = 1; return true;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 32: `case 1: /* UWOP_ALLOC_LARGE */ if (opinfo > 1) return false; *out_slots = (uint8_t)(2 + opinfo); return true;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 33: `case 2: /* UWOP_ALLOC_SMALL */ *out_slots = 1; return true;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 34: `case 3: /* UWOP_SET_FPREG */ *out_slots = 1; return true;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 35: `case 4: /* UWOP_SAVE_NONVOL */ *out_slots = 2; return true;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 36: `case 5: /* UWOP_SAVE_NONVOL_FAR */ *out_slots = 3; return true;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 37: `case 6: /* v2 UWOP_EPILOG, v1 UWOP_SAVE_XMM (undoc) */` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 38: `if (version == 2) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 39: `/* The first long-form epilog descriptor has one operand slot.` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 40: `Short-form and subsequent descriptors occupy one slot. */` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 41: `*out_slots = (!v2_seen_epilog && (opinfo & 1u) == 0u) ? 2u : 1u;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 42: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 43: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 44: `*out_slots = 2; return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 45: `case 7: /* v2 UWOP_SPARE, v1 UWOP_SAVE_XMM_FAR (undoc) */` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 46: `if (version == 2) { *out_slots = 1; return true; }` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 47: `*out_slots = 3; return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 48: `case 8: /* UWOP_SAVE_XMM128 */ *out_slots = 2; return true;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 49: `case 9: /* UWOP_SAVE_XMM128_FAR */ *out_slots = 3; return true;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 50: `case 10: /* UWOP_PUSH_MACHFRAME */ *out_slots = 1; return true;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 51: `default:` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 52: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 53: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 54: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 56: `static bool parse_unwindinfo_header(const Buf* b, const PeImage* img, uint32_t unwind_rva,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 57: `uint32_t* out_fileoff, uint8_t hdr[4], StrBuf* err) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 58: `uint32_t off = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 59: `if (!pe_rva_to_fileoff(img, unwind_rva, &off)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 60: `sb_appendf(err, "UnwindInfo RVA 0x%08x not mappable to file offset.\n", unwind_rva);` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 61: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 62: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 63: `if ((size_t)off + 4 > b->size) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 64: `sb_append(err, "UnwindInfo header out of file range.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 65: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 66: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 67: `if (!buf_read(b, off, hdr, 4)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 68: `sb_append(err, "Failed to read UnwindInfo header bytes.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 69: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 70: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 71: `*out_fileoff = off;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 72: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 73: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 75: `static bool read_u16_slot(const Buf* b, size_t off, uint16_t* out) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 76: `uint8_t tmp[2];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 77: `if (!buf_read(b, off, tmp, 2)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 78: `*out = (uint16_t)(tmp[0] | ((uint16_t)tmp[1] << 8));` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 79: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 80: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 83: `static uint32_t clamp_size_to_u32(size_t value) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 84: `return value > UINT32_MAX ? UINT32_MAX : (uint32_t)value;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 85: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 87: `static size_t min_size(size_t a, size_t b) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 88: `return a < b ? a : b;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 89: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 91: `static size_t safe_add_size(size_t a, size_t b) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 92: `if (SIZE_MAX - a < b) return SIZE_MAX;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 93: `return a + b;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 94: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 96: `static size_t unwind_record_limit(const Buf* b, const PeImage* img, uint32_t unwind_rva) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 97: `size_t limit = b ? b->size : 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 98: `const PeSection* section = pe_find_section_for_rva(img, unwind_rva);` - Эта строка связывает адресную модель PE с фактическими bytes файла. Она помогает не перепутать RVA, VA и file offset.

Строка 99: `if (section && section->raw_size > 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 100: `size_t raw_start = (size_t)section->raw_ptr;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 101: `size_t raw_end = safe_add_size(raw_start, (size_t)section->raw_size);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 102: `limit = min_size(limit, raw_end);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 103: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 104: `return limit;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 105: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 107: `static size_t available_until_limit(const Buf* b, size_t off, size_t limit) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 108: `if (!b) return 0;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 109: `size_t end = min_size(b->size, limit);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 110: `if (off >= end) return 0;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 111: `return end - off;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 112: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 114: `static void unwind_diag_capture_bytes(UnwindSummary* out,` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 115: `const Buf* b,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 116: `size_t file_offset,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 117: `size_t max_count) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 118: `if (!out || !b || file_offset >= b->size || max_count == 0) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 119: `size_t count = b->size - file_offset;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 120: `if (count > max_count) count = max_count;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 121: `if (count > UNWIND_DIAG_MAX_BYTES) count = UNWIND_DIAG_MAX_BYTES;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 122: `memcpy(out->diagnostic_bytes, b->data + file_offset, count);` - Эта строка копирует указанное число bytes. `memcpy` не знает границ буфера, поэтому корректность должна быть доказана проверками раньше.

Строка 123: `out->diagnostic_bytes_count = (uint8_t)count;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 124: `out->diagnostic_bytes_file_offset = clamp_size_to_u32(file_offset);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 125: `out->diagnostic_has_bytes = count > 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 126: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 128: `static void unwind_diag_set(UnwindSummary* out,` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 129: `const Buf* b,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 130: `const char* field,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 131: `bool has_file_offset,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 132: `size_t file_offset,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 133: `bool has_size,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 134: `size_t expected_size,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 135: `size_t available_size,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 136: `bool has_slots,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 137: `uint32_t expected_slots,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 138: `uint32_t available_slots) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 139: `if (!out) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 140: `if (field && field[0]) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 141: `size_t n = strlen(field);` - Здесь используется функция работы со строками C. Нужно помнить, что строка C завершается нулевым байтом, а указатель на строку сам по себе не хранит длину.

Строка 142: `if (n >= sizeof(out->diagnostic_field)) n = sizeof(out->diagnostic_field) - 1;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 143: `memcpy(out->diagnostic_field, field, n);` - Эта строка копирует указанное число bytes. `memcpy` не знает границ буфера, поэтому корректность должна быть доказана проверками раньше.

Строка 144: `out->diagnostic_field[n] = '\0';` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 145: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 146: `out->diagnostic_has_file_offset = has_file_offset;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 147: `if (has_file_offset) out->diagnostic_file_offset = clamp_size_to_u32(file_offset);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 148: `out->diagnostic_has_size = has_size;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 149: `if (has_size) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 150: `out->diagnostic_expected_size = clamp_size_to_u32(expected_size);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 151: `out->diagnostic_available_size = clamp_size_to_u32(available_size);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 152: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 153: `out->diagnostic_has_slots = has_slots;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 154: `if (has_slots) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 155: `out->diagnostic_expected_slots = expected_slots;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 156: `out->diagnostic_available_slots = available_slots;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 157: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 158: `if (has_file_offset) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 159: `size_t max_bytes = has_size ? available_size : (size_t)UNWIND_DIAG_MAX_BYTES;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 160: `unwind_diag_capture_bytes(out, b, file_offset, max_bytes);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 161: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 162: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 164: `static void unwind_diag_set_read_failure(UnwindSummary* out,` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 165: `const Buf* b,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 166: `const char* field,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 167: `size_t file_offset,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 168: `size_t expected_size,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 169: `size_t limit) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 170: `size_t available = available_until_limit(b, file_offset, limit);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 171: `unwind_diag_set(out, b, field, true, file_offset, true,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 172: `expected_size, available, false, 0, 0);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 173: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 175: `static uint32_t read_u32_from_slots(uint16_t lo, uint16_t hi) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 176: `return (uint32_t)lo | ((uint32_t)hi << 16);` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 177: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 179: `static bool is_unwindable_gpr(uint8_t reg) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 180: `/* Real MSVC, Rust and OpenSSL records also describe volatile registers. */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 181: `return reg < 16 && reg != 4;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 182: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 184: `static bool is_unwindable_xmm(uint8_t reg) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 185: `return reg < 16;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 186: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 188: `static bool is_abi_nonvolatile_gpr(uint8_t reg) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 189: `return reg == 3 || reg == 5 || reg == 6 || reg == 7 || reg >= 12;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 190: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 192: `static bool is_abi_nonvolatile_xmm(uint8_t reg) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 193: `return reg >= 6 && reg < 16;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 194: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 196: `static void add_fixed_stack(uint32_t* total, uint32_t amount, uint32_t* anomalies) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 197: `if (!total || !anomalies) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 198: `if (UINT32_MAX - *total < amount) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 199: `*total = UINT32_MAX;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 200: `*anomalies |= UA_OPERAND_SANITY;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 201: `return;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 202: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 203: `*total += amount;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 204: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 206: `static bool alloc_large_mode0_suspicious(uint16_t scaled8) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 207: `uint32_t size = (uint32_t)scaled8 * 8u;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 208: `return size == 0 || size <= 128u || (size % 8u) != 0;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 209: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 211: `static bool alloc_large_mode1_suspicious(uint32_t size) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 212: `return size == 0 || (size % 8u) != 0 || size < (512u * 1024u);` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 213: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 215: `static bool save_nonvol_far_suspicious(uint32_t off) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 216: `return off == 0 || (off % 8u) != 0 || off < (512u * 1024u);` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 217: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 219: `static bool save_xmm_far_suspicious(uint32_t off) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 220: `return off == 0 || (off % 16u) != 0 || off < (1024u * 1024u);` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 221: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 223: `static bool parse_chain_rf(const Buf* b, const PeImage* img, size_t fileoff_after_codes,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 224: `RuntimeFunction* out_chain) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 225: `uint32_t a=0, c=0, u=0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 226: `if (!buf_read_u32(b, fileoff_after_codes + 0, &a)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 227: `if (!buf_read_u32(b, fileoff_after_codes + 4, &c)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 228: `if (!buf_read_u32(b, fileoff_after_codes + 8, &u)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 229: `out_chain->begin_rva = a;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 230: `out_chain->end_rva = c;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 231: `out_chain->unwind_rva = u;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 232: `(void)img;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 233: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 234: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 236: `static bool unwind_parse_fail(UnwindSummary* out,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 237: `StrBuf* err,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 238: `size_t err_start,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 239: `const char* fallback) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 240: `if (!out) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 241: `out->valid = false;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 243: `const char* message = fallback ? fallback : "UNWIND_INFO parse failed.";` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 244: `if (err && err->s && err->len > err_start) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 245: `message = err->s + err_start;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 246: `} else if (err && fallback) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 247: `sb_appendf(err, "%s\n", fallback);` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 248: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 250: `size_t n = strlen(message);` - Здесь используется функция работы со строками C. Нужно помнить, что строка C завершается нулевым байтом, а указатель на строку сам по себе не хранит длину.

Строка 251: `while (n > 0 && (message[n - 1] == '\n' || message[n - 1] == '\r')) n--;` - Это цикл с условием продолжения. При чтении нужно понять, что изменяется внутри цикла и почему цикл обязательно завершится.

Строка 252: `if (n >= sizeof(out->parse_error)) n = sizeof(out->parse_error) - 1;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 253: `memcpy(out->parse_error, message, n);` - Эта строка копирует указанное число bytes. `memcpy` не знает границ буфера, поэтому корректность должна быть доказана проверками раньше.

Строка 254: `out->parse_error[n] = '\0';` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 255: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 256: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 258: `bool unwind_parse(const Buf* b,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 259: `const PeImage* img,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 260: `uint32_t unwind_rva,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 261: `const UnwindParseOpts* opts,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 262: `UnwindSummary* out,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 263: `StrBuf* ops_text,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 264: `StrBuf* err) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 265: `if (!b || !img || !out || !err) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 266: `memset(out, 0, sizeof(*out));` - Эта строка заполняет область памяти байтом, обычно нулём. В проекте это часто приводит структуру к известному начальному состоянию перед заполнением.

Строка 267: `size_t err_start = err->len;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 268: `if (ops_text) { ops_text->len = 0; if (ops_text->s) ops_text->s[0]=0; }` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 270: `int max_depth = 8;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 271: `bool follow_chain = true;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 272: `bool strict_abi = false;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 273: `if (opts) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 274: `if (opts->max_chain_depth > 0) max_depth = opts->max_chain_depth;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 275: `follow_chain = opts->follow_chain;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 276: `strict_abi = opts->strict_abi;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 277: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 278: `if (max_depth > 16) max_depth = 16;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 280: `uint8_t hdr[4];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 281: `uint32_t uoff = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 282: `uint32_t attempted_uoff = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 283: `bool attempted_mapped = pe_rva_to_fileoff(img, unwind_rva, &attempted_uoff);` - Эта строка связывает адресную модель PE с фактическими bytes файла. Она помогает не перепутать RVA, VA и file offset.

Строка 284: `if ((unwind_rva & 3u) != 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 285: `out->anomalies |= UA_ALIGNMENT;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 286: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 287: `if (!parse_unwindinfo_header(b, img, unwind_rva, &uoff, hdr, err)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 288: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 289: `unwind_diag_set(out, b, "UnwindInfoAddress / UNWIND_INFO header",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 290: `attempted_mapped, attempted_mapped ? (size_t)attempted_uoff : 0u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 291: `attempted_mapped, 4u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 292: `attempted_mapped ? available_until_limit(b, (size_t)attempted_uoff, b->size) : 0u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 293: `false, 0, 0);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 294: `return unwind_parse_fail(out, err, err_start, "Failed to read UNWIND_INFO header.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 295: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 296: `size_t record_limit = unwind_record_limit(b, img, unwind_rva);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 298: `uint8_t ver = (uint8_t)(hdr[0] & 0x7);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 299: `uint8_t flags = (uint8_t)(hdr[0] >> 3);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 300: `uint8_t prolog = hdr[1];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 301: `uint8_t count = hdr[2];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 302: `uint8_t fr = (uint8_t)(hdr[3] & 0x0f);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 303: `uint8_t foff = (uint8_t)(hdr[3] >> 4);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 305: `out->version = ver;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 306: `out->flags = flags;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 307: `out->prolog_size = prolog;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 308: `out->count_codes = count;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 309: `out->frame_reg = fr;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 310: `out->frame_off = foff;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 312: `if (!(ver == 1 || ver == 2)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 313: `out->anomalies |= UA_VERSION_UNKNOWN;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 314: `unwind_diag_set(out, b, "Version", true, (size_t)uoff, true, 1u, 1u, false, 0, 0);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 315: `return unwind_parse_fail(out, err, err_start, "Unsupported UNWIND_INFO version.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 316: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 317: `if (flags & (uint8_t)~(UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER | UNW_FLAG_CHAININFO)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 318: `out->anomalies |= UA_BAD_OPERAND;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 319: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 320: `if ((flags & UNW_FLAG_CHAININFO) && (flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER))) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 321: `out->anomalies |= UA_BAD_OPERAND;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 322: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 323: `if (fr == 0 && foff != 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 324: `out->anomalies |= UA_BAD_OPERAND;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 325: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 326: `if (fr != 0 && !is_unwindable_gpr(fr)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 327: `out->anomalies |= UA_BAD_OPERAND;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 328: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 329: `if (strict_abi && fr != 0 && !is_abi_nonvolatile_gpr(fr)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 330: `out->anomalies |= UA_ABI_REGISTER_CLASS;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 331: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 333: `/* total slots stored on disk (aligned to even number) */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 334: `uint32_t slots_even = (uint32_t)((count + 1u) & ~1u);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 335: `size_t codes_off = (size_t)uoff + 4;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 336: `size_t codes_bytes = (size_t)slots_even * 2u;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 337: `size_t codes_available = available_until_limit(b, codes_off, record_limit);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 338: `if (codes_bytes > codes_available) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 339: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 340: `unwind_diag_set(out, b, "CountOfCodes / UNWIND_CODE array",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 341: `true, codes_off, true, codes_bytes, codes_available,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 342: `true, slots_even, (uint32_t)(codes_available / 2u));` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 343: `sb_append(err, "Unwind codes array out of file range.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 344: `return unwind_parse_fail(out, err, err_start, "Unwind codes array out of file range.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 345: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 347: `uint32_t fixed_stack = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 348: `uint32_t pushed = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 349: `uint16_t nonvol_mask = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 350: `uint16_t xmm_mask = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 351: `bool have_prev_code_offset = false;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 352: `uint8_t prev_code_offset = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 353: `bool saw_set_fpreg = false;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 354: `bool saw_push_nonvol = false;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 355: `bool v2_seen_epilog = false;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 357: `/* parse slots 0..count-1 as unwind code starts */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 358: `for (uint32_t i = 0; i < count; ) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 359: `uint16_t slot = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 360: `size_t slot_fileoff = codes_off + (size_t)i * 2u;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 361: `if (!read_u16_slot(b, slot_fileoff, &slot)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 362: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 363: `unwind_diag_set_read_failure(out, b, "UNWIND_CODE slot", slot_fileoff, 2u, record_limit);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 364: `sb_append(err, "Failed to read unwind slot.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 365: `return unwind_parse_fail(out, err, err_start, "Failed to read unwind slot.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 366: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 368: `uint8_t code_offset = (uint8_t)(slot & 0xff);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 369: `uint8_t op_byte = (uint8_t)(slot >> 8);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 370: `uint8_t op = (uint8_t)(op_byte & 0x0f);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 371: `uint8_t opinfo = (uint8_t)(op_byte >> 4);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 373: `uint8_t slots = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 374: `if (!slots_for_code(ver, op, opinfo, v2_seen_epilog, &slots)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 375: `out->anomalies |= UA_BAD_OPCODE;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 376: `/* try to continue safely as 1 slot */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 377: `slots = 1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 378: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 379: `if (i + slots > count) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 380: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 381: `unwind_diag_set(out, b, "CountOfCodes / UWOP operand slots",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 382: `true, slot_fileoff, true, (size_t)slots * 2u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 383: `(size_t)(count - i) * 2u, true, slots, count - i);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 384: `sb_append(err, "Unwind opcode spans beyond CountOfCodes.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 385: `return unwind_parse_fail(out, err, err_start, "Unwind opcode spans beyond CountOfCodes.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 386: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 387: `bool v2_descriptor = ver == 2 && (op == 6 || op == 7);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 388: `if (!v2_descriptor) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 389: `if (code_offset > prolog) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 390: `out->anomalies |= UA_CODEOFFSET_OUTSIDE;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 391: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 392: `if (have_prev_code_offset && code_offset > prev_code_offset) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 393: `out->anomalies |= UA_CODEOFFSET_ORDER;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 394: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 395: `have_prev_code_offset = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 396: `prev_code_offset = code_offset;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 397: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 398: `if (strict_abi && !v2_descriptor) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 399: `if (saw_push_nonvol && op != 0 && op != 10) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 400: `out->anomalies |= UA_ABI_PUSH_ORDER;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 401: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 402: `if (op == 0) saw_push_nonvol = true;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 403: `if ((flags & UNW_FLAG_CHAININFO) && op != 4) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь,

фиксирует ошибку или включает признак.

Строка 404: `out->anomalies |= UA_ABI_CHAIN_RESTRICTION;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 405: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 406: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 408: `if (ops_text) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 409: `/* print one-line per op start */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 410: `switch (op) {` - Это выбор поведения по дискретному коду. В проекте `switch` особенно важен для UWOP: номер операции определяет, сколько slots читать и как интерпретировать operands.

Строка 411: `case 0: sb_appendf(ops_text, " @%02x PUSH_NONVOL %s\n", code_offset, unwind_reg_name(opinfo)); break;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 412: `case 1: sb_appendf(ops_text, " @%02x ALLOC_LARGE mode=%u\n", code_offset, (unsigned)opinfo); break;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 413: `case 2: sb_appendf(ops_text, " @%02x ALLOC_SMALL %u\n", code_offset, (unsigned)(opinfo*8u + 8u)); break;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 414: `case 3: sb_appendf(ops_text, " @%02x SET_FPREG %s + %u\n", code_offset, unwind_reg_name(fr), (unsigned)(foff*16u)); break;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 415: `case 4: sb_appendf(ops_text, " @%02x SAVE_NONVOL %s\n", code_offset, unwind_reg_name(opinfo)); break;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 416: `case 5: sb_appendf(ops_text, " @%02x SAVE_NONVOL_FAR %s\n", code_offset, unwind_reg_name(opinfo)); break;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 417: `case 6:` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 418: `if (ver == 2) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 419: `if (!v2_seen_epilog) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 420: `if ((opinfo & 1u) == 0u) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 421: `uint16_t epilog_slot = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 422: `if (read_u16_slot(b, codes_off + (size_t)(i + 1) * 2,` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 423: `&epilog_slot)) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 424: `unsigned epilog_offset =` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 425: `(unsigned)(epilog_slot & 0xffu) |` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 426: `((unsigned)(epilog_slot >> 12) << 8);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 427: `sb_appendf(ops_text,` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 428: `" EPILOG size=%u offset_from_end=%u flags=%u (v2)\n",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 429: `(unsigned)code_offset, epilog_offset,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 430: `(unsigned)opinfo);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 431: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 432: `} else {` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 433: `sb_appendf(ops_text, " EPILOG size=%u flags=%u (v2)\n",` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 434: `(unsigned)code_offset, (unsigned)opinfo);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 435: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 436: `} else {` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 437: `unsigned epilog_offset =` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 438: `(unsigned)code_offset | ((unsigned)opinfo << 8);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 439: `sb_appendf(ops_text, " EPILOG offset_from_end=%u (v2)\n",` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 440: `epilog_offset);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 441: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 442: `} else {` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 443: `sb_appendf(ops_text, " @%02x SAVE_XMM (undoc/v1)\n", code_offset);` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 444: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 445: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 446: `case 7:` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 447: `if (ver == 2) sb_appendf(ops_text, " @%02x SPARE (v2)\n", code_offset);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 448: `else sb_appendf(ops_text, " @%02x SAVE_XMM_FAR (undoc/v1)\n", code_offset);` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 449: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 450: `case 8: sb_appendf(ops_text, " @%02x SAVE_XMM128 XMM%u\n", code_offset, (unsigned)opinfo); break;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 451: `case 9: sb_appendf(ops_text, " @%02x SAVE_XMM128_FAR XMM%u\n", code_offset, (unsigned)opinfo); break;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-a или formatter-a.

Строка 452: `case 10: sb_appendf(ops_text, " @%02x PUSH_MACHFRAME err=%u\n", code_offset, (unsigned)opinfo); break;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-a или formatter-a.

Строка 453: `default: sb_appendf(ops_text, " @%02x OP_%u info=%u\n", code_offset, (unsigned)op, (unsigned)opinfo); break;` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-a или formatter-a.

Строка 454: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 455: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 457: `/* compute effects */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 458: `switch (op) {` - Это выбор поведения по дискретному коду. В проекте `switch` особенно важен для UWOP: номер операции определяет, сколько slots читать и как интерпретировать operands.

Строка 459: `case 0: /* push nonvol */` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-a или formatter-a.

Строка 460: `add_fixed_stack(&fixed_stack, 8, &out->anomalies);` - Это проверка битовой маски. Код выясняет, включён ли конкретный флаг или признак, не требуя, чтобы он был единственным включенным битом.

Строка 461: `pushed++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 462: `if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 463: `if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 464: `out->anomalies |= UA_ABI_REGISTER_CLASS;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 465: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 466: `if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 467: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 468: `case 1: { /* alloc large */` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-a или formatter-a.

Строка 469: `if (opinfo == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 470: `uint16_t scaled = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 471: `if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 472: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 473: `unwind_diag_set_read_failure(out, b, "UWOP_ALLOC_LARGE operand",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 474: `codes_off + (size_t)(i + 1) * 2u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 475: `2u, record_limit);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 476: `return unwind_parse_fail(out, err, err_start, "Truncated UWOP_ALLOC_LARGE operand.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 477: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 478: `add_fixed_stack(&fixed_stack, (uint32_t)scaled * 8u, &out->anomalies);` - Это проверка битовой маски. Код выясняет, включён ли конкретный флаг или признак, не требуя, чтобы он был единственным включенным битом.

Строка 479: `if (alloc_large_mode0_suspicious(scaled)) out->anomalies |= UA_OPERAND_SANITY;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 480: `if (ops_text) sb_appendf(ops_text, " size=%u\n", (unsigned)((uint32_t)scaled * 8u));` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 481: `} else if (opinfo == 1) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 482: `uint16_t lo=0, hi=0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 483: `if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 484: `!read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 485: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 486: `unwind_diag_set_read_failure(out, b, "UWOP_ALLOC_LARGE operand",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 487: `codes_off + (size_t)(i + 1) * 2u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 488: `4u, record_limit);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 489: `return unwind_parse_fail(out, err, err_start, "Truncated UWOP_ALLOC_LARGE operand.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 490: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 491: `uint32_t size = read_u32_from_slots(lo, hi);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 492: `add_fixed_stack(&fixed_stack, size, &out->anomalies);` - Это проверка битовой маски. Код выясняет, включён ли конкретный флаг или признак, не требуя, чтобы он был единственным включенным битом.

Строка 493: `if (alloc_large_mode1_suspicious(size)) out->anomalies |= UA_OPERAND_SANITY;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 494: `if (ops_text) sb_appendf(ops_text, " size=%u\n", (unsigned)size);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 495: `} else {` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 496: `out->anomalies |= UA_BAD_OPERAND;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 497: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 498: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 499: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 500: `case 2: /* alloc small */` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-a или formatter-a.

Строка 501: `add_fixed_stack(&fixed_stack, (uint32_t)opinfo * 8u + 8u, &out->anomalies);` - Это проверка битовой маски. Код выясняет, включён ли конкретный флаг или признак, не требуя, чтобы он был единственным включенным битом.

Строка 502: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 503: `case 3: /* set frame reg */` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-a или formatter-a.

Строка 504: `if (fr == 0 || saw_set_fpreg) out->anomalies |= UA_BAD_OPERAND;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 505: `if (strict_abi && opinfo != 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 506: `out->anomalies |= UA_ABI_RESERVED_OPINFO;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 507: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 508: `saw_set_fpreg = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 509: `/* no fixed stack delta */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 510: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 511: `case 4: { /* save nonvol */` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-a или formatter-a.

Строка 512: `uint16_t scaled = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 513: `if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 514: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 515: `unwind_diag_set_read_failure(out, b, "UWOP_SAVE_NONVOL operand",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 516: `codes_off + (size_t)(i + 1) * 2u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 517: `2u, record_limit);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 518: `return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_NONVOL operand.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 519: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 520: `if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 521: `if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 522: `out->anomalies |= UA_ABI_REGISTER_CLASS;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 523: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 524: `if (strict_abi && scaled == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 525: `out->anomalies |= UA_ABI_SAVE_OFFSET;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 526: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 527: `if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 528: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 529: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 530: `case 5: { /* save nonvol far */` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-a или formatter-a.

Строка 531: `uint16_t lo=0, hi=0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 532: `if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 533: `!read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 534: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 535: `unwind_diag_set_read_failure(out, b, "UWOP_SAVE_NONVOL_FAR operand",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 536: `codes_off + (size_t)(i + 1) * 2u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 537: `4u, record_limit);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 538: `return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_NONVOL_FAR operand.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 539: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 540: `uint32_t save_offset = read_u32_from_slots(lo, hi);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 541: `if (save_nonvol_far_suspicious(save_offset)) out->anomalies |= UA_OPERAND_SANITY;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 542: `if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 543: `if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 544: `out->anomalies |= UA_ABI_REGISTER_CLASS;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 545: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 546: `if (strict_abi && save_offset == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 547: `out->anomalies |= UA_ABI_SAVE_OFFSET;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 548: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 549: `if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 550: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 551: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 552: `case 6: { /* save xmm (v1, undocumented encoding) or epilog (v2) */` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 553: `if (ver != 2) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 554: `uint16_t scaled = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 555: `if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 556: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 557: `unwind_diag_set_read_failure(out, b, "UWOP_SAVE_XMM operand",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 558: `codes_off + (size_t)(i + 1) * 2u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 559: `2u, record_limit);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 560: `return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM operand.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 561: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 562: `if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 563: `if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 564: `out->anomalies |= UA_ABI_REGISTER_CLASS;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 565: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 566: `if (strict_abi && scaled == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 567: `out->anomalies |= UA_ABI_SAVE_OFFSET;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 568: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 569: `if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 570: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 571: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 572: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 573: `case 7: { /* save xmm far (v1, undocumented encoding) or spare (v2) */` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 574: `if (ver != 2) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 575: `uint16_t lo=0, hi=0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 576: `if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 577: `!read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 578: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 579: `unwind_diag_set_read_failure(out, b, "UWOP_SAVE_XMM_FAR operand",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 580: `codes_off + (size_t)(i + 1) * 2u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 581: `4u, record_limit);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 582: `return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM_FAR operand.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 583: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 584: `uint32_t save_offset = read_u32_from_slots(lo, hi);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 585: `if (save_xmm_far_suspicious(save_offset)) out->anomalies |= UA_OPERAND_SANITY;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 586: `if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 587: `if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 588: `out->anomalies |= UA_ABI_REGISTER_CLASS;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 589: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 590: `if (strict_abi && save_offset == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 591: `out->anomalies |= UA_ABI_SAVE_OFFSET;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 592: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 593: `if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 594: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 595: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 596: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 597: `case 8: { /* save xmm128 */` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-a или formatter-a.

Строка 598: `uint16_t scaled = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 599: `if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 600: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 601: `unwind_diag_set_read_failure(out, b, "UWOP_SAVE_XMM128 operand",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 602: `codes_off + (size_t)(i + 1) * 2u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 603: `2u, record_limit);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 604: `return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM128 operand.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 605: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 606: `if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 607: `if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 608: `out->anomalies |= UA_ABI_REGISTER_CLASS;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 609: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 610: `if (strict_abi && scaled == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 611: `out->anomalies |= UA_ABI_SAVE_OFFSET;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 612: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 613: `if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 614: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 615: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 616: `case 9: { /* save xmm128 far */` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-a или formatter-a.

Строка 617: `uint16_t lo=0, hi=0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 618: `if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 619: `!read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 620: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 621: `unwind_diag_set_read_failure(out, b, "UWOP_SAVE_XMM128_FAR operand",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 622: `codes_off + (size_t)(i + 1) * 2u,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 623: `4u, record_limit);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 624: `return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM128_FAR operand.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 625: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 626: `uint32_t save_offset = read_u32_from_slots(lo, hi);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 627: `if (save_xmm_far_suspicious(save_offset)) out->anomalies |= UA_OPERAND_SANITY;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 628: `if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 629: `if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 630: `out->anomalies |= UA_ABI_REGISTER_CLASS;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 631: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 632: `if (strict_abi && save_offset == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 633: `out->anomalies |= UA_ABI_SAVE_OFFSET;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 634: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 635: `if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 636: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 637: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 638: `case 10: /* push machframe */` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-a или formatter-a.

Строка 639: `if (opinfo > 1) out->anomalies |= UA_BAD_OPERAND;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 640: `add_fixed_stack(&fixed_stack, (opinfo == 0) ? 40u : 48u, &out->anomalies);` - Это проверка битовой маски. Код выясняет, включён ли конкретный флаг или признак, не требуя, чтобы он был единственным включенным битом.

Строка 641: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 642: `default:` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-a или formatter-a.

Строка 643: `/* ignore */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 644: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 645: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 647: `if (ver == 2 && op == 6) v2_seen_epilog = true;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 648: `i += slots;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 649: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 650: `if (fr != 0 && !saw_set_fpreg && !(flags & UNW_FLAG_CHAININFO)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 651: `out->anomalies |= UA_BAD_OPERAND;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 652: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 654: `/* tail (handler or chained info) */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 655: `size_t after_codes = codes_off + codes_bytes;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 657: `if (flags & UNW_FLAG_CHAININFO) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 658: `out->has_chain = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 659: `RuntimeFunction chain = {0};` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 660: `if (!parse_chain_rf(b, img, after_codes, &chain)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 661: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 662: `unwind_diag_set_read_failure(out, b, "CHAININFO RUNTIME_FUNCTION tail", after_codes, 12u, record_limit);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 663: `return unwind_parse_fail(out, err, err_start, "Truncated CHAININFO RUNTIME_FUNCTION.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 664: `} else {` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 665: `out->chain = chain;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 666: `if (chain.end_rva <= chain.begin_rva ||` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 667: `chain.begin_rva >= img->size_of_image ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 668: `chain.end_rva > img->size_of_image ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 669: `chain.unwind_rva == 0 ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 670: `chain.unwind_rva >= img->size_of_image) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 671: `out->anomalies |= UA_CHAIN_INVALID;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 672: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 673: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 674: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 676: `if (flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 677: `out->has_handler = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 678: `uint32_t handler_rva = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 679: `if (!buf_read_u32(b, after_codes + 0, &handler_rva)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 680: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 681: `unwind_diag_set_read_failure(out, b, "exception handler RVA tail", after_codes, 4u, record_limit);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена,

типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 682: `return unwind_parse_fail(out, err, err_start, "Truncated exception handler RVA.");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 683: `} else {` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 684: `out->handler_rva = handler_rva;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 685: `if (handler_rva >= img->size_of_image) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 686: `out->anomalies |= UA_HANDLER_OOB;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 687: `unwind_diag_set(out, b, "exception handler RVA", true, after_codes,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 688: `true, 4u, 4u, false, 0, 0);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 689: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 690: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 691: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 693: `out->fixed_stack_bytes = fixed_stack;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 694: `out->pushed_nonvol_count = pushed;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 695: `out->saved_nonvol_mask = nonvol_mask;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 696: `out->saved_xmm_mask = xmm_mask;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 698: `/* follow chain (lightly): detect loops and depth. */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 699: `if (follow_chain && out->has_chain) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 700: `uint32_t seen[16]; /* depth max 16 */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 701: `int seen_n = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 702: `out->chain_path[0] = unwind_rva;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 703: `out->chain_path_count = 1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 704: `uint32_t cur = out->chain.unwind_rva;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 705: `for (int depth = 0; depth < max_depth; depth++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 706: `bool loop = false;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 707: `for (int j = 0; j < seen_n; j++) if (seen[j] == cur) loop = true;` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 708: `if (out->chain_path_count < UNWIND_DIAG_MAX_CHAIN_PATH) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 709: `out->chain_path[out->chain_path_count++] = cur;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 710: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 711: `if (loop) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 712: `out->anomalies |= UA_CHAIN_LOOP;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 713: `out->chain_loop_detected = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 714: `out->chain_loop_rva = cur;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 715: `unwind_diag_set(out, b, "CHAININFO chain path", true, (size_t)uoff,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 716: `false, 0, 0, false, 0, 0);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 717: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 718: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 719: `if (seen_n < (int)(sizeof(seen)/sizeof(seen[0]))) seen[seen_n++] = cur;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 721: `UnwindSummary tmp;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 722: `StrBuf tmp_ops = {0};` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 723: `StrBuf tmp_err = {0};` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 724: `sb_init(&tmp_err, 256);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 725: `sb_init(&tmp_ops, 256);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 726: `UnwindParseOpts o2 = {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 727: `.max_chain_depth = 0,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 728: `.follow_chain = false,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 729: `.strict_abi = strict_abi` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 730: `};` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 731: `if (!unwind_parse(b, img, cur, &o2, &tmp, NULL, &tmp_err)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает

безопасный путь, фиксирует ошибку или включает признак.

Строка 732: `out->anomalies |= UA_TRUNCATED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 733: `sb_free(&tmp_err);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 734: `sb_free(&tmp_ops);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 735: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 736: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 737: `/* combine fixed stack heuristically */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 738: `add_fixed_stack(&out->fixed_stack_bytes, tmp.fixed_stack_bytes, &out->anomalies);` - Это проверка битовой маски. Код выясняет, включён ли конкретный флаг или признак, не требуя, чтобы он был единственным включенным битом.

Строка 739: `out->pushed_nonvol_count += tmp.pushed_nonvol_count;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 740: `out->saved_nonvol_mask |= tmp.saved_nonvol_mask;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 741: `out->saved_xmm_mask |= tmp.saved_xmm_mask;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 742: `if (tmp.frame_reg != out->frame_reg || tmp.frame_off != out->frame_off) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 743: `out->anomalies |= UA_BAD_OPERAND;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 744: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 745: `if (!out->has_handler && tmp.has_handler) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 746: `out->has_handler = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 747: `out->handler_rva = tmp.handler_rva;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 748: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 749: `sb_free(&tmp_err);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 750: `sb_free(&tmp_ops);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 752: `if (!tmp.has_chain) break;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 753: `cur = tmp.chain.unwind_rva;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 754: `if (depth == max_depth-1) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 755: `out->anomalies |= UA_CHAIN_DEPTH_EXCEEDED;` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 756: `unwind_diag_set(out, b, "CHAININFO max depth", true, (size_t)uoff,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 757: `false, 0, 0, true, (uint32_t)max_depth, (uint32_t)depth + 1u);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 758: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 759: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 760: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 762: `out->valid = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 763: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 764: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 766: `RuntimeFunctionReadResult unwind_read_runtime_functions_ex(` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 767: `const Buf* b,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 768: `const PeImage* img,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 769: `RuntimeFunction** out_rf,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 770: `size_t* out_count,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 771: `StrBuf* err) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 772: `if (!b || !img || !out_rf || !out_count || !err) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 773: `return RUNTIME_FUNCTION_READ_MALFORMED;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 774: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 775: `*out_rf = NULL;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 776: `*out_count = 0;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 778: `if (img->exception_rva == 0 && img->exception_size == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 779: `return RUNTIME_FUNCTION_READ_OK;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 780: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 781: `if (img->exception_rva == 0 || img->exception_size == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 782: `sb_append(err, "Malformed exception directory: RVA and size must both be zero or non-zero.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за

нехватки памяти или переполнения capacity.

Строка 783: `return RUNTIME_FUNCTION_READ_MALFORMED;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 784: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 785: `uint32_t off = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 786: `if (!pe_rva_to_fileoff(img, img->exception_rva, &off)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 787: `sb_appendf(err, "Exception directory RVA 0x%08x not mappable.\n", img->exception_rva);` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 788: `return RUNTIME_FUNCTION_READ_MALFORMED;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 789: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 790: `if ((size_t)off + img->exception_size > b->size) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 791: `sb_append(err, "Exception directory out of file range.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 792: `return RUNTIME_FUNCTION_READ_MALFORMED;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 793: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 794: `if (img->exception_size < 12) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 795: `sb_append(err, "Exception directory size too small.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 796: `return RUNTIME_FUNCTION_READ_MALFORMED;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 797: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 798: `if ((img->exception_size % 12u) != 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 799: `sb_append(err, "Exception directory size is not a multiple of RUNTIME_FUNCTION size.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 800: `return RUNTIME_FUNCTION_READ_MALFORMED;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 801: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 802: `size_t n = img->exception_size / 12;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 803: `RuntimeFunction* rf = (RuntimeFunction*)calloc(n, sizeof(RuntimeFunction));` - Эта строка выделяет память в heap. После неё нужно проверить результат на NULL и определить, кто будет освобождать ресурс.

Строка 804: `if (!rf) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 805: `sb_append(err, "Out of memory.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 806: `return RUNTIME_FUNCTION_READ_OOM;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 807: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 809: `for (size_t i = 0; i < n; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 810: `size_t e = (size_t)off + i*12;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 811: `uint32_t bva=0, eva=0, uva=0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 812: `if (!buf_read_u32(b, e + 0, &bva) ||` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 813: `!buf_read_u32(b, e + 4, &eva) ||` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 814: `!buf_read_u32(b, e + 8, &uva)) {` - Эта строка читает bytes через безопасную обёртку проекта. Важный смысл: модуль не обращается к сырой памяти напрямую, а просит нижний слой проверить границы.

Строка 815: `free(rf);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 816: `sb_append(err, "Failed to read runtime function entry.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 817: `return RUNTIME_FUNCTION_READ_MALFORMED;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 818: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 819: `rf[i].begin_rva = bva;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 820: `rf[i].end_rva = eva;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 821: `rf[i].unwind_rva = uva;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 822: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 824: `*out_rf = rf;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 825: `*out_count = n;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 826: `return RUNTIME_FUNCTION_READ_OK;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 827: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 829: `bool unwind_read_runtime_functions(const Buf* b,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 830: `const PeImage* img,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 831: `RuntimeFunction** out_rf,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 832: `size_t* out_count,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 833: `StrBuf* err) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 834: `return unwind_read_runtime_functions_ex(` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 835: `b, img, out_rf, out_count, err) == RUNTIME_FUNCTION_READ_OK;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 836: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Файл src/heuristics.c

Файл содержит 1044 строк. Ниже идут непустые строки с учебным объяснением. Смысл упражнения - не заучить комментарий, а научиться самостоятельно выполнять такой разбор.

Строка 1: `#include "heuristics.h"` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 2: `#include <stdlib.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 3: `#include <string.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 4: `#include <stdio.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 5: `#include <stdarg.h>` - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 7: `#ifndef IMAGE_SCN_MEM_EXECUTE` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 8: `#define IMAGE_SCN_MEM_EXECUTE 0x20000000u` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 9: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 10: `#ifndef IMAGE_SCN_MEM_WRITE` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 11: `#define IMAGE_SCN_MEM_WRITE 0x80000000u` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 12: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 14: `#define UA_ABI_MASK \` - Это макроопределение. В проекте такие константы дают именованный смысл числам формата PE, Windows flags или размерам внутренних ограничений.

Строка 15: `(UA_ABI_REGISTER_CLASS | UA_ABI_RESERVED_OPINFO | UA_ABI_PUSH_ORDER | \` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 16: `UA_ABI_CHAIN_RESTRICTION | UA_ABI_SAVE_OFFSET)` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 19: `/*` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 20: `* Heuristic calibration notes` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 21: `* ----------------------------` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 22: `* These constants intentionally form a conservative engineering model rather` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 23: `* than a mathematically validated classifier. The weights are grouped by the` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 24: `* evidence family discussed in the thesis so the rationale stays stable and` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 25: `* reviewable:` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 26: `* - table/unwind integrity violations => strongest static weights` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 27: `* - placement anomalies => medium/high depending on prevalence` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 28: `* - reuse/dominance patterns => moderate unless they are very strong` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 29: `* - mitigations reduce confidence when evidence is isolated or sample size is tiny` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 30: `*/` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 31: `static const int SCORE_EMPTY_PDATA = 10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 32: `static const int SCORE_PE_SECTION_RAW_OOB = 25;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 33: `static const int SCORE_PE_SECTION_VIRTUAL_OOB = 20;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 34: `static const int SCORE_PE_SECTION_RAW_OVERLAP = 15;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 35: `static const int SCORE_PE_SECTION_VIRTUAL_OVERLAP = 20;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 36: `static const int SCORE_PE_SECTION_MISALIGNED = 8;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 37: `static const int SCORE_PE_EXCEPTION_DIRECTORY_OOB = 25;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 38: `static const int SCORE_INVALID_RANGE_LOW = 10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 39: `static const int SCORE_INVALID_RANGE_HIGH = 20;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 40: `static const int SCORE_RF_OOB_LOW = 10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 41: `static const int SCORE_RF_OOB_HIGH = 25;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 42: `static const int SCORE_UNWIND_RVA_OOB_LOW = 10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 43: `static const int SCORE_UNWIND_RVA_OOB_HIGH = 25;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 44: `static const int SCORE_RF_UNSORTED = 15;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 45: `static const int SCORE_RF_OVERLAP = 25;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 46: `static const int SCORE_DUPLICATE_BEGIN = 10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 47: `static const int SCORE_UNWIND_PARSE_FAIL_LOW = 10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 48: `static const int SCORE_UNWIND_PARSE_FAIL_HIGH = 25;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 49: `static const int SCORE_UNWIND_ANOM_LOW = 10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 50: `static const int SCORE_UNWIND_ANOM_HIGH = 20;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 51: `static const int SCORE_UNWIND_REUSE_MEDIUM = 10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 52: `static const int SCORE_UNWIND_REUSE_HIGH = 20;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 53: `static const int SCORE_HANDLER_DOMINANCE = 15;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 54: `static const int SCORE_UNWIND_EXEC_SECTION = 20;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 55: `static const int SCORE_UNWIND_UNUSUAL_SECTION_LOW = 8;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 56: `static const int SCORE_UNWIND_UNUSUAL_SECTION_HIGH = 15;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 57: `static const int SCORE_UNWIND_WRITABLE_SECTION_LOW = 10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 58: `static const int SCORE_UNWIND_WRITABLE_SECTION_HIGH = 18;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 59: `static const int SCORE_HANDLER_NONEXEC = 15;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 60: `static const int SCORE_HANDLER_WRITABLE = 10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 61: `static const int SCORE_COMBO_PLACEMENT_EXEC_AND_HANDLER = 10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 62: `static const int SCORE_COMBO_PLACEMENT_AND_INTEGRITY = 8;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 63: `static const int SCORE_COMBO_REUSE_AND_DOMINANCE = 8;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 64: `static const int SCORE_COMBO_WRITABLE_METADATA_AND_HANDLER = 10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 66: `static const double CONFIDENCE_BASE = 0.15;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 67: `static const double CONFIDENCE_ZERO_SCORE_BASE = 0.20;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 68: `static const double CONFIDENCE_PER_FAMILY = 0.11;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 69: `static const double CONFIDENCE_PER_STRONG = 0.08;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 70: `static const double CONFIDENCE_PER_MEDIUM = 0.04;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 71: `static const double CONFIDENCE_BONUS_TABLE_HARD = 0.08;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 72: `static const double CONFIDENCE_BONUS_UNWIND_HARD = 0.08;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 73: `static const double CONFIDENCE_PENALTY_SINGLE_SIGNAL = 0.12;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 74: `static const double CONFIDENCE_PENALTY_SMALL_SAMPLE = 0.06;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 75: `static const double CONFIDENCE_PENALTY_PLACEMENT_ONLY = 0.10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 76: `static const double CONFIDENCE_PENALTY_SINGLE_FAMILY = 0.08;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 77: `static const double CONFIDENCE_MIN = 0.05;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 78: `static const double CONFIDENCE_MAX = 0.90;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 80: `static const double THRESHOLD_HIGH_PERCENT = 0.01; /* >1% of entries => not just a one-off glitch */` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 81: `static const double THRESHOLD_UNWIND_ANOM_HIGH = 0.05; /* >5% anomalies => broad integrity issue */` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 82: `static const double THRESHOLD_UNWIND_REUSE_MEDIUM = 0.10;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 83: `static const double THRESHOLD_UNWIND_REUSE_HIGH = 0.30;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 84: `static const double THRESHOLD_HANDLER_DOMINANCE = 0.80;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 86: `static bool sec_name_eq(const char* a, const char* b) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 87: `if (!a || !b) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 88: `while (*a && *b) {` - Это цикл с условием продолжения. При чтении нужно понять, что изменяется внутри цикла и почему цикл обязательно завершится.

Строка 89: `char ca = *a;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 90: `char cb = *b;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 91: `if (ca >= 'A' && ca <= 'Z') ca = (char)(ca - 'A' + 'a');` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 92: `if (cb >= 'A' && cb <= 'Z') cb = (char)(cb - 'A' + 'a');` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 93: `if (ca != cb) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 94: `a++; b++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 95: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 96: `return *a == 0 && *b == 0;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 97: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 99: `static bool sec_name_contains_ci(const char* haystack, const char* needle) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 100: `if (!haystack || !needle || !*needle) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 101: `size_t nlen = strlen(needle);` - Здесь используется функция работы со строками C. Нужно помнить, что строка C завершается нулевым байтом, а указатель на строку сам по себе не хранит длину.

Строка 102: `for (const char* h = haystack; *h; h++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 103: `size_t i = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 104: `while (i < nlen) {` - Это цикл с условием продолжения. При чтении нужно понять, что изменяется внутри цикла и почему цикл обязательно завершится.

Строка 105: `char ca = h[i];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 106: `char cb = needle[i];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 107: `if (!ca) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 108: `if (ca >= 'A' && ca <= 'Z') ca = (char)(ca - 'A' + 'a');` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 109: `if (cb >= 'A' && cb <= 'Z') cb = (char)(cb - 'A' + 'a');` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 110: `if (ca != cb) break;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 111: `i++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 112: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 113: `if (i == nlen) return true;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 114: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 115: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 116: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 118: `static bool sec_name_allowed_for_unwind(const char* name8) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 119: `/* Common placements: .xdata (MSVC), .rdata/.pdata in practice, and toolchain-specific xdata-like sections. */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 120: `return sec_name_eq(name8, ".xdata") || sec_name_eq(name8, ".rdata") || sec_name_eq(name8, ".pdata") ||` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 121: `sec_name_eq(name8, ".gnu_xdata") || sec_name_eq(name8, ".mrdata") ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 122: `sec_name_eq(name8, "xdata") || sec_name_eq(name8, "rdata") || sec_name_eq(name8, "pdata");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 123: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 125: `static bool sec_name_plausibly_unwind_named(const char* name8) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 126: `return sec_name_contains_ci(name8, "xdata") || sec_name_contains_ci(name8, "pdata") || sec_name_contains_ci(name8, "rdata");` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 127: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 129: `static bool sec_name_code_like(const char* name8) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 130: `return sec_name_eq(name8, ".text") || sec_name_eq(name8, "text") ||` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 131: `sec_name_contains_ci(name8, "text") || sec_name_contains_ci(name8, "code") || sec_name_contains_ci(name8, "init");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 132: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 134: `static bool image_has_executable_section(const PeImage* img) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 135: `if (!img || !img->sections) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и

копирования памяти.

Строка 136: `for (size_t i = 0; i < img->num_sections; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 137: `const PeSection* section = &img->sections[i];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 138: `if ((section->characteristics & IMAGE_SCN_MEM_EXECUTE) != 0 &&` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 139: `(section->virtual_size != 0 || section->raw_size != 0)) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 140: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 141: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 142: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 143: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 144: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 146: `/* Very small u32->count hash map (open addressing, linear probing). */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 147: `typedef struct HEntry {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 148: `uint32_t key;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 149: `uint32_t val;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 150: `bool used;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 151: `} HEntry;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 153: `typedef struct HMap {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 154: `HEntry* e;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 155: `size_t cap;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 156: `size_t len;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 157: `} HMap;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 159: `static uint32_t h32(uint32_t x) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 160: `/* xorshift-ish */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 161: `x ^= x >> 16;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 162: `x *= 0x7feb352dU;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 163: `x ^= x >> 15;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 164: `x *= 0x846ca68bU;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 165: `x ^= x >> 16;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 166: `return x;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 167: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 169: `static bool hmap_init(HMap* m, size_t cap) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 170: `if (!m) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 171: `m->cap = cap < 64 ? 64 : cap;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 172: `m->len = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 173: `m->e = (HEntry*)calloc(m->cap, sizeof(HEntry));` - Эта строка выделяет память в heap. После неё нужно проверить результат на NULL и определить, кто будет освобождать ресурс.

Строка 174: `return m->e != NULL;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 175: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 177: `static void hmap_free(HMap* m) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 178: `if (!m) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 179: `free(m->e);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 180: `m->e = NULL;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 181: `m->cap = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 182: `m->len = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 183: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 185: `static bool hmap_grow(HMap* m);` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 187: `static bool hmap_inc(HMap* m, uint32_t key, uint32_t* out_newval) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 188: `if (!m || !m->e) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 189: `if ((m->len + 1) * 10 >= m->cap * 7) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 190: `if (!hmap_grow(m)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 191: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 192: `uint32_t hv = h32(key);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 193: `size_t idx = (size_t)hv % m->cap;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 194: `for (size_t step = 0; step < m->cap; step++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 195: `HEntry* ent = &m->e[idx];` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 196: `if (!ent->used) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 197: `ent->used = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 198: `ent->key = key;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 199: `ent->val = 1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 200: `m->len++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 201: `if (out_newval) *out_newval = 1;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 202: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 203: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 204: `if (ent->key == key) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 205: `ent->val++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 206: `if (out_newval) *out_newval = ent->val;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 207: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 208: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 209: `idx = (idx + 1) % m->cap;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 210: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 211: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 212: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 214: `static bool hmap_grow(HMap* m) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 215: `size_t oldcap = m->cap;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 216: `HEntry* old = m->e;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 217: `if (oldcap > (SIZE_MAX - 1u) / 2u) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 218: `size_t newcap = oldcap * 2 + 1;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 219: `HEntry* ne = (HEntry*)calloc(newcap, sizeof(HEntry));` - Эта строка выделяет память в heap. После неё нужно проверить результат на NULL и определить, кто будет освобождать ресурс.

Строка 220: `if (!ne) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 222: `m->e = ne;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 223: `m->cap = newcap;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 224: `m->len = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 226: `for (size_t i = 0; i < oldcap; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 227: `if (!old[i].used) continue;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 228: `/* reinsert */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 229: `uint32_t key = old[i].key;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 230: `uint32_t val = old[i].val;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 231: `uint32_t hv = h32(key);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 232: `size_t idx = (size_t)hv % m->cap;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 233: `for (;;) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 234: `HEntry* ent = &m->e[idx];` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 235: `if (!ent->used) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 236: `ent->used = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 237: `ent->key = key;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 238: `ent->val = val;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 239: `m->len++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 240: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 241: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 242: `idx = (idx + 1) % m->cap;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 243: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 244: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 245: `free(old);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 246: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 247: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 249: `static uint32_t hmap_max_val(const HMap* m) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 250: `uint32_t mx = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 251: `if (!m || !m->e) return 0;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 252: `for (size_t i = 0; i < m->cap; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 253: `if (m->e[i].used && m->e[i].val > mx) mx = m->e[i].val;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 254: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 255: `return mx;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 256: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 258: `static bool hmap_max_entry(const HMap* m, uint32_t* out_key, uint32_t* out_val) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 259: `uint32_t best_key = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 260: `uint32_t best_val = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 261: `if (!m || !m->e) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 262: `for (size_t i = 0; i < m->cap; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 263: `if (!m->e[i].used) continue;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 264: `if (m->e[i].val > best_val) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 265: `best_key = m->e[i].key;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 266: `best_val = m->e[i].val;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 267: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 268: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 269: `if (best_val == 0) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 270: `if (out_key) *out_key = best_key;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 271: `if (out_val) *out_val = best_val;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 272: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 273: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 275: `static size_t hmap_unique(const HMap* m) { return m ? m->len : 0; }` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 277: `typedef struct RfIdx {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 278: `uint32_t begin;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 279: `uint32_t end;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 280: `} RfIdx;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 282: `typedef struct FindingLocation {` - Здесь объявляется новый составной тип. `struct` собирает связанные поля в один объект, а `typedef` позволяет использовать короткое имя типа без повторения слова `struct`.

Строка 283: `bool set;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 284: `uint32_t rva;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 285: `} FindingLocation;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 287: `static void remember_location(FindingLocation* location, uint32_t rva) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 288: `if (!location || location->set) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 289: `location->set = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 290: `location->rva = rva;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 291: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 293: `enum PeSectionIssue {` - Здесь объявляется перечисление именованных констант. В проекте enum часто используется для кодов результата, flags или битовых признаков, чтобы не писать магические числа.

Строка 294: `PE_ISSUE_RAW_OOB,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 295: `PE_ISSUE_VIRTUAL_OOB,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 296: `PE_ISSUE_RAW_OVERLAP,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 297: `PE_ISSUE_VIRTUAL_OVERLAP,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 298: `PE_ISSUE_MISALIGNED` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 299: `};` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 301: `static FindingLocation first_pe_section_issue(const PeImage* img, enum PeSectionIssue issue) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 302: `FindingLocation result = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 303: `if (!img || !img->sections) return result;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 305: `for (size_t i = 0; i < img->num_sections; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 306: `const PeSection* s = &img->sections[i];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 307: `uint64_t span = s->virtual_size > s->raw_size ? s->virtual_size : s->raw_size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 308: `bool match = false;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 309: `if (issue == PE_ISSUE_RAW_OOB) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 310: `match = s->raw_size > 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 311: `((uint64_t)s->raw_ptr + (uint64_t)s->raw_size > (uint64_t)img->file_size);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 312: `} else if (issue == PE_ISSUE_VIRTUAL_OOB) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 313: `match = span > 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 314: `((uint64_t)s->virtual_address + span > (uint64_t)img->size_of_image);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 315: `} else if (issue == PE_ISSUE_MISALIGNED) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 316: `match = (img->section_alignment > 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 317: `(s->virtual_address % img->section_alignment) != 0) ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 318: `(s->raw_size > 0 && img->file_alignment > 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 319: `(s->raw_ptr % img->file_alignment) != 0);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 320: `} else {` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 321: `for (size_t j = 0; j < i && !match; j++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 322: `const PeSection* p = &img->sections[j];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 323: `uint64_t pspan = p->virtual_size > p->raw_size ? p->virtual_size : p->raw_size;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 324: `if (issue == PE_ISSUE_RAW_OVERLAP) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 325: `match = s->raw_size > 0 && p->raw_size > 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 326: `(uint64_t)s->raw_ptr < (uint64_t)p->raw_ptr + p->raw_size &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 327: `(uint64_t)p->raw_ptr < (uint64_t)s->raw_ptr + s->raw_size;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 328: `} else if (issue == PE_ISSUE_VIRTUAL_OVERLAP) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 329: `match = span > 0 && pspan > 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 330: `(uint64_t)s->virtual_address < (uint64_t)p->virtual_address + pspan &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 331: `(uint64_t)p->virtual_address < (uint64_t)s->virtual_address + span;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 332: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 333: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 334: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 335: `if (match) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 336: `remember_location(&result, s->virtual_address);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 337: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 338: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 339: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 340: `return result;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 341: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 343: `static int cmp_rfidx(const void* a, const void* b) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 344: `const RfIdx* x = (const RfIdx*)a;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 345: `const RfIdx* y = (const RfIdx*)b;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 346: `if (x->begin < y->begin) return -1;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 347: `if (x->begin > y->begin) return 1;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 348: `if (x->end < y->end) return -1;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 349: `if (x->end > y->end) return 1;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 350: `return 0;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 351: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 353: `static int clamp_int(int v, int lo, int hi) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 354: `if (v < lo) return lo;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 355: `if (v > hi) return hi;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 356: `return v;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 357: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 359: `static void add_score(int* score, int add) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 360: `if (!score) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 361: `*score = clamp_int(*score + add, 0, 100);` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 362: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 364: `static double safe_ratio(double a, double b) { return (b == 0.0) ? 0.0 : (a / b); }` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 367: `const char* heuristics_evidence_family_name(uint32_t bit) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 368: `switch (bit) {` - Это выбор поведения по дискретному коду. В проекте `switch` особенно важен для UWOP: номер операции определяет, сколько slots читать и как интерпретировать operands.

Строка 369: `case HEF_TABLE_INTEGRITY: return "table_integrity";` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 370: `case HEF_UNWIND_INTEGRITY: return "unwind_integrity";` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 371: `case HEF_METADATA_PLACEMENT: return "metadata_placement";` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 372: `case HEF_UNWIND_REUSE: return "unwind_reuse";` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 373: `case HEF_HANDLER_DOMINANCE: return "handler_dominance";` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 374: `default: return "unknown";` - Это ветка внутри `switch`. Она связывает конкретное значение кода с конкретным действием parser-а или formatter-а.

Строка 375: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 376: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 378: `size_t heuristics_evidence_family_count(uint32_t mask) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 379: `size_t n = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 380: `while (mask) {` - Это цикл с условием продолжения. При чтении нужно понять, что изменяется внутри цикла и почему цикл обязательно завершится.

Строка 381: `n += (mask & 1u) ? 1u : 0u;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 382: `mask >>= 1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 383: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 384: `return n;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 385: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 387: `static uint32_t family_bit_from_name(const char* family) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 388: `if (!family) return 0;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 389: `if (strcmp(family, "table_integrity") == 0) return HEF_TABLE_INTEGRITY;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 390: `if (strcmp(family, "unwind_integrity") == 0) return HEF_UNWIND_INTEGRITY;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 391: `if (strcmp(family, "metadata_placement") == 0) return HEF_METADATA_PLACEMENT;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 392: `if (strcmp(family, "unwind_reuse") == 0) return HEF_UNWIND_REUSE;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 393: `if (strcmp(family, "handler_dominance") == 0) return HEF_HANDLER_DOMINANCE;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 394: `return 0;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 395: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 397: `static void safe_copy(char* dst, size_t dst_cap, const char* src) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 398: `if (!dst || dst_cap == 0) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 399: `if (!src) src = "";` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 400: `snprintf(dst, dst_cap, "%s", src);` - Эта строка форматирует текст с ограничением размера буфера. В C это важнее, чем неограниченная запись, потому что фиксированные массивы `char` нельзя переполнять.

Строка 401: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 403: `#if defined(__GNUC__) || defined(__clang__)` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 404: `#pragma GCC diagnostic push` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 405: `#pragma GCC diagnostic ignored "-Wformat-nonliteral"` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 406: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 408: `static HeuristicFinding* add_finding(HeuristicReport* out,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 409: `const char* code,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 410: `const char* family,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 411: `const char* severity,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 412: `const char* fmt,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 413: `...) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 414: `if (!out || !fmt) return NULL;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 416: `char msg[192];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 417: `va_list ap;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 418: `va_start(ap, fmt);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 419: `(void)vsnprintf(msg, sizeof(msg), fmt, ap);` - Эта строка форматирует текст с ограничением размера буфера. В C это важнее, чем неограниченная запись, потому что фиксированные массивы `char` нельзя переполнять.

Строка 420: `va_end(ap);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 422: `out->evidence_families_mask |= family_bit_from_name(family);` - Это включение бита или накопление признака в маске. Старые включенные признаки сохраняются, новый признак добавляется через bitwise OR.

Строка 424: `HeuristicFinding* stored = NULL;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 425: `if (out->finding_count < HEURISTICS_MAX_FINDINGS) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 426: `HeuristicFinding* f = &out->findings[out->finding_count++];` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 427: `memset(f, 0, sizeof(*f));` - Эта строка заполняет область памяти байтом, обычно нулём. В проекте это часто приводит структуру к известному начальному состоянию перед заполнением.

Строка 428: `safe_copy(f->code, sizeof(f->code), code ? code : "unspecified");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 429: `safe_copy(f->family, sizeof(f->family), family ? family : "unknown");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 430: `safe_copy(f->severity, sizeof(f->severity), severity ? severity : "info");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 431: `safe_copy(f->message, sizeof(f->message), msg);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 432: `stored = f;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 433: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 435: `sb_appendf(&out->details, "- [%s/%s] %s\n",` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 436: `severity ? severity : "info",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 437: `code ? code : "unspecified",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 438: `msg);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 439: `return stored;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 440: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 442: `static void annotate_finding(HeuristicFinding* finding,` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 443: `FindingLocation location,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 444: `size_t affected_count) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 445: `if (!finding) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 446: `finding->has_location = location.set;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 447: `finding->representative_rva = location.rva;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 448: `finding->affected_count = affected_count;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 449: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 451: `static void append_note(HeuristicReport* out, const char* fmt, ...) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 452: `if (!out || !fmt) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 454: `char msg[192];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 455: `va_list ap;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 456: `va_start(ap, fmt);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 457: `(void)vsnprintf(msg, sizeof(msg), fmt, ap);` - Эта строка форматирует текст с ограничением размера буфера. В C это важнее, чем неограниченная запись, потому что фиксированные массивы `char` нельзя переполнять.

Строка 458: `va_end(ap);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 460: `sb_appendf(&out->details, "- [info] %s\n", msg);` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 461: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 463: `static size_t count_findings_with_severity(const HeuristicReport* out, const char* sev) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 464: `size_t n = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 465: `if (!out || !sev) return 0;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 466: `for (size_t i = 0; i < out->finding_count; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 467: `if (strcmp(out->findings[i].severity, sev) == 0) n++;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 468: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 469: `return n;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 470: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 472: `static bool has_finding_code(const HeuristicReport* out, const char* code) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 473: `if (!out || !code) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 474: `for (size_t i = 0; i < out->finding_count; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 475: `if (strcmp(out->findings[i].code, code) == 0) return true;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 476: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 477: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 478: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 480: `static void add_mitigation(HeuristicReport* out, const char* code, const char* fmt, ...) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 481: `if (!out || !fmt) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 482: `if (out->mitigation_count >= HEURISTICS_MAX_MITIGATIONS) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 484: `char msg[192];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 485: `va_list ap;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 486: `va_start(ap, fmt);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 487: `(void)vsnprintf(msg, sizeof(msg), fmt, ap);` - Эта строка форматирует текст с ограничением размера буфера. В C это важнее, чем неограниченная запись, потому что фиксированные массивы `char` нельзя переполнять.

Строка 488: `va_end(ap);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 490: `HeuristicMitigation* m = &out->mitigations[out->mitigation_count++];` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 491: `memset(m, 0, sizeof(*m));` - Эта строка заполняет область памяти байтом, обычно нулём. В проекте это часто приводит структуру к известному начальному состоянию перед заполнением.

Строка 492: `safe_copy(m->code, sizeof(m->code), code ? code : "unspecified");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 493: `safe_copy(m->message, sizeof(m->message), msg);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 495: `sb_appendf(&out->details, " [mitigation/%s] %s\n",` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 496: `code ? code : "unspecified",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 497: `msg);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 498: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 500: `#if defined(__GNUC__) || defined(__clang__)` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 501: `#pragma GCC diagnostic pop` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 502: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 504: `static const char* evidence_level_from_counts(size_t weak_count,` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 505: `size_t medium_count,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 506: `size_t strong_count,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 507: `int score) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 508: `if (strong_count >= 2 || score >= 70) return "strong";` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 509: `if (strong_count >= 1 || medium_count >= 2 || score >= 30) return "medium";` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 510: `if (weak_count || medium_count || strong_count || score > 0) return "weak";` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 511: `return "none";` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 512: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 514: `static void finalize_verdict(HeuristicReport* out) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 515: `if (!out) return;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 517: `out->suspicion = safe_ratio((double)out->score, 100.0);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 519: `size_t family_count = heuristics_evidence_family_count(out->evidence_families_mask);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 520: `size_t strong_count = count_findings_with_severity(out, "high") + count_findings_with_severity(out, "critical");` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 521: `size_t medium_count = count_findings_with_severity(out, "medium");` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 522: `size_t weak_count = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 523: `if (out->finding_count >= strong_count + medium_count) weak_count = out->finding_count - strong_count - medium_count;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 525: `out->strong_finding_count = strong_count;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 526: `out->medium_finding_count = medium_count;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 527: `out->weak_finding_count = weak_count;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 528: `safe_copy(out->evidence_level, sizeof(out->evidence_level),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 529: `evidence_level_from_counts(weak_count, medium_count, strong_count, out->score));` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 531: `double confidence = CONFIDENCE_BASE;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 532: `confidence += (double)family_count * CONFIDENCE_PER_FAMILY;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 533: `confidence += (double)strong_count * CONFIDENCE_PER_STRONG;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 534: `confidence += (double)medium_count * CONFIDENCE_PER_MEDIUM;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 536: `if (out->rf_invalid_ranges || out->rf_oob || out->rf_unwind_oob || out->rf_overlaps) confidence += CONFIDENCE_BONUS_TABLE_HARD;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 537: `if (out->unwind_parse_fail || out->unwind_anomalies) confidence += CONFIDENCE_BONUS_UNWIND_HARD;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 538: `if (out->score == 0) confidence = CONFIDENCE_ZERO_SCORE_BASE;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 540: `/* Weakening rules: lower confidence when only isolated / non-correlated static signals exist. */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 541: `if (out->finding_count == 1 && strong_count == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 542: `add_mitigation(out, "single_signal_only",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 543: `"Only one non-strong signal contributed; treat as weak standalone evidence.");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 544: `confidence -= CONFIDENCE_PENALTY_SINGLE_SIGNAL;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 545: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 546: `if (out->total < 32 && strong_count == 0 && out->finding_count > 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 547: `add_mitigation(out, "small_sample_size",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 548: `"Very small .pdata sample can overstate ratios; confidence reduced until more functions are observed.");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 549: `confidence -= CONFIDENCE_PENALTY_SMALL_SAMPLE;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 550: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 551: `if (out->unwind_unusual_section > 0 &&` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 552: `out->unwind_exec_section == 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 553: `out->unwind_writable_section == 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 554: `out->handler_nonexec_section == 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 555: `out->handler_writable_section == 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 556: `out->rf_invalid_ranges == 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 557: `out->rf_oob == 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 558: `out->rf_unwind_oob == 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 559: `out->unwind_parse_fail == 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 560: `out->unwind_anomalies == 0) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 561: `add_mitigation(out, "placement_only_signal",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 562: `"Placement-only anomalies may come from uncommon but benign layouts; manual verification required.");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 563: `confidence -= CONFIDENCE_PENALTY_PLACEMENT_ONLY;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 564: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 565: `if ((out->strong_finding_count == 0) && (family_count <= 1) && out->finding_count > 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 566: `add_mitigation(out, "single_family_only",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 567: `"Evidence comes from only one family; cross-layer corroboration is recommended before strong conclusions.");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 568: `confidence -= CONFIDENCE_PENALTY_SINGLE_FAMILY;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 569: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 571: `if (confidence < CONFIDENCE_MIN) confidence = CONFIDENCE_MIN;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 572: `if (confidence > CONFIDENCE_MAX) confidence = CONFIDENCE_MAX;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 573: `out->confidence = confidence;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 575: `out->requires_manual_review =` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 576: `(out->score >= 20) ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 577: `(out->unwind_exec_section > 0) ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 578: `(out->unwind_writable_section > 0) ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 579: `(out->handler_nonexec_section > 0) ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 580: `(out->handler_writable_section > 0) ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 581: `(out->unwind_parse_fail > 0) ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 582: `(out->mitigation_count > 0);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 584: `if (out->score >= 70) safe_copy(out->verdict, sizeof(out->verdict), "high_suspicion");` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 585: `else if (out->score >= 40) safe_copy(out->verdict, sizeof(out->verdict), "moderate_suspicion");` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 586: `else if (out->score >= 15) safe_copy(out->verdict, sizeof(out->verdict), "low_suspicion");` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 587: `else safe_copy(out->verdict, sizeof(out->verdict), "no_strong_static_anomalies");` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 588: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 590: `bool heuristics_analyze(const PeImage* img,` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 591: `const RuntimeFunction* rf,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 592: `const UnwindSummary* us,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 593: `const bool* us_ok,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 594: `size_t n,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 595: `HeuristicReport* out,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 596: `StrBuf* err) {` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 597: `if (!img || !out || !err) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 598: `if (n > 0 && (!rf || !us || !us_ok)) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 599: `memset(out, 0, sizeof(*out));` - Эта строка заполняет область памяти байтом, обычно нулём. В проекте это часто приводит структуру к известному начальному состоянию перед заполнением.

Строка 600: `if (!sb_init(&out->details, 1024)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или

включает признак.

Строка 601: `sb_append(err, "Out of memory (details buffer).\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 602: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 603: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 604: `out->total = n;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 605: `out->pe_section_raw_oob = img->section_raw_oob;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 606: `out->pe_section_virtual_oob = img->section_virtual_oob;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 607: `out->pe_section_raw_overlaps = img->section_raw_overlaps;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 608: `out->pe_section_virtual_overlaps = img->section_virtual_overlaps;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 609: `out->pe_section_misaligned = img->section_misaligned;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 610: `out->pe_exception_directory_oob = img->exception_directory_oob;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 612: `/* RUNTIME_FUNCTION sanity */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 613: `uint32_t prev_begin = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 614: `bool prev_valid = false;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 615: `size_t unsorted = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 616: `FindingLocation invalid_range_loc = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 617: `FindingLocation rf_oob_loc = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 618: `FindingLocation unwind_oob_loc = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 619: `FindingLocation unsorted_loc = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 620: `FindingLocation overlap_loc = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 621: `FindingLocation duplicate_loc = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 622: `FindingLocation parse_fail_loc = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 623: `FindingLocation anomaly_loc = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 624: `FindingLocation unusual_unwind_loc = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 625: `FindingLocation executable_unwind_loc = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 626: `FindingLocation writable_unwind_loc = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 627: `FindingLocation nonexec_handler_loc = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 628: `FindingLocation writable_handler_loc = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 630: `HMap unwind_counts = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 631: `HMap handler_counts = {0};` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 632: `if (n > (SIZE_MAX - 1u) / 2u) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 633: `sb_append(err, "Runtime function count is too large for reuse analysis.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 634: `sb_free(&out->details);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 635: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 636: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 637: `if (!hmap_init(&unwind_counts, n * 2 + 1u) ||` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 638: `!hmap_init(&handler_counts, 128)) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 639: `sb_append(err, "Out of memory (hash maps).\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 640: `hmap_free(&unwind_counts);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 641: `hmap_free(&handler_counts);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 642: `sb_free(&out->details);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 643: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 644: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 646: `for (size_t i = 0; i < n; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 647: `uint32_t b = rf[i].begin_rva;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 648: `uint32_t e = rf[i].end_rva;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 649: `uint32_t u = rf[i].unwind_rva;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 651: `if (e <= b) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 652: `out->rf_invalid_ranges++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 653: `remember_location(&invalid_range_loc, b);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 654: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 655: `if (b >= img->size_of_image || e > img->size_of_image) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 656: `out->rf_oob++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 657: `remember_location(&rf_oob_loc, b);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 658: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 659: `if (u >= img->size_of_image || u == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 660: `out->rf_unwind_oob++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 661: `remember_location(&unwind_oob_loc, u);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 662: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 664: `if (prev_valid && b < prev_begin) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 665: `unsorted++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 666: `remember_location(&unsorted_loc, b);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 667: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 668: `prev_begin = b;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 669: `prev_valid = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 671: `if (!hmap_inc(&unwind_counts, u, NULL)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 672: `sb_append(err, "Out of memory while counting unwind metadata reuse.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 673: `hmap_free(&unwind_counts);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 674: `hmap_free(&handler_counts);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 675: `sb_free(&out->details);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 676: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 677: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 679: `if (us_ok[i] && us[i].has_handler) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 680: `out->handler_count++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 681: `if (!hmap_inc(&handler_counts, us[i].handler_rva, NULL)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 682: `sb_append(err, "Out of memory while counting handler reuse.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 683: `hmap_free(&unwind_counts);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 684: `hmap_free(&handler_counts);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 685: `sb_free(&out->details);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 686: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 687: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 688: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 690: `/* Section placement sanity: unwind info is typically in non-executable data (.xdata/.rdata).` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 691: `Placing unwind metadata in executable / unusual sections is a common interception/tampering artifact. */` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 692: `if (us_ok[i]) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 693: `const PeSection* su = pe_find_section_for_rva(img, u);` - Эта строка связывает адресную модель PE с фактическими bytes файла. Она помогает не перепутать RVA, VA и file offset.

Строка 694: `if (!su) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 695: `out->unwind_unusual_section++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 696: `remember_location(&unusual_unwind_loc, u);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 697: `} else {` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 698: `bool allowed = sec_name_allowed_for_unwind(su->name);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 699: `bool plausible = sec_name_plausibly_unwind_named(su->name);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 700: `if (!allowed && !plausible) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 701: `out->unwind_unusual_section++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 702: `remember_location(&unusual_unwind_loc, u);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 703: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 704: `if (su->characteristics & IMAGE_SCN_MEM_EXECUTE) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 705: `out->unwind_exec_section++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 706: `remember_location(&executable_unwind_loc, u);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 707: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 708: `if (su->characteristics & IMAGE_SCN_MEM_WRITE) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 709: `out->unwind_writable_section++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 710: `remember_location(&writable_unwind_loc, u);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 711: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 712: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 714: `if (us[i].has_handler) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 715: `const PeSection* sh = pe_find_section_for_rva(img, us[i].handler_rva);` - Эта строка связывает адресную модель PE с фактическими bytes файла. Она помогает не перепутать RVA, VA и file offset.

Строка 716: `if (!sh || !(sh->characteristics & IMAGE_SCN_MEM_EXECUTE)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 717: `out->handler_nonexec_section++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 718: `remember_location(&nonexec_handler_loc, us[i].handler_rva);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 719: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 720: `if (sh && (sh->characteristics & IMAGE_SCN_MEM_WRITE)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 721: `out->handler_writable_section++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 722: `remember_location(&writable_handler_loc, us[i].handler_rva);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 723: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 724: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 725: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 727: `if (!us_ok[i]) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 728: `out->unwind_parse_fail++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 729: `remember_location(&parse_fail_loc, u);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 730: `} else if (us[i].anomalies & (uint32_t)~UA_ABI_MASK) {` - Это проверка битовой маски. Код выясняет, включён ли конкретный флаг или признак, не требуя, чтобы он был единственным включенным битом.

Строка 731: `out->unwind_anomalies++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 732: `remember_location(&anomaly_loc, u);` - Это проверка битовой маски. Код выясняет, включён ли конкретный флаг или признак, не требуя, чтобы он был единственным включенным битом.

Строка 733: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 734: `if (us_ok[i] && (us[i].anomalies & UA_ABI_MASK)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 735: `out->unwind_abi_warnings++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 736: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 737: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 738: `out->rf_unsorted = unsorted;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 740: `/* Overlap / duplicates check on sorted array */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 741: `RfIdx* idx = NULL;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 742: `if (n > 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 743: `idx = (RfIdx*)malloc(n * sizeof(RfIdx));` - Эта строка выделяет память в heap. После неё нужно проверить результат на NULL и определить, кто будет освобождать ресурс.

Строка 744: `if (!idx) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 745: `sb_append(err, "Out of memory (sort array).\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 746: `hmap_free(&unwind_counts);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 747: `hmap_free(&handler_counts);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 748: `sb_free(&out->details);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 749: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 750: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 751: `for (size_t i = 0; i < n; i++) { idx[i].begin = rf[i].begin_rva; idx[i].end = rf[i].end_rva; }` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime

functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 752: `qsort(idx, n, sizeof(RfIdx), cmp_rfidx);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 754: `uint32_t max_end = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 755: `uint32_t sorted_prev_begin = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 756: `bool have_prev = false;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 757: `for (size_t i = 0; i < n; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 758: `if (have_prev) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 759: `if (idx[i].begin == sorted_prev_begin) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 760: `out->rf_duplicate_begins++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 761: `remember_location(&duplicate_loc, idx[i].begin);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 762: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 763: `if (idx[i].begin < max_end) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 764: `out->rf_overlaps++;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 765: `remember_location(&overlap_loc, idx[i].begin);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 766: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 767: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 768: `sorted_prev_begin = idx[i].begin;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 769: `if (!have_prev || idx[i].end > max_end) max_end = idx[i].end;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 770: `have_prev = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 771: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 772: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 773: `free(idx);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 775: `uint32_t unwind_max = hmap_max_val(&unwind_counts);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 776: `uint32_t dominant_unwind_rva = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 777: `(void)hmap_max_entry(&unwind_counts, &dominant_unwind_rva, NULL);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 778: `size_t unwind_unique = hmap_unique(&unwind_counts);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 779: `out->unwind_reuse_ratio = safe_ratio((double)unwind_max, (double)n);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 780: `out->unwind_unique_ratio = safe_ratio((double)unwind_unique, (double)n);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 782: `uint32_t handler_max = hmap_max_val(&handler_counts);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 783: `uint32_t dominant_handler_rva = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 784: `(void)hmap_max_entry(&handler_counts, &dominant_handler_rva, NULL);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 785: `out->handler_reuse_ratio = (out->handler_count > 0) ? safe_ratio((double)handler_max, (double)out->handler_count) : 0.0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 787: `/* Scoring */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 788: `int score = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 789: `if (out->pe_section_raw_oob) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 790: `add_score(&score, SCORE_PE_SECTION_RAW_OOB);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 791: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 792: `add_finding(out, "pe_section_raw_oob", "table_integrity", "high",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 793: `"Section raw ranges exceed the input file: %zu", out->pe_section_raw_oob),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 794: `first_pe_section_issue(img, PE_ISSUE_RAW_OOB), out->pe_section_raw_oob);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 795: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 796: `if (out->pe_section_virtual_oob) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 797: `add_score(&score, SCORE_PE_SECTION_VIRTUAL_OOB);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 798: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 799: `add_finding(out, "pe_section_virtual_oob", "table_integrity", "high",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 800: "Section virtual ranges exceed SizeOfImage: %zu", out->pe_section_virtual_oob), - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 801: first_pe_section_issue(img, PE_ISSUE_VIRTUAL_OOB), out->pe_section_virtual_oob); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 802: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 803: if (out->pe_section_raw_overlaps) { - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 804: add_score(&score, SCORE_PE_SECTION_RAW_OVERLAP); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 805: annotate_finding(- Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 806: add_finding(out, "pe_section_raw_overlap", "table_integrity", "medium", - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 807: "Overlapping section raw ranges: %zu", out->pe_section_raw_overlaps), - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 808: first_pe_section_issue(img, PE_ISSUE_RAW_OVERLAP), out->pe_section_raw_overlaps); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 809: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 810: if (out->pe_section_virtual_overlaps) { - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 811: add_score(&score, SCORE_PE_SECTION_VIRTUAL_OVERLAP); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 812: annotate_finding(- Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 813: add_finding(out, "pe_section_virtual_overlap", "table_integrity", "high", - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 814: "Overlapping section virtual ranges: %zu", out->pe_section_virtual_overlaps), - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 815: first_pe_section_issue(img, PE_ISSUE_VIRTUAL_OVERLAP), out->pe_section_virtual_overlaps); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 816: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 817: if (out->pe_section_misaligned) { - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 818: add_score(&score, SCORE_PE_SECTION_MISALIGNED); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 819: annotate_finding(- Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 820: `add_finding(out, "pe_section_misaligned", "table_integrity", "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 821: `"PE alignment/header invariants or section alignment rules are violated: %zu",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 822: `out->pe_section_misaligned),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 823: `first_pe_section_issue(img, PE_ISSUE_MISALIGNED), out->pe_section_misaligned);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 824: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 825: `if (out->pe_exception_directory_oob) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 826: `FindingLocation location = { true, img->exception_rva };` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 827: `add_score(&score, SCORE_PE_EXCEPTION_DIRECTORY_OOB);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 828: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 829: `add_finding(out, "pe_exception_directory_oob", "table_integrity", "high",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 830: `"Exception directory is malformed, unreadable, incomplete, or out of bounds."),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 831: `location, 1);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 832: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 833: `if (n == 0 &&` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 834: `img->exception_rva == 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 835: `img->exception_size == 0 &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 836: `image_has_executable_section(img)) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 837: `add_score(&score, SCORE_EMPTY_PDATA);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 838: `add_finding(out, "no_runtime_functions", "table_integrity", "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 839: `"No RUNTIME_FUNCTION entries found; this can be valid for a leaf-only image, so confirm code complexity manually.");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 840: `} else if (n == 0 && img->exception_rva == 0 && img->exception_size == 0) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 841: append_note(out, "No RUNTIME_FUNCTION entries and no executable sections; this is consistent with a resource-only or forwarding image."); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 842: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 844: double invalid_ranges_pct = safe_ratio((double)out->rf_invalid_ranges, (double)n); - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 845: double oob_pct = safe_ratio((double)out->rf_oob, (double)n); - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 846: double unwind_oob_pct = safe_ratio((double)out->rf_unwind_oob, (double)n); - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 847: double unwind_fail_pct = safe_ratio((double)out->unwind_parse_fail, (double)n); - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 848: double unwind_anom_pct = safe_ratio((double)out->unwind_anomalies, (double)n); - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 850: if (out->rf_invalid_ranges) { - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 851: add_score(&score, invalid_ranges_pct > THRESHOLD_HIGH_PERCENT ? SCORE_INVALID_RANGE_HIGH : SCORE_INVALID_RANGE_LOW); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 852: annotate_finding(- Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 853: add_finding(out, "invalid_ranges", "table_integrity", invalid_ranges_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium", - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 854: "Invalid ranges (End<=Begin): %zu (%.2f%%)", out->rf_invalid_ranges, invalid_ranges_pct*100.0), - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 855: invalid_range_loc, out->rf_invalid_ranges); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 856: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 857: if (out->rf_oob) { - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 858: add_score(&score, oob_pct > THRESHOLD_HIGH_PERCENT ? SCORE_RF_OOB_HIGH : SCORE_RF_OOB_LOW); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 859: annotate_finding(- Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 860: add_finding(out, "rf_oob", "table_integrity", oob_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium", - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 861: "Begin/End out of SizeOfImage: %zu (%.2f%%)", out->rf_oob, oob_pct*100.0), - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 862: rf_oob_loc, out->rf_oob); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 863: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 864: `if (out->rf_unwind_oob) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 865: `add_score(&score, unwind_oob_pct > THRESHOLD_HIGH_PERCENT ? SCORE_UNWIND_RVA_OOB_HIGH : SCORE_UNWIND_RVA_OOB_LOW);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 866: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 867: `add_finding(out, "unwind_rva_oob", "table_integrity", unwind_oob_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 868: `"UnwindInfoRVA missing/out-of-bounds: %zu (%.2f%%)", out->rf_unwind_oob, unwind_oob_pct*100.0),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 869: `unwind_oob_loc, out->rf_unwind_oob);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 870: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 871: `if (out->rf_unsorted) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 872: `add_score(&score, out->rf_unsorted > 0 ? SCORE_RF_UNSORTED : 0);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 873: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 874: `add_finding(out, "rf_unsorted", "table_integrity", "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 875: `"Table not sorted by BeginAddress: %zu inversions", out->rf_unsorted),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 876: `unsorted_loc, out->rf_unsorted);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 877: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 878: `if (out->rf_overlaps) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 879: `add_score(&score, out->rf_overlaps > 0 ? SCORE_RF_OVERLAP : 0);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 880: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 881: `add_finding(out, "rf_overlaps", "table_integrity", "high",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 882: `"Overlapping ranges (after sorting): %zu", out->rf_overlaps),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 883: `overlap_loc, out->rf_overlaps);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 884: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 885: `if (out->rf_duplicate_begins) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 886: `add_score(&score, SCORE_DUPLICATE_BEGIN);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 887: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 888: `add_finding(out, "duplicate_begin", "table_integrity", "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 889: `"Duplicate BeginAddress (after sorting): %zu", out->rf_duplicate_begins),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 890: `duplicate_loc, out->rf_duplicate_begins);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 891: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 893: `if (out->unwind_parse_fail) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 894: `add_score(&score, unwind_fail_pct > THRESHOLD_HIGH_PERCENT ? SCORE_UNWIND_PARSE_FAIL_HIGH : SCORE_UNWIND_PARSE_FAIL_LOW);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 895: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 896: `add_finding(out, "unwind_parse_fail", "unwind_integrity", unwind_fail_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 897: `"UnwindInfo parse failures: %zu (%.2f%%)", out->unwind_parse_fail, unwind_fail_pct*100.0),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 898: `parse_fail_loc, out->unwind_parse_fail);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 899: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 900: `if (out->unwind_anomalies) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 901: `add_score(&score, unwind_anom_pct > THRESHOLD_UNWIND_ANOM_HIGH ? SCORE_UNWIND_ANOM_HIGH : SCORE_UNWIND_ANOM_LOW);` - Это проверка битовой маски. Код выясняет, включён ли конкретный флаг или признак, не требуя, чтобы он был единственным включенным битом.

Строка 902: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 903: `add_finding(out, "unwind_anomalies", "unwind_integrity", unwind_anom_pct > THRESHOLD_UNWIND_ANOM_HIGH ? "high" : "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 904: `"UnwindInfo anomalies (version/opcode/chain/etc): %zu (%.2f%%)", out->unwind_anomalies, unwind_anom_pct*100.0),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 905: `anomaly_loc, out->unwind_anomalies);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 906: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 908: `bool moderate_reuse_candidate =` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 909: `out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_MEDIUM && n > 200;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 910: `if (out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_HIGH && n > 200) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 911: `add_score(&score, SCORE_UNWIND_REUSE_HIGH);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 912: `FindingLocation location = { true, dominant_unwind_rva };` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 913: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 914: `add_finding(out, "unwind_reuse_strong", "unwind_reuse", "high",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 915: `"Strong UnwindInfo reuse: max %u entries share same UnwindInfoRVA (ratio=%.2f)",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 916: `(unsigned)unwind_max, out->unwind_reuse_ratio),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 917: `location, unwind_max);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 918: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 920: `if (out->handler_count > 50 && out->handler_reuse_ratio > THRESHOLD_HANDLER_DOMINANCE) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 921: `bool has_static_context =` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 922: `out->rf_invalid_ranges > 0 || out->rf_oob > 0 || out->rf_unwind_oob > 0 || out->rf_unsorted > 0 ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 923: `out->rf_overlaps > 0 || out->rf_duplicate_begins > 0 || out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 924: `out->unwind_exec_section > 0 || out->unwind_unusual_section > 0 || out->unwind_writable_section > 0 ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 925: `out->handler_nonexec_section > 0 || out->handler_writable_section > 0 ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 926: `moderate_reuse_candidate;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 927: `const PeSection* dominant_sec = dominant_handler_rva ? pe_find_section_for_rva(img, dominant_handler_rva) : NULL;` - Эта строка связывает адресную модель PE с фактическими bytes файла. Она помогает не перепутать RVA, VA и file offset.

Строка 928: `bool dominant_runtime_like = dominant_sec &&` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 929: `(dominant_sec->characteristics & IMAGE_SCN_MEM_EXECUTE) &&` - Это проверка битовой маски. Код выясняет, включён ли конкретный флаг или признак, не требуя, чтобы он был единственным включенным битом.

Строка 930: `sec_name_code_like(dominant_sec->name) &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 931: `out->handler_nonexec_section == 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 933: `if (!(dominant_runtime_like && !has_static_context)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 934: `add_score(&score, SCORE_HANDLER_DOMINANCE);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 935: `FindingLocation location = { true, dominant_handler_rva };` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 936: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 937: `add_finding(out, "handler_dominance", "handler_dominance", "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 938: `"One handler dominates: max %u entries share same HandlerRVA (ratio=%.2f%s%s)",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 939: `(unsigned)handler_max, out->handler_reuse_ratio,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 940: `dominant_sec ? ", section=" : "",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 941: `dominant_sec ? dominant_sec->name : ""),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 942: `location, handler_max);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 943: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 944: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 946: `if (moderate_reuse_candidate &&` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 947: `!has_finding_code(out, "unwind_reuse_strong") &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 948: `(has_finding_code(out, "handler_dominance") ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 949: `out->rf_invalid_ranges > 0 || out->rf_oob > 0 || out->rf_unwind_oob > 0 ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 950: `out->rf_unsorted > 0 || out->rf_overlaps > 0 || out->rf_duplicate_begins > 0 ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 951: `out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 952: `out->unwind_exec_section > 0 || out->unwind_unusual_section > 0 ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и

связь строки с предыдущими проверками.

Строка 953: `out->unwind_writable_section > 0 || out->handler_nonexec_section > 0 ||` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 954: `out->handler_writable_section > 0)) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 955: `add_score(&score, SCORE_UNWIND_REUSE_MEDIUM);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 956: `FindingLocation location = { true, dominant_unwind_rva };` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 957: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 958: `add_finding(out, "unwind_reuse_moderate", "unwind_reuse", "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 959: `"Moderate UnwindInfo reuse is corroborated by another static signal: max %u entries share same UnwindInfoRVA (ratio=%.2f)",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 960: `(unsigned)unwind_max, out->unwind_reuse_ratio),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 961: `location, unwind_max);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 962: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 964: `/* Placement heuristics */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 965: `if (out->unwind_exec_section) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 966: `add_score(&score, SCORE_UNWIND_EXEC_SECTION);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 967: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 968: `add_finding(out, "unwind_in_exec_section", "metadata_placement", "high",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 969: `"UnwindInfo located in executable section(s): %zu", out->unwind_exec_section),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 970: `executable_unwind_loc, out->unwind_exec_section);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 971: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 972: `if (out->unwind_unusual_section) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 973: `int add = (out->unwind_unusual_section > (n / 20)) ? SCORE_UNWIND_UNUSUAL_SECTION_HIGH : SCORE_UNWIND_UNUSUAL_SECTION_LOW; /* >5% => broader placement pattern */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 974: `add_score(&score, add);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 975: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 976: `add_finding(out, "unwind_unusual_section", "metadata_placement", add >= 15 ? "high" : "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 977: `"UnwindInfo located in unusual sections (not .xdata/.rdata/.pdata): %zu", out->unwind_unusual_section),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 978: `unusual_unwind_loc, out->unwind_unusual_section);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 979: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 980: `if (out->unwind_writable_section) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 981: `int add = (out->unwind_writable_section > (n / 20)) ? SCORE_UNWIND_WRITABLE_SECTION_HIGH : SCORE_UNWIND_WRITABLE_SECTION_LOW;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 982: `add_score(&score, add);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 983: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 984: `add_finding(out, "unwind_writable_section", "metadata_placement", add >= 18 ? "high" : "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 985: `"UnwindInfo located in writable section(s): %zu", out->unwind_writable_section),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 986: `writable_unwind_loc, out->unwind_writable_section);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 987: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 988: `if (out->handler_nonexec_section) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 989: `add_score(&score, SCORE_HANDLER_NONEXEC);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 990: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 991: `add_finding(out, "handler_nonexec_section", "metadata_placement", "high",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 992: `"Exception handler RVA in non-executable/unknown section: %zu", out->handler_nonexec_section),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 993: `nonexec_handler_loc, out->handler_nonexec_section);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 994: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 995: `if (out->handler_writable_section) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 996: `add_score(&score, SCORE_HANDLER_WRITABLE);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 997: `annotate_finding(` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 998: `add_finding(out, "handler_writable_section", "metadata_placement", "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 999: `"Exception handler RVA in writable section(s): %zu", out->handler_writable_section),` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1000: `writable_handler_loc, out->handler_writable_section);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1001: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 1003: `if (out->unwind_exec_section > 0 && out->handler_nonexec_section > 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 1004: `add_score(&score, SCORE_COMBO_PLACEMENT_EXEC_AND_HANDLER);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1005: `add_finding(out, "placement_exec_nonexec_handler_combo", "metadata_placement", "high",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1006: `"Unwind metadata in executable section(s) coincides with handler targets in non-executable/unknown section(s); the combined placement pattern is much harder to explain benignly.");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1007: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 1008: `if ((out->unwind_exec_section > 0 || out->handler_nonexec_section > 0) &&` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 1009: `(out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 || out->rf_unwind_oob > 0)) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1010: `add_score(&score, SCORE_COMBO_PLACEMENT_AND_INTEGRITY);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1011: `add_finding(out, "placement_integrity_combo", "unwind_integrity", "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1012: `"Suspicious metadata placement is corroborated by UNWIND_INFO / UnwindInfoRVA integrity problems; treat the combination as stronger than either signal alone.");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1013: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 1014: `if (out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_MEDIUM && has_finding_code(out, "handler_dominance")) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 1015: `add_score(&score, SCORE_COMBO_REUSE_AND_DOMINANCE);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1016: `add_finding(out, "reuse_dominance_combo", "unwind_reuse", "medium",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1017: `"Heavy UnwindInfo reuse coincides with a dominant handler target, which is consistent with centralized dispatcher-style metadata reuse.");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1018: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 1019: `if (out->unwind_writable_section > 0 && out->handler_writable_section > 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 1020: `add_score(&score, SCORE_COMBO_WRITABLE_METADATA_AND_HANDLER);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1021: `add_finding(out, "writable_metadata_handler_combo", "metadata_placement", "high",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1022: `"Writable unwind metadata coincides with writable handler targets; this combination is uncommon for normal compiler-produced images and deserves close review.");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1023: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 1025: `if (score == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 1026: `append_note(out, "No strong anomalies detected by heuristics.");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1027: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 1028: `out->score = score;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1029: `finalize_verdict(out);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 1031: `hmap_free(&unwind_counts);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 1032: `hmap_free(&handler_counts);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 1033: `if (!sb_ok(&out->details)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 1034: `sb_append(err, "Out of memory while building heuristic details.\n");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 1035: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 1036: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 1037: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 1038: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 1040: void heuristics_free(HeuristicReport* r) { - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 1041: if (!r) return; - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 1042: sb_free(&r->details); - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 1043: memset(r, 0, sizeof(*r)); - Эта строка заполняет область памяти байтом, обычно нулём. В проекте это часто приводит структуру к известному начальному состоянию перед заполнением.

Строка 1044: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Файл src/main.c

Файл содержит 527 строк. Ниже идут непустые строки с учебным объяснением. Смысл упражнения - не заучить комментарий, а научиться самостоятельно выполнять такой разбор.

Строка 1: #include <stdio.h> - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 2: #include <stdlib.h> - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 3: #include <string.h> - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 4: #include <stdbool.h> - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 5: #include <errno.h> - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 6: #include <limits.h> - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 8: #include "util.h" - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 9: #include "pe.h" - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 10: #include "unwind.h" - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 11: #include "heuristics.h" - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 12: #include "report.h" - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 13: #include "dynamic.h" - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 14: #include "app_info.h" - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 16: #if defined(_WIN32) - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 17: #include <windows.h> - Это директива препроцессора. Она подключает объявления типов, функций или констант, чтобы компилятор мог проверить дальнейший код. Такая строка не выполняется во время работы программы.

Строка 18: #endif - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 20: typedef struct Args { - Здесь объявляется новый составной тип. struct собирает связанные поля в один объект, а typedef позволяет использовать короткое имя типа без повторения слова struct.

Строка 21: `const char* input;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 22: `bool dump;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 23: `bool show_ops;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 24: `bool heuristics;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 25: `const char* csv_path;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 26: `const char* ghidra_csv_path;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 27: `const char* json_path;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 28: `const char* html_path;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 29: `int dump_limit; /* -1 => all */` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 30: `bool follow_chain;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 31: `bool strict_abi;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 32: `int max_chain_depth;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 33: `bool dynamic;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 34: `uint32_t timeout_ms;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 35: `uint32_t trace_steps;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 36: `const char** child_argv;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 37: `int child_argc;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 38: `} Args;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 40: `typedef enum ParseResult {` - Здесь объявляется перечисление именованных констант. В проекте enum часто используется для кодов результата, flags или битовых признаков, чтобы не писать магические числа.

Строка 41: `PARSE_OK = 0,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 42: `PARSE_HELP = 1,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 43: `PARSE_VERSION = 2,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 44: `PARSE_ERR = 3` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 45: `} ParseResult;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 47: `static char* dup_cstr(const char* s) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 48: `if (!s) return NULL;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 49: `size_t n = strlen(s);` - Здесь используется функция работы со строками C. Нужно помнить, что строка C завершается нулевым байтом, а указатель на строку сам по себе не хранит длину.

Строка 50: `char* out = (char*)malloc(n + 1u);` - Эта строка выделяет память в heap. После неё нужно проверить результат на NULL и определить, кто будет освобождать ресурс.

Строка 51: `if (!out) return NULL;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 52: `memcpy(out, s, n + 1u);` - Эта строка копирует указанное число bytes. `memcpy` не знает границ буфера, поэтому корректность должна быть доказана проверками раньше.

Строка 53: `return out;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 54: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 56: `static char* argv_text_to_utf8(const char* s) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 57: `if (!s) return NULL;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 58: `#if defined(_WIN32)` - Это условная компиляция. Препроцессор решает, попадёт ли следующий код в сборку. В проекте так отделяются Windows-зависимые части и защитные объявления.

Строка 59: `int wide_len = MultiByteToWideChar(CP_ACP, 0, s, -1, NULL, 0);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 60: `if (wide_len <= 0) return dup_cstr(s);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 61: `wchar_t* wide = (wchar_t*)calloc((size_t)wide_len, sizeof(wchar_t));` - Эта строка выделяет память в heap. После неё нужно проверить результат на NULL и определить, кто будет освобождать ресурс.

Строка 62: `if (!wide) return NULL;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 63: `if (MultiByteToWideChar(CP_ACP, 0, s, -1, wide, wide_len) <= 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 64: `free(wide);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 65: `return dup_cstr(s);` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 66: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 67: `int utf8_len = WideCharToMultiByte(CP_UTF8, 0, wide, -1, NULL, 0, NULL, NULL);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 68: `if (utf8_len <= 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 69: `free(wide);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 70: `return dup_cstr(s);` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 71: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 72: `char* out = (char*)malloc((size_t)utf8_len);` - Эта строка выделяет память в heap. После неё нужно проверить результат на NULL и определить, кто будет освобождать ресурс.

Строка 73: `if (!out) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 74: `free(wide);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 75: `return NULL;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 76: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 77: `if (WideCharToMultiByte(CP_UTF8, 0, wide, -1, out, utf8_len, NULL, NULL) <= 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 78: `free(out);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 79: `free(wide);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 80: `return dup_cstr(s);` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 81: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 82: `free(wide);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 83: `return out;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 84: `#else` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 85: `return dup_cstr(s);` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 86: `#endif` - Это продолжение блока условной компиляции. Оно влияет на то, какой код увидит компилятор для конкретной платформы или конфигурации.

Строка 87: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 89: `static void print_version(void) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 90: `printf("EH/Unwind Analyzer %s\n", app_version());` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 91: `printf("Report schema: %s\n", app_report_schema_version());` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 92: `printf("Build: %s, %s, %s\n", app_build_compiler(), app_build_platform(), app_build_mode());` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 93: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 95: `static void usage(const char* argv0) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 96: `printf("EH/Unwind Analyzer %s (PE x64)\n", app_version());` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 97: `printf("Usage: %s <pefile> [options]\n\n", argv0);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 98: `printf("Options:\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 99: `printf("  --dump  Dump all RUNTIME_FUNCTION entries\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 100: `printf("  --limit N  Limit dump to first N entries (default: all when --dump)\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 101: `printf("  --ops  With --dump: also print unwind ops list per function\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 102: `printf("  --heuristics  Run EH/Unwind anomaly heuristics and print report\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 103: `printf("  --csv <path>  Write analysis CSV (full fields)\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 104: `printf("  --ghidra-csv <path>  Write CSV for Ghidra import (begin/end/name/comment)\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 105: `printf("  --json <path>  Write JSON (functions + heuristic/dynamic assessment)\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 106: `printf("  --html <path>  Write a self-contained interactive HTML report\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 107: `printf("  --no-follow-chain  Do not follow UNW_FLAG_CHAININFO\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 108: `printf("  --strict-abi  Flag formal Microsoft x64 unwind ABI violations\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 109: `printf("  --max-chain-depth N  Max chain depth, 1..16 (default 8)\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 110: `printf("  --dynamic  Run Windows-only dynamic exception validation (debugger)\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 111: `printf("  --timeout-ms N  Dynamic mode: hard timeout, N>0 (default 15000)\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 112: `printf("  --trace-steps N  Dynamic mode: single-step samples after handler hit (default 64, range 0..256)\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 113: `printf("  --  Separator: remaining args are passed to the target program\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 114: `printf("  --version  Show version and build information\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 115: `printf("  --help  Show this help\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 116: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 118: `static bool append_invocation_arg(StrBuf* out, const char* arg) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 119: `if (!out || !arg) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 120: `bool quote = (*arg == '\0') || strpbrk(arg, " \t\"") != NULL;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 121: `if (!quote) return sb_append(out, arg);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 122: `if (!sb_append(out, "\"")) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 123: `for (const char* p = arg; *p; p++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 124: `if (*p == '"' || *p == '\\') {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 125: `if (!sb_appendn(out, "\\", 1)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 126: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 127: `if (!sb_appendn(out, p, 1)) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 128: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 129: `return sb_append(out, "\"");` - Эта строка добавляет текст в динамический `StrBuf`. Нужно учитывать, что append может не удаться из-за нехватки памяти или переполнения capacity.

Строка 130: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 132: `static bool build_invocation(int argc, char** argv, StrBuf* out) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 133: `if (!out) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 134: `for (int i = 0; i < argc; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 135: `if (strcmp(argv[i], "--") == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 136: `int redacted = argc - i - 1;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 137: `if (i && !sb_appendn(out, " ", 1)) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 138: `if (!sb_appendf(out, "-- [%d target argument%s redacted]",` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 139: `redacted, redacted == 1 ? "" : "s")) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 140: `return false;` - Функция сообщает вызывающему коду, что действие не выполнено безопасно. После такого результата вызывающий код не должен использовать выходные данные как корректные.

Строка 141: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 142: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 143: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 144: `if (i && !sb_appendn(out, " ", 1)) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 145: `char* utf8_arg = argv_text_to_utf8(argv[i]);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 146: `if (!utf8_arg) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 147: `bool ok = append_invocation_arg(out, utf8_arg);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 148: `free(utf8_arg);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 149: `if (!ok) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 150: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 151: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 152: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 154: `static bool parse_long_range(const char* text, long min_value, long max_value, long* out) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 155: `if (!text || !*text || !out) return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 156: `errno = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 157: `char* end = NULL;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 158: `long value = strtol(text, &end, 10);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 159: `if (errno == ERANGE || end == text || *end != '\0') return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 160: `if (value < min_value || value > max_value) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 161: `*out = value;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 162: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 163: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 165: `static bool parse_u32_range(const char* text, uint32_t min_value, uint32_t max_value, uint32_t* out) {` - Это начало функции, которая возвращает логический статус. `true` означает успешное выполнение договора функции, `false` означает ошибку или невозможность безопасно продолжать.

Строка 166: `if (!text || !*text || !out || text[0] == '-') return false;` - Это защитная проверка входа. Код отсекает нулевой указатель или недопустимое состояние до опасного действия. В системном C такие проверки стоят раньше разыменования и копирования памяти.

Строка 167: `errno = 0;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 168: `char* end = NULL;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 169: `unsigned long value = strtoul(text, &end, 10);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 170: `if (errno == ERANGE || end == text || *end != '\0') return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 171: `if (value < (unsigned long)min_value || value > (unsigned long)max_value) return false;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 172: `*out = (uint32_t)value;` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 173: `return true;` - Функция сообщает успешное выполнение договора. Обычно к этому моменту все выходные параметры уже заполнены и проверки пройдены.

Строка 174: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 176: `static ParseResult parse_args(int argc, char** argv, Args* a) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 177: `memset(a, 0, sizeof(*a));` - Эта строка заполняет область памяти байтом, обычно нулём. В проекте это часто приводит структуру к известному начальному состоянию перед заполнением.

Строка 178: `a->dump_limit = -1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 179: `a->follow_chain = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 180: `a->max_chain_depth = 8;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 181: `a->timeout_ms = 15000;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 182: `a->trace_steps = 64;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 184: `if (argc < 2) return PARSE_ERR;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 185: `if (strcmp(argv[1], "--help") == 0 || strcmp(argv[1], "-h") == 0) return PARSE_HELP;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 186: `if (strcmp(argv[1], "--version") == 0) return PARSE_VERSION;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 187: `a->input = argv[1];` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 189: `for (int i = 2; i < argc; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 190: `const char* s = argv[i];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 191: `if (strcmp(s, "--") == 0) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 192: `a->child_argc = argc - (i + 1);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 193: `a->child_argv = (const char**)&argv[i + 1];` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 194: `break;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 195: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 196: `if (strcmp(s, "--help") == 0 || strcmp(s, "-h") == 0) return PARSE_HELP;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 197: `else if (strcmp(s, "--version") == 0) return PARSE_VERSION;` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 198: `else if (strcmp(s, "--dump") == 0) a->dump = true;` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 199: `else if (strcmp(s, "--ops") == 0) a->show_ops = true;` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 200: `else if (strcmp(s, "--heuristics") == 0) a->heuristics = true;` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 201: `else if (strcmp(s, "--dynamic") == 0) a->dynamic = true;` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 202: `else if (strcmp(s, "--no-follow-chain") == 0) a->follow_chain = false;` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 203: `else if (strcmp(s, "--strict-abi") == 0) a->strict_abi = true;` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 204: `else if (strcmp(s, "--limit") == 0 && i + 1 < argc) {` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 205: `long value = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 206: `if (!parse_long_range(argv[++i], 0, INT_MAX, &value)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 207: `fprintf(stderr, "Invalid value for --limit: %s (expected 0..%d)\n", argv[i], INT_MAX);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 208: `return PARSE_ERR;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 209: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 210: `a->dump_limit = (int)value;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 211: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 212: `else if (strcmp(s, "--max-chain-depth") == 0 && i + 1 < argc) {` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 213: `long value = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 214: `if (!parse_long_range(argv[++i], 1, 16, &value)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 215: `fprintf(stderr, "Invalid value for --max-chain-depth: %s (expected 1..16)\n", argv[i]);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 216: `return PARSE_ERR;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 217: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 218: `a->max_chain_depth = (int)value;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 219: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 220: `else if (strcmp(s, "--timeout-ms") == 0 && i + 1 < argc) {` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 221: `if (!parse_u32_range(argv[++i], 1, UINT32_MAX, &a->timeout_ms)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 222: `fprintf(stderr, "Invalid value for --timeout-ms: %s (expected 1..%u)\n",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 223: `argv[i], (unsigned)UINT32_MAX);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 224: `return PARSE_ERR;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 225: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 226: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 227: `else if (strcmp(s, "--trace-steps") == 0 && i + 1 < argc) {` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 228: `if (!parse_u32_range(argv[++i], 0, 256, &a->trace_steps)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 229: `fprintf(stderr, "Invalid value for --trace-steps: %s (expected 0..256)\n", argv[i]);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 230: `return PARSE_ERR;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 231: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 232: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 233: `else if (strcmp(s, "--csv") == 0 && i + 1 < argc) { a->csv_path = argv[++i]; }` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 234: `else if (strcmp(s, "--ghidra-csv") == 0 && i + 1 < argc) { a->ghidra_csv_path = argv[++i]; }` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 235: `else if (strcmp(s, "--json") == 0 && i + 1 < argc) { a->json_path = argv[++i]; }` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 236: `else if (strcmp(s, "--html") == 0 && i + 1 < argc) { a->html_path = argv[++i]; }` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и

понять, какое состояние данных соответствует этой ветке.

Строка 237: `else {` - Это альтернативная ветка ранее проверенного условия. При чтении нужно вернуться к предыдущему `if` и понять, какое состояние данных соответствует этой ветке.

Строка 238: `fprintf(stderr, "Unknown/invalid option: %s\n", s);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 239: `return PARSE_ERR;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 240: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 241: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 242: `return PARSE_OK;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 243: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 245: `static void print_one(size_t idx, const RuntimeFunction* r, const UnwindSummary* u, bool ok) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 246: `uint32_t size = (r->end_rva >= r->begin_rva) ? (r->end_rva - r->begin_rva) : 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 247: `char fbuf[64];` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 248: `unwind_flags_to_string(u->flags, fbuf);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 250: `printf("[%zu] 0x%08X..0x%08X sz=%-6u unwind=0x%08X ",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 251: `idx, (unsigned)r->begin_rva, (unsigned)r->end_rva, (unsigned)size, (unsigned)r->unwind_rva);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 253: `if (!ok) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 254: `printf("UNWIND: <parse failed%s%s>\n",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 255: `u->parse_error[0] ? ": " : "",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 256: `u->parse_error);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 257: `return;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 258: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 260: `printf("v%u flags=%s prolog=%u stack=%uB",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 261: `(unsigned)u->version, fbuf, (unsigned)u->prolog_size, (unsigned)u->fixed_stack_bytes);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 263: `if (u->frame_reg) printf(" fp=%s+%u", unwind_reg_name(u->frame_reg), (unsigned)(u->frame_off * 16u));` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 264: `if (u->has_handler) printf(" handler=0x%08X", (unsigned)u->handler_rva);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 265: `if (u->has_chain) printf(" chain_u=0x%08X", (unsigned)u->chain.unwind_rva);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 266: `if (u->anomalies) printf(" anom=0x%08X", (unsigned)u->anomalies);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 267: `printf("\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 268: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 270: `int main(int argc, char** argv) {` - Это объявление или определение функции. Сначала указан тип результата, затем имя функции, затем параметры. По этой строке нужно понять входы, выход и ответственность функции.

Строка 271: `Args a;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 272: `int exit_code = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 273: `ParseResult pres = parse_args(argc, argv, &a);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 274: `if (pres != PARSE_OK) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 275: `if (pres == PARSE_VERSION) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 276: `print_version();` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 277: `return 0;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 278: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 279: `usage(argv[0]);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 280: `return (pres == PARSE_HELP) ? 0 : 2;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 281: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 283: `uint8_t* owned = NULL;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 284: `Buf b;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 285: `if (!buf_init_from_file(&b, a.input, &owned)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 286: `fprintf(stderr, "Failed to read file: %s\n", a.input);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 287: `return 1;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 288: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 290: `StrBuf err;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 291: `sb_init(&err, 1024);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 293: `PeImage img;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 294: `if (!pe_parse(&img, &b, &err)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 295: `fprintf(stderr, "PE parse error:\n%s", err.s ? err.s : "");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 296: `sb_free(&err);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 297: `buf_free_owned(owned);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 298: `return 1;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 299: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 301: `RuntimeFunction* rf = NULL;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 302: `size_t rf_count = 0;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 303: `StrBuf err2;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 304: `sb_init(&err2, 1024);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 306: `RuntimeFunctionReadResult rf_read = unwind_read_runtime_functions_ex(` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 307: `&b, &img, &rf, &rf_count, &err2);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 308: `if (rf_read != RUNTIME_FUNCTION_READ_OK) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 309: `fprintf(stderr, "Failed to read exception directory:\n%s", err2.s ? err2.s : "");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 310: `if (rf_read == RUNTIME_FUNCTION_READ_OOM) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 311: `pe_free(&img);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 312: `sb_free(&err);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 313: `sb_free(&err2);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 314: `buf_free_owned(owned);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 315: `return 1;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 316: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 317: /* - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 318: * Keep the established report field/finding code for compatibility. - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 319: * It now covers every structurally unreadable exception directory. - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 320: */ - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 321: img.exception_directory_oob = true; - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 322: fprintf(stderr, "Continuing with a partial structural report.\n"); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 323: exit_code = 1; - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 324: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 326: report_print_brief(a.input, &img, rf_count); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 328: UnwindSummary* us = (UnwindSummary*)calloc(rf_count ? rf_count : 1, sizeof(UnwindSummary)); - Эта строка выделяет память в heap. После неё нужно проверить результат на NULL и определить, кто будет освобождать ресурс.

Строка 329: bool* us_ok = (bool*)calloc(rf_count ? rf_count : 1, sizeof(bool)); - Эта строка выделяет память в heap. После неё нужно проверить результат на NULL и определить, кто будет освобождать ресурс.

Строка 330: if (!us || !us_ok) { - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 331: fprintf(stderr, "Out of memory.\n"); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 332: free(us); - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 333: free(us_ok); - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 334: free(rf); - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 335: pe_free(&img); - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 336: sb_free(&err); - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 337: sb_free(&err2); - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 338: buf_free_owned(owned); - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 339: return 1; - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 340: } - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 342: UnwindParseOpts uopts = { - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 343: `.max_chain_depth = a.max_chain_depth,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 344: `.follow_chain = a.follow_chain,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 345: `.strict_abi = a.strict_abi` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 346: `};` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 348: `StrBuf ops;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 349: `sb_init(&ops, 1024);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 351: `for (size_t i = 0; i < rf_count; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 352: `StrBuf perr;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 353: `sb_init(&perr, 256);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 354: `if (a.show_ops) { ops.len = 0; if (ops.s) ops.s[0]=0; }` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 355: `bool ok = unwind_parse(&b, &img, rf[i].unwind_rva, &uopts, &us[i], a.show_ops ? &ops : NULL, &perr);` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 356: `us_ok[i] = ok;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 357: `if (a.dump && a.show_ops && ok) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 358: `/* ops will be printed in dump loop below */` - Это комментарий разработчика. Он не выполняется, но объясняет намерение кода или связь с форматом/ABI. Комментарий полезен, но корректность всё равно должна подтверждаться кодом.

Строка 359: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 360: `sb_free(&perr);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 361: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 363: `if (a.dump) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 364: `size_t limit = (a.dump_limit < 0) ? rf_count : (size_t)a.dump_limit;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 365: `if (limit > rf_count) limit = rf_count;` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 366: `printf("\n[Dump] Showing %zu / %zu entries\n", limit, rf_count);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 367: `for (size_t i = 0; i < limit; i++) {` - Это цикл с явным счётчиком или проходом по массиву. В проекте такие циклы читают секции, runtime functions, unwind slots, findings или entries map. Важно проверить границы индекса.

Строка 368: `print_one(i, &rf[i], &us[i], us_ok[i]);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 369: `if (a.show_ops && us_ok[i]) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 370: `StrBuf ops2;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 371: `StrBuf perr;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 372: `sb_init(&ops2, 1024);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 373: `sb_init(&perr, 256);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 374: `(void)unwind_parse(&b, &img, rf[i].unwind_rva, &uopts, &us[i], &ops2, &perr);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 375: `if (ops2.s && ops2.len) printf("%s", ops2.s);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 376: `sb_free(&ops2);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 377: `sb_free(&perr);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 378: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 379: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 380: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 382: `HeuristicReport hr;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 383: `HeuristicReport* hr_ptr = NULL;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 384: `DynamicReport dr;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 385: `memset(&dr, 0, sizeof(dr));` - Эта строка заполняет область памяти байтом, обычно нулём. В проекте это часто приводит структуру к известному начальному состоянию перед заполнением.

Строка 386: `DynamicReport* dr_ptr = NULL;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 387: `bool dr_initialized = false;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 388: `if (a.heuristics) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 389: `StrBuf perr;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 390: `sb_init(&perr, 512);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 391: `if (heuristics_analyze(&img, rf, us, us_ok, rf_count, &hr, &perr)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 392: `report_print_heuristics(&hr);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 393: `hr_ptr = &hr;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 394: `} else {` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 395: `fprintf(stderr, "Heuristic analysis failed:\n%s", perr.s ? perr.s : "");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 396: `exit_code = 1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 397: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 398: `sb_free(&perr);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 399: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 401: `if (a.dynamic) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 402: `StrBuf derr;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 403: `sb_init(&derr, 1024);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 404: `printf("\n[Dynamic] Starting debugger evidence collection (Windows only)...\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 405: `bool ran = dynamic_collect_windows(a.input, a.child_argv, a.child_argc,` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 406: `&img, rf, us, us_ok, rf_count,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 407: `a.timeout_ms, a.trace_steps,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 408: `&dr, &derr);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 409: `dr_initialized = true;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 410: `dr_ptr = &dr;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 411: `if (!ran) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 412: `fprintf(stderr, "[Dynamic] Failed: %s", derr.s ? derr.s : "");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 413: `exit_code = 1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 414: `} else {` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 415: `printf("[Dynamic] Done. exceptions_seen=%u unwind_handler_hits=%u veh_hits=%u dyn_ftable_calls=%u\n",` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 416: `(unsigned)dr.exceptions_seen,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 417: `(unsigned)dr.unwind_handler_hits,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 418: `(unsigned)dr.veh_hits,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 419: `(unsigned)dr.rtl_add_function_table_calls);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 420: `if (dr.details.s && dr.details.len) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 421: `printf("%s\n", dr.details.s);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 422: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 423: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 424: `sb_free(&derr);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 425: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 427: `if (a.csv_path) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 428: `StrBuf e;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 429: `sb_init(&e, 256);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 430: `if (!report_write_csv(a.csv_path, &img, rf, us, us_ok, rf_count, false, &e)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 431: `fprintf(stderr, "CSV write failed:\n%s", e.s ? e.s : "");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 432: `exit_code = 1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 433: `} else {` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 434: `printf("\nWrote CSV: %s\n", a.csv_path);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 435: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 436: `sb_free(&e);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 437: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 438: `if (a.ghidra_csv_path) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 439: `StrBuf e;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 440: `sb_init(&e, 256);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 441: `if (!report_write_csv(a.ghidra_csv_path, &img, rf, us, us_ok, rf_count, true, &e)) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 442: `fprintf(stderr, "Ghidra CSV write failed:\n%s", e.s ? e.s : "");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 443: `exit_code = 1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 444: `} else {` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 445: `printf("\nWrote Ghidra CSV: %s\n", a.ghidra_csv_path);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 446: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 447: `sb_free(&e);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 448: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 450: `StrBuf invocation = {0};` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 451: `char timestamp[32] = "";` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 452: `char input_sha256[65] = "";` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 453: `char* input_display = NULL;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 454: `ReportMetadata metadata;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 455: `memset(&metadata, 0, sizeof(metadata));` - Эта строка заполняет область памяти байтом, обычно нулём. В проекте это часто приводит структуру к известному начальному состоянию перед заполнением.

Строка 456: `ReportMetadata* metadata_ptr = NULL;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 457: `if (a.json_path || a.html_path) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 458: `input_display = argv_text_to_utf8(a.input);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 459: `bool metadata_ok = sb_init(&invocation, 256) &&` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 460: `input_display &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 461: `build_invocation(argc, argv, &invocation) &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 462: `app_utc_timestamp(timestamp, sizeof(timestamp)) &&` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 463: `sha256_hex(b.data, b.size, input_sha256);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 464: `if (!metadata_ok) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 465: `fprintf(stderr, "Failed to prepare report metadata.\n");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 466: `exit_code = 1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 467: `} else {` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 468: `metadata.tool_version = app_version();` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 469: `metadata.report_schema_version = app_report_schema_version();` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 470: `metadata.generated_at_utc = timestamp;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 471: `metadata.input_sha256 = input_sha256;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 472: `metadata.input_size = (uint64_t)b.size;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 473: `metadata.command_line = invocation.s ? invocation.s : "";` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 474: `metadata.build_compiler = app_build_compiler();` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 475: `metadata.build_platform = app_build_platform();` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 476: `metadata.build_mode = app_build_mode();` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 477: `metadata.strict_abi = a.strict_abi;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 478: `metadata.follow_chain = a.follow_chain;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 479: `metadata.max_chain_depth = (uint32_t)a.max_chain_depth;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 480: `metadata_ptr = &metadata;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 481: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 482: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 484: `if (a.json_path) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 485: `StrBuf e;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 486: `sb_init(&e, 256);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 487: `if (metadata_ptr &&` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 488: `!report_write_json_ex(a.json_path, input_display ? input_display : a.input,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 489: `&img, rf, us, us_ok, rf_count,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 490: `hr_ptr, dr_ptr, metadata_ptr, &e)) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 491: `fprintf(stderr, "JSON write failed:\n%s", e.s ? e.s : "");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 492: `exit_code = 1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 493: `} else if (metadata_ptr) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 494: `printf("\nWrote JSON: %s\n", a.json_path);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 495: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 496: `sb_free(&e);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 497: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 498: `if (a.html_path) {` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 499: `StrBuf e;` - Это объявление переменной или объекта. При чтении нужно определить тип, имя, начальное значение и срок жизни переменной внутри текущего блока.

Строка 500: `sb_init(&e, 256);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 501: `if (metadata_ptr &&` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 502: `!report_write_html(a.html_path, input_display ? input_display : a.input,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 503: `&img, rf, us, us_ok, rf_count,` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 504: `hr_ptr, dr_ptr, metadata_ptr, &e)) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 505: `fprintf(stderr, "HTML write failed:\n%s", e.s ? e.s : "");` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 506: `exit_code = 1;` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 507: `} else if (metadata_ptr) {` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 508: `printf("\nWrote HTML: %s\n", a.html_path);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 509: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 510: `sb_free(&e);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 511: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.

Строка 512: `sb_free(&invocation);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 513: `free(input_display);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 515: `if (a.heuristics && hr_ptr) heuristics_free(&hr);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 516: `if (dr_initialized) dynamic_report_free(&dr);` - Это условное ветвление. Нужно прочитать условие как проверяемый инвариант: если он нарушен или выполнен, программа выбирает безопасный путь, фиксирует ошибку или включает признак.

Строка 518: `sb_free(&ops);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 519: `free(us_ok);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 520: `free(us);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 521: `free(rf);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 522: `pe_free(&img);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 523: `sb_free(&err);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 524: `sb_free(&err2);` - Эта строка освобождает heap-память. После освобождения объект больше нельзя использовать как живой ресурс.

Строка 525: `buf_free_owned(owned);` - Эта строка является частью текущего алгоритма. Чтобы понять её, нужно определить используемые имена, типы этих имён, операторы и связь строки с предыдущими проверками.

Строка 526: `return exit_code;` - Это выход из функции с результатом. Чтобы понять строку, нужно знать тип функции и смысл возвращаемого значения для вызывающего кода.

Строка 527: `}` - Эта строка задаёт границу блока. В C границы блока важны для области видимости переменных и для того, какие инструкции относятся к условию, циклу, функции или структуре.